物联网综合网关使用指南

UM2224 V2.01 data:2024/3/8

类别	内容
关键字	物联网万能综合网关 IOTGateway
摘要	边沿计算网关

©2022-2024 Guangdong Wangxuntong IoT Technology Co., Ltd

目录

1. 产品简介 8
2. 网关参数 8
2.1. 功能特点及应用特色 8
2.2. 接口说明 9
2.3. 性能参数 10
2.4. 电源接口 12
3. 网关配置流程 14
3.1. 修改网关 IP 14
3.2. 配置网关 IP 参数 15
3.3. 配置网关工作的脚本 17
4. 网关脚本编程 20
4.1. 网关编程语言介绍 20
4.2. Lua 基本语法介绍 21
5. AI 主机边缘计算网关 C#开发使用说明 25
5.1. 网关 C#开发简介 25
5.2. 开发流程介绍 26
5.2.1. 搭建 windows 开发环境 26
5.2.2. 测试程序编译 26
5.2.3. 将生成的程序拷进网关 27
5.2.4. 编辑运行脚本 28
6. 网关 VPN 远程调试 29
7. 小智 MCP 配置流程 33
7.1. 官方小智 MCP 接入点获取 33
7.2. 案例说明 35
7.3. 代码函数说明 36
7.4. 官方小智 MCP 接入点获取 39
7.5. 代码填写并烧入 39
8. AI 主机操作说明 39
8.1. 视频 AI 分析系统的使用 39

8.1.1. 识别区域画框 41
8.1.2. 抓拍记录 43
8.1.3. 人脸添加 44
8.2. AI 识别接口介绍 45
8.2.1. 寄存器映射表 45
8.2.2. 目标检测类型列表 46
8.3. 各识别类型详细参数（按编号排序） 48
8.3.1. 标签识别 48
8.3.2. 安全帽检测 48
8.3.3. 行人检测 50
8.3.4. 睡岗识别 51
8.3.5. 跌倒检测 54
8.3.6. 反光衣检测 56
8.3.7. 火焰烟雾检测 57
8.3.8. 人头检测 60
8.3.9. 电梯非机动车检测 61
8.3.10. 电动车头盔识别 65
8.3.11. 重型机械车辆检测 66
8.3.12. 井盖识别 75
8.3.13. 灭火器摆放识别 77
8.3.14. 通用打电话抽烟检测 79
8.3.15. 车辆检测 84
8.3.16. 疲劳检测 85
8.3.17. 垃圾桶满溢检测 85
8.3.18. 口罩佩戴检测 87
8.3.19. 非机动车检测 88
8.4. Modbus 寄存器通信规范 92
8.4.1. 核心寄存器地址定义 92
8.4.2. 数据读取规则 92
8.5. 典型应用案例代码 93

9. 脚本生成工具功能 95
10. 脚本函数接口 97
10.1. 配置串口通讯参数 97
10.2. Modbus/TCP 服务（从站） 98
10.2.1. 启动 modbus/tcp server（从站） 98
10.3. Modbus/TCP 数据读写（主站） 98
10.3.1. 从 01H 号功能码(coil status/线圈状态寄存器)中读取数据 98
10.3.2. 从 02H 号功能码(input status/输入状态寄存器)中读取数据 99
10.3.3. 从 03H 功能码(holding register/保持寄存器)中读取数据 100
10.3.4. 从 04H 号功能码(input register/输入寄存器) 中读取数据 100
10.3.5. 从 05H 号功能码(coil status/线圈寄存器) 中写数据 101
10.3.6. 从 06H 号功能码中写寄存器 102
10.4. Modbus/RTU 从站服务（从站） 102
10.4.1. 启动 modbus/RTU 从站服务（从站） 102
10.5. Modbus/RTU 数据读写（主站） 103
10.5.1. 从 01H 号功能码(coil status/线圈状态) 中读取数据 103
10.5.2. 从 02H 号功能码(input status/输入状态) 中读取数据 103
10.5.3. 从 03H 功能码(holding register/保持寄存器)中读取数据 104
10.5.4. 从 04H 号功能码(input register/输入寄存器) 中读取数据 105
10.5.5. 从 05H 号功能码(coil status/线圈寄存器) 中写数据 105
10.5.6. 从 06H 号功能码中写寄存器 106
10.5.7. 从 10H 号功能码中写多个寄存器 106
10.5.8. 从 0FH 号功能码中写多个线圈 107
10.5.9. 设置 modbus RTU 设备地址 107
10.6. LED 显示屏操作 108
10.6.1. LED 点阵屏初始化函数 108
10.6.2. LED 点阵屏显示 16x16 字符 109
10.6.3. 桌面式 LED 点阵屏驱动 110
10.6.4. LED 屏亮度调节函数 111
10.6.5. 新 LED 显示屏显示函数 111

10.6.6. 新 LED 显示屏清屏函数 112
10.6.7. 新 LED 屏亮度调节函数 112
10.7. dlt645-2007 协议智能电表驱动 112
10.7.1. 读电表状态信息 112
10.8. RFID 读卡器驱动 113
10.8.1. ID.读卡器驱动函数 113
10.8.2. IC 读卡器（带蜂鸣器）服务器对接函数 114
10.8.3. UHF 服务函数 115
10.9. 中移动云平台对接驱动 117
10.9.1. 添加云变量函数 117
10.9.2. 读云变量函数 117
10.9.3. 写云变量函数 118
10.9.4. 设置云平台对接函数 118
10.9.5. 向云平台获取某个网关云变量值 119
10.9.6. 向服务器发起 GET 请求 120
10.10. WOL 网卡唤醒函数 120
10.10.1. WOL 网卡唤醒函数 120
10.11. 网关系统函数 121
10.11.1. 发布脚本 121
10.11.2. 系统延时 121
10.11.3. 读网关寄存器 122
10.11.4. 写网关寄存器 122
10.11.5. 读网关线圈值 123
10.11.6. 写网关线圈值 123
10.11.7. 启动 web server 服务 124
10.11.8. 设置网关重启 125
10.11.9. 禁用网关上传下载功能 125
10.11.10. 创建线程函数 125
10.11.11. 添加变量事件函数 126
10.12. 串口调试函数 127

10.12.1. 字符数据发送 127
10.12.2. 16 进制数据发送 128
10.12.3. 串口数据接收字符型显示 128
10.12.4. 串口数据接收 16 进制显示 128
10.13. Zigbee 模块驱动 129
10.13.1. 获取 Zigbee 节点 AD 值 129
10.13.2. 获取 Zigbee 节点 DI 值 129
10.13.3. 设置 Zigbee 节点继电器输出 130
10.14. GSM 短信驱动 130
10.15. 超声波车位探测驱动 131
10.15.1. 超声波车位探测模块 131
10.16. 三菱 PLC 通信控制 132
10.16.1. FX3U 写驱动(PLC 设备 ID 固定位为 1) 132
10.16.2. FX3U 写驱动 133
10.16.3. FX3U 读驱动 134
10.16.4. FX2N 写驱动 135
10.16.5. FX2N 读驱动 136
10.17. dlt645-97 协议水表驱动 137
10.17.1. 读水表数信息 137
10.17.2. M-bus 水表采集器操作 137
10.18. 小米智能家居产品对接 138
10.18.1. 开启小米智能家居网关服务 138
10.18.2. 读智能家居产品状态 138
10.18.3. 写智能家居产品状态 139
10.19. Tlink 云平台对接 142
10.20. 阿里云平台对接 143
10.21. 私有云（MQTT）平台对接 143
10.21.1. 开启 MQTT 服务函数 143
10.21.2. 设置主题密码函数 144
10.22. 迅通云（TCP）平台对接 144

10.23. Home Assistant（MQTT）平台对接 145
10.23.1. 开启 MQTT 服务函数 145
10.23.2. 添加 HA 云变量 146
10.23.3. HA 更新上报数据 148
10.23.4. Home Assistant 代码例程 149
10.24. 窗帘电机驱动 151
10.24.1. 窗帘电机控制 151
10.24.2. 读取窗帘电机位置状态 152
10.25. LoRa 网关驱动 152
10.25.1. 网关启动 TCP 模式 152
10.25.2. 设定 LoRa 数据接收响应时长 153
10.26. 小智官方 MCP 连接 154
10.26.1. 网关 MCP 客户端连接到官方 MCP 接入点 154
10.27. AI 主机扬声器播放 155
10.27.1. 音频播放函数 155
11. 网关高级编程 158
12. 网关固件更新 160
13. 脚本编写案例 161
13.1. 远程抄表案例 161
14. Node-red 对接 164
14.1. 基于 Modbus 实现 Node-red 组态监测 164
14.1.1. 实验介绍 164
14.1.2. 实验准备 164
14.1.3. 实验步骤 164
14.1.4. 实验总结 177
14.2. 基于 zigbee2mqtt 实现 Node-red 组态监测 179
14.2.1. 实验介绍 179
14.2.2. 实验准备 179
14.2.3. 实验过程 179
14.2.4. 实验总结 190

14.3. 基于红外实现 node-red 组态控制 191
14.3.1. 实验介绍 191
14.3.2. 实验准备 191
14.3.3. 实验过程 195
14.3.4. 实验总结 208
14.4. Node-RED 实现西门子 S7 PLC 的数据采集及反控 210
14.4.1. 实验介绍 210
14.4.2. 实验准备 210
14.4.3. 实验过程 211
14.4.4. 实验总结 218
14.5. 采集 modbus 设备数据到 mqtt 服务器，APP inventor 通过 mqtt 进行采集与反控 219
14.5.1. 实验介绍 219
14.5.2. 实验准备 219
14.5.3. 实验过程 220
14.5.4. 实验总结 236
14.6. node-red 采集 modbus 设备数据到 mqtt 服务器，云平台通过 mqtt 进行采集与反控 237
14.6.1. 实验介绍 237
14.6.2. 实验准备 237
14.6.3. 实验过程 239
14.6.4. 实验总结 251

1. 产品简介

HWXT-A60 边缘计算网关，CPU 采用成熟的高性能双核心 A35 800MHz 64 位工业处理器。

网关电源输入采用业内领先的超宽降压 DC/DC 方案，为用户提供 DC6~28V 超宽压电源输入并支持交流供电。采用的同步整流降压技术能保持高效率并有效减少发热，整机运行温度的下降，能进一步提升长期不断电运行的可靠性和寿命。能抵抗 GB/T17626.5-2008 标准中最高等级 4 级（4KV）8/20uS 的雷击测试。多重电源防护，抗静电、过流、防反接等保护能有效保证野外等恶劣环境下的可靠运行。

网关集成 1 个 100M/10M 高速自适应网卡，网卡采用双级防雷防静电保护，能抵抗 2KV 雷击。

网关支持 2 路 RS485 通信接口，全部采用全独立电气隔离设计，RS485 电路采用三级防雷防静电保护，支持 GB/T17626.5-2008 标准中 10/700uS 测试的最高等级 4KV 防护。每路 RS485 全部采用完全独立隔离电源设计，每一个对外 RS485 接口都和其它 RS485 接口完全电气隔离，有效减少复杂电磁干扰环境下对 RS485 总线的串扰问题，提升整个系统通讯可靠性；同时配备 2 个 DO（数字输出）接口，可直接驱动继电器、指示灯等外部控制设备，满足边缘侧本地控制场景需求。

HWXT-A60 边缘计算网关内部集成可靠的工业 WIFI 模块和 RF 模块，支持灵活无线组网与远程数据传输；内置 MIC（麦克风）接口，可接入音频采集设备并搭配外部扬声器实现音频播放功能，适配语音提示、音频交互等场景；同时集成完善的异常恢复处理电路，有效解决无人值守应用中的故障自恢复需求。

稳定的硬件设计能够保证系统长时间正常运行不宕机，支持纯硬件定时看门狗，适合无人值守 7X24 小时运行的应用场景。

软件系统上，HWXT-A60 边缘计算网关运行最新的 linux5.10 内核，移植多种通信协议和物联网平台，支持 lua 脚本编程、python 脚本编程、node-red 流程化编程、c# 编程等开发配置模式，配套网关配置工具，可快速实现各项参数配置。

2. 网关参数

2.1. 功能特点及应用特色

1. 支持 10M/100M 以太网接入，灵活组网；
2. 支持 WIFI 无线联网，无需额外硬件，短距离通信便捷，信号稳定可靠；
3. 内置标准 Modbus TCP 服务器，支持多客户端并发访问；
4. 配备智能看门狗芯片，确保网关在恶劣环境下长期稳定运行；

©2022-2024 Guangdong Wangxuntong IoT Technology Co., Ltd

5. 具备强大的远程软件升级能力；
6. 提供一键恢复功能，可快速恢复系统出厂设置；
7. 支持 2 路 DO 输出，可连接负载实现设备控制及其他物联网操作；
8. 提供 1 路 RS485 通信接口（工作模式可配置），支持驱动电表、水表、Zigbee 终端、Modbus RTU 设备、PLC 设备、RFID 设备等；
9. 支持 1 路 USB 输出，可以连接多种 USB 扩展设备；
10. 支持通过脚本语言配置及管理网关参数；
11. 支持通过脚本语言采集与控制电表、水表、Zigbee 设备、Modbus RTU 设备、PLC 设备、RFID 设备等，各类设备均有对应脚本函数支持；
12. 支持通过脚本语言对接云服务器，网关厂商提供完善云服务平台，同时兼容中国移动云服务平台；
13. 支持通过脚本语言实现物联网系统基础逻辑判断与处理；
14. 提供网关厂商自主研发的脚本编写软件及脚本下载工具。

2.2. 接口说明

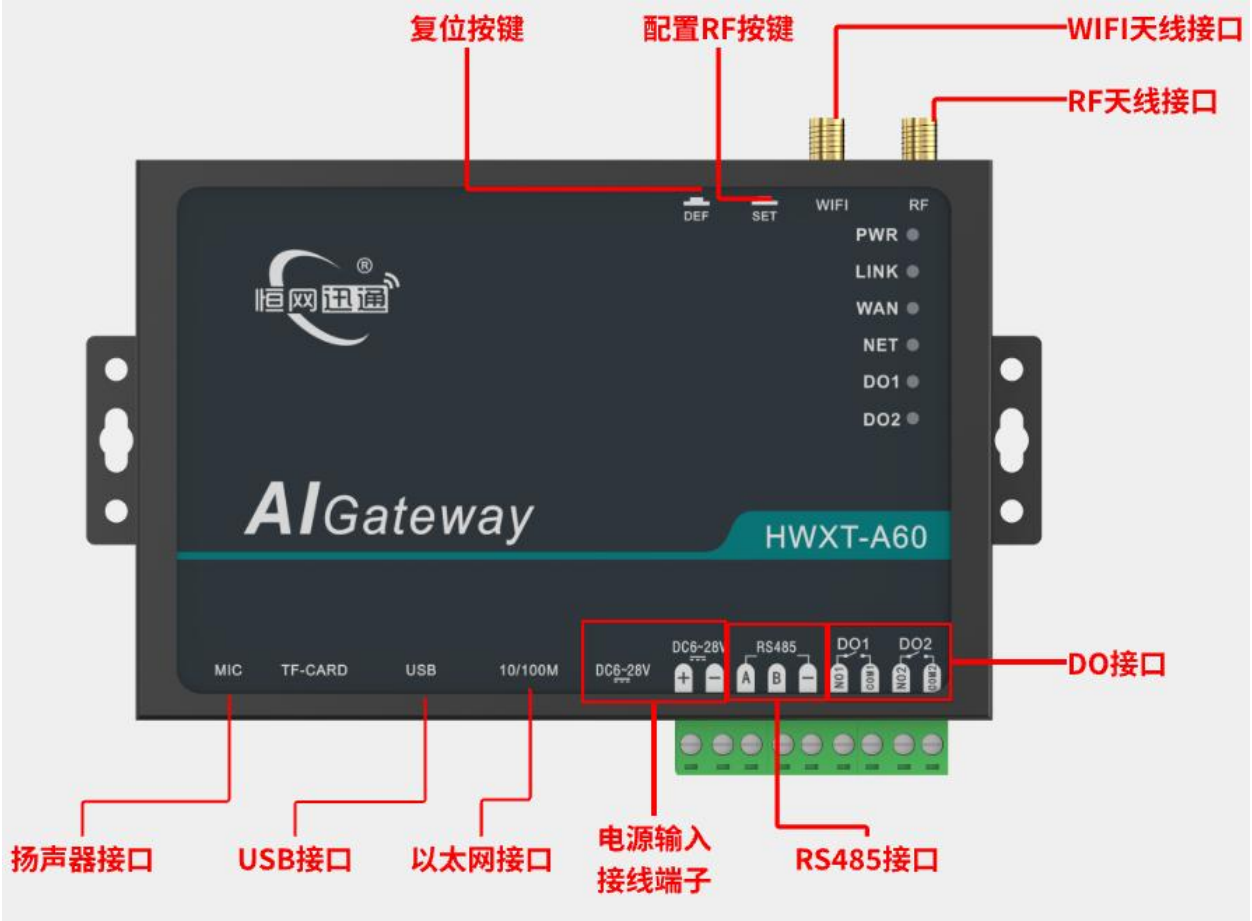


图 1 接口说明

2.3. 性能参数

CPU	双核心 A3564 位处理器,主频 800MH
内存	512MbyteDDR3 高性能内存
存储	8GByteeMMC
网口	1 个 100M/10M 以太网接口支持 AUTOMDI/MDIX 双级抗雷防护 ◆支持 GB/T17626.5-2008 标准中 10/700uS 测试的 3 级 2KV 防护 ◆±15kVHumanBodyModel ◆±15kVIEC61000-4-2AirDischarge
WIFI	频段支持：2.4~2.4835GHz 或 5.15~5.85GHz 协议支持：IEEE 802.11a/b/g/n 特性：工业级稳定性能，无需额外硬件，短距离无线组网便捷
RS485 接口	1 个全独立隔离 RS485 接口（支持收发数据指示灯）RS485 采用三级 防护 ◆支持 GB/T17626.5-2008 标准中 10/700uS 测试的最高等级 4KV 防护 ◆±15kVHumanBodyModel ◆±15kVIEC61000-4-2AirDischarge
电源接口	支持电源接口与 DC 电源插口 ◆标准 5.08mm 间距 3PIN 欧式端子接口 输入电压:直流 DC6~28V 电源防护 ◆GB/T17626.5-2008 标准 4 级(4KV)8/20uS 雷击测试 ◆防反接保护 ◆过压保护 ◆抗脉冲群保护 ◆抗静电: ◆±15kVHumanBodyModel ◆±15kVIEC61000-4-2AirDischarge
USBHOST 接口	板载 1 个 USBHOST 接口支持扩展 USB 外设 ◆对外提供 5V500mA 电流输出能力 ◆支持输出过载保护功能 ◆对外输出电源支持可编程开关控制 ◆±6kVHumanBodyModel ◆±8kVIEC61000-4-2AirDischarge 支持连接多种 USB 扩展设备（如外设模块、存储设备等）

其它拓展接口	DO（数字输出）接口：2 个，可直接驱动继电器、指示灯等外部控制设备，满足边缘侧本地控制需求 MIC（麦克风）接口：支持接入音频采集设备，搭配外部扬声器可实现音频播放，适配语音提示、音频交互场景
尺寸	128.8mm×117mm×28mm(L×W×H)
WIFI 参数	频段支持：2.4~2.4835GHzor5.15~5.85GHz WiFi 协议支持：IEEE802.11a/b/g/n/a
Zigbee 参数	无线标准：Zigbee3.0(1EEE802.15.4) 安全机制：WPA/WPA2 加密类型：AES-128 最大发射功率：+20dBm 接收灵敏度：-100dBm@802.15.4(2.4GHZ)
编程方法	支持 lua 脚本编程、python 脚本编程、c#.netcore 编程、node-red 流程化编程 提供脚本函数：支持设备采集控制、云服务器对接
环境参数	工作温度：-40～85℃ 储运温度：-40～85℃ 工作相对湿度：20%～90%无凝露 储运相对湿度：15%～95%无凝露

2.4. 电源接口

HWXT-A60 边缘计算网关电源接口为 3PIN5.08mm 端子

网络定义	说明	备注
⏚	Earth（安全地线）	连接大地
V-	电源输入负极/交流输入	电源输入负极/交流输入
V+	电源输入正极/交流输入	电源输入正极/交流输入

电源输入支持（直流）DC6V～28V、（交流）AC12V～30V 宽压输入，支持过流、抗雷保护，

支持反接保护。

电源防护

◆GB/T17626.5-2008 标准 4 级(4KV)8/20uS 雷击测试

◆防反接保护

◆过压保护

◆抗脉冲群保护

◆抗静电:±15kVHumanBodyModel

±15kVIEC61000-4-2AirDischarge

关于 Earth(安全地线)使用说明，为获得更好的防雷、防静电、抗干扰的性能，在条件允许的情况下 Earth 一定要可靠与大地进行连接。对于使用 DC 头进行供电的用户，可以通过金属外壳固定螺丝与大地进行可靠连接。Earth 如果不能可靠接地，抗干扰和防雷性能会受到影响。如有一些机房连接外部地线是悬空的，地线上带有强干扰信号或者高压时，可能会通过接地线串入机器内部影响机器正常工作（严重时可能会烧坏机器），此时如果现场实在没有可靠外部地线连接时，可以断开与本机连接的地线，让本机地线和外壳与外部地线绝缘。在没有可靠对外地线可用时，采用本机地线和外部地线绝缘，反而还会更安全，能有效减少故障的发生（注：现场施工请用户一定注意地线连接问题）。

RS485 接口

网关对外总共有 1 个串口采用完全独立隔离电源设计。并且 RS485 接口全部采用多级防雷防静电设计，非常适合对稳定性要求苛刻的工业与电力系统应用。接口采用标准 9PIN5.08mm 间距连接器，脚位定义如下表：

网关总共为 1 个串口

端口号	信号定义	信号说明	端口说明	备注
COM1	A	RS485A	RS485 接口	支持 GB/T17626.5-2008 标准 10/700uS 雷击测试 4KV 防护 ±15kVHumanBodyModel ±15kVIEC61000-4-2AirDischarge
	B	RS485B		

指示灯

网关共有 6 个指示灯，具体定义说明如下：

POW----电源指示灯，用于指示电源供电是否正常

LINK-----系统可编程指示灯 1，用于系统状态指示或者用户自定义功能指

WAN-----无线 4G/WiFi 模块工作状态指示

NET-----网络连接是否正常

DO1-----用于指示 DO1 闭合和断开状态

DO2-----用于指示 DO2 闭合和断开状态

WAN 指示灯状态说明

WAN 指示灯工作状态	所指示的网络状态
慢闪（200mS 亮/1800mS 灭）	找网络状态
慢闪（1800mS 亮/200mS 灭）	待机状态
快闪（125mS 亮/125mS 灭）	数据传输模式

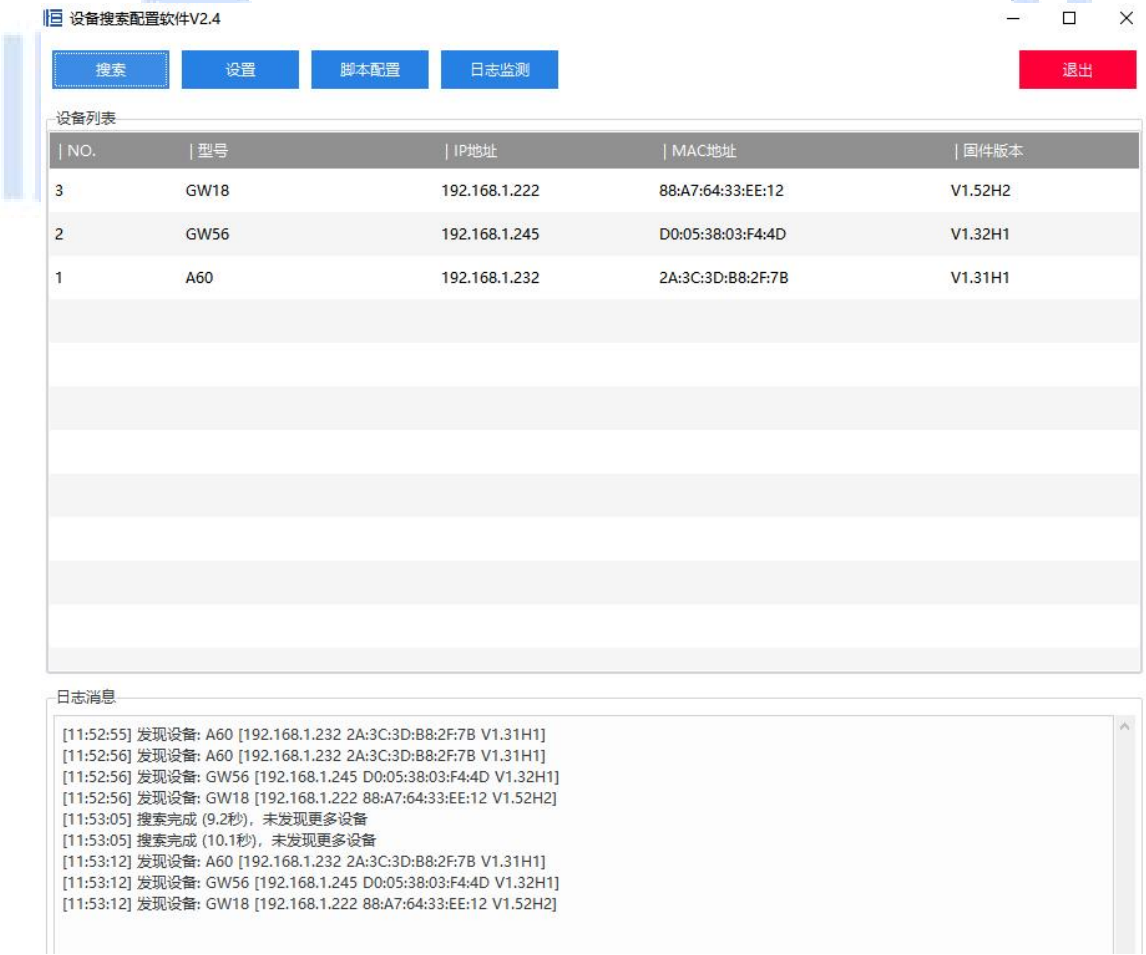
亮	通话中

3. 网关配置流程

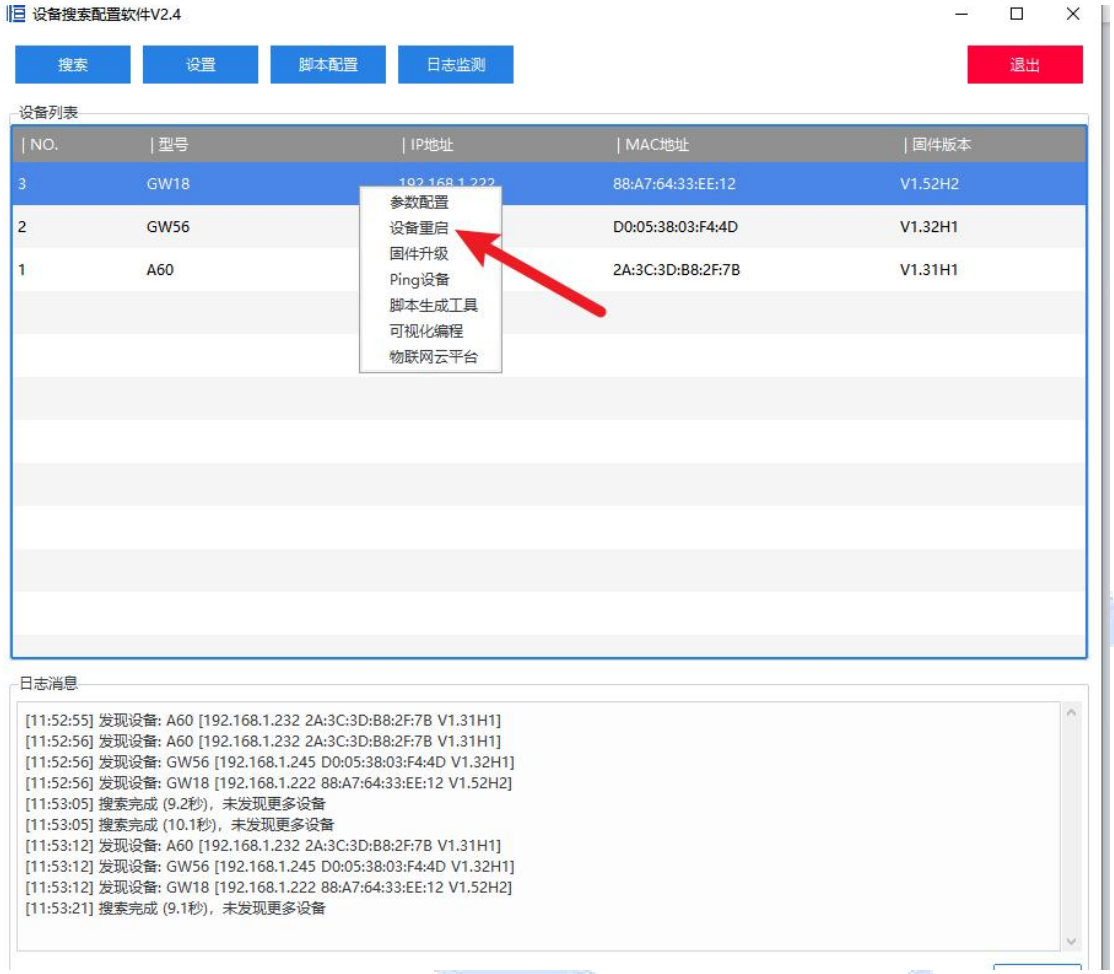
使用网线将网关网口连接电脑网口（或网络互通的路由器），如图 3-1。在没有配置 ip 参数前，默认网关 ip：192.168.1.80，子网掩码：255.255.255.0，默认网关：192.168.1.1。计算机 IP 地址必须和网关在同一网段，确保网络互通。

3.1. 修改网关 IP

试用网关之前，首先必须保证网关正确配置 IP，有连接外网的情况还需要设置正确的默认网关、DNS 等信息。打开 “IOTGatewayConfig.exe”软件。选择正确的网卡（确定电脑哪个网口和网关的网口有物理连接，可以是直连或者经过交换机、路由器）。点击“搜索”按钮，如有系统弹出允许软件访问网络，应点击允许。如果网关正常工作，将可以搜索到网关设备（支持跨网段搜索）。

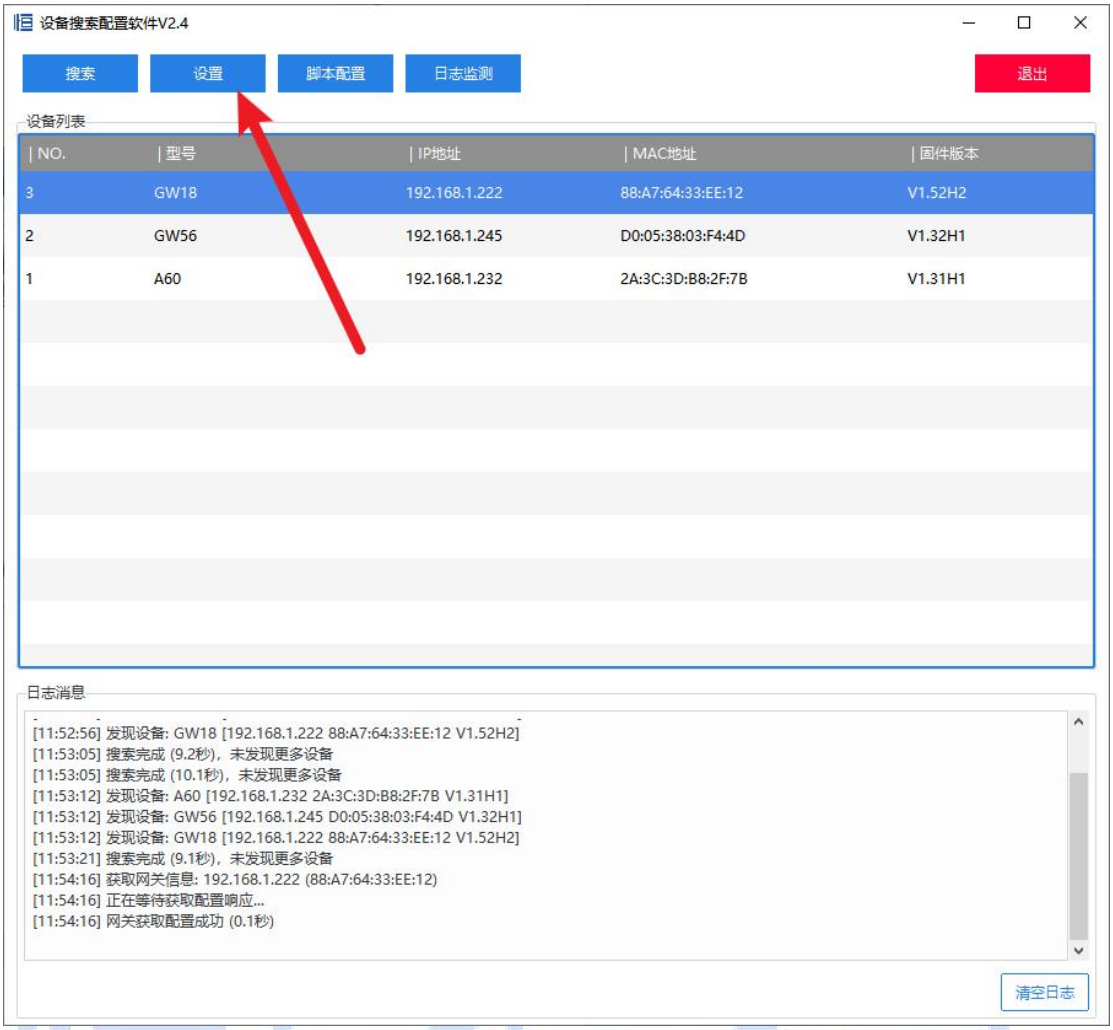


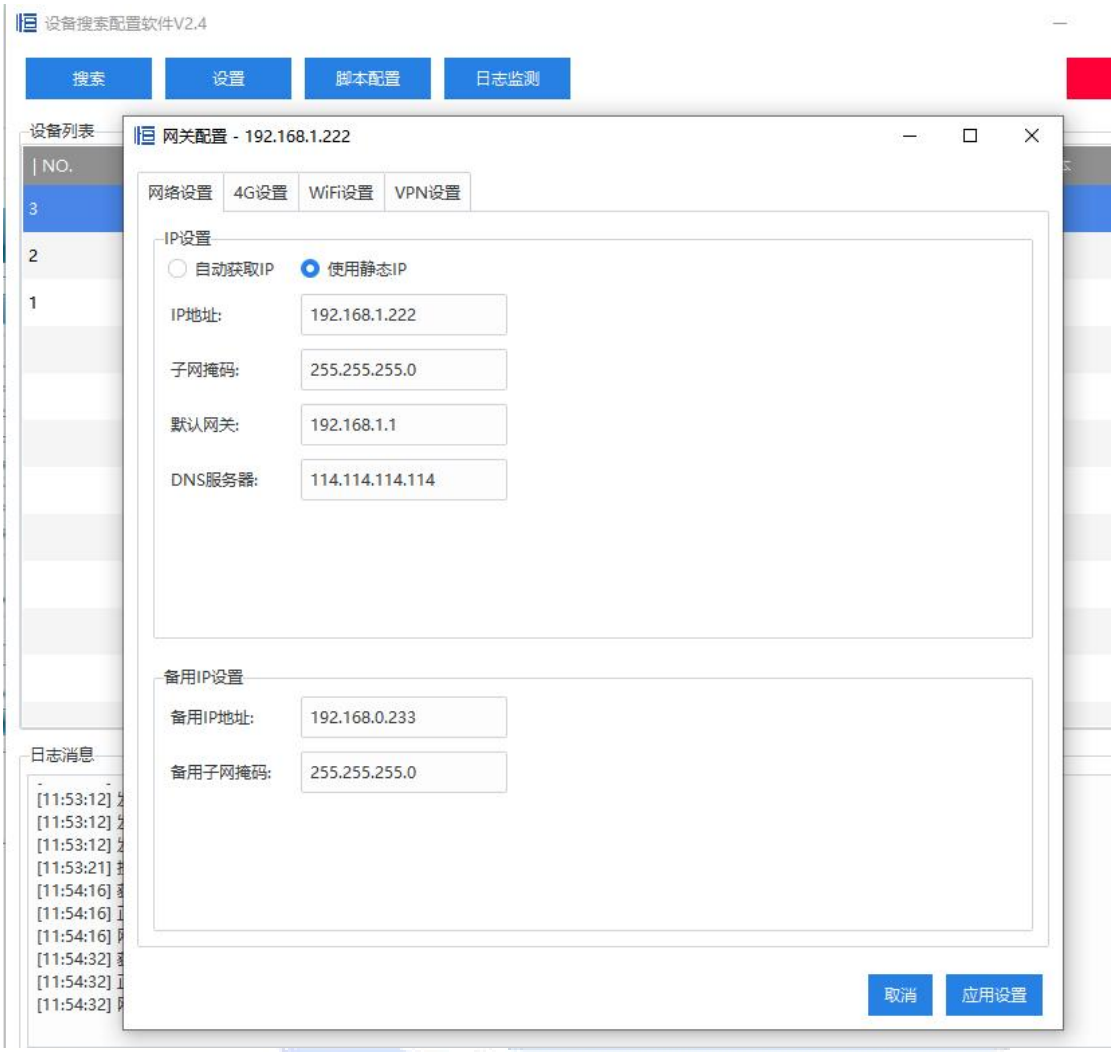
选中要配置的设备，点击“设置”按钮，将弹出 IP 参数配置窗口，根据需要修改 IP 相关配置，修改后点击“设置”按钮（支持跨网段设置），设置后，如果要配置生效，需要重启一下网关。操作如下，右击要复位的网关，在弹出的下拉列表选择“设备重启”等待 15 秒左右，听到滴一声表示网关重启成功。



3.2. 配置网关 IP 参数

点开设置，即可配置网关 IP 参数

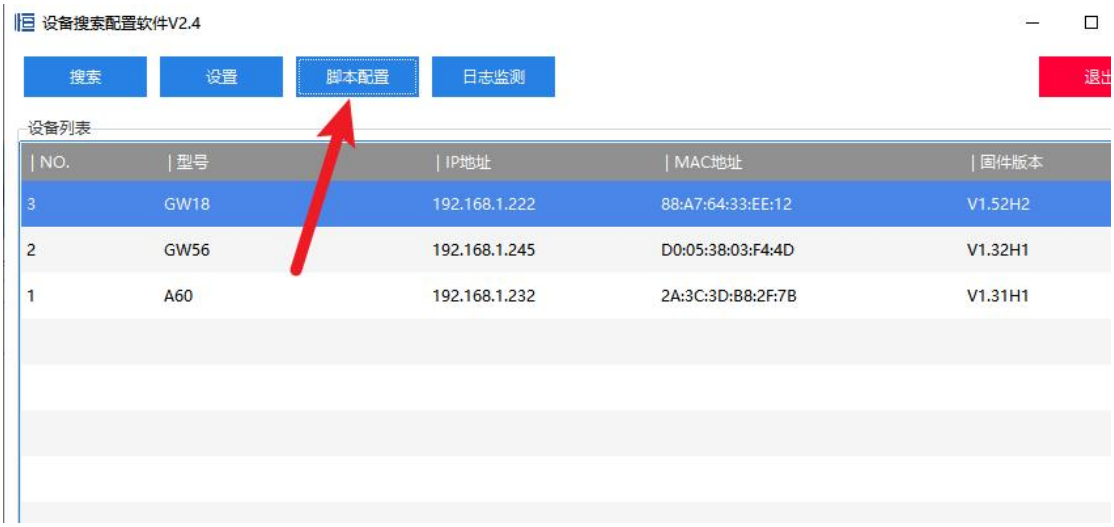

设备搜索配置软件V2.4
搜索
设置
脚本配置
日志监测
退出
设备列表
| NO.
| 型号
| IP地址
| MAC地址
| 固件版本
3 GW18 192.168.1.222 88:A7:64:33:EE:12 V1.52H2
2 GW56 192.168.1.245 D0:05:38:03:F4:4D V1.32H1
1 A60 192.168.1.232 2A:3C:3D:B8:2F:7B V1.31H1
日志消息
[11:52:56] 发现设备: GW18 [192.168.1.222 88:A7:64:33:EE:12 V1.52H2]
[11:53:05] 搜索完成 (9.2秒)，未发现更多设备
[11:53:05] 搜索完成 (10.1秒)，未发现更多设备
[11:53:12] 发现设备: A60 [192.168.1.232 2A:3C:3D:B8:2F:7B V1.31H1]
[11:53:12] 发现设备: GW56 [192.168.1.245 D0:05:38:03:F4:4D V1.32H1]
[11:53:12] 发现设备: GW18 [192.168.1.222 88:A7:64:33:EE:12 V1.52H2]
[11:53:21] 搜索完成 (9.1秒)，未发现更多设备
[11:54:16] 获取网关信息: 192.168.1.222 (88:A7:64:33:EE:12)
[11:54:16] 正在等待获取配置响应...
[11:54:16] 网关获取配置成功 (0.1秒)
清空日志

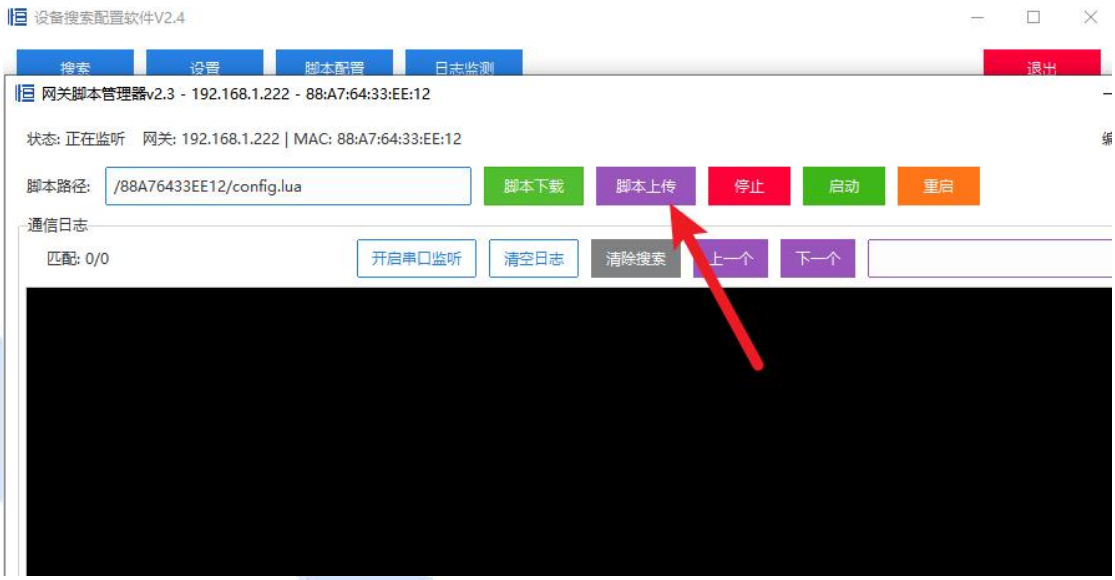


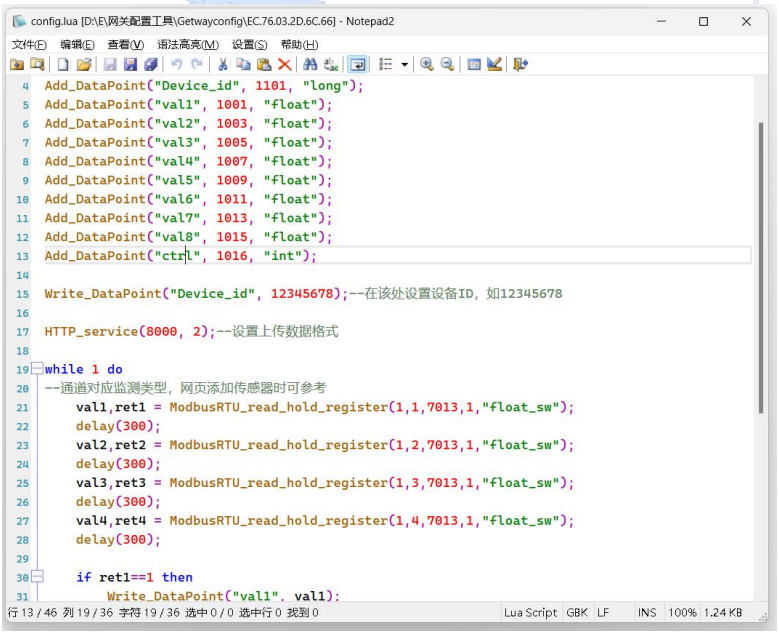
设置完成后，重启网关即修改成功

3.3. 配置网关工作的脚本

在“IOTGatewayConfig.exe”软件上选择要配置脚本的网关，点击“脚本配置”按钮，弹出的脚本配置窗口，能进行 lua 编程和 python 编程，在弹出的窗口上点击“脚本上传”，网关如果正常连接软件将弹出脚本编辑器。

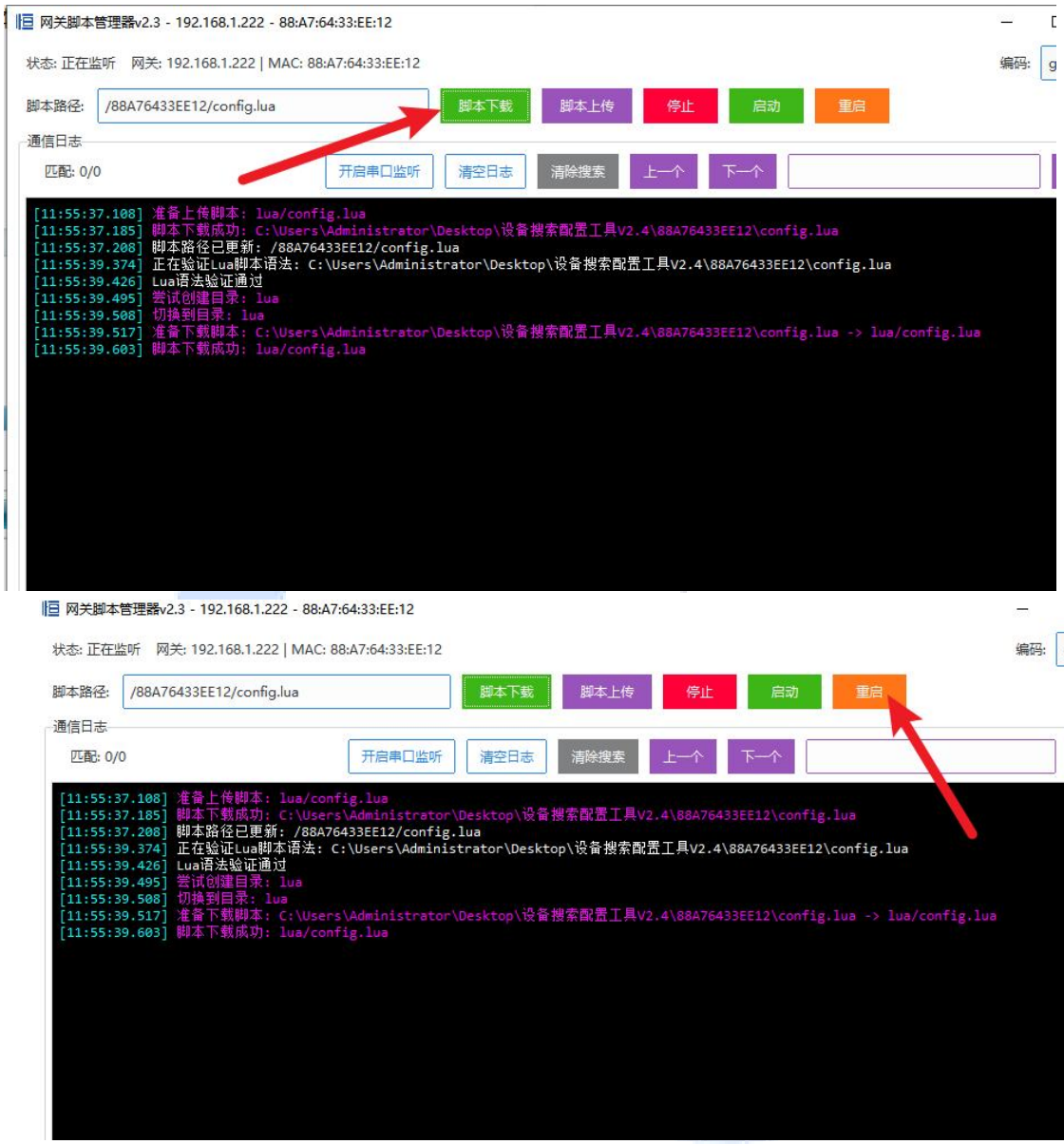






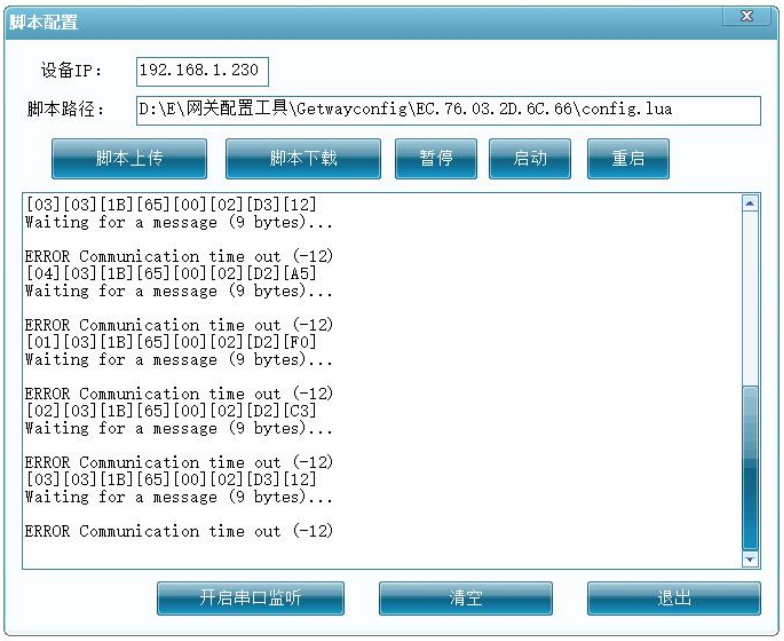
根据下文中的脚本语法和函数说明，修改完脚本后，在编辑器点保存，保存后在脚本

配置窗口上点击“脚本下载”，如果要脚本生效，则点“重启”按钮，脚本程序自动重新运行。



如果脚本出现语法错误，或者运行出错，调试信息窗口上可以看到报错的位置，如图 3-8。根据提示修改脚本即可。

在和硬件联机调试时，可以点击“开启串口监听”按钮，即可以看到串口采集的 485 通信日志。



4. 网关脚本编程

4.1. 网关编程语言介绍

网关配置脚本基于 Lua 5.3.1 语法。Lua 是一个小巧的脚本语言。是巴西里约热内卢天主教大学（Pontifical Catholic University of Rio de Janeiro）里的一个研究小组，由 Roberto

Ierusalimschy、Waldemar Celes 和 Luiz Henrique de Figueiredo 所组成并于 1993 年开发。其设计目的是为了嵌入应用程序中，从而为应用程序提供灵活的扩展和定制功能，提供在特定平台上的即时编译功能。

注：网关服务启动后将会按顺序执行脚本文件，当配置脚本运行结束后，服务程序将自动退出，所以设计配置脚本注意使用循环结构，处理连续处理的事件。

4. 2. Lua 基本语法介绍

1) 注释：在 Lua 中，可以使用单行注释和多行注释。
   a) 单行注释：连续两个减号"--"表示注释的开始，一直延续到行末为止。相当于 C++语言中的"//"。
   b) 多行注释：由"--[["表示注释开始，并且一直延续到"]]"为止。这种注释相当于 C 语言中的"/*...*/"。在注释当中，"[["和"]]"可嵌套。
2) 词法约定
   a) Lua 中用到的名字（也称作标识符）可以是任何非数字开头的字母、数字、下划线组成的字符串
   b) 下面关键字为 Lua 保留字，不能用作名字：

and break do else elseif
end false for functionif
in local nil not or
repeat return then true until
while

   c) Lua 是一个大小写敏感的语言： and 是一个保留字，但是 And 和 AND 则是两个不同的合法的名字。
   d) 下面是 Lua 的另外一些关键字及运算符

+ - * / % ^ #
== ~= <= >= <> =
( ) { } [ ]
; : , ...

3) 值与类型
   a) Lua 是一种动态类型语言。这意味着变量没有类型，只有值才有类型。Lua 中有八种基本类型：nil,boolean,number，string，function，userdata，thread，and table。Nil 类型只有一种值 nil，它的主要用途用于标表识和别的任何值的差异；通常，当需要描述一个无意义的值时会用到它。Boolea 类型只有两种值：false 和 true。nil 和 false 都能导致条件为假；而另外所有的值都被当作真。Number 表示实数（双

精度浮点数）。String 表示一串字符的数组。 Lua 是 8-bit clean 的：字符串可以包含任何 8 位字符，包括零结束符('\0')在类型定义。而所有的值本身携带它们自己的类型信息。

例如:

定义字符串类型(string)变量 a 值为"gateway":

```
a = " gateway "
```

定义数字类型(number)变量 b 值为 123

```
b = 123
```

定义 boolean 类型变量 c 值为假(false):

```
c = false
```

定义函数类型变量 test，函数名为 test，参数为 a、b，返回值为 a + b：

```
functiontest(a, b)
    return    a + b
end
```

4) 调用函数:

将函数 test，参数依次为 1， 2 的返回值赋给变量 a

```
a    = test(1, 2)
```

由于 Lua 是弱类型语言,所以您可以在先定义 a 为字符串类型(string)，然后再定义它为数字类型(number)

例如:

```
a    =    "zhiqian"        --    此时 a 为字符串类型
a    =    123              --    此时 a 被覆盖，为整数类型
```

5) 比较运算:

比较符号	<	>	<=	>=	==	~=
含义	小于	大于	小于或等于	大于或等于	相等	不相等

a) 所有这些操作符总是返回 true 或 false

例如:

```
a = 50    b = 60
print(a == b) --返回 false
print(a ~= b) --返回 true
```

6) 逻辑运算：

a) and，or，not 其中，and 和 or 与 C 语言区别特别大。

b) 在这里，请先记住，在 Lua 中，只有 false 和 nil 才计算为 false，其它任何数据都计算为 true，0 也是 true！

c) and 和 or 的运算结果不是 true 和 false，而是和它的两个操作数相关。

d) a and b：如果 a 为 false，则返回 a；否则返回 b

e) a or b：如果 a 为 true，则返回 a；否则返回 b

例如:

```
print(4 and 5 输出 5)
print(nil and 13) --输出 nil
print(false and 13) --输出 false
print(4 or 5) --输出 4
print(falseor 5) --输出 5
```

7) 常见控制语句

a) If/while/repeat 这些控制结构符合通常的意义，而且也有类似的语法

b) while 语句语法:

while 条件 do 操作 end

例如:

```
定义数字型变量 a = 123，当 a > 100 时
a= 110
while   a>100
do
   a = a -1
   print(a)
end
```

c) if 语句语法:

if *条件*
then *操作*
elseif *条件*
then *操作*
else *操作*
end

例如:

定义数字 a = 100，当 a >100 时打印"a 大于 100 "，当 a <100 时打印"a 小于

100 "，否则打印"a 等于 100 "，代码如下：

```
a = 100
    if a>100
then
    print("a 大于 100")
elseif a<100
then
    print("a 小于 100")
else
    print("a 等于 100")
end
```

d) repeat 语句语法:

repeat *操作*

until *条件*

例如:

定义变量 a = 105，重复执行 a = a -1 操作，直到 a<=100 为止

```
a = 105
repeat
    a=a-1
    print(a)
until   a<= 100
```

e) for 语句语法:

for v = 起始值，终止条件，步长 do 操作 end

当起始值不满足终止条件时，执行操作，对起始值执行步长操作

例如:

V 的起始值为 10，当 v < 20，打印 v ，每次给 v 加 1 时

```
for v = 10, 20, 1
   do   print(v )
end
```

5. AI 主机边缘计算网关 C#开发使用说明

5.1. 网关 C#开发简介

为了应对多种场景下网关的开发需求，AI 主机边缘计算网关设计了 C#开发的功能。可以让用户开发 C#的程序，然后放入网关上运行。C#的开发使用了. NET Core 的框架。NET Core 是一个跨平台的开发平台，可以在 Windows、Linux 和 macOS 等多个操作系统上运行。

在嵌入式系统开发中，.NET Core（现已改名为 .NET）可以提供许多有价值的应用，包括但不限于以下几点：

1. **跨平台支持**:.NET Core 是一个跨平台的开发平台,可以在多种操作系统上运行,包括 Windows、Linux 和 macOS。这使得开发人员可以更轻松地在不同的嵌入式系统上开发和部署应用程序。

2. **强大的开发工具**:.NET Core 提供了强大的开发工具和框架,如 Visual Studio 和 Visual Studio Code，这些工具可以极大地简化嵌入式系统的开发过程，提高开发效率。

3. **高性能**：.NET Core 具有优化的性能特性，包括 JIT（Just-In-Time）编译、多线程支持和异步编程模型，可以提供高性能的嵌入式应用程序。

4. **丰富的库和组件**：.NET Core 拥有丰富的类库和组件，提供了许多现成的功能模块，开发人员可以通过这些库快速构建复杂的嵌入式应用程序。

5. **安全性和稳定性**：.NET Core 具有良好的安全性和稳定性，可以帮助开发人员构建 安全可靠的嵌入式系统，保护系统和数据的安全。

总的来说，.NET Core 在嵌入式系统开发中的应用价值在于提供了跨平台支持、强大的开发工具、高性能、丰富的库和组件，以及安全性和稳定性，帮助开发人员更轻松地构建高质量的嵌入式应用程序。

5.2. 开发流程介绍

5.2.1. 搭建 windows 开发环境

进入微软官网下载.NET6.0 的 SDK。下载 .NET 6.0 (Linux、macOS 和 Windows) (microsoft.com)

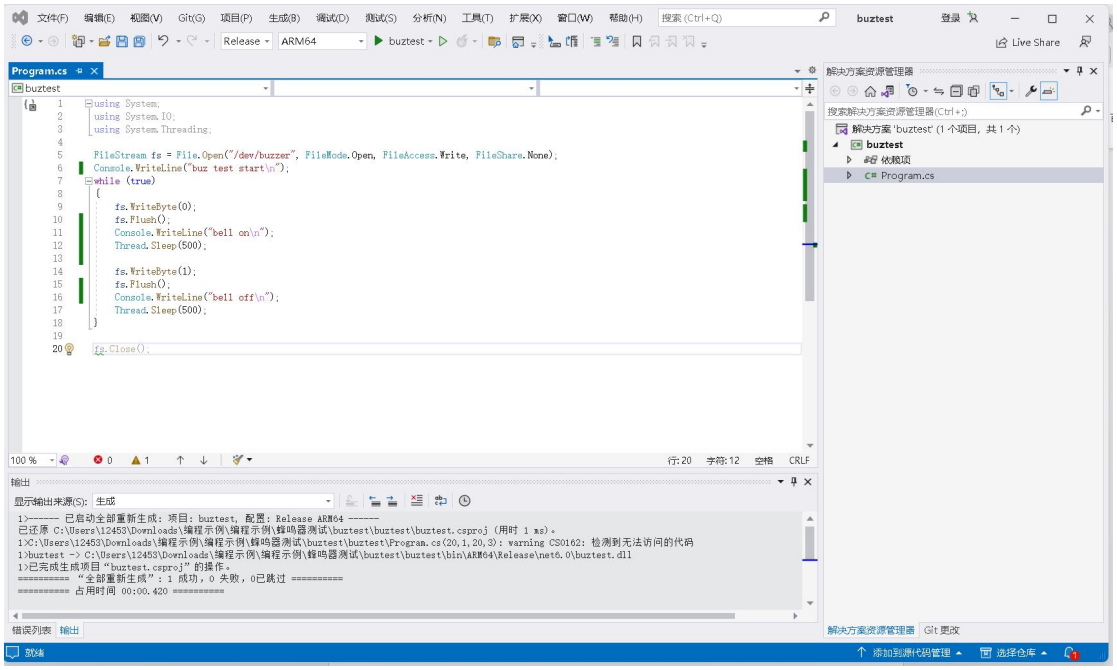
SDK 6.0.131

OS	安装程序	二进制文件
Linux	包管理器说明	Arm32 | Arm32 Alpine | Arm64 | Arm64 Alpine | x64 | x64 Alpine
macOS	Arm64 | x64	Arm64 | x64
Windows	Arm64 | x64 | x86 | winget 指令	Arm64 | x64 | x86
全部	dotnet-install scripts	

安装下载的 SDK，根装提示安装 VS2002。

5.2.2. 测试程序编译

打开网关提供的例子程序，这里以控制网关蜂鸣器的程序为例。打开工程文件 buztest.sln，如下图:

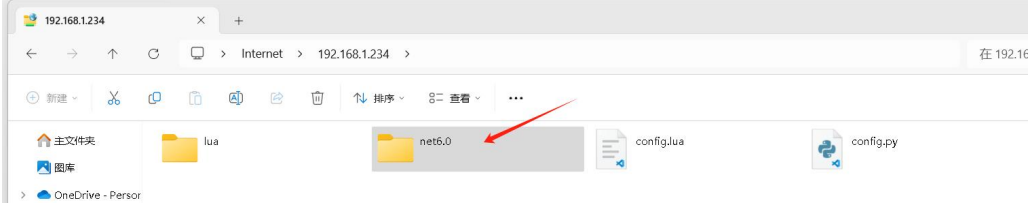


AI 主机网关采用 64 位 ARM 处理器，这里就选择 ARM64

最后在 VS2022 的菜单里，点击生成->生成解决方案，完成编译。编译完成后，在项目目录下：\buztest\buztest\bin\ARM64\Release\net6.0\ buztest.dll 为生成的程序文件。

5.2.3. 将生成的程序拷进网关

打开网关的 FTP 文件服务 ftp://192.168.1.234。如需要提示输入密码则通过匿名登录即可（anonymous）。将 net6.0 整个文件夹拷贝到 ftp，如下图：



5.2.4. 编辑运行脚本

拷贝了运行程序后，需要通过 lua 脚本来启动 C#的程序运行。只需在网关的脚本上，加入一行代码：run_dotnet("/net6.0/buztest.dll")，即会启动程序的运行。其中 run_dotnet 函数的输入参数，必须和 FTP 上拷贝进去的文件名称对应。

```
run_dotnet("/net6.0/buztest.dll")
release(); --配置为发布版本
--设置网口信息
--在该处设置网络参数ip、掩码、网关
sys_set_com(1, 9600, "none", 8, 1); --usb串口
HTTP_service(8000, 4);--设置上传数据格式
ModbusTCP_service(502,1)
 mode1();
```

注意：运行 run_dotnet(),lua 代码会阻塞。等待 c#代码运行结束。如需要同时运行 lua 其他代码，需要使用多线程编程。具体查看网关多线程编程说明。

修改好 lua 脚本后，在配置工具上点“脚本下载”，下载完成后再点“暂停”，再点“启动”。这时可以听到网关的蜂鸣器响 500 毫秒，停 500 毫秒，在配置工具上，可以看到 C#代码的运行日志，如下图。同时也可以通过配置工具，控制 C#程序的启动、停止、重启等操作。C#程序的 Console.WriteLine()信息打印，可以定向输出到调试软件，开发调试比较方便。

6. 网关 VPN 远程调试

AI 主机网关内置 VPN 远程调试功能，VPN 是一款基于互联网，通过加密通道将企业数据中心、企业办公网络、或互联网终端和专有网络安全可靠地连接起来的服务。构建一个局域网。AI 主机网关的 VPN 功能做了特别的设计，将传统复杂繁琐的 VPN 建立过程，进行了精简设计，用户只需要一个识别码，即可以实现 VPN 连接的搭建。可以用于网关的远程调试，远程维护，同时也可以对网关所在网络下的其他设备进行远程维护。如 PLC 远程上下载程序等功能。具体操作如下：

1. 点击左上角工具，点击驱动安装，安装虚拟网卡（只需安装一次）。

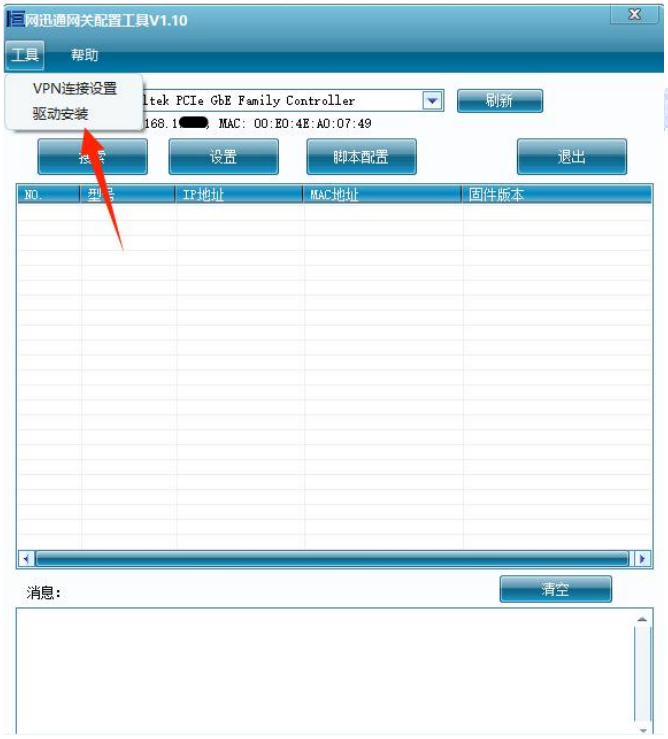


图 4 安装虚拟网卡

2. 安装 VPN 只需要一直点击 Next（下一步）即可安装成功

图 5 安装 VPN

3. 安装好后，选择对应的网关点击设置，开启 VPN 并填写识别码（一个网关一个识别码），并自主设置密码。

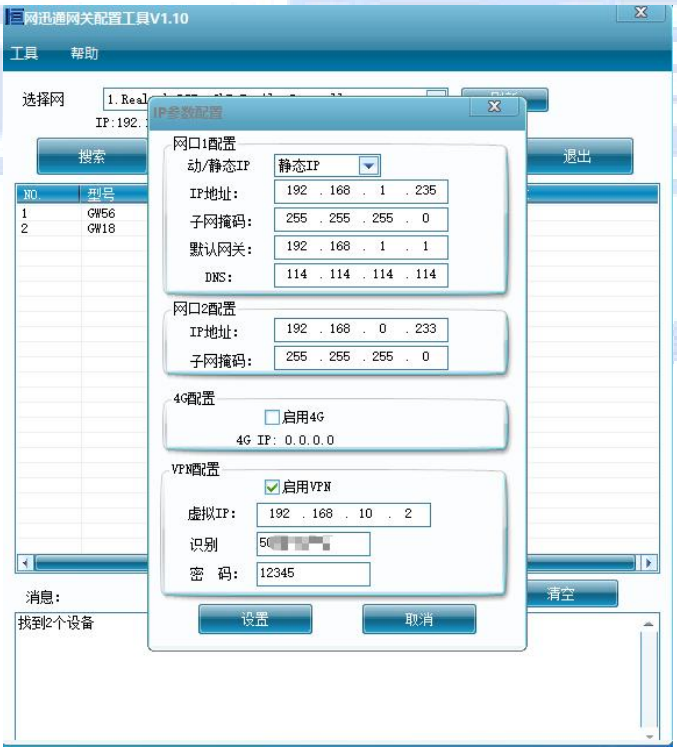


图 6 设置 VPN

4. 进入网络连接，查看本地 VPN 的 IP 地址。

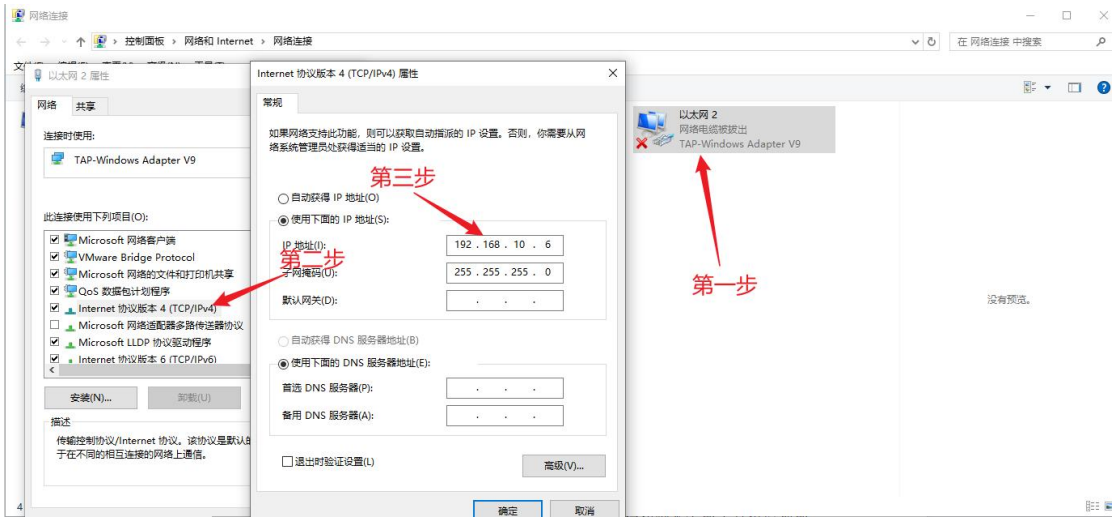


图 7 查看本地 VPN 地址

5. 设置好后，点击左上角工具，VPN 连接设置填入网关识别码和密码。

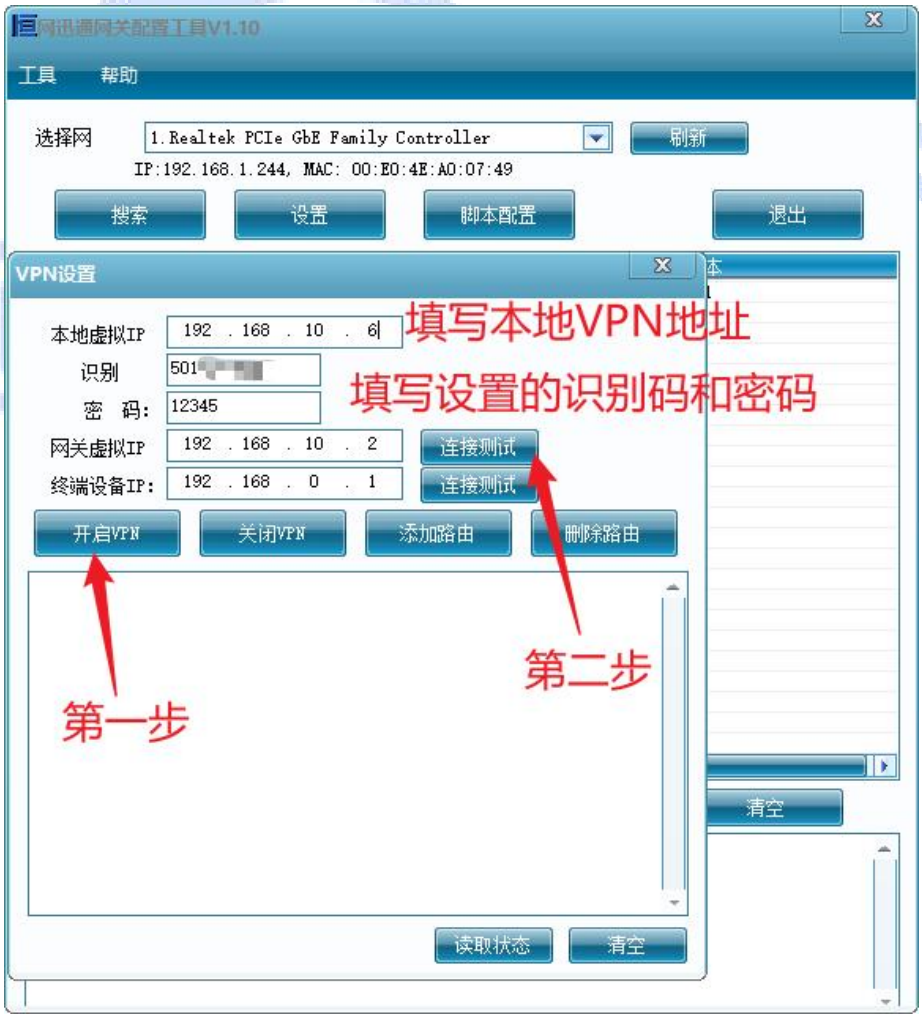


图 8 VPN 设置

打开 VPN

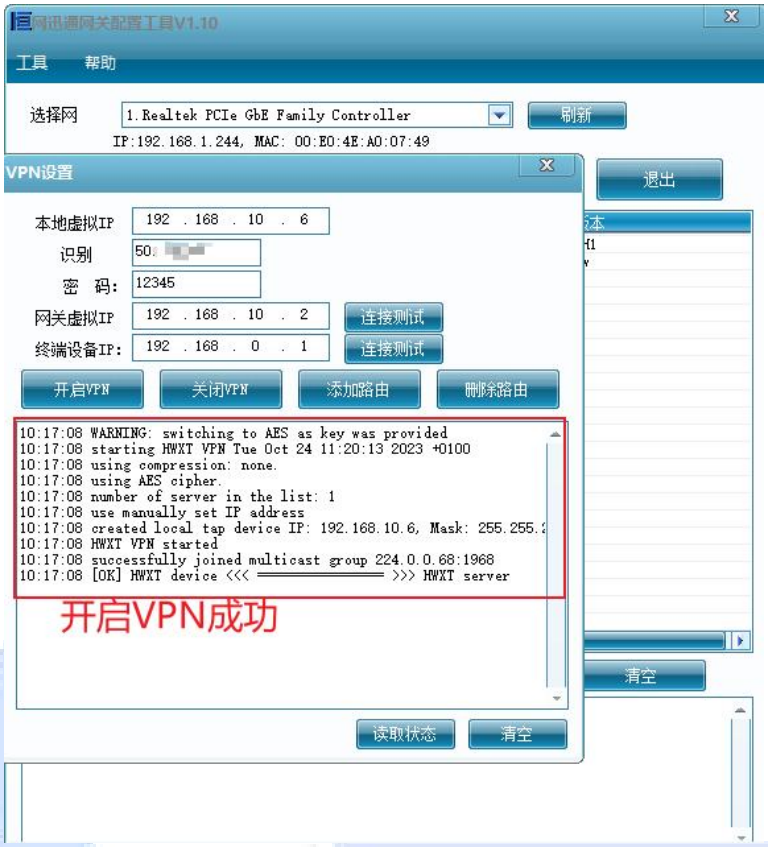


图 9 开启 VPN

打开连接测试，PING 成功即可搜索使用

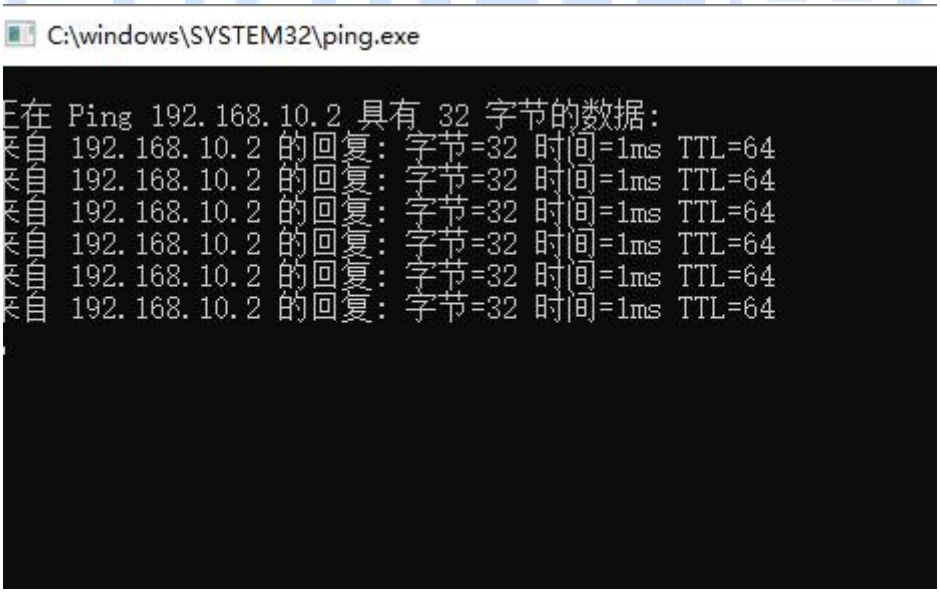


图 10 连接测试

6. 下列选择框选择 VPN 的网段，进行搜索

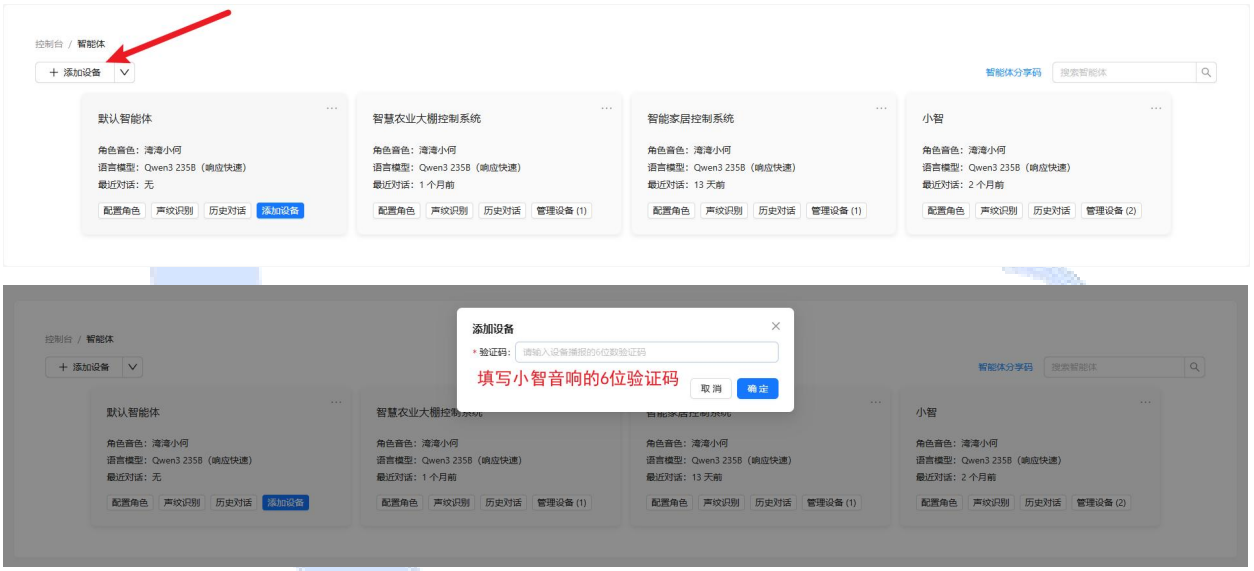
图 11 搜索 VPN 网段

7. 小智 MCP 配置流程

7.1. 官方小智 MCP 接入点获取

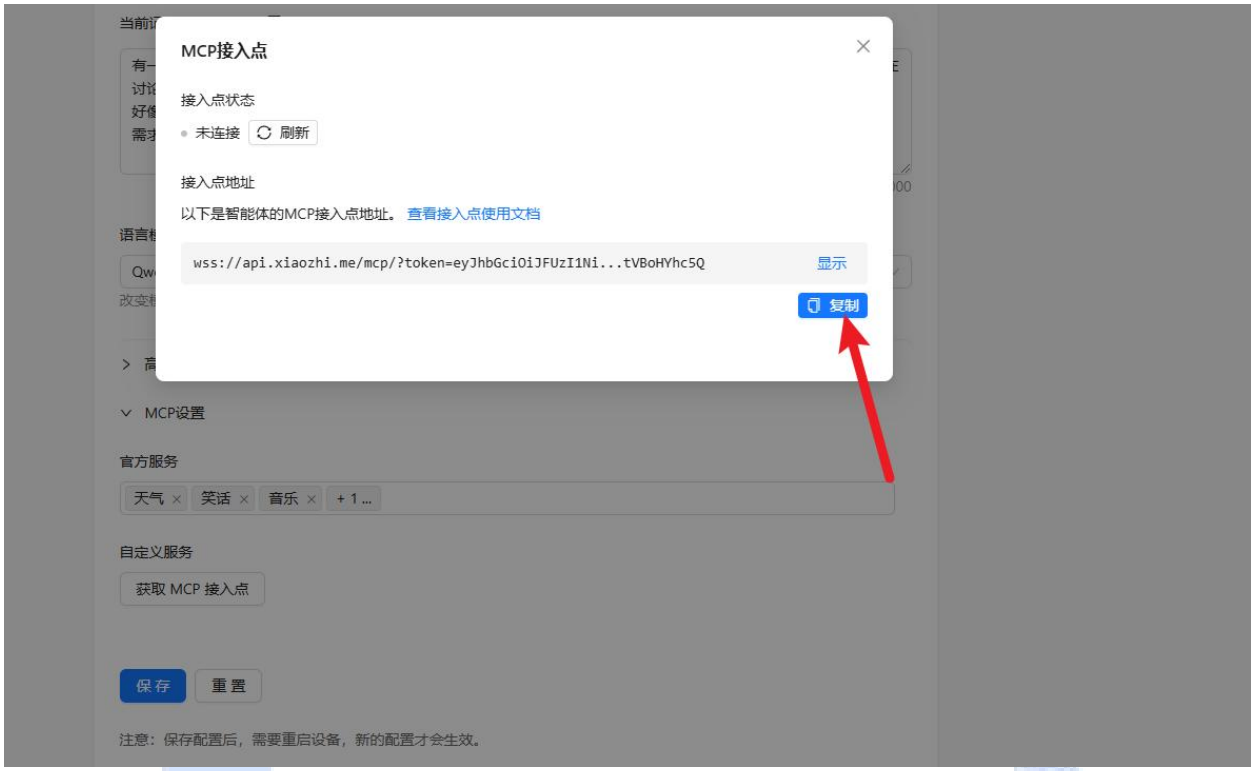
1. 访问小智官方网站中直接通过手机号登录小智 AI。

2．添加智能体设备，填写硬件小智播报出来的 6 位验证码，名称自定义命名



3．创建成功后，点击新创建的小智智能体，配置角色

4．找到 MCP 设置，点击获取 MCP 接入点，即可查看 mcp 的接入状态与 mcp 接入点地址。



7.2. 案例说明

案例核心硬件清单及连接说明

脚本代码及文件点击下载：MCP 案例脚本.lua

本脚本控制的核心硬件为“卫生间 + 客厅 + 厨房 + 卧室”关键设备，均通过网关**串口 1（RS485 接口）** 通信，寄存器地址范围 1000-2000，具体清单如下：

区域	设备类型	设备功能	网关寄存器地址	通信参数（串口 1）	引用文档章节
卫生间	照明灯	开关控制（0 = 关，1 = 开）	1001	波特率 9600、无校验、8 数据位、1 停止位	9.1、9.5.5
卫生间	温湿度传感器	采集温度 / 湿度数据	1004（温度）、1005（湿度）	同上	9.5.4
客厅	主灯	开关控制	1006	同上	9.5.5
客厅	窗帘电机	开合控制（0 = 关，100 = 开）	1008	同上	9.24.1

区域	设备类型	设备功能	网关寄存器地址	通信参数（串口 1）	引用文档章节
客厅	火焰传感器	检测火焰（1 = 触发，0 = 正常）	1011	同上	9.5.2
厨房	风扇	开关控制	1013	同上	9.5.5
厨房	照明灯	开关控制	1015	同上	9.5.5
厨房	烟雾传感器	检测烟雾（1 = 触发，0 = 正常）	1016	同上	9.5.2
卧室	主灯	开关控制	1019	同上	9.5.5
卧室	光照度传感器	采集光照强度数据	1021	同上	9.5.4
卧室	窗帘电机	开合控制	1022	同上	9.24.1
卧室	人体传感器	检测人体（1 = 触发，0 = 无）	1023	同上	9.5.2

连接注意：RS485 接口需区分 A/B 线，网关 RS485 接口的“A”接传感器 / 执行器的“A”，“B”接“B”；电源需符合 DC6~28V 宽压输入，建议接地以提升防雷抗干扰能力。

7.3. 代码函数说明

本脚本核心函数，按代码执行顺序说明如下：

脚本初始化函数

函数名	功能	参数说明	引用文档章节
release()	配置脚本为“发布版本”，烧入后掉电不丢失；无此函数则为“内存调试模式”，掉电丢失	无参数	10.11.1
sys_set_com(1, 9600,	初始化串口 1 通信参数	1：串口号；9600：波	10.1

函数名	功能	参数说明	引用文档章节
"none", 8, 1)		特率；"none"：无校验；8：数据位；1：停止位	
HTTP_service(8000, 4)	启动网关 Web Server 服务，支持 HTTP 读写云变量	8000：服务端口；4：数据格式（标准 JSON）	10.11.7
ModbusTCP_service(502, 1)	启动 Modbus TCP 从站服务，允许外部主站（如组态软件）访问网关寄存器	502：Modbus 默认端口；1：网关设备 ID	10.2.1

云变量与映射配置函数

函数名	功能	参数说明	引用文档章节
Add_DataPoint("led1", 1001, "int")	定义云变量，关联网关寄存器地址与数据类型，支持云平台上报与控制	"led1"：云变量名；1001：寄存器地址（1000-2000 范围）；"int"：数据类型	10.9.1
local config = {["卫生间照明灯"] = "led1", ...}	配置云变量中英文映射，供小智平台识别设备类型	键：中文设备名；值：云变量英文标识（需与 Add_DataPoint 的变量名一致）	10.26.1
MCP_service("wss://api.xiaozhi.me/mcp/?token=xxx", config)	接入小智 MCP 平台，实现设备数据上报与远程控制	第 1 参数: 小智 MCP 接入点 WebSocket 地址；第 2 参数：变量映射表	10.26.1

数据采集与设备控制函数

函数名	功能	参数说明	引用文档章节

函数名	功能	参数说明	引用文档章节
ModbusRTU_read_input_register(1, 15, 0x012b, 2, "int")	读取 Modbus RTU 输入寄存器数据（如温湿度、光照度）	1：串口号；15：传感器设备地址；0x012b：传感器寄存器地址；2：读取数量；"int"：类型	10.5.4
ModbusRTU_read_input_status(1, 1, 0, 3)	读取 Modbus RTU 输入状态（如火焰、烟雾、人体传感器）	1：串口号；1：控制器地址；0：起始地址；3：读取数量	10.5.2
Read_sys_register(1008, "int")	读取网关内部寄存器值（如窗帘位置、设备开关状态）	1008：寄存器地址；"int"：数据类型	10.11.3
curtain_485_control(1, 0xfdfe, 0x02)	控制窗帘电机开合	1：串口号；0xfdfe：窗帘电机设备地址；0x02：命令（0x01 = 开，0x02 = 关）	10.24.1
ModbusRTU_force_single_coil(1, 1, 0, led1)	控制 Modbus RTU 线圈输出（如灯、风扇开关）	1：串口号；1：控制器地址；0：线圈地址；led1：控制值（0 = 关，1 = 开）	10.5.5
mode1()	核心功能函数，包含云变量定义、死循环数据采集与控制逻辑	无参数，内部通过 while 1 do...end 实现持续运行	/

辅助函数

函数名	功能	参数说明	引用文档章节
delay(100)	系统延时，避免通信频繁导致资源占用过高	100：延时时间（单位：ms）	10.11.2
Write_DataPoint("wendu", wd)	将采集到的数据写入云变量，供上报或控制使用	"wendu"：云变量名；wd：写入的值	10.9.3

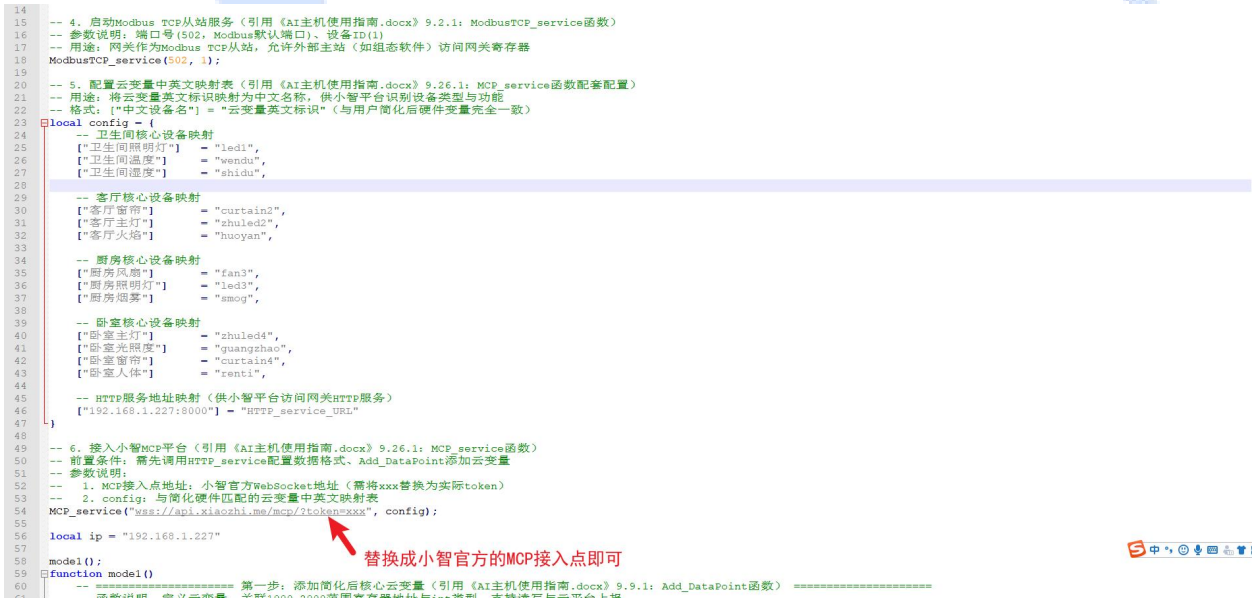
函数名	功能	参数说明	引用文档章节
playwav("/wav/yanwu.wav", 1)	播放报警音频（如烟雾、火焰报警）	"/wav/yanwu.wav"：音频路径；1：阻塞模式	10.27.1

7.4. 官方小智 MCP 接入点获取

小智 MCP 接入点需从**小智官方渠道获取**，具体流程查看官方小智 MCP 接入点获取的操作步骤

7.5. 代码填写并烧入

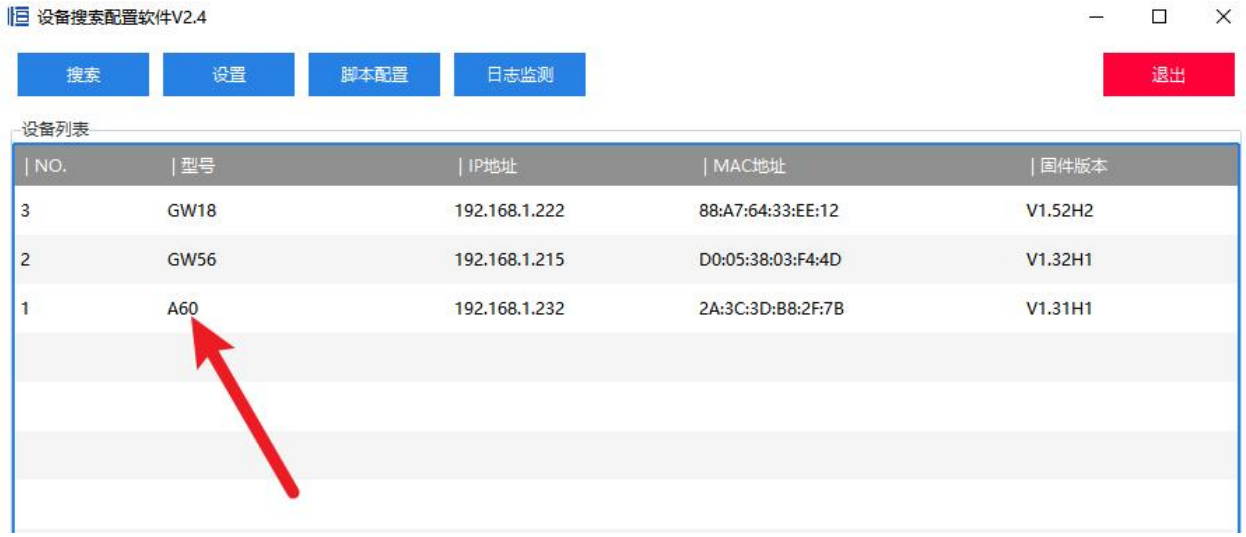
代码填写与烧入需使用提到的“IOTGatewayConfig.exe”工具，分 4 步操作，具体查看配置网关工作的脚本的操作步骤



8. AI 主机操作说明

8.1. 视频 AI 分析系统的使用

1．使用网关搜索型号为 A60 的网关 IP 地址



2．浏览器搜索：网关 IP:8889/ai/，即可进入视频 AI 分析系统

3．在视频 AI 分析系统内，点击左边栏目中的任何一种算法，即可启动算法，在摄像头中框出来。

8.1.1. 识别区域画框

为了避免区域范围过大，容易误触的问题，我们的 AI 分析系统做了区域画框的处理。

返回

通道1 清除全部 绘制区域

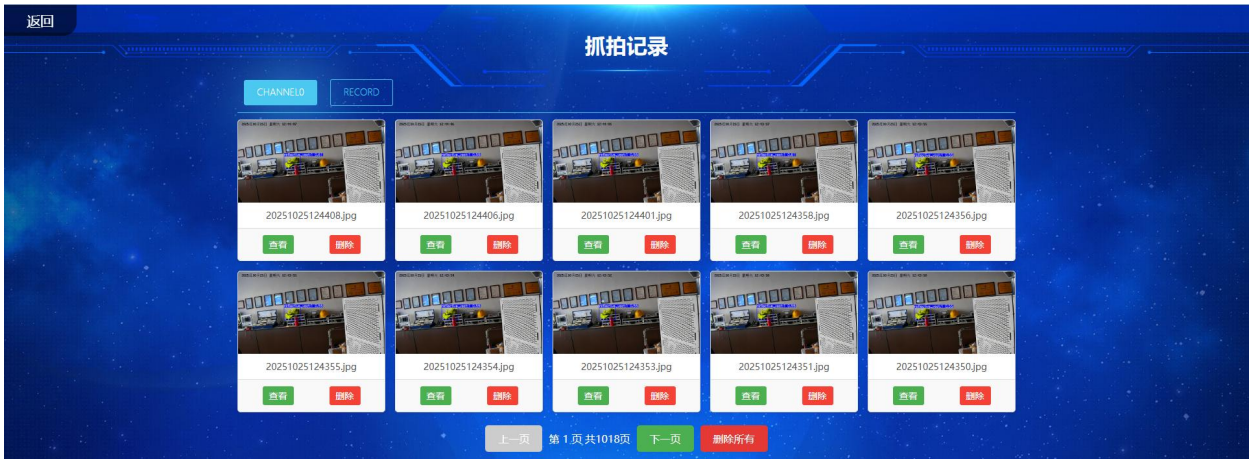
返回

通道1 清除全部 绘制区域

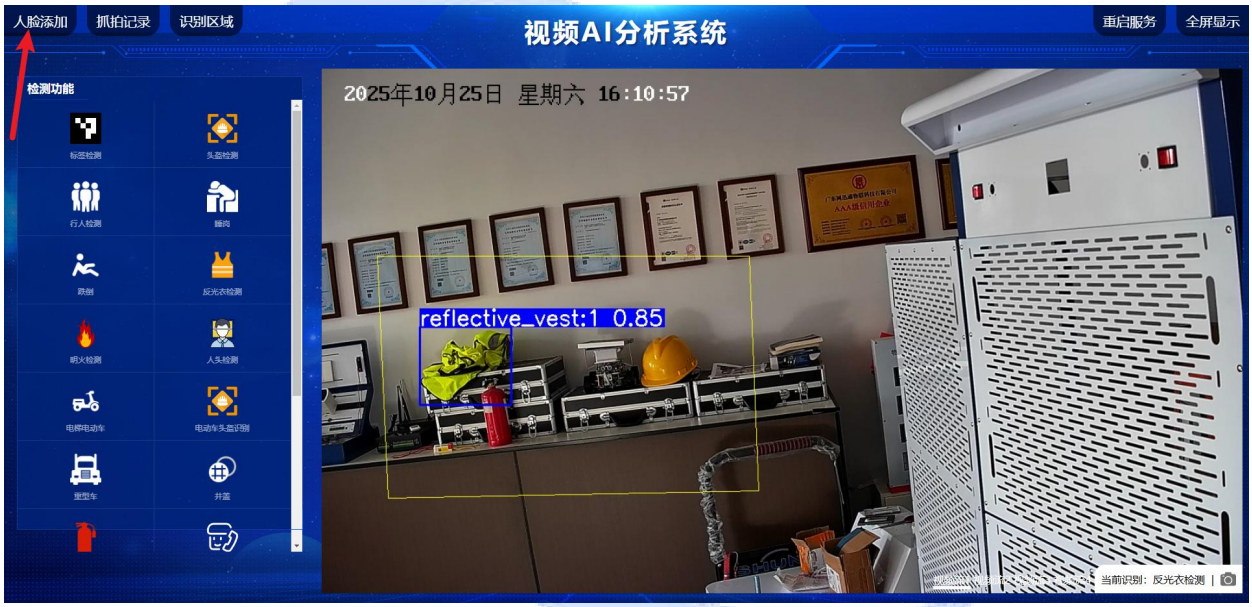
人脸添加
抓拍记录
识别区域
视频AI分析系统
重启服务
全屏显示
检测功能
2025年10月25日 星期六 16:10:14
reflective_vest:1 0.85
当前识别：反光衣检测

8.1.2. 抓拍记录

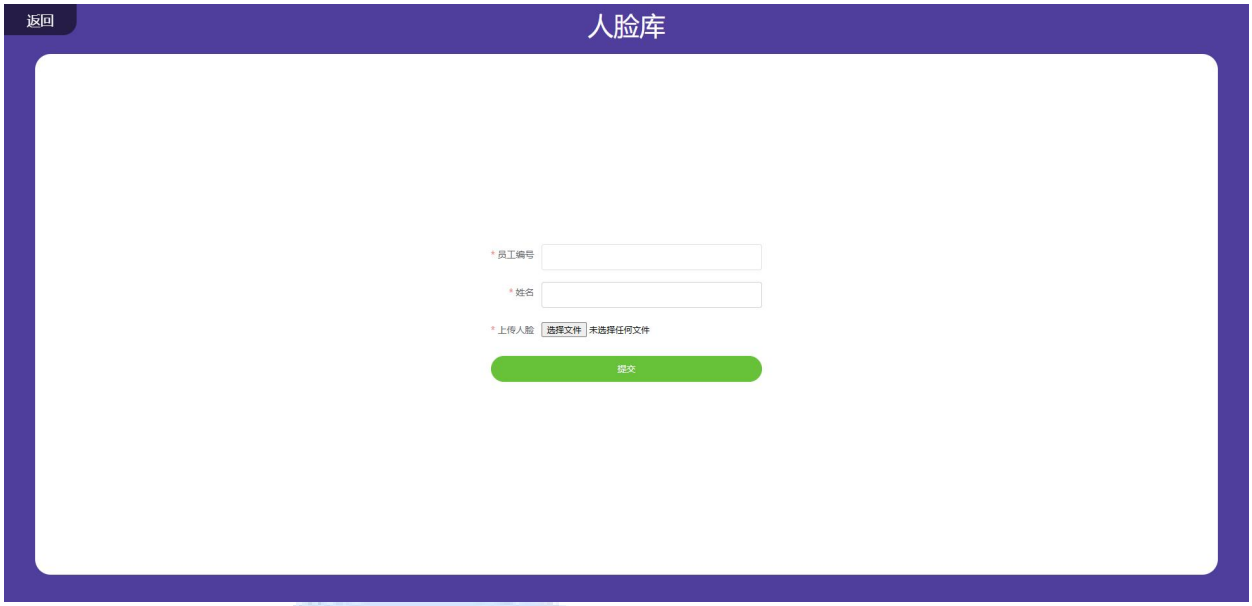
人脸添加
抓拍记录
识别区域
视频AI分析系统
重启服务
全屏显示
检测功能
2025年10月25日 星期六 15:14:28
当前识别：


返回
抓拍记录
CHANNEL0
RECORD
20251025124408.jpg
20251025124406.jpg
20251025124401.jpg
20251025124358.jpg
20251025124356.jpg
20251025124355.jpg
20251025124354.jpg
20251025124353.jpg
20251025124351.jpg
20251025124350.jpg
查看
删除
上一页
第 1 页 共1018页
下一页
删除所有

8.1.3. 人脸添加


人脸添加
抓拍记录
识别区域
视频AI分析系统
重启服务
全屏显示
检测功能
2025年10月25日 星期六 16:10:57
reflective_vest:1 0.85
当前识别：反光衣检测

返回
人脸库
添加人脸
员工编号
姓名
人脸
操作
No Data



8.2. AI 识别接口介绍

8.2.1. 寄存器映射表

地址范围	字段说明	数据类型	说明
10	目标监测类型	int	0-18（对应 19 种检测模式）
1000	目标数量	int	当前帧检测到的目标总数

地址范围	字段说明	数据类型	说明
1001+	目标数据组（每 6 寄存器一组）	-	每组包含 6 个字段：
1001	目标类型	int	对应具体目标分类
1002	置信度	Int	0-100%
1003	X 坐标	int	像素坐标
1004	Y 坐标	int	像素坐标
1005	宽度	int	像素单位
1006	高度	int	像素单位

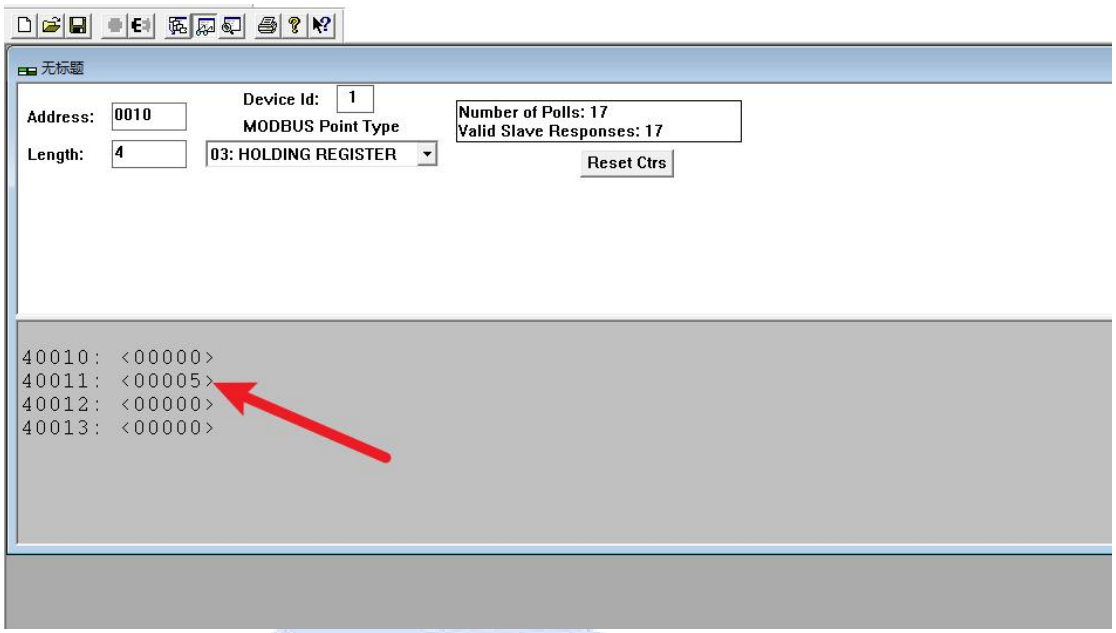
8.2.2. 目标检测类型列表

对应寄存器映射表中的寄存器地址 10。

识别编号	识别类型名	核心用途
0	标签识别	自定义标签类目标识别
1	安全帽检测	作业区域人员安全帽佩戴合规性检测
2	行人检测	场景内实时人数统计（非累计）
3	睡岗识别	值守岗位人员睡觉行为监测
4	跌倒检测	老人 / 儿童跌倒行为识别与救援提醒
5	反光衣检测	作业人员反光衣穿戴合规性检测
6	火焰烟雾检测	监控区域内明火、烟雾危险监测

识别编号	识别类型名	核心用途
7	人头检测	场景内实时人头数量统计（非累计）
8	电梯非机动车检测	禁止电动车、自行车进入电梯
9	电动车头盔识别	电动车驾驶员头盔佩戴合规性检测
10	重型机械车辆检测	施工现场 13 类工程车辆检测与计数
11	井盖识别	城市道路井盖缺失 / 松动危险监测
12	灭火器摆放识别	消防区域灭火器摆放合规性检测
13	通用打电话抽烟检测	禁烟 / 保密场景内打电话、抽烟行为监测
14	车辆检测	通道区域过线车流量统计（进 / 出）
15	疲劳检测	人员闭眼、张嘴等疲劳状态识别
16	垃圾桶满溢检测	室外垃圾桶垃圾满溢状态识别
17	口罩佩戴检测	医院 / 后厨等场景口罩佩戴合规性检测
18	非机动车检测	室外场景电动车、自行车等检测

例如当我在视频 AI 分析系统中选择的是反光衣检测，使用 Modscan 中，读取到的寄存器 10 中，modbus 数值为 05。



这为后续编辑代码，判断此目标类型为最高优先级。

8.3.各识别类型详细参数（按编号排序）

8.3.1. 标签识别

- **场景描述**：用于自定义标签类目标识别
- **适用场所**：无明确限制（需根据标签类型适配）

8.3.2. 安全帽检测

- **场景描述**：自动识别作业区域人员安全帽佩戴情况，支持红 / 白 / 蓝 / 黄 / 橘 / 酱紫 6 种颜色，不区分颜色
- **适用场所**：白天 / 室内外、工地、各类需佩戴安全帽的作业区域
- **目标类型定义**：

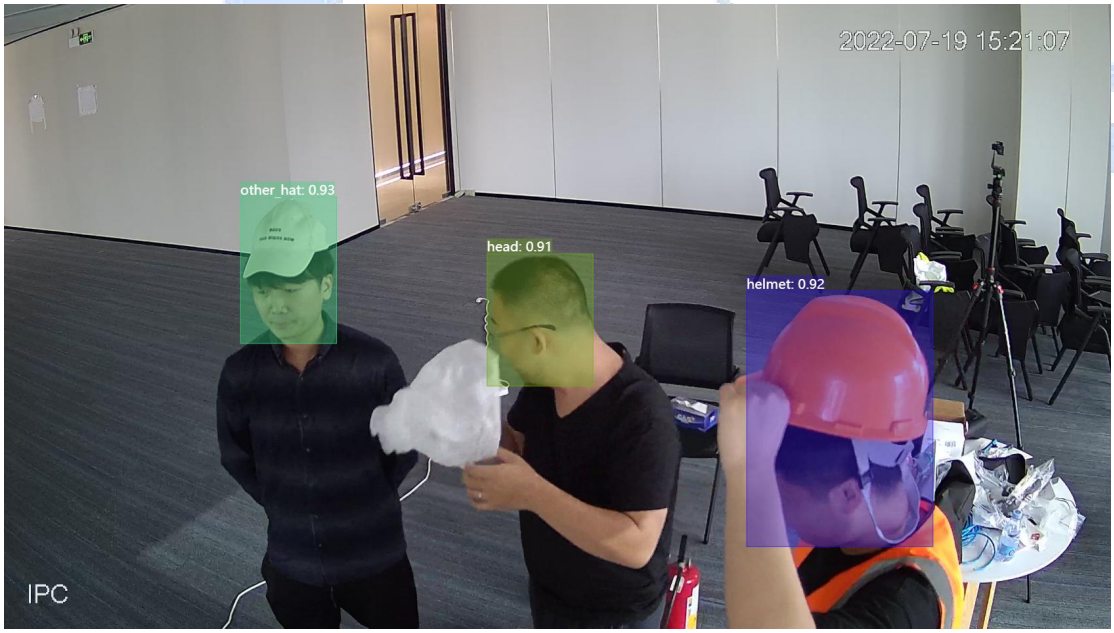
目标序号	目标名称	说明

目标序号	目标名称	说明
0	head	未佩戴安全帽（仅识别头部）
1	helmet	佩戴安全帽
2	other_hat	佩戴非安全帽类帽子

0-head 未佩戴安全帽（仅识别头部）

1-helmet 佩戴安全帽

2-other_hat(其它帽子)



8.3.3. 行人检测

- **场景描述**：自动检测场景内人员，支持实时人数统计（单次入镜人员有唯一 ID，出镜后清空，不累计）
- **适用场所**：白天 / 夜间光照良好、室内外环境
- **目标类型定义**：

目标序号	目标名称	说明
0	person	行人

0-person(人)

8.3.4. 睡岗识别

- **场景描述**：监测值守岗位人员睡觉行为（躺着睡、趴着睡），防止脱岗
- **适用场所**：保安亭、监控室、机房、贵重物品存放区等固定值守岗位
- **目标类型定义**：

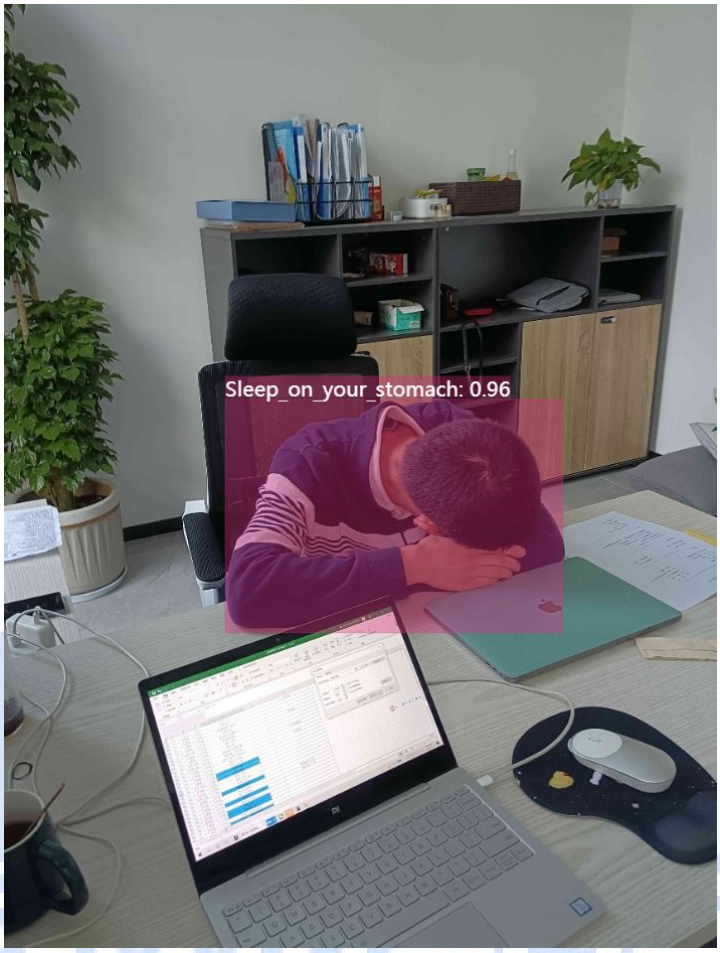
目标序号	目标名称	说明

目标序号	目标名称	说明
0	Lie_down_and_sleep	躺着睡
1	Normal	未睡觉（正常状态）
2	Sleep_on_your_stomach	趴着睡

0-Lie_down_and_sleep(躺着睡)

1-Normal(没睡)

2-Sleep_on_your_stomach(趴着睡)

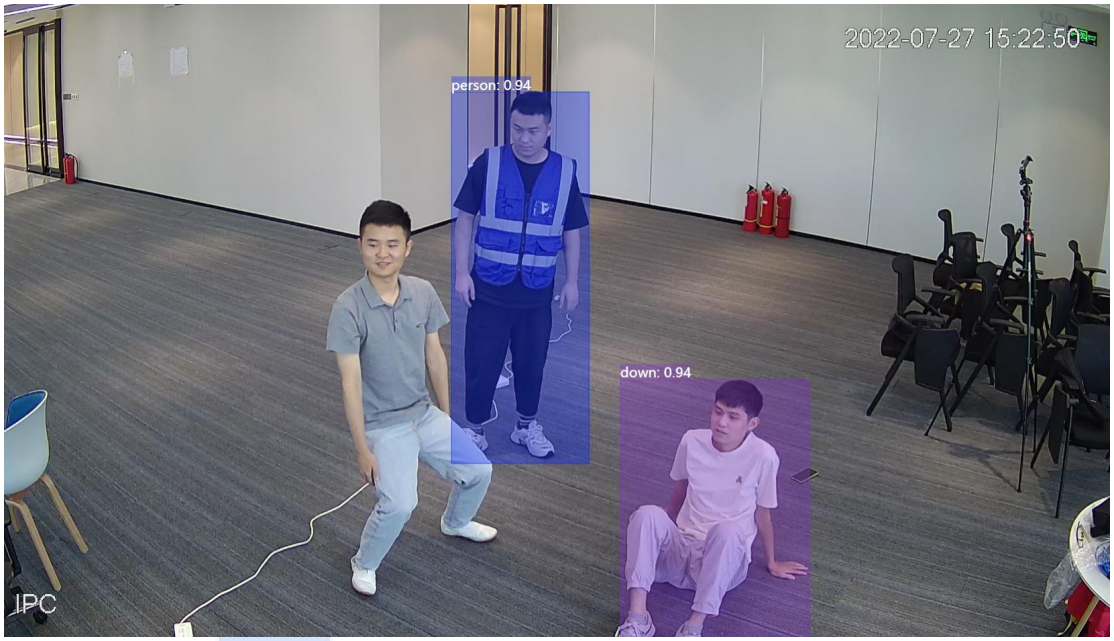


8.3.5. 跌倒检测

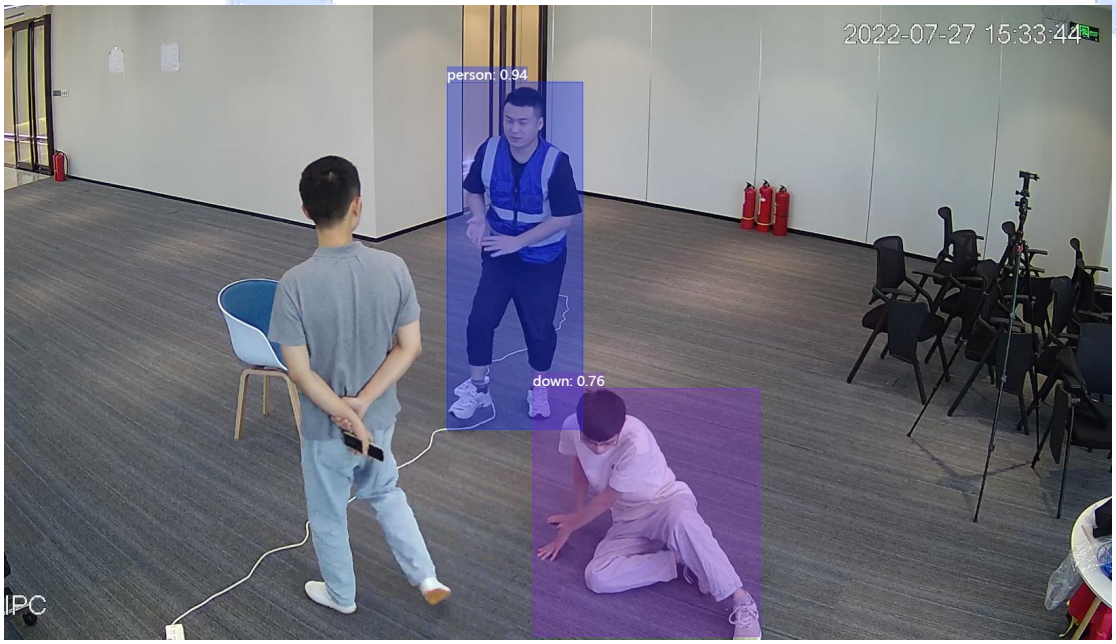
- **场景描述**：识别老年人 / 儿童跌倒行为，便于及时救援
- **适用场所**：白天 / 夜间光照良好、室内外（小区老人活动室、独居老人家中、学校）
- **目标类型定义**：

目标序号	目标名称	说明
0	down	跌倒在地
1	person	正常站立人员
2	sit	坐着（非跌倒）

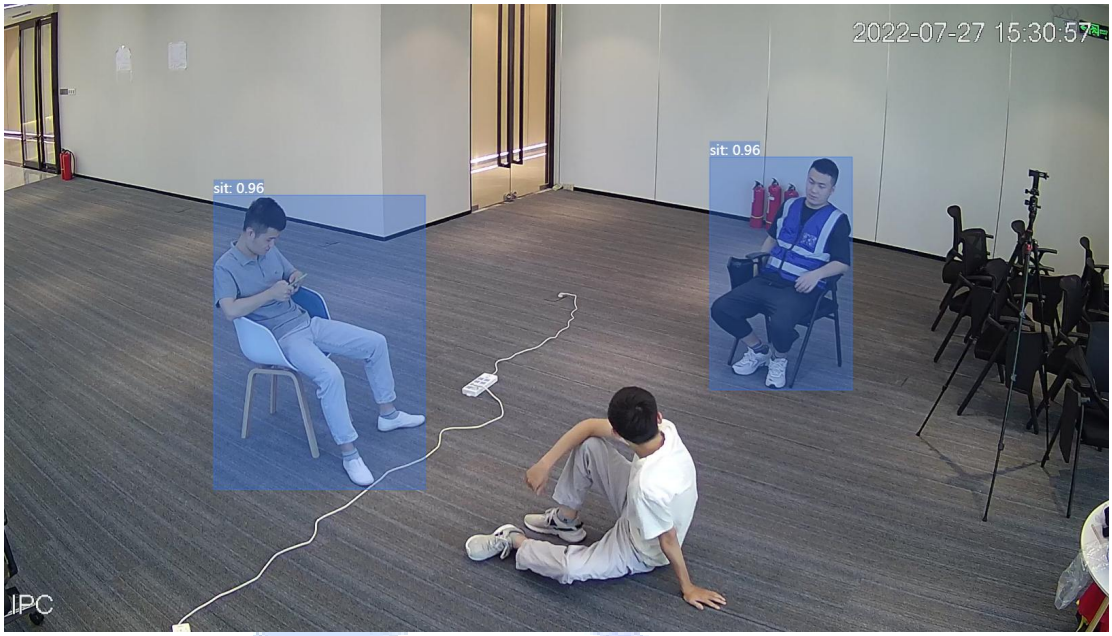
0-down(跌倒)



1-person(人)



2-sit(坐着)

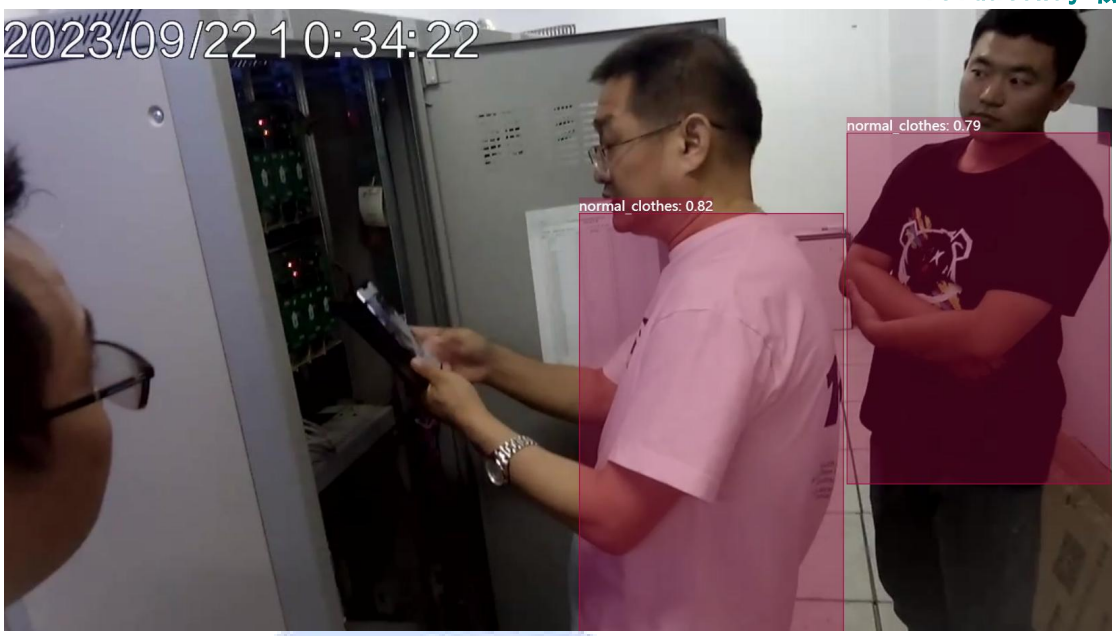


8.3.6. 反光衣检测

- **场景描述**：自动识别作业人员反光衣穿戴情况，不区分反光衣颜色
- **适用场所**：白天 / 室外、工地、城市道路（施工 / 环卫 / 交通作业人员）
- **目标类型定义**：

目标序号	目标名称	说明
0	normal_clothes	未穿反光衣
1	reflective_vest	已穿反光衣

0-normal_clothes(没穿反光衣)



1-reflective_vest(穿反光衣)

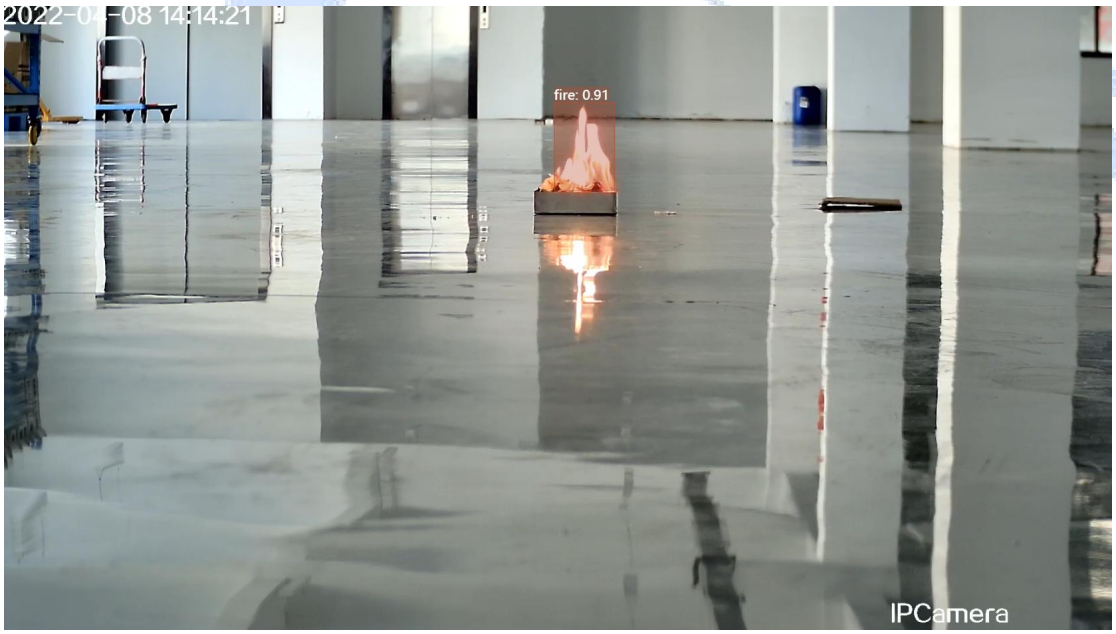
8.3.7. 火焰烟雾检测

- **场景描述**: 实时识别监控区域内明火 / 烟雾, 不区分火焰 / 烟雾颜色及来源; **不支持**夜间 / 红外补光 / 非彩色画面
- **适用场所**: 白天正常光照、室内外（油站、应急管理、秸秆焚烧、社区、写字楼）

- **目标类型定义**:

目标序号	目标名称	说明
0	fire	明火
1	other_fire	其它火焰
2	other_smoke	其它烟雾
3	smoke	烟雾

0-fire(明火)



1-other_fire(其它火焰)

2-other_smoke(其它烟雾)

3-smoke(烟雾)

8.3.8. 人头检测

- **场景描述**：自动检测场景内人头，支持实时人头数量统计（非累计）
- **适用场所**：白天 / 夜间光照良好、室内外环境
- **目标类型定义**：

目标序号	目标名称	说明
0	head	人头

0-head(头部)

8.3.9. 电梯非机动车检测

- **场景描述**：禁止电动车、自行车进入电梯，避免消防安全隐患
- **适用场所**：社区、园区、楼宇等电梯内
- **目标类型定义**：

目标序号	目标名称	说明
0	Baby-carriage	婴儿车（允许）
1	bicycle	自行车（禁止）
2	Electric-bicycle	电动自行车（禁止）
3	wheelchair	轮椅（允许）

0-Baby-carriage(婴儿车)

1-bicycle(自行车)

2-Electric-bicycle(电动自行车)

3-wheelchair(轮椅)

8.3.10. 电动车头盔识别

- **场景描述**：检测电动车驾驶员头盔佩戴情况，符合多地 “骑乘电动车须戴头盔” 法规
- **适用场所**：电动车可通行的道路
- **目标类型定义**：

目标序号	目标名称	说明
0	head	未戴头盔
1	motor_helmet	已戴头盔

0-head(头部)

1-motor_helmet(电动车头盔)

8.3.11. 重型机械车辆检测

- **场景描述**：检测施工现场 13 类工程车辆，支持车辆位置监测、数量统计及阈值设定
- **适用场所**：白天 / 夜间光照良好、室内外施工现场
- **目标类型定义**：

目标序号	目标名称	说明
0	big_truck	大卡车
1	Bulldozer	推土机
2	crane	起重机
3	Crushing_drill	破碎钻机
4	excavating_machinery	挖掘机
5	Forklift	叉车
6	fuel_tank_car	油罐车
7	Lift_truck	起重车
8	oil_car	油槽车
9	Other_heavy_vehicles	其它重型车辆
10	Part_of_the_car	泥头车
11	Pile_driving_drill	打桩钻机
12	Roller	滚轧机

0-big_truck(大卡车)

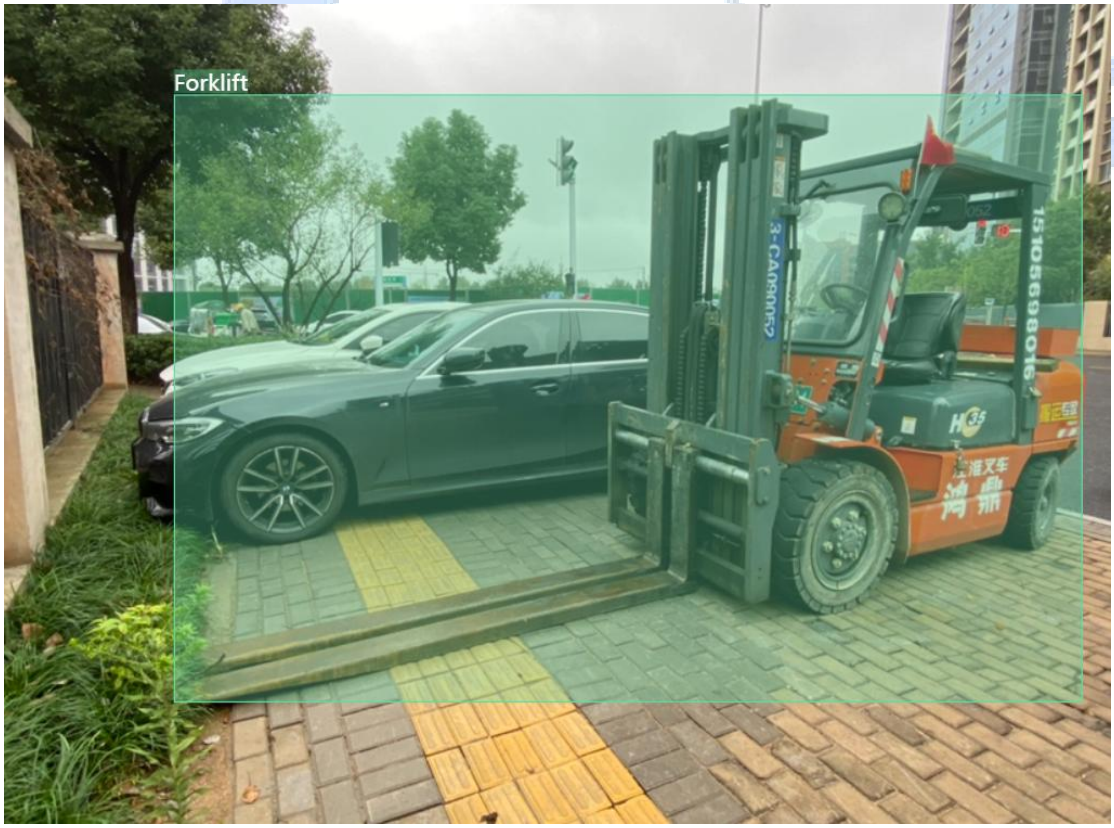
1-Bulldozer(推土机)

2-crane(起重机)

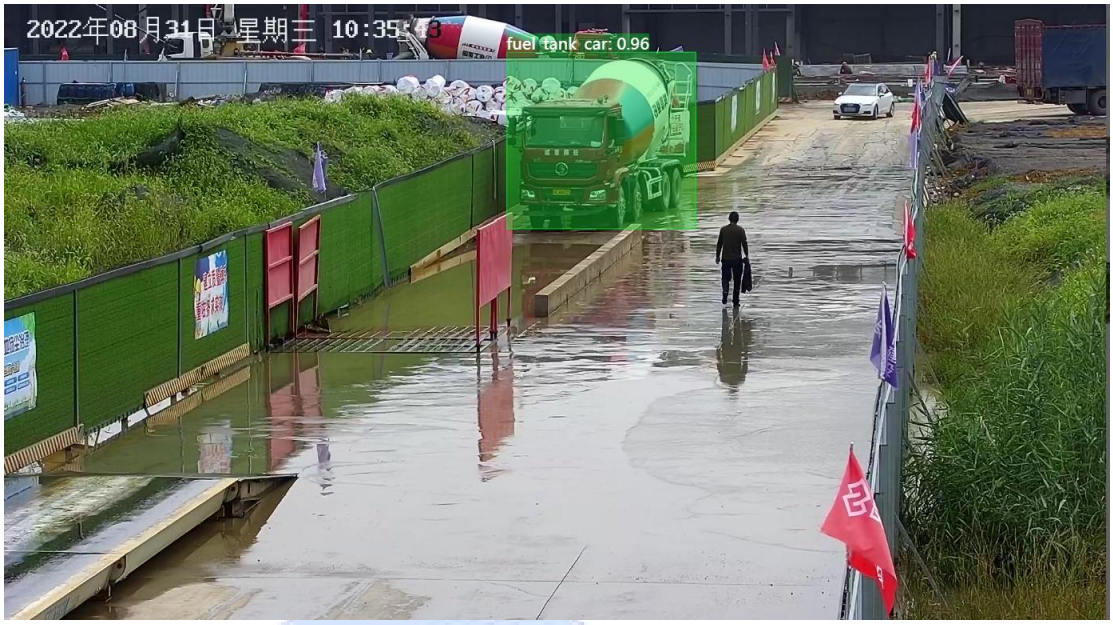
3-Crushing_drill(破碎钻机)

4-excavating_machinery(挖掘机)

5-Forklift(叉车)

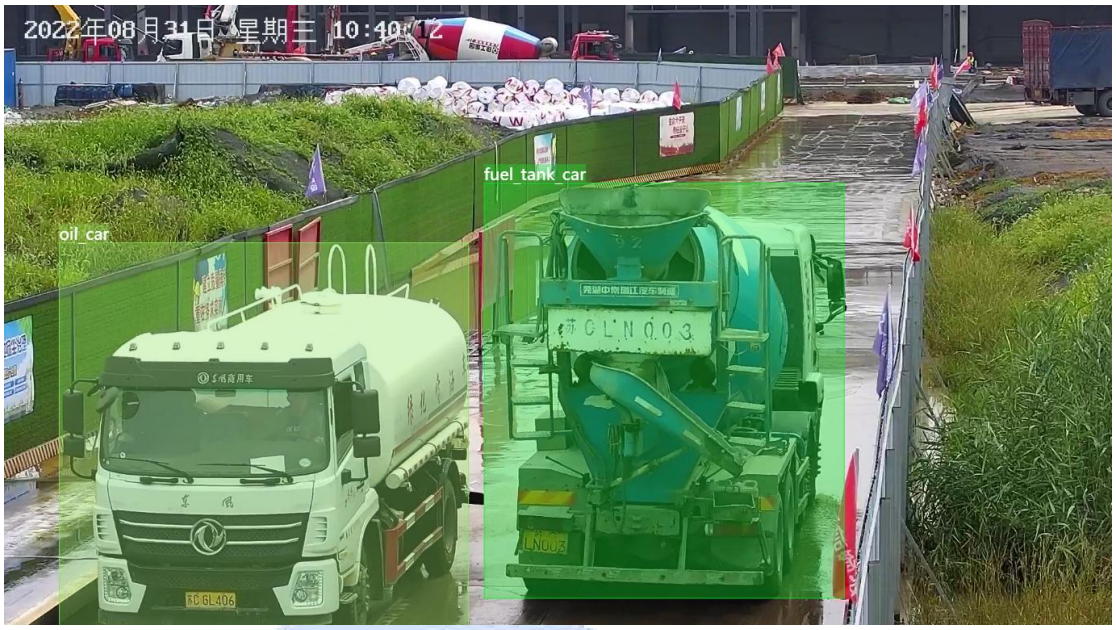


6-fuel_tank_car(油罐车)



7-Lift_truck(起重车)

8-oil_car(油槽车)



9-Other_heavy_vehicles(其它重型车辆)

10-Part_of_the_car(卡车)

11-Pile_driving_drill(打桩钻机)

12-Roller(滚轧机)

8.3.12. 井盖识别

- **场景描述**：识别井盖位置，检测井盖缺失、松动、未盖好等危险情况
- **适用场所**：白天 / 夜间光照良好、室外城市道路
- **目标类型定义**：

目标序号	目标名称	说明
0	Covered	有井盖（安全）
1	Damaged	危险井盖（损坏）
2	Missing	井盖打开 / 缺失

目标序号	目标名称	说明
3	Non_convered	没盖好井盖

0-Covered(安全井盖)

1-Damaged(危险井盖)

2-Missing(井盖打开)

3-Non_convered(没盖好井盖)

8.3.13. 灭火器摆放识别

- **场景描述**：识别指定区域灭火器是否存在及摆放是否正确（禁止倒放）
- **适用场所**：室内消防区域（社区 / 园区楼道消防间、作业区域消防点）
- **目标类型定义**：

目标序号	目标名称	说明
0	Correct_placement	正确摆放
1	Wrong_placement	错误摆放（倒放 / 缺失）

0-Correct_placement(正确摆放灭火器)

1-Wrong_placement(错误摆放灭火器)

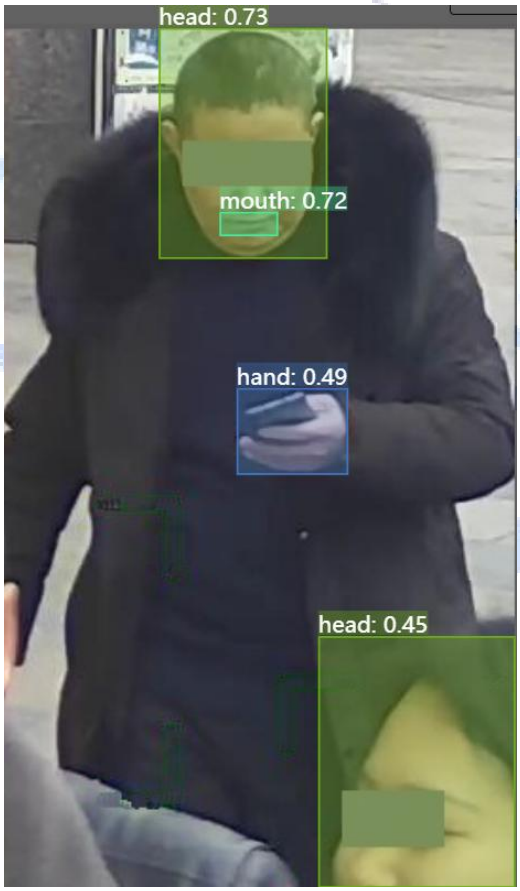
8.3.14. 通用打电话抽烟检测

- **场景描述**:
  - 打电话：人头 1.5 倍面积内出现手机，定义为打电话；手机放桌上 / 戴耳机通话定义为“玩手机”
  - 抽烟：人头 1.2 倍范围内出现香烟且处于 “抽” 的状态；手持香烟不抽不定义为抽烟
- **适用场所**:
  - 打电话：室内正常光照（禁烟防爆场景、保密场景如教室 / 考场）
  - 抽烟：室内正常光照（各类禁烟公共区域）
- **目标类型定义**:

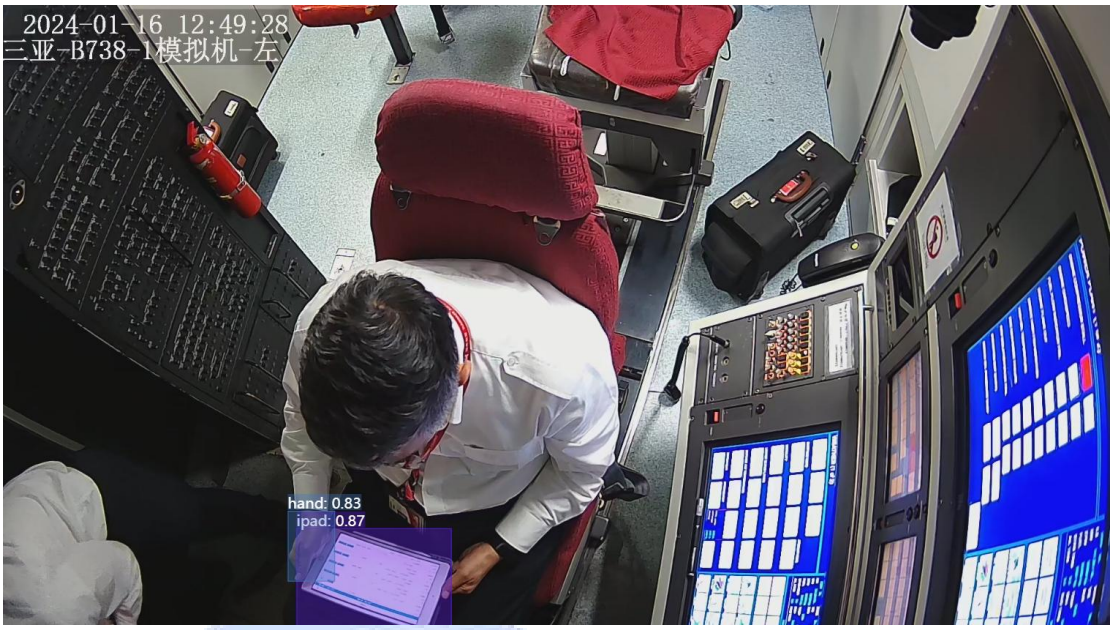
目标序号	目标名称	说明
0	hand	手部
1	head	头部
2	ipad	平板电脑
3	mask	口罩
4	mouth	嘴巴
5	phone	手机（打电话关联）
6	smoke	烟（抽烟关联）

0-hand(手部)

1-head(头部)



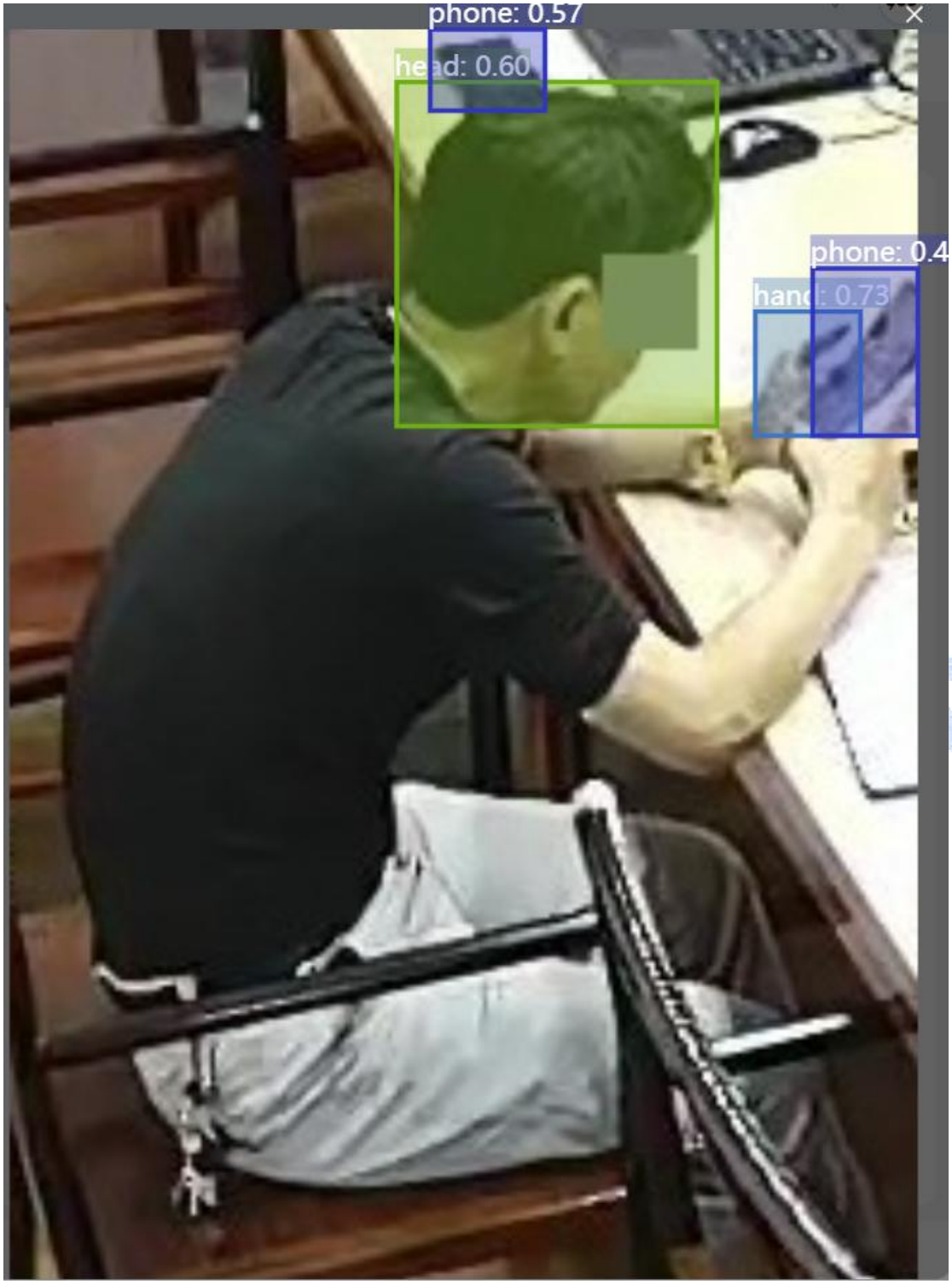
2-ipad(平板电脑)



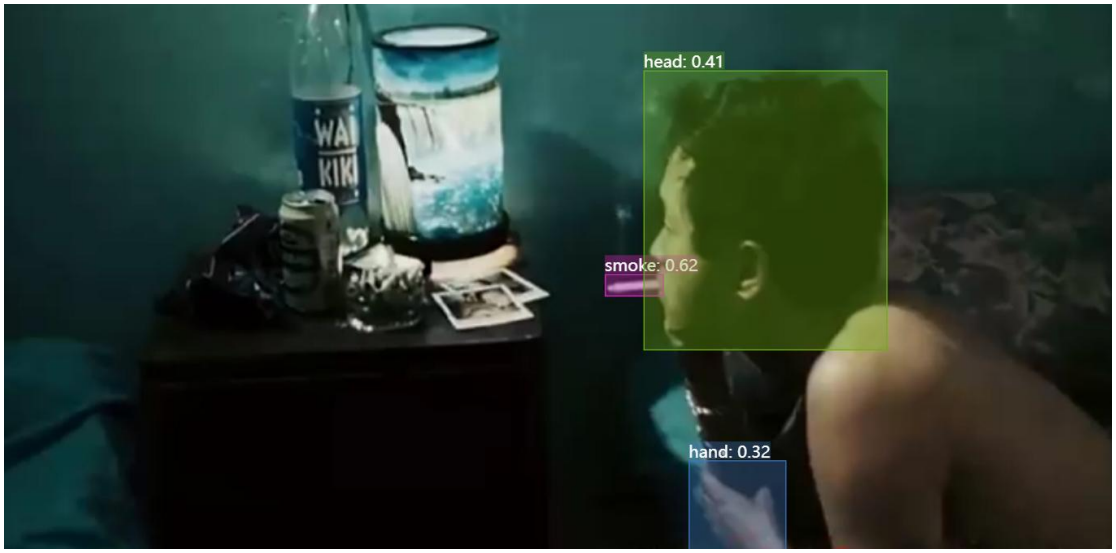
3-mask(口罩)

4-mouth(嘴巴)

5-phone(手机)



6-smoke(烟)



8.3.15. 车辆检测

- **场景描述**：在监控画面通道区域划线，统计过线车流量（进 / 出方向分别计数）
- **适用场所**：常规道路、通道区域（非路口）
- **目标类型定义**：

目标序号	目标名称	说明
0	car	车辆

0-car(车)

8.3.16. 疲劳检测

- **场景描述**：通过表情识别检测人员疲劳状态（闭眼、张嘴打瞌睡）
- **适用场所**：白天 / 夜间光照良好、室内外（驾驶舱、值守岗位）
- **目标类型定义**：

目标序号	目标名称	说明
0	closed_eye	闭眼（疲劳）
1	closed_mouth	闭嘴（正常）
2	open_eye	睁眼（正常）
3	open_mouth	张嘴（疲劳）

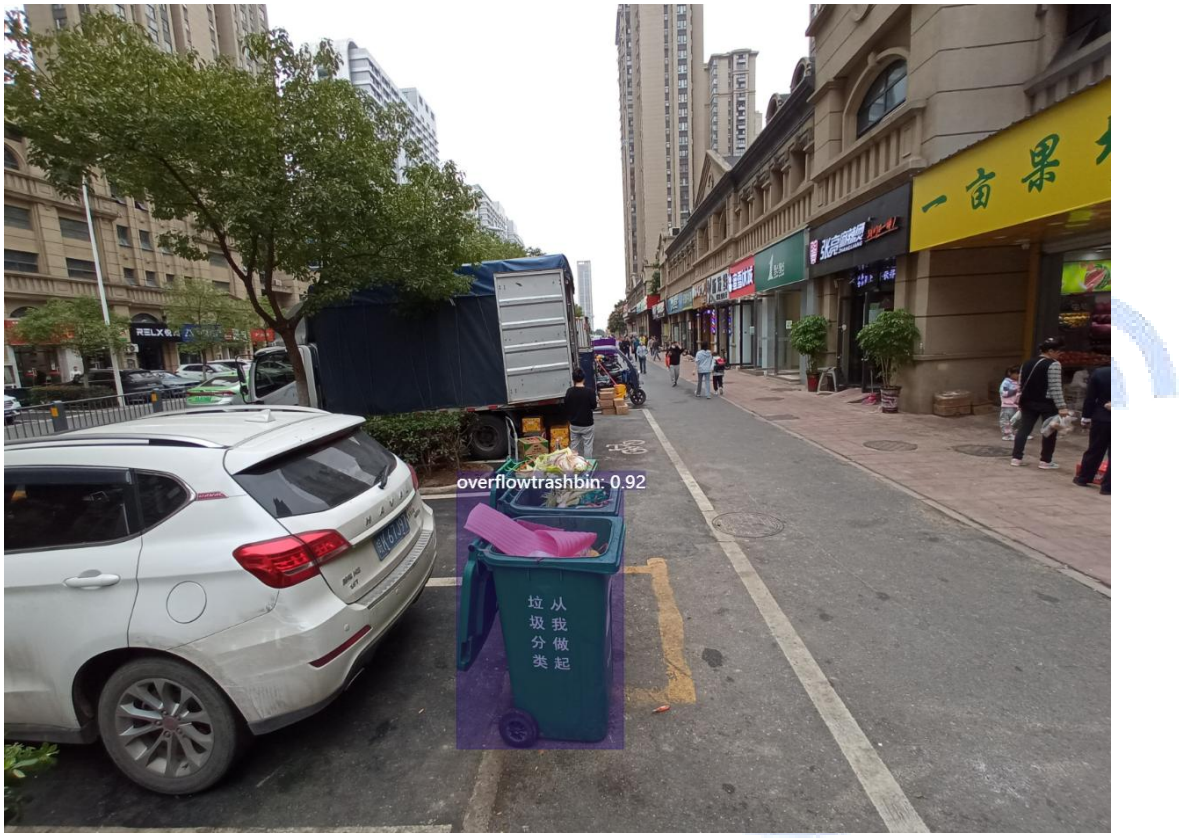
8.3.17. 垃圾桶满溢检测

- **场景描述**：识别室外垃圾桶内垃圾是否满溢，提醒清理
- **适用场所**：白天 / 夜间光照良好、室外城市道路

- **目标类型定义**:

目标序号	目标名称	说明
0	overflowtrashbin	垃圾桶满溢
1	trashbin	垃圾桶正常（未满）

0-overflowtrashbin(垃圾桶满溢)



1-trashbin(垃圾桶正常)

8.3.18. 口罩佩戴检测

- **场景描述**：自动识别人员是否佩戴口罩
- **适用场所**：医院、后厨、公共场所等需佩戴口罩的场景
- **目标类型定义**：

目标序号	目标名称	说明
0	face	未戴口罩
1	face_mask	已戴口罩

0-face(没戴口罩)

1-face_mask(戴口罩)

8.3.19. 非机动车检测

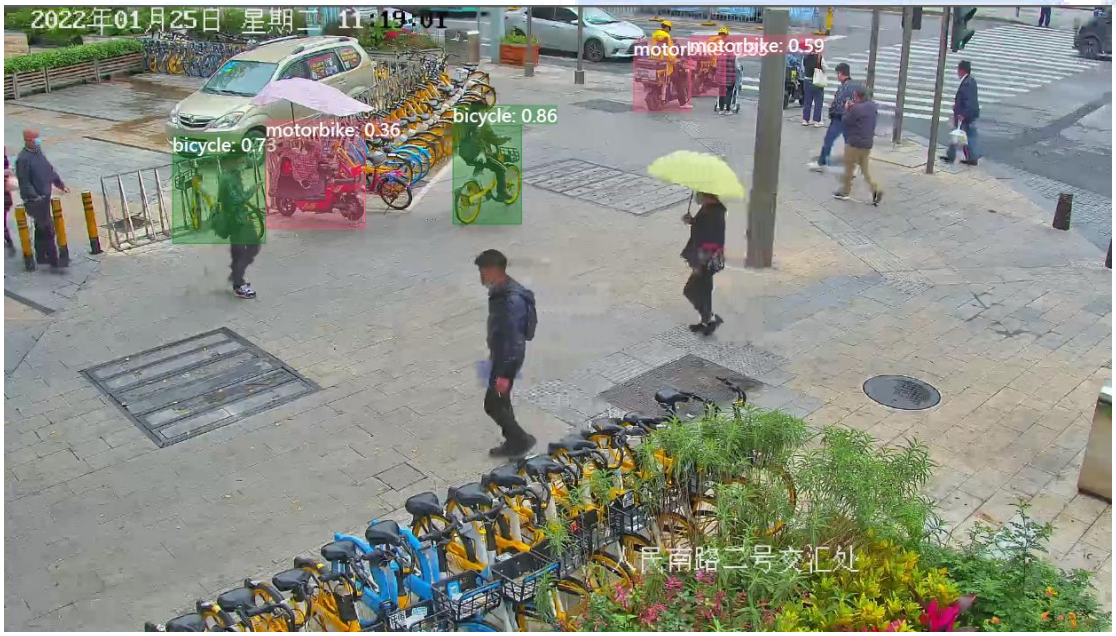
- **场景描述**：识别室外场景内非机动车，避免消防安全隐患（如电动车违规停放）

- **适用场所**：白天 / 夜间光照良好、室外（社区、园区、道路旁）

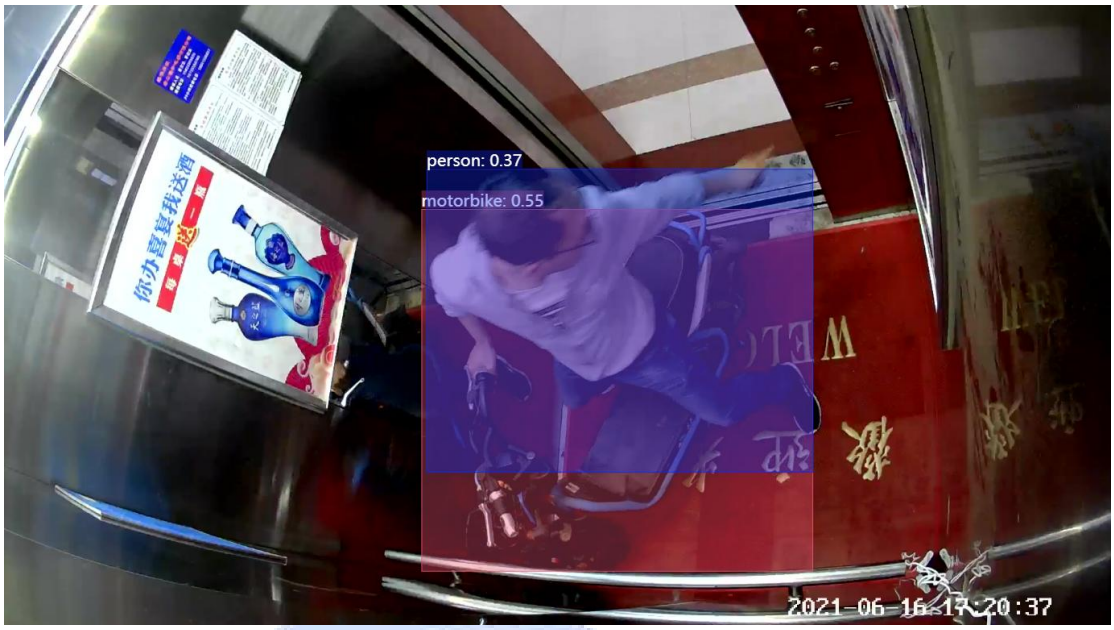
- **目标类型定义**：

目标序号	目标名称	说明
0	bicycle	自行车
1	motorbike	摩托车
2	person	人员
3	stroller	婴儿车
4	wheelchair	轮椅

0-bicycle(自行车)

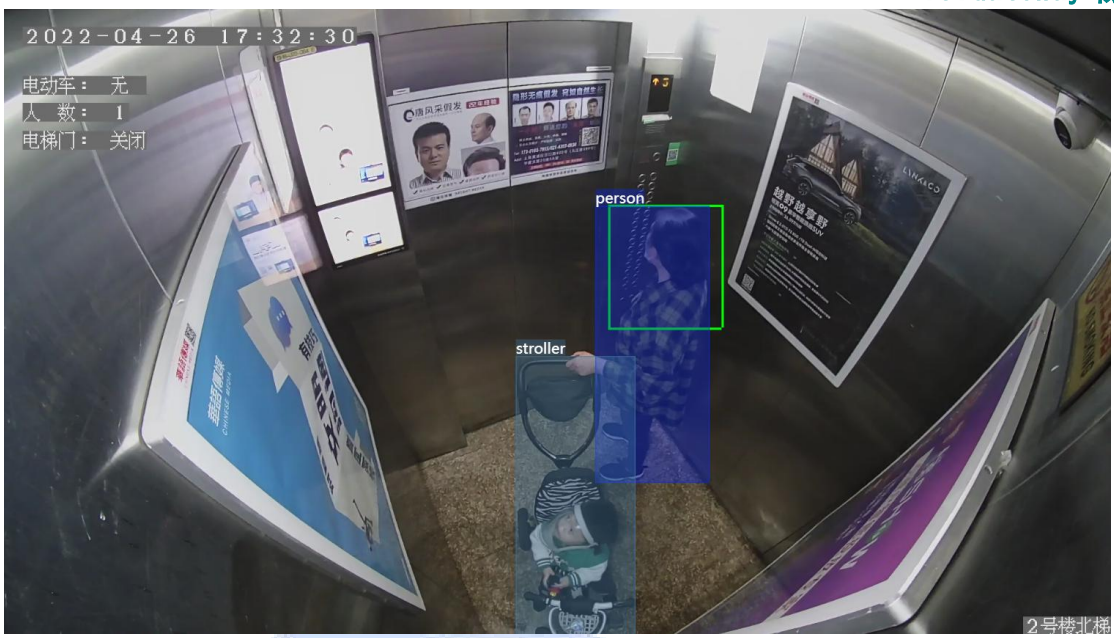


1-motorbike(摩托车)

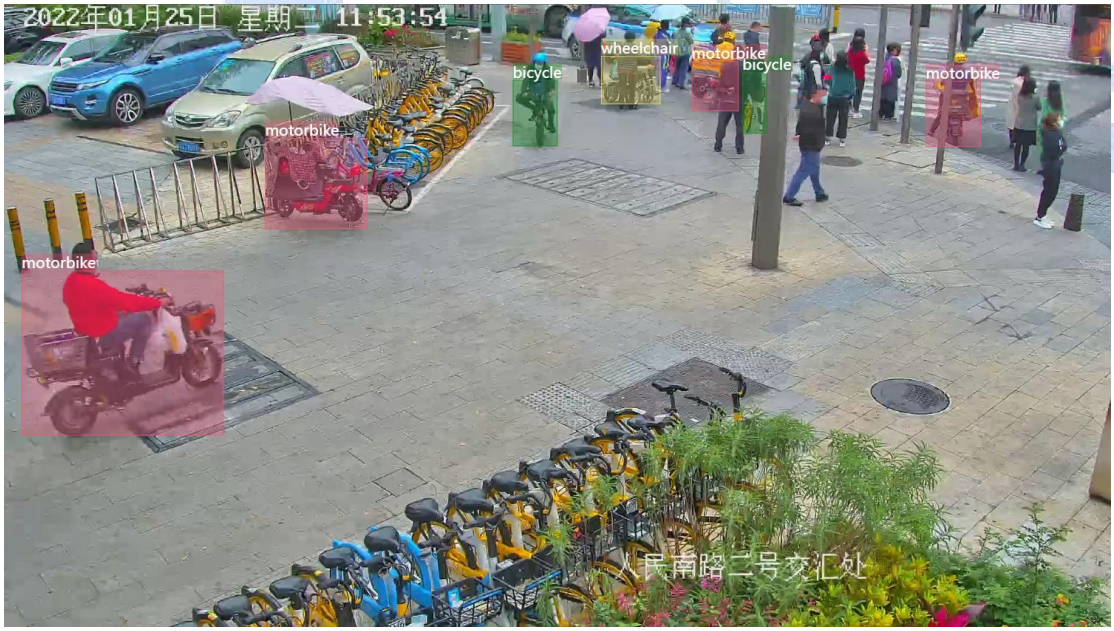


2-person(人)

3-stroller(婴儿车)



4-wheelchair(轮椅)



8.4. Modbus 寄存器通信规范

8.4.1. 核心寄存器地址定义

寄存器地址	寄存器含义	数据类型	取值范围/说明	关联功能
10	模型类型（识别类型编号）	int	0~18（对应 2.1 节的识别编号）	所有识别类型
1000	目标数量	int	- 通用场景：目标数量	所有识别类型
1001~N	目标详细数据（每 6 个一组）	int	每组对应 1 个目标，顺序为： 1. 目标类型 2. 目标可信度（0~100，越高越准） 3. 目标 x 坐标 4. 目标 y 坐标 5. 目标宽度 6. 目标高度	所有识别类型（需获取目标位置）

8.4.2. 数据读取规则

1. **第一步：读取识别类型**

读取地址 10，获取当前启用的识别类型编号（0~18），确定后续读取的寄存器逻辑。

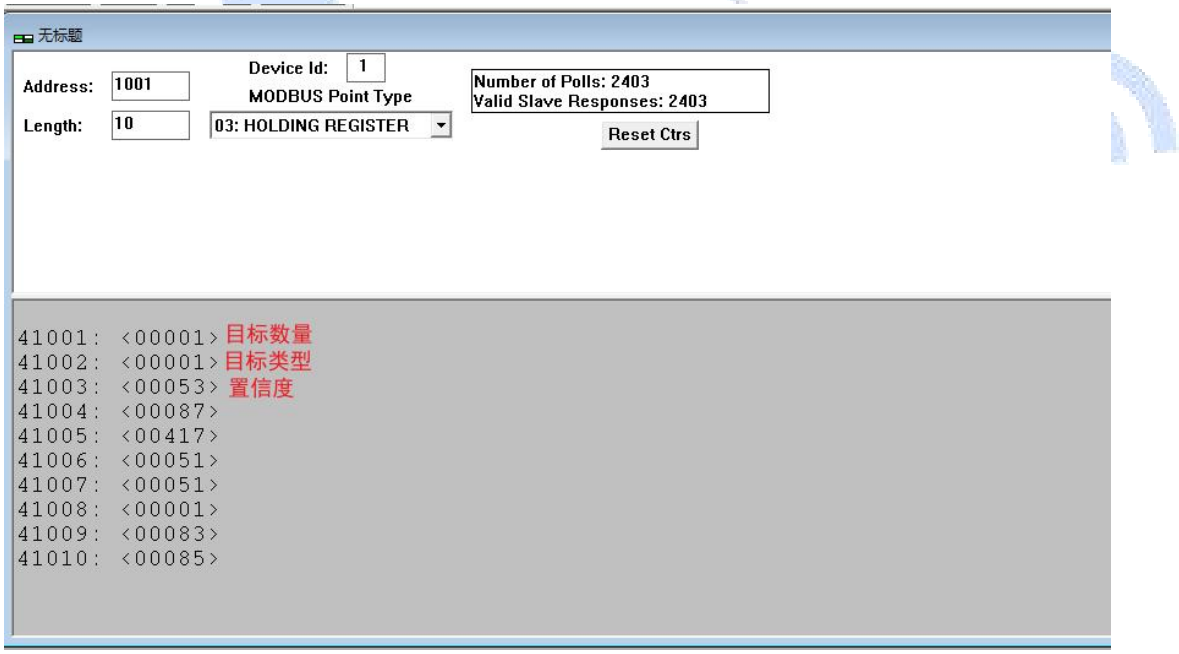
2. **第二步：读取目标数量**

   - 通用场景：读取地址 1001，获取当前目标数量。

3. **第三步：读取目标详细信息（可选）**

若需目标位置 / 可信度，按“1002+6*(n-1)”读取（n 为目标序号，从 1 开始），每组 6 个寄存器对应 1 个目标。

例如我选择的是反光衣的算法并且识别到反光衣，打开测试工具看状态。



查看反光衣的目标类型显示为，目标类型为 1 时，检测到反光衣，为 0 时未检测到反光衣。

8.5. 典型应用案例代码

```
release();
ip = "192.168.3.202"          -- AI 主机 的 IP，根据现场网段自行修改
-------------------------------------------------------------------
-- 主循环：每 1 s 去查询一次 Modbus 寄存器，根据当前“监测类型”做不同判断
-------------------------------------------------------------------
while 1 do
```

```
    -- 1. 读取“当前通道的监测类型”
    --    功能码 03（读保持寄存器）
    --    modbus 端口为5002，
    --    设备地址为1，起始地址 10，长度 1，读取一个寄存器
    --    返回值 ret：1=成功，0=失败；mode：具体监测类型
    ret, mode = ModbusTCP_read_hold_register(ip, 5002, 1, 10, 1, "int")
    if ret == 1 then            -- 读成功才继续，否则直接跳到下 1 s
        ------------------------------------------------------------------
        -- mode=3  趴桌子（睡觉）检测
        ------------------------------------------------------------------
        if mode == 3 then
            -- 地址 1001：该通道的“检测结果计数器”
            -- 大于 1 表示连续检测到趴桌子，需要提醒
            ret, s = ModbusTCP_read_hold_register(ip, 5002, 1, 1001, 1, "int")
            if s > 1 then
                playwav("/wav/nosleep.wav", 1)    -- 播放“请勿睡觉”提示音
            end
        ------------------------------------------------------------------
        -- mode=5  反光衣（安全背心）检测
        ------------------------------------------------------------------
        elseif mode == 5 then
            -- 一次读 2 个寄存器：
            --   1000：检测到的“反光衣目标数量”
            --   1001：是否合规（1=合规，0=不合规）
            ret, num, s = ModbusTCP_read_hold_register(ip, 5002, 1, 1000, 2, "int
")
            -- 只要检测到人(num>0) 却 没穿反光衣(s==0) 就报警
            if num > 0 and s == 0 then
                playwav("/wav/novest.wav", 1)     -- 播放“请穿反光衣”提示音
            end
        ------------------------------------------------------------------
        -- mode=6  明火（火焰）检测
        ------------------------------------------------------------------
        elseif mode == 6 then
            -- 地址 1000：火焰检测置信度/计数，非 0 即视为有火
            ret, fire = ModbusTCP_read_hold_register(ip, 5002, 1, 1000, 1, "int")
            if fire ~= 0 then
                playwav("/wav/nofire.wav", 1)     -- 播放“禁止明火”提示音
            end
        end
    end
    delay(1000)    -- 轮询间隔 1 s
end
```

9. 脚本生成工具功能

拖拽式界面：

用户可以通过拖拽界面上的预定义元素（如函数块、循环结构、条件判断等来构建 Lua 脚本的逻辑。
界面元素通常包含清晰的图标和标签，以帮助用户理解其功能。

自动代码生成：

当用户通过拖拽构建完程序逻辑后，工具会自动将用户的操作转换为 Lua 脚本代码。
用户可以在需要时查看或编辑生成的代码，以确保其符合需求。

调试与测试：

大多数可视化编程工具都提供调试功能，允许用户逐步执行脚本以查找和修复错误。
用户还可以直接在工具中测试脚本，查看其运行结果是否符合预期。

导出与分享：

用户可以将生成的 Lua 脚本导出为文件，以便在其他环境中使用。
某些工具还可能支持将脚本分享到在线平台或社区，以便与其他用户共享和交流。

教育与学习：

学生和初学者可以使用这种工具来学习 Lua 编程和可视化编程的基本概念，而无需深入学习复杂的语法和规则。

1. **打开网关配置工具，右键网关打开脚本生成工具即可进入。**

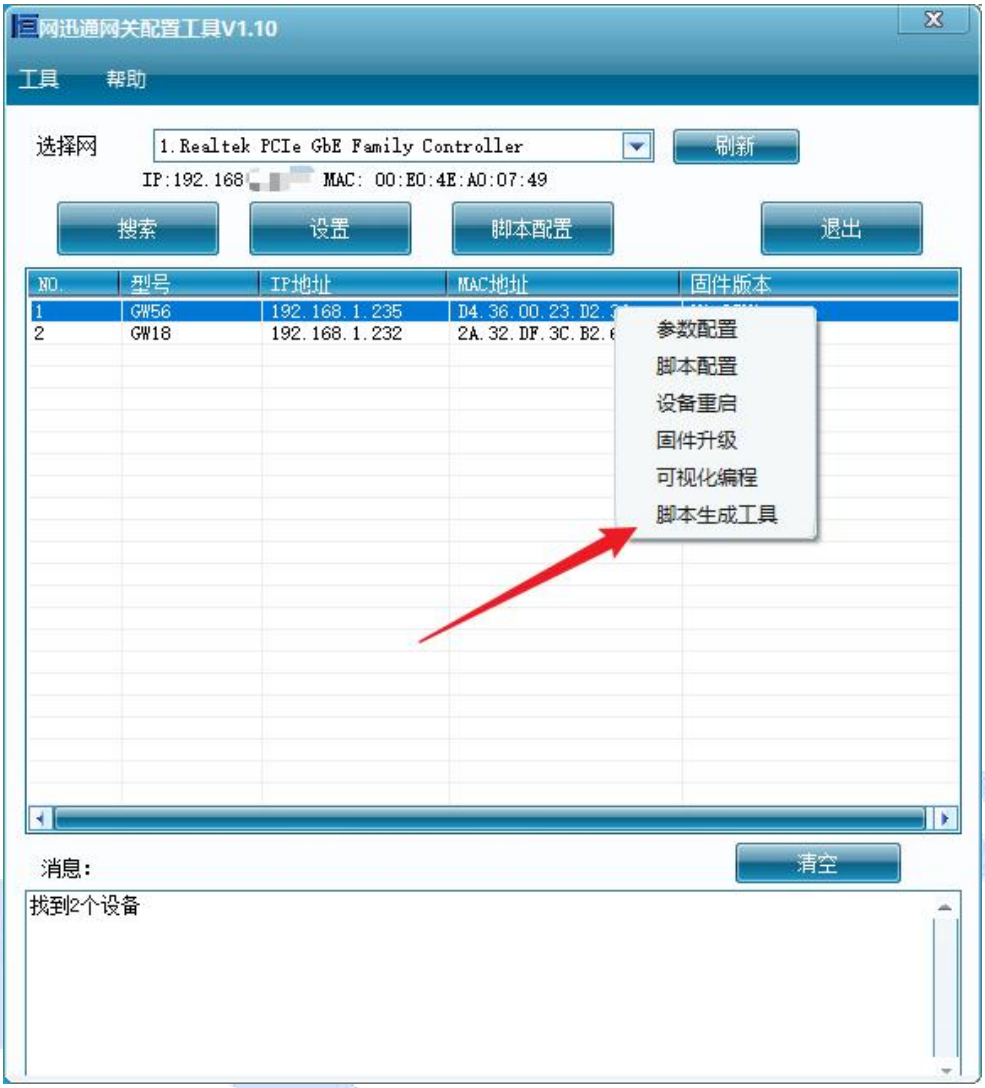


图 12 进入脚本生成工具

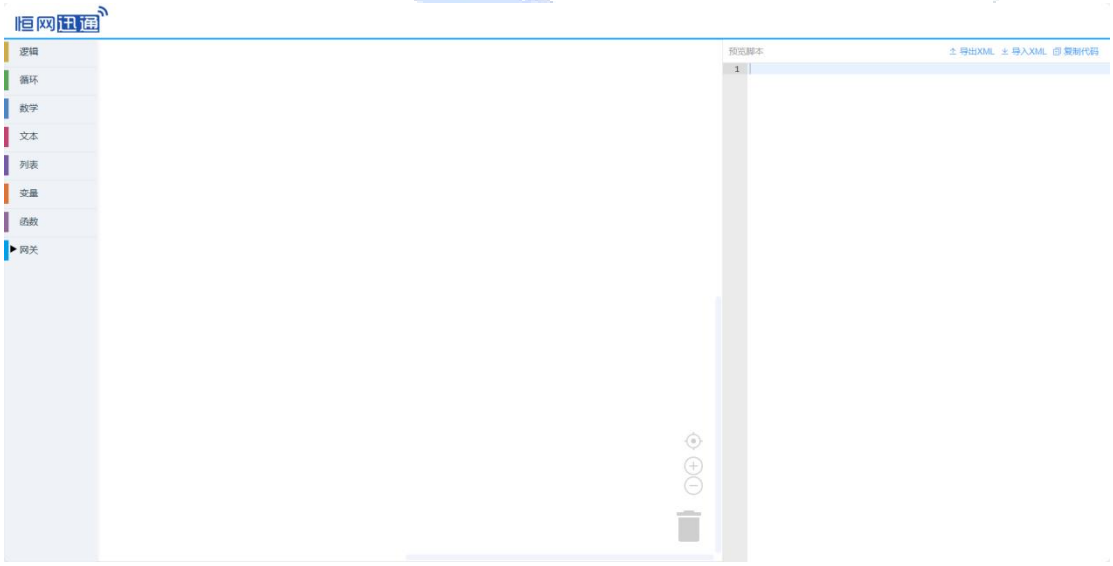


图 13 进入脚本生成工具

2. 案例

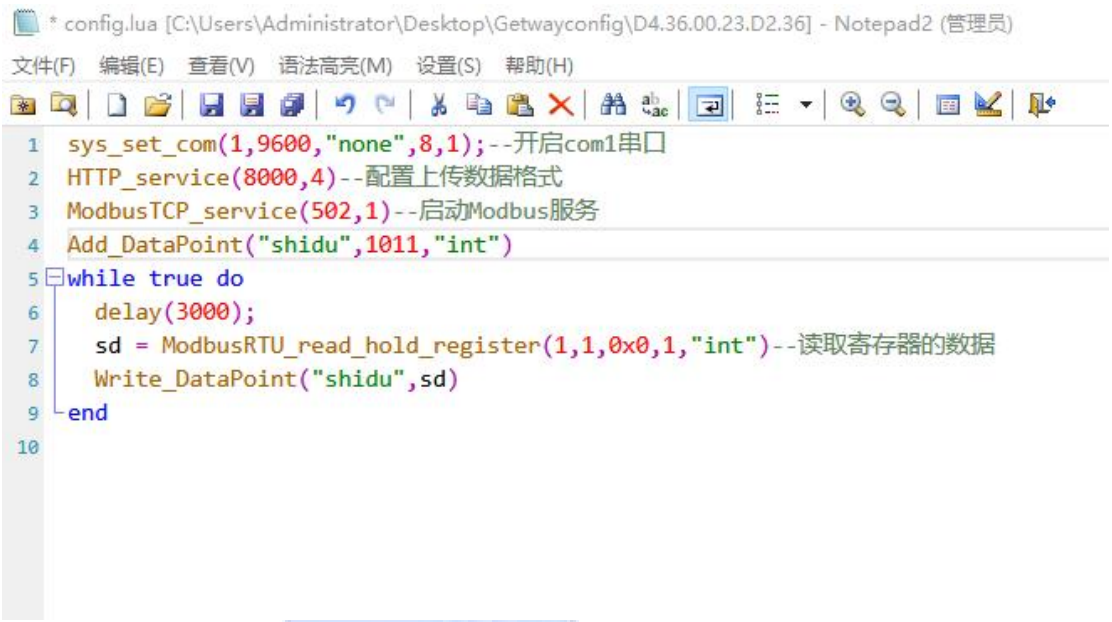


图 14 要编写的代码

根据此 lua 脚本，用脚本生成工具生成。

图 15 脚本生成工具使用

生成代码后，即可直接复制使用

总结

基于简易拖拽式可视化编程的工具为 Lua 脚本的生成提供了一种直观、便捷的方式。通过图形界面构建程序逻辑并自动生成代码，这种工具大大降低了 Lua 编程的门槛，使得更多用户能够轻松上手并享受编程的乐趣。

10. 脚本函数接口

10.1. 配置串口通讯参数

sys_set_com		
设置网关 com 通讯参数		
参数	**类型**	**说明**
Com	int	串口，1，2，3，4

Baud	int	波特率，9600，19200， 57600，115200，etc
Parity	char	校验，"even"，"odd" or "none"
Dbit	int	数据位，5，6，7，8
Sbit	int	停止位，1，2
返回值	void	Void
示例代码：		
-- 设置 com0，19200，N，8，1 sys_set_com(0, 19200, "none", 8, 1)		

10.2. Modbus/TCP 服务（从站）

10.2.1. 启动 modbus/tcp server（从站）

ModbusTCP_service		
启动 ModbusTCP 服务		
参数	**类型**	**说明**
port	Int	port > 0
id	Int	Modbus 设备 id
返回值	void	
函数说明		
示例代码：		
-- 启动 tcp server 端口为 502, 设备 id 为 1 **ModbusTCP_service** (502, 1)		

10.3. Modbus/TCP 数据读写（主站）

10.3.1. 从 01H 号功能码(coil status/线圈状态寄存器)中读取数据

ModbusTCP_read_coil_status		
从 1 号功能码(coil status/线圈状态寄存器) 中读取数据		
参数	**类型**	**说明**
Ip	Str	ModbusTCP 从机 ip
Port	Int	ModbusTCP 从机端口

Id	Int	设备地址 1 <= id<= 254
addr	Int	寄存器取值范围 0-0xffff
Num	Int	线圈寄存器数量，1-8 范围
返回值	Int	第 1 个返回值为读取状态，读取寄存器数量超过 8 个返回-1，连接不上服务器返回-2，连接上服务器没有收到服务器返回返回-3,后续参数根据读取数量返回多个数值
函数说明		
示例代码：		
--向 ip 为 192.168.1.100 端口为 502，设备地址为 15 的从机，读 0x07cf 地址开始的 2 个线圈状态值 Stat,reg1,reg2= ModbusTCP_read_coil_status (“192.168.1.100”,502, 15, 0x07cf, 2);		

10.3.2. 从 02H 号功能码(input status/输入状态寄存器)中读取数据

ModbusTCP_read_input_status		
从 2 号功能码(input status/输入状态寄存器) 中读取数据		
参数	**类型**	**说明**
Ip	Str	ModbusTCP 从机 ip
Port	Int	ModbusTCP 从机端口
Id	Int	设备地址 1 <= id<= 254
addr	Int	寄存器取值范围 0-0xffff
Num	Int	输入状态寄存器数量，1-8 范围
返回值	Int	第 1 个返回值为读取状态，读取寄存器数量超过 8 个返回-1，连接不上服务器返回-2，连接上服务器没有收到服务器返回返回-3,后续参数根据读取数量返回多个数值
函数说明		
示例代码：		

--向 ip 为 192.168.1.100 端口为 502，设备地址为 15 的从机，读 0x07cf 地址开始的 2 个输入状态值 Stat,reg1,reg2= ModbusTCP_read_input_status (“192.168.1.100”,502, 15, 0x07cf, 2);

10.3.3. 从 03H 功能码(holding register/保持寄存器)中读取数据

ModbusTCP_read_hold_register		
根据功能码从 3 号功能码(hold register/状态寄存器) 中读取数据		
参数	**类型**	**说明**
Ip	Str	ModbusTCP 从机 ip
Port	Int	ModbusTCP 从机端口
Id	Int	设备地址 1 <= id<= 254
Addr	Int	寄存器取值范围 0-0xffff
Num	Int	保持寄存器数量
Type	Str	读取变量的类型，int、float、BCD
返回值	Int	第 1 个返回值为读取状态，读取寄存器数量超过 8 个返回-1，连接不上服务器返回-2，连接上服务器没有收到服务器返回返回-3,后续参数根据读取数量返回多个数值
函数说明		
示例代码：		
--向 ip 为 192.168.1.100 端口为 502，设备 id 为 15 的从机，读 100 地址开始的 8 个保持寄存器值，类型为 float Stat,v1,v2,v3,v4,v5,v6,v7,v8=ModbusTCP_read_hold_register("192.168.1.100",502,15, 100, 8, "float")		

10.3.4. 从 04H 号功能码(input register/输入寄存器) 中读取数据

ModbusTCP_read_input_register		
根据功能码从 4 号功能码(input register /输入寄存器) 中读取数据		
参数	**类型**	**说明**
Ip	Str	ModbusTCP 从机 ip
Port	Int	ModbusTCP 从机端口

id	Int	设备地址 1 <= id<= 254
addr	Int	寄存器取值范围 0-0xffff
Num	Int	输入寄存器数量，1-8 范围
type	Str	读取变量的类型，int、float、BCD
返回值	Int	第 1 个返回值为读取状态，读取寄存器数量超过 8 个返回-1，连接不上服务器返回-2，连接上服务器没有收到服务器返回返回-3,后续参数根据读取数量返回多个数值
函数说明		
示例代码：		
--向 ip 为 192.168.1.100 端口为 502，设备 id 为 15 的从机，读 100 地址开始的 2 个输入寄存器值，类型为 float Stat,reg1,reg2= ModbusTCP_read_input_register ("192.168.1.100",502,15,100,2,"float")		

10.3.5. 从 05H 号功能码(coil status/线圈寄存器) 中写数据

ModbusTCP_force_single_coil		
从 5 号功能码(coil status/线圈寄存器) 中写数据		
参数	**类型**	**说明**
Ip	Str	ModbusTCP 从机 ip
Port	Int	ModbusTCP 从机端口
Id	Int	设备地址 1 <= id<= 254
Addr	Int	寄存器取值范围 0-0xffff
Value	Int	线圈值，0 或 1
返回值	Int	第 1 个返回值为读取状态，读取寄存器数量超过 8 个返回-1，连接不上服务器返回-2，连接上服务器没有收到服务器返回返回-3
函数说明		
示例代码：		
--向 ip 为 192.168.1.100 端口为 502 设备 id 为 15 的从机，写 0x03e9 地址线圈值为 1 Stat= ModbusTCP_force_single_coil ("192.168.1.100",502,15, 0x03e9, 1);		

10.3.6. 从 06H 号功能码中写寄存器

ModbusTCP_write_register		
从 06H 号功能码中写寄存器		
参数	**类型**	**说明**
Ip	Str	ModbusTCP 从机 ip
Port	Int	ModbusTCP 从机端口
Id	Int	设备地址 1 <= id<= 254
addr	Int	寄存器地址取值范围 0-0xffff
Value	Int	寄存器值取值范围 0-0xffff
Type	Str	写入变量的类型：int
返回值	Int	第 1 个返回值为读取状态，读取寄存器数量超过 8 个返回-1，连接不上服务器返回-2，连接上服务器没有收到服务器返回返回-3
函数说明		
示例代码：		
--向 ip 为 192.168.1.100 端口为 502,设备 id 为 14 的从机，设置 0x0063 寄存器地址数值为 0x0001，数据类型为”int” Stat= ModbusTCP_write_register (“192.168.1.100”,502, 14, 0x0063, 0x0001, "int");		

10.4. Modbus/RTU 从站服务（从站）

10.4.1. 启动 modbus/RTU 从站服务（从站）

ModbusRTU_slave_service		
启动网关某一串口作为 modbusRTU 从站设备，供主设备连接获取寄存器数值。如与工业组态触摸屏、组态软件等设备连接		
参数	**类型**	**说明**
Com	Com	1，2，3，4 分别对应网关的串口 1-4 串口
返回值	void	
函数说明	使用该函数前，需要先调用 sys_set_com 初始化串口参数。启用该服务后，modbusRTU 从站设备 ID 自动识别。 启动该服务后，可读写网关保持寄存器和线圈状态值，注意读写寄存器范围为 1000-2000，线圈范围为 1000-1100	

示例代码：
-- 启动串口 1 作为 modbus/RTU 从站设备 sys_set_com(1, 9600, "none", 8, 1); ModbusRTU_slave_service (0);

10.5. Modbus/RTU 数据读写（主站）

10.5.1. 从 01H 号功能码(coil status/线圈状态) 中读取数据

ModbusRTU_read_coil_status		
从 1 号功能码(coil status/线圈状态) 中读取数据		
参数	**类型**	**说明**
Com	Int	1，2，3，4 分别对应网关的串口 1-串口 4
Id	Int	设备地址 1 <= id<= 254
Addr	Int	寄存器取值范围 0-0xffff
Num	Int	线圈状态寄存器数量，1-8 范围
返回值	Int	根据读取数量返回多个数值，最后返回一个状态值，返回读取返回的字节数，读失败返回 0（V3.9 以上固件支持），如果不需判断该状态，无需赋值即可
函数说明		
示例代码：		
-- 向 com1 的设备 id 为 15 的从机，读 0x07cf 线圈地址开始的 2 个线圈值 reg1,reg2,state=ModbusRTU_read_coil_status(1, 15, 0x07cf, 2);		

10.5.2. 从 02H 号功能码(input status/输入状态) 中读取数据

ModbusRTU_read_input_status		
从 2 号功能码(input status/输入状态) 中读取数据		
参数	**类型**	**说明**
Com	Int	1，2，3，4 分别对应网关的串口 1-串口 4
Id	Int	设备地址 1 <= id<= 254

Addr	Int	输入状态寄存器取值范围 0-0xffff
Num	Int	输入状态寄存器数量，1-8 范围
返回值	Int	根据读取数量返回多个数值，最后返回一个状态值，返回读取返回的字节数，读失败返回 0（V3.9 以上固件支持），如果不需判断该状态，无需赋值即可
函数说明		
示例代码：		
-- 向 com1 的设备 id 为 15 的从机，读 0x07cf 线圈地址开始的 2 个线圈值 reg1,reg2,state=ModbusRTU_read_input_status(1, 15, 0x07cf, 2);		

10.5.3. 从 03H 功能码(holding register/保持寄存器)中读取数据

ModbusRTU_read_hold_register		
根据功能码从 3 号功能码(hold status/状态寄存器) 中读取数据		
参数	**类型**	**说明**
Com	Int	1，2，3，4 分别对应网关的串口 1-串口 4
Id	Int	设备地址 1 <= id<= 254
Addr	Int	寄存器取值范围 0-0xffff
Num	Int	线圈寄存器数量
Type	Str	读取变量的类型：int、float、BCD
返回值	int、float、BCD	根据参数 Type 类型返回对应类型，根据参数 Num，数量返回多个数值，最后返回一个状态值，返回读取返回的字节数，读失败返回 0（V3.9 以上固件支持），如果不需判断该状态，无需赋值即可
函数说明		
示例代码：		
-- 向 com1 的设备 id 为 15 的从机，读 100 地址开始的 8 个保持寄存器值，类型为 float v1,v2,v3,v4,v5,v6,v7,v8,state =ModbusRTU_read_hold_register(1, 15, 100, 8, "float")		

10.5.4. 从 04H 号功能码(input register/输入寄存器) 中读取数据

ModbusRTU_read_input_register		
根据功能码从 4 号功能码(input register/输入寄存器) 中读取数据		
参数	**类型**	**说明**
Com	Int	1，2，3，4 分别对应网关的串口 1-串口 4
Id	Int	设备地址 1 <= id<= 254
Addr	Int	寄存器取值范围 0-0xffff
Num	Int	线圈寄存器数量，1-8 范围
Type	Str	读取变量的类型，int、float、BCD
返回值	int、float、BCD	根据参数 Type 类型返回对应类型，根据参数 Num，数量返回多个数值，最后返回一个状态值，返回读取返回的字节数，读失败返回 0（V3.9 以上固件支持），如果不需判断该状态，无需赋值即可
函数说明		
示例代码：		
-- 向 com1 的设备 id 为 15 的从机，读 100 地址开始的 8 个输入寄存器值，类型为 float v1,v2,v3,v4,v5,v6,v7,v8,state=ModbusRTU_read_input_register(1, 15, 100, 8, "float")		

10.5.5. 从 05H 号功能码(coil status/线圈寄存器) 中写数据

ModbusRTU_force_single_coil		
从 5 号功能码(coil status/线圈寄存器) 中写数据		
参数	**类型**	**说明**
Com	Int	1，2，3，4 分别对应网关的串口 1-串口 4
Id	Int	设备地址 1 <= id<= 254
Addr	Int	寄存器取值范围 0-0xffff
Value	Int	线圈值，0 或 1
返回值	Int	写成功返回 1，失败返回 0
函数说明		
示例代码：		

--向 com1 的设备 id 为 15 的从机，写 0x03e9 地址的线圈值为 1 State=ModbusRTU_force_single_coil(1, 15, 0x03e9, 1);

10.5.6. 从 06H 号功能码中写寄存器

ModbusRTU_write_register		
从 06H 号功能码中写寄存器		
参数	**类型**	**说明**
Com	Int	1，2，3，4 分别对应网关的串口 1-串口 4
Id	Int	设备地址 1 <= id<= 254
Addr	Int	寄存器地址取值范围 0-0xffff
Value	Int	寄存器值取值范围 0-0xffff
Type	Str	写入变量的类型：int
返回值	Int	写成功返回 1，失败返回 0
函数说明		
示例代码：		
-- 向 com1 的设备 id 为 14 的从机，写 0x0063 地址寄存器数值为 0x0001，数据类型为"int" ModbusRTU_write_register(1, 14, 0x0063, 0x0001, "int");		

10.5.7. 从 10H 号功能码中写多个寄存器

ModbusRTU_preset_multiple_registers		
参数	**类型**	**说明**
com	Int	1,2,3,4 分别对应网关的串口 1-串口 4
Slave_addr	Int	设备地址 1 <= id<= 254
start_addr	Int	寄存器开始地址
nb	Int	写入的寄存器数量
Value1	Int	寄存器的数值
Value2	Int	寄存器的数值
…	Int	寄存器的数值
Value_nb	Int	寄存器的数值(根据 nb 长度，填入对应寄存器值)
返回值	Int	写成功寄存器数量，失败返回 0
函数说明	寄存器值类型固定为 16 位整形数，根据写入寄存器长度 nb，对应填入的寄存器值的数量，写其他类型数值请转换后填入	

示例代码：
-- 向 com1 的设备 id 为 1 的从机，写 0x0057 地址寄存器开始 2 个寄存器数值分别为 0x01, 0x00 State=ModbusRTU_preset_multiple_registers(1, 0x01, 0x0057, 2, 0x01, 0x00);

10.5.8. 从 0FH 号功能码中写多个线圈

ModbusRTU_force_multiple_coils		
参数	**类型**	**说明**
com	Int	1，2，3，4 分别对应网关的串口 1-串口 4
Slave_addr	Int	设备地址 1 <= id<= 254
start_addr	Int	线圈开始地址
nb	Int	写入的线圈数量
Value1	Int	线圈的数值
Value2	Int	线圈器的数值
…	Int	线圈器的数值
Value_nb	Int	线圈器的数值(根据 nb 长度，填入对应线圈器值)
返回值	Int	写成功线圈数量，失败返回 0
函数说明	寄存器值类型固定为 8 位整形数，根据写入寄存器长度 nb，对应填入的线圈值的数量，线圈值 0 为断开线圈，非 0 为打开线圈	
示例代码：		
-- 向 com1 的设备 id 为 1 的从机，写 0x0057 地址线圈开始 2 个线圈数值分别为打开和关闭 State= ModbusRTU_force_multiple_coils(1, 0x01, 0x0057, 2, 0x01, 0x00);		

10.5.9. 设置 modbus RTU 设备地址

set_device_addr		
参数	**类型**	**说明**
Com	Int	1，2，3，4 分别对应网关的串口 1-串口 4

dev_type	Int	设备类型 0x01：红外伴侣模块 0x02：PM2.5 传感器 0x03：模拟传感器 0x04：8AI2DI 数据采集器 0x05：4DI4DO 数据采集控制器 0x06：8UI2DI 数据采集器 0x07：智能触摸开关模块 0x08：光照度传感器 0x09：温湿度传感器 0x0A：自来水网监测实训模块 0x0B：激光车位探测传感器
Address	Int	要数值的设备地址 0-0xff
返回值	Void	Void
备注：	该函数只能驱动智嵌系列模块，注意调用该函数的时候 485 口只能接一个同类设备，否则接在总线上的所有同类设备地址都会被设置	
示例代码：		
-- 设置温湿度传感器地址为 0x0e set_device_addr(1, 0x09, 0x0e)		

10.6. LED 显示屏操作

10.6.1. LED 点阵屏初始化函数

LED_config		
参数	**类型**	**说明**
wide	Int	wide=24; //led 显示屏区域宽度 8 个像素为 1 单位
hight	Int	hight=64;//led 显示屏区域高度 1 个像素为 1 单位，16 个像数为 1 行。
line	Int	line=1; //01-单行显示 02-多行显示
autoline	Int	autoline=1;//01-不自动换行 02-自动换行

返回值	Void	Void
备注：		
示例代码：		
--初始化 LED 点阵屏的配置 LED_config(24, 64, 1, 1);		

10.6.2. LED 点阵屏显示 16x16 字符

LED_DS		
参数	类型	说明
Com	Int	1，2，3，4 分别对应网关的串口 1-串口 4
txt	Str	LED 屏显示的字符
x	Int	显示 x 位置（0-24）
y	Int	显示 y 位置（0-64）
DisplayMode	Int	显示方式，其定义如下： 0x01——静止显示 0x02——快速打出 0x03——向左移动 0x04——向右移动 0x05——向上移动 0x06——向下移动
Speed	Int	显示速度，定义如下： 0x00——最快 0x01——0x17 依次减速 0x18 ——最慢
num	Int	动态区域编号 动态区必须统一编号，编号从 0 开始递增。
返回值	Void	Void
备注：	该函数只能驱动智嵌 192x64 和 192x32 电子 led 屏，使用该函数无需做串口初始化操作	
示例代码：		
-- 驱动 led 屏在 0,0 位置显示“物联网综合网关”，以 0x10 的速度向左移动 LED_DS(1, "物联网综合网关",0 ,0, 0x03,0x10,0)		

10.6.3. 桌面式 LED 点阵屏驱动

LED_DS96x16		
参数	**类型**	**说明**
Com	Int	1，2，3，4 分别对应网关的串口 1-串口 4
Txt	Str	LED 屏显示的字符
DisplayMode	Int	显示方式，其定义如下： 0，自动 1，立即 2，跑马灯左 3，跑马灯右 4，百叶窗 5，飘雪 6，渐显 7，三角 8，宽字 9，左进 10， 右进 11， 上进 12， 下进 13， 左擦 14， 右擦 15， 下擦 16， 上擦
Speed	int	显示速度 0，1 级（最快） 1，2 级 2，3 级 3，4 级 4，5 级 5，6 级
返回值	void	Void
备注：	该函数只能驱动智嵌 9616 点阵屏	
示例代码：		

```
-- 驱动 led 屏固定显示“智能控制”
sys_set_com(2, 57600, "none", 8, 1);
LED_DS96x16(2, "智能控制", 1, 1);
```

显示屏显示效果

10.6.4. LED 屏亮度调节函数

LED_Brightness		
参数	**类型**	**说明**
com	int	串口号 1-4
Brightness	int	设置亮度 亮度值为 0-15 共 16 级。亮度值为 15 时亮度最高 t
返回值：无		
备注：		
示例代码：		
sys_set_com(1, 9600, "none", 8, 1) LED_Brightness(10)		

10.6.5. 新 LED 显示屏显示函数

LED_DSV3		
参数	**类型**	**说明**
com	int	串口号 1-4
x	int	X 坐标 0-128
y	int	Y 坐标 0-32
size	int	字体（12/16/24/32）
mode	int	模式（0 静止显示 1 右移显示 2 左移显示 3 上移显示 4 下移）
txt	str	要显示的字符串
返回值：无		
备注：		
示例代码：		
sys_set_com(1, 9600, "none", 8, 1)		

LED_DSV3(1,0,0,24,0,"物联网")

10.6.6. 新 LED 显示屏清屏函数

LED_ClearV3		
参数	类型	说明
com	int	串口号 1-4
返回值：无		
备注：		
示例代码：		
sys_set_com(1, 9600, "none", 8, 1) LED_ClearV3(1);		

10.6.7. 新 LED 屏亮度调节函数

LED_BrightnessV3		
参数	类型	说明
com	int	串口号 1-4
Brightness	int	设置亮度 亮度值为 0-400。亮度值为 400 时亮度最高
返回值：无		
备注：		
示例代码：		
sys_set_com(1, 9600, "none", 8, 1) LED_BrightnessV3(1,400)		

10.7. dlt645-2007 协议智能电表驱动

10.7.1. 读电表状态信息

DLT645_07_read		
参数	类型	说明

Com	Int	1，2，3，4 分别对应网关的串口 1-串口 4
dev_id	Str	电表 ID 号，12 字节，不足应补 0，如 "000000850249"
Code	Str	功能码，要读取信息的功能码： "02010100"--A 相电压值 "02010200"--B 相电压值 "02010300"--C 相电压值 "02020100"--A 相电流 "02020200"--B 相电流 "02020300"--C 相电流 "02060000"--总功率因数 "00010000"--正向有功总量 "00010100"--正向有功尖电量 "00010200"--正向有功峰电量 "00010300"--正向有功平电量 "00010400"--正向有功谷电量 更多功能代码请查阅 dlt645-2007 协议文档
返回值	Int	功能码对应数值
备注：	调用 DLT645_07_read 函数前，应先调用 sys_set_com 函数配置串口，电表一般数值波特率为：sys_set_com(3, 2400, "even", 8, 1);	
示例代码：		

--将编号为"000000850249"的电表连接 com3，读取当前电流值
sys_set_com(3, 2400, "even", 8, 1);
curent = DLT645_07_read(3,"000000850249", "02020100");		

10.8. RFID 读卡器驱动

10.8.1. ID.读卡器驱动函数

read_RFID

参数	类型	说明
Com	Int	1，2，3，4 分别对应网关的串口 1-串口 4
dev_addr	Int	ID 读卡器地址
返回值	Str	读卡的 ID 号，如果读不到卡则返回："Not ID"
备注：		
示例代码：		
--读地址为 1 的 ID 读卡器卡号 number = **read_RFID** (3,1);		

10.8.2. IC 读卡器（带蜂鸣器）服务器对接函数

door_inout_service()		
参数	**类型**	**说明**
URL	Str	服务器服务端 URL
Com	Int	1，2，3，4 分别对应网关的串口 1-串口 4
Relay	Int	继电器组，网关同服务器交互后，如果收到服务器正确返回，将会闭合继电器 500ms 时间，继电器为网关自带继电器，开启一个服务为占用 2 个继电器（一进一出，可以区分进出），取值 1 为继电器组 1,即继电器 1、2，取值 2 为继电器组 2，即继电器 3、4
返回值	Str	无
备注：	该函数执行后，将在后台开启一个服务，定时查询读卡器卡号，如果监测到刷卡，将连卡号封装是 http 请求包推送给服务器，推送的格式如下： POST /DoorControl/passHandler.ashx HTTP/1.1 Content-Type: application/x-www-form-urlencoded Host: 192.168.1.155:80 Content-Length: 57 { Opflag:"GetAccess", ID:"3471879774",	

	status:"in"
	}
	服务器端可以设计软件判断该卡号，如果合法则返回带”true”字符的返回包，继电器自动会闭合，否则读卡器将滴 2 声，如果服务器连接不通读卡器将长鸣 1s。
	服务器返回格式可以参考如下：
	HTTP/1.1 200 OK
	Cache-Control: private
	Content-Type: text/html; charset=utf-8
	Server: Microsoft-IIS/7.5
	X-AspNet-Version: 4.0.30319
	X-Powered-By: ASP.NET
	Date: Thu, 17 Nov 2016 10:02:20 GMT
	Content-Length: 44
	{
	"Msg": false,
	"tip": "not record"
	}

示例代码：	

--开启读卡器连接服务，读卡器连接串口 1
sys_set_com(1, 19200, "none", 8, 1);
door_inout_service("192.168.1.200/DoorControl/passHandler.ashx", 1, 1);	

10.8.3. UHF 服务函数

UHF_service ()		
参数	**类型**	**说明**
URL	Str	服务器服务端 URL
Com	Int	1，2，3，4 分别对应网关的串口 1-串口 4
Addr	Int	服务器返回状态寄存器地址，服务器返回 true，该值为 1，返回非 true 该值为 0，服务器无法连接该值为-1
返回值	Str	无

备注：	该函数执行后，将在后台开启一个服务，定时查询 UHF 卡号，如果监测到刷卡，将连卡号封装是 http 请求包推送给服务器，推送的格式如下：
	POST /DoorControl/passHandler.ashx HTTP/1.1
	Content-Type: application/x-www-form-urlencoded
	Host: 192.168.1.155:80
	Content-Length: 68
	{
	Opflag:"GetAccess",
	ID:E20062D4D5B2F9B156EA0BE6,
	status:"in"
	}
	服务器端可以设计软件判断该卡号，如果合法则返回带”true”字符的返回包，Addr 地址对应寄存器值为 1，否则为 0，如果服务器连接不通该值为-1。
	服务器返回格式可以参考如下：
	HTTP/1.1 200 OK
	Cache-Control: private
	Content-Type: text/html; charset=utf-8
	Server: Microsoft-IIS/7.5
	X-AspNet-Version: 4.0.30319
	X-Powered-By: ASP.NET
	Date: Thu, 17 Nov 2016 10:02:20 GMT
	Content-Length: 44
	{
	"Msg": false,
	"tip": "not record"
	}

示例代码：

```
--开启读卡器连接服务，读卡器连接串口 1，返回状态写到 1001 寄存器
sys_set_com(1, 9600, "none", 8, 1);
UHF_service("192.168.1.155/DoorControl/passHandler.ashx", 1, 1001);
while 1 do
reg1 = Read_sys_register (1001, "int");
if reg1 == 1 then
```

```
    SetDO(2,1);
    delay(1000);
    SetDO(2,0);
    Write_sys_register (1001,0,"int");
end
end
```

10.9. 中移动云平台对接驱动

10.9.1. 添加云变量函数

Add_DataPoint		
参数	**类型**	**说明**
Name	Str	变量名称
addr	Int	变量对应内存寄存器地址，1000-2000
Type	Str	变量类型：“int”、“float”、“str” 每个寄存器地址有 2 个字节，“int”类型为 1 个寄存器空间，“float”类型为 2 个寄存器空间,“str”类型根据字符串的大小决定占用的寄存器空间大小，最大不超过 25 个寄存器空间，即 50 字节。
返回值	Void	
备注：	根据项目需求添加云变量，重复调用则重复添加变量，变量的数值对应，其数值寄存器的地址	
示例代码：		
--添加变量名为"dianliu"的 float 类型云变量，数值对应 1001 寄存器 Add_DataPoint ("dianliu", 1001, "float");		

10.9.2. 读云变量函数

Read_DataPoint		
参数	**类型**	**说明**

Name	Str	变量名称
返回值	自动,int	有 2 个返回值，第 1 个返回值为云变量值，根据 Add_DataPoint 函数设定的变量类型返回，第 2 个返回值为云变量值是否改变状态值，本地脚本操作改变云变量值即返回 1，上层应用操作改变云变量值即返回 2，否则返回 0。
备注：	通过变量名称读取变量数值	
示例代码：		
--读取名为"dianliu"的云变量数值 Value ,state= Read_DataPoint ("dianliu");		

10.9.3. 写云变量函数

Write_DataPoint		
参数	**类型**	**说明**
Name	Str	变量名称
Value	Str	变量数值
返回值	Int	返回：1，赋值成功， 2 变量不存在， 3 变量值格式不合法
备注：	通过变量名称写变量数值	
示例代码：		
--写名为"dianliu"的云变量值为 1.23 stat = Write_DataPoint("dianliu", 1.23);		

10.9.4. 设置云平台对接函数

OneNet_service		
参数	**类型**	**说明**
IP	Str	云服务器 IP 地址
Port	Int	云服务器服务端口
devID	Str	设备 ID 号（在云平台添加设备后会自动生成设备 ID 号）
api-key	Str	设备或项目 api-key 号

time	Int	上传云平台时间周期，单位为秒
返回值	Void	
备注：	使用该函数前应先在云平台添加设备，根据云平台生成的 devID 和 api-key 填入该函数，如何在云平台添加设备请查看智嵌提供的其他文档。	
示例代码：		
--设置网关以网口方式联网，设备号为"283507"，连接到云平台，变量上传服务器周期为 10 秒 OneNet_service ("183.230.40.33", 876, "283507", "zSBY2M5qhptNXP6JT4rsH5Bz7EgA", 10);		

10.9.5. 向云平台获取某个网关云变量值

GET_cloud		
参数	**类型**	**说明**
IP	Str	云服务器 IP 地址，固定为：api.heclouds.com"
Port	Int	云服务器服务端口,固定为：80
devID	Str	设备 ID 号（在云平台添加设备后会自动生成设备 ID 号）
api-key	Str	设备或项目 api-key 号
Num	int	要获取数据流值的数量，最大 10 个，如果超过 10 个可以分次获取
Name	Str	数据流的名字，如果有多个名字需顺序填写
返回值	Str	云变量的值，1-10 个返回值，如果指定的数据流名称不存在，将会返回 none
备注：	该函数实现向云平台获取指定网关指定数据流的数值，函数执行一次，获取一次	
示例代码：		

```
--向云平台获取 ID 为 3013193 的设备的数据流"count",、"shidu"的值
HTTP_service(8000,1);
Add_DataPoint("count",1001,"str")--启动云变量
Add_DataPoint("shidu",1031,"str")
while 1 do
count,shidu=GET_cloud("api.heclouds.com",80,"3013193",
"CLEGObo8=ezNVVXw=TRzEgqkDdU=", 2, "count", "shidu" );
Write_DataPoint("count", count);
Write_DataPoint("shidu", shidu);
delay(2000);
```

```
end
```

10.9.6. 向服务器发起 GET 请求

HTTP_GET		
参数	**类型**	**说明**
Type	Int	0 为网口连接 1 为 gprs 连接
Ip	Str	服务器 ip
Port	Int	服务器服务端口
URL	Str	请求服务器的 URL 地址
返回值	Void	服务器应答数据
备注：	该函数实现向服务器获发起 GET 请求，URL 可以通过编写脚本任意指定	
示例代码：		

```
HTTP_service (8000, 1);
Add_DataPoint("relpy ",1001,"str")--启动云变量
while 1 do
relpy=HTTP_GET(0,"192.168.1.112",8001,
"/aquaculture/collect/get?sn=abcd&ow=3&ph=2&an=1&te=1&tr=1&verify=verify")
Write_DataPoint("count", relpy);
delay(2000);
end
```

10.10. WOL 网卡唤醒函数

10.10.1. WOL 网卡唤醒函数

WOL		
参数	**类型**	**说明**
Mac	Str	要唤醒的计算机 mac 地址
返回值	Void	
备注：	远程唤醒技术(WOL，Wake-on-LAN) 是由网卡配合其他软	

	硬件，通过给处于待机状态的网卡发送特定的数据祯，实现电脑从停机状态启动的一种技术。使用该函数前需确保计算机已经启用 WOL 功能并知道该计算机 mac 地址，当计算机处于关机状态下，调用该函数即能远程开启指定计算机。注意该功能只能使用在网关和计算机在同一路由下。
示例代码：	
--远程唤醒 mac 为“00.0a.0c.0d.0e.0f”的计算机 WOL("00.0a.0c.0d.0e.0f");	

10.11. 网关系统函数

10.11.1. 发布脚本

release		
参数	类型	说明
无		
返回值	Void	
备注：	在调试脚本的时候，往往需要反复下载脚本配置，一定程度上影响了网关的使用寿命，建议在调试程序时，使用内存调试模式，等脚本调试成功后再设置脚本为发布版本，掉电不丢失。 如果脚本配置文件没有 release()函数则脚本会进入内存调试模式，这时可无限次数反复下载，网关掉电后脚本配置将会丢失。添加 release()函数后，每次下载程序会自动保存，掉点重启后程序有效。	
示例代码：		
--发布当前脚本 release();		

10.11.2. 系统延时

delay		
参数	类型	说明

Time	Int	延时时间，单位为 ms
返回值	Void	
备注：	系统时间延时	
示例代码：		
--系统延时 1 秒 delay(1000);		

10.11.3. 读网关寄存器

Read_sys_register		
参数	**类型**	**说明**
Addr	Int	网关内部寄存器地址 1000-2000
Type	Str	寄存器类型：“int”、“float”、“str”每个寄存器地址有 2 个字节，“int”类型为 1 个寄存器空间，“float”类型为 2 个寄存器空间，“str”类型根据字符串的大小决定占用的寄存器空间大小，最大不超过 25 个寄存器空间，即 50 字节。
返回值	Int	寄存器数值
备注：	网关一位寄存器为 2 个字节长度，int 类型为一个寄存器长度（即 2 个字节），float 类型为二个寄存器长度（即 4 个字节）	
示例代码：		
--读取网关地址为 1001 的寄存器数值，int 类型 reg1 = Read_sys_register (1021, "int");		

10.11.4. 写网关寄存器

Write_sys_register		
参数	**类型**	**说明**

Addr	Int	网关内部寄存器地址 1000-2000
Value	int	寄存器数值
Type	Str	寄存器类型：“int”、“float”、“str”每个寄存器地址有 2 个字节，“int”类型为 1 个寄存器空间，“float”类型为 2 个寄存器空间，“str”类型根据字符串的大小决定占用的寄存器空间大小，最大不超过 25 个寄存器空间，即 50 字节。
返回值	void	
备注：	网关一位寄存器为 2 个字节长度，int 类型为一个寄存器长度（即 2 个字节），float 类型为二个寄存器长度（即 4 个字节）	
示例代码：		
--写网关地址为 1001 的寄存器数值为 100，int 类型 Write_sys_register (1001, 100, "int");		

10.11.5. 读网关线圈值

Read_sys_coil		
参数	**类型**	**说明**
Addr	Int	网关内部线圈地址 1000-1100
返回值	Int	寄存器数值 0 或 1
备注：		
示例代码：		
--读取网关地址为 1001 的线圈数值 reg1 = Read_sys_coil (1001);		

10.11.6. 写网关线圈值

Write_sys_coil		
参数	**类型**	**说明**
Addr	Int	网关内部线圈地址 1000-1100

Value	Int	线圈值 0 或 1
返回值	void	
备注：		
示例代码：		
--写网关地址为 1001 的线圈值为 1 Write_sys_coil (1001, 1);		

10.11.7. 启动 web server 服务

HTTP_service		
参数	**类型**	**说明**
Port	Int	Web 服务端口号
Type	int	0 为精简格式，1 为 json 格式，4 为标准 json 格式
返回值	Void	
备注：	启动 web server 服务端口，可以通过 HTTP 方式发送设置或读取云变量数值，设置云变量数值使用 POST 方法，POST 数据内容为：“云变量名称=数值;”，如设置云变量 Open1 数值为 1，则命令为：“ Open1=1;”，设置多个云变量以;号分隔，大小写不限制。 读取云变量数值状态，使用 GET 方法，使用该功能需要使用 Add_DataPoint 函数添加要获取的变量。 设置为精简格式网关响应 GET 请求格式为： HTTP/1.1 200 OK ,;count,0;light,0;air,0;psw,0; 设置为 json 格式网关响应 GET 请求格式为： HTTP/1.1 200 OK {"datastreams":[{"id":"count","datapoints":[{"value":0}]}, {"id":"light","datapoints":[{"value":0}]}, {"id":"air","datapoints":[{"value":0}]}, {"id":"psw","datapoints":[{"value":""}]}]}	
示例代码：		
--启动 8000 端口 web server，以 json 格式显示 HTTP_service(8000,1)		

10.11.8. 设置网关重启

set_reboot_time ()		
参数	**类型**	**说明**
Time	int	重启时间，单位：分钟；重启时间从该函数执行时开始计数
返回值	Void	
备注：		
示例代码：		
--设置网关工作 10 小时后重启一次 set_reboot_time (600); --10*60 分钟		

10.11.9. 禁用网关上传下载功能

disable_debug ()		
参数	**类型**	**说明**
无		
返回值	Void	
备注：	在做工程项目时，当完成脚本配置调试后，为了防止被其他用户误操作导致脚本被篡改等情况，可以用该函数禁用脚本上传和下载功能。禁用网关上传下载功能后，如果要重新开启该功能，必须通过恢复出厂设置，这时会导致原脚本丢失。请慎重使用该功能，使用前要做好脚本备份。	
示例代码：		
--禁用网关上传下载功能 disable_debug ();		

10.11.10. 创建线程函数

pthread_create ()		
参数	**类型**	**说明**

Name	Char*	线程函数名称 注意要加上“”号
返回值	Void	
备注：	请参考第六章网关高级编程说明	
示例代码：		

```
--创建线程 thread1
      function thread1()
       k=0;
       while 1 do
      print("thread1 thread "..k.."\n" );
       delay(500);
       k= k+1;
       end
      end

      function main()
         sys_set_eth0("192.168.1.80", "255.255.255.0", "192.168.1.1");
         pthread_create("thread1");
         k=0;
         while 1 do
            print("main thread "..k.."\n" );
            delay(500);
            k= k+1;
         end
      end
```

10.11.11. 添加变量事件函数

add_cloud_event ()		
参数	**类型**	**说明**
Name	Char*	变量名称 注意要加上“”号
event_name	Char*	响应函数名称 注意要加上“”号
返回值	Void	
备注：	请参考第六章网关高级编程说明	
示例代码：		

```
--创建  onoff 变量响应事件
function onoff_handle( value,cloud_name )
      if    value == 257 then
          SetDO(1, 0);
```

```
      Write_DataPoint( cloud_name, 0);
      elseif   value== 256 then
          SetDO(1, 1);
          Write_DataPoint( cloud_name, 1);
      end
end

function main()
   release();
   sys_set_eth0("192.168.1.80", "255.255.255.0", "192.168.1.1");
   HTTP_service(8000,1);
   Add_DataPoint("onoff", 1001, "int");
   add_cloud_event("onoff", "onoff_handle");
   k=0;
   while 1 do
print("main thread "..k.."\n" );
      delay(500);
      k= k+1;
   end
end
```

10.12. 串口调试函数

10.12.1. 字符数据发送

uart_send_str()		
参数	**类型**	**说明**
Com	int	1，2，3，4 分别对应网关的串口 1-串口 4
Data	Char *	发送数据内容
返回值	Void	
备注：	使用前先设置串口参数	
示例代码：		

--网关串口 1 发送“uart test”字符串
sys_set_com(1, 9600, "none", 8, 1);
uart_send_str(1, "uart test\r\n");		

10.12.2. 16 进制数据发送

uart_send_hex()		
参数	**类型**	**说明**
Com	int	1，2，3，4 分别对应网关的串口 1-串口 4
Data	Char *	发送数据内容，字节之间通过空格隔开
返回值	Void	
备注：	使用前先设置串口参数	
示例代码：		

--网关串口 1 发送“12345”字符串
sys_set_com(1, 9600, "none", 8, 1);
uart_send_hex(1, "31 32 33 34 35");		

10.12.3. 串口数据接收字符型显示

uart_read_str()		
参数	**类型**	**说明**
Com	int	1，2，3，4 分别对应网关的串口 1-串口 4
Time	Int	延时读数据时间，单位为毫秒
返回值	Str	串口数据
备注：	使用前先设置串口参数，确保串口返回数据为字符型数据，否则请使用 uart_read_hex()函数	
示例代码：		

--网关以字符方式读串口 1 数据，延时 1 秒
sys_set_com(1, 9600, "none", 8, 1);
data = uart_read_str(1, 1000);		

10.12.4. 串口数据接收 16 进制显示

uart_read_hex()		
参数	**类型**	**说明**
Com	Int	1，2，3，4 分别对应网关的串口 1-串口 4

Time	Int	延时读数据时间，单位为毫秒
返回值	Str	串口数据，16 进制数以字符串方式显示
备注：	使用前先设置串口参数	
示例代码：		

--网关读串口 1 数据以 16 进制方式返回，延时 1 秒
sys_set_com(1, 9600, "none", 8, 1);
data = uart_read_hex(1, 1000);
--如果串口数据为 1234，则 data="31 32 33 34"。		

10.13. Zigbee 模块驱动

10.13.1. 获取 Zigbee 节点 AD 值

zigbee_get_ad()		
参数	**类型**	**说明**
Com	int	1，2，3，4 分别对应网关的串口 1-串口 4
Addr	Int	Zigbee 节点地址
Channel	Int	AD 通道号，ad0 和 ad1 对应 1，2
Time	Int	延时读数据时间，单位为毫秒，根据实际网络延时设定
返回值	Int	Ad 值，0-0xffff
备注：	使用前先设置串口参数，并配置好 Zigbee 模块参数	
示例代码：		
--读取地址为 0x2001 的 Zigbee 模块通道 0 的 AD 值 sys_set_com(2, 115200, "none", 8, 1); ad = zigbee_get_ad(2, 0x2001,1, 100); --100ms 延时		

10.13.2. 获取 Zigbee 节点 DI 值

zigbee_get_di()		
参数	**类型**	**说明**

Com	Int	1，2，3，4 分别对应网关的串口 1-串口 4
Addr	Int	Zigbee 节点地址
Time	Int	延时读数据时间，单位为毫秒，根据实际网络延时设定
返回值	Int	Di 状态值，0x00-0x07,该字节低 3 位分别表示继电器、DI1，DI0 三个开关状态
备注：	使用前先设置串口参数，并配置好 Zigbee 模块参数	
示例代码：		
--读取地址为 0x2001 的 Zigbee 模块 DI 值 sys_set_com(2, 115200, "none", 8, 1); DO0,DI1,DI0 = zigbee_get_di(2, 0x2001, 100); --100ms 延时		

10.13.3. 设置 Zigbee 节点继电器输出

zigbee_set_do()		
参数	**类型**	**说明**
Com	Int	1，2，3，4 分别对应网关的串口 1-串口 4
Addr	Int	Zigbee 节点地址
Stat	Int	继电器状态，1 为打开，0 为关闭
Time	Int	延时读数据时间，单位为毫秒，根据实际网络延时设定
返回值	Int	设置成功返回 1,否则返回 0
备注：	使用前先设置串口参数，并配置好 Zigbee 模块参数	
示例代码：		
--设置地址为 0x2001 的 Zigbee 模块继电器闭合 sys_set_com(2, 115200, "none", 8, 1); stat = zigbee_set_do(2, 0x2001, 0, 100); --100ms 延时		

10.14. GSM 短信驱动

GSM_send_msg()

参数	类型	说明
Number		报警电话号码
Msg	Str	短信报警发送内容
返回值	Int	设置成功返回 1,否则返回 0
备注：	使用前先保证网关正确接入资费正常的 SIM 卡，并将 GSM 天线接入完好。 注意：调用该函数编写脚本时请注意脚本逻辑，避免重复调用该函数而导致批量发送短信。	
示例代码：		
--向电话 15914359625 发送“短信报警测试 123”短信 GSM_send_msg("15914359625", "短信报警测试 123");		

10.15. 超声波车位探测驱动

10.15.1. 超声波车位探测模块

get_parking_lot()		
参数	类型	说明
Com	int	1，2，3，4 分别对应网关的串口 1-串口 4
Addr	Int	1）超声波车位探测模块 地址
返回值	Int	有车：01 无车：02 通信失败：00
备注：	使用前先设置串口参数	
示例代码：		

--读取接在网关串口 2 上地址为 1 的车位传感器的开关状态
sys_set_com(2, 9600, "none", 8, 1);
di = get_parking_lot(2, 1);		

10.16. 三菱 PLC 通信控制

10.16.1. FX3U 写驱动(PLC 设备 ID 固定位为 1)

<table>
<tr><td colspan="4">fx3u_write();</td></tr>
<tr><td>参数</td><td>类型</td><td colspan="2">说明</td></tr>
<tr><td>Com</td><td>int</td><td colspan="2">1，2，3，4 分别对应网关的串口 1-串口 4</td></tr>
<tr><td>Code</td><td>Str</td><td colspan="2">
<table>
<tr><td>数据区名称</td><td>数据区类型</td><td>读写类型</td><td>推荐使用数据格式</td></tr>
<tr><td>开关量输入 (X)</td><td>位继电器</td><td>只读</td><td>位操作</td></tr>
<tr><td>开关量输出 (Y)</td><td>位继电器</td><td>读写</td><td>位操作</td></tr>
<tr><td>辅助继电器 (M)</td><td>位继电器</td><td>读写</td><td>位操作</td></tr>
<tr><td>状态继电器 (S)</td><td>位继电器</td><td>读写</td><td>位操作</td></tr>
<tr><td>特殊辅助继电器 (M)</td><td>位继电器</td><td>只读</td><td>位操作</td></tr>
<tr><td>定时器接点 (TS)</td><td>位继电器</td><td>只读</td><td>位操作</td></tr>
<tr><td>计数器接点 (CS)</td><td>位继电器</td><td>只读</td><td>位操作</td></tr>
<tr><td>数据寄存器 (D)</td><td>16 位寄存器</td><td>读写</td><td>16 位有符号数、位操作</td></tr>
<tr><td>文件寄存器（D）</td><td>16 位寄存器</td><td>读写</td><td>16 位有符号数、位操作</td></tr>
<tr><td>特殊数据寄存器（D）</td><td>16 位寄存器</td><td>只读</td><td>16 位有符号数、位操作</td></tr>
<tr><td>定时器当前值 (TN)</td><td>16 位寄存器</td><td>只读</td><td>16 位有符号数</td></tr>
<tr><td>计数器当前值 (CN)</td><td>16 位寄存器</td><td>只读</td><td>16 位有符号数</td></tr>
</table>
目前只支持：“M”
</td></tr>
<tr><td>RegAddr</td><td>Int</td><td colspan="2">寄存器地址</td></tr>
<tr><td>Value</td><td>Int</td><td colspan="2">寄存器数值</td></tr>
<tr><td>返回值</td><td>Int</td><td colspan="2">发送失败返回：-1；
成功：返回反馈字符数，一般为 5</td></tr>
<tr><td>备注：</td><td colspan="3">使用前先设置串口参数，PLC 设备地址固定为 0x01</td></tr>
<tr><td colspan="4">示例代码：</td></tr>
<tr><td colspan="4">--数值 PLC 地址为 400 的辅助继电器，值为 1
sys_set_com(1, 9600, "none",8, 1);
fx3u_write(1, "M", 400, 1);</td></tr>
</table>

10.16.2. FX3U 写驱动

<table>
<tr><td colspan="4">fx3u_writeV2( );</td></tr>
<tr><td>参数</td><td>类型</td><td colspan="2">说明</td></tr>
<tr><td>Com</td><td>int</td><td colspan="2">1，2，3，4 分别对应网关的串口 1-串口 4</td></tr>
<tr><td>ID</td><td>Int</td><td colspan="2">PLC 设备 id，一般为 1</td></tr>
<tr><td>Code</td><td>Str</td><td colspan="2">
<table>
<tr><td>数据区名称</td><td>数据区类型</td><td>读写类型</td><td>推荐使用数据格式</td></tr>
<tr><td>开关量输入 (X)</td><td>位继电器</td><td>只读</td><td>位操作</td></tr>
<tr><td>开关量输出 (Y)</td><td>位继电器</td><td>读写</td><td>位操作</td></tr>
<tr><td>辅助继电器 (M)</td><td>位继电器</td><td>读写</td><td>位操作</td></tr>
<tr><td>状态继电器 (S)</td><td>位继电器</td><td>读写</td><td>位操作</td></tr>
<tr><td>特殊辅助继电器 (M)</td><td>位继电器</td><td>只读</td><td>位操作</td></tr>
<tr><td>定时器接点 (TS)</td><td>位继电器</td><td>只读</td><td>位操作</td></tr>
<tr><td>计数器接点 (CS)</td><td>位继电器</td><td>只读</td><td>位操作</td></tr>
<tr><td>数据寄存器 (D)</td><td>16 位寄存器</td><td>读写</td><td>16 位有符号数、位操作</td></tr>
<tr><td>文件寄存器（D）</td><td>16 位寄存器</td><td>读写</td><td>16 位有符号数、位操作</td></tr>
<tr><td>特殊数据寄存器（D）</td><td>16 位寄存器</td><td>只读</td><td>16 位有符号数、位操作</td></tr>
<tr><td>定时器当前值 (TN)</td><td>16 位寄存器</td><td>只读</td><td>16 位有符号数</td></tr>
<tr><td>计数器当前值 (CN)</td><td>16 位寄存器</td><td>只读</td><td>16 位有符号数</td></tr>
</table>
目前只支持：“M”、“D”
</td></tr>
<tr><td>RegAddr</td><td>Int</td><td colspan="2">寄存器地址</td></tr>
<tr><td>Value</td><td>Int</td><td colspan="2">寄存器数值</td></tr>
<tr><td>返回值</td><td>Int</td><td colspan="2">发送失败返回：-1；
成功：返回反馈字符数，一般为 5</td></tr>
<tr><td>备注：</td><td colspan="3">使用前先设置串口参数 ，PLC 设备地址固定为 0x01</td></tr>
<tr><td colspan="4">示例代码：</td></tr>
<tr><td colspan="4">--数值 PLC 地址为 400 的辅助继电器值为 1。
sys_set_com(1, 9600, "none", 8, 1);
fx3u_write(1, 1,"M", 400, 1);</td></tr>
</table>

10.16.3. FX3U 读驱动

<table>
<tr><td colspan="4">fx3u_readV2( );</td></tr>
<tr><td>参数</td><td>类型</td><td colspan="2">说明</td></tr>
<tr><td>Com</td><td>int</td><td colspan="2">1，2，3，4 分别对应网关的串口 1-串口 4</td></tr>
<tr><td>ID</td><td>Int</td><td colspan="2">PLC 设备 id，一般为 1</td></tr>
<tr><td>Code</td><td>Str</td><td colspan="2">
<table>
<tr><td>数据区名称</td><td>数据区类型</td><td>读写类型</td><td>推荐使用数据格式</td></tr>
<tr><td>开关量输入 (X)</td><td>位继电器</td><td>只读</td><td>位操作</td></tr>
<tr><td>开关量输出 (Y)</td><td>位继电器</td><td>读写</td><td>位操作</td></tr>
<tr><td>辅助继电器 (M)</td><td>位继电器</td><td>读写</td><td>位操作</td></tr>
<tr><td>状态继电器 (S)</td><td>位继电器</td><td>读写</td><td>位操作</td></tr>
<tr><td>特殊辅助继电器 (M)</td><td>位继电器</td><td>只读</td><td>位操作</td></tr>
<tr><td>定时器接点 (TS)</td><td>位继电器</td><td>只读</td><td>位操作</td></tr>
<tr><td>计数器接点 (CS)</td><td>位继电器</td><td>只读</td><td>位操作</td></tr>
<tr><td>数据寄存器 (D)</td><td>16 位寄存器</td><td>读写</td><td>16 位有符号数、位操作</td></tr>
<tr><td>文件寄存器（D）</td><td>16 位寄存器</td><td>读写</td><td>16 位有符号数、位操作</td></tr>
<tr><td>特殊数据寄存器（D）</td><td>16 位寄存器</td><td>只读</td><td>16 位有符号数、位操作</td></tr>
<tr><td>定时器当前值 (TN)</td><td>16 位寄存器</td><td>只读</td><td>16 位有符号数</td></tr>
<tr><td>计数器当前值 (CN)</td><td>16 位寄存器</td><td>只读</td><td>16 位有符号数</td></tr>
</table>
目前只支持：“M”、“X”、“Y”、“D”
</td></tr>
<tr><td>RegAddr</td><td>Int</td><td colspan="2">寄存器地址</td></tr>
<tr><td>Times</td><td>Int</td><td colspan="2">等待消息时间数值范围 0-150，以 10ms 为增量</td></tr>
<tr><td>返回值</td><td>Int</td><td colspan="2">发送失败返回：-1；
成功：返回读到的数值</td></tr>
<tr><td>备注：</td><td colspan="3">使用前先设置串口参数 ，PLC 设备地址固定为 0x01</td></tr>
<tr><td colspan="4">示例代码：</td></tr>
<tr><td colspan="4">--数值 PLC 地址为 400 的辅助继电器，值为 1
sys_set_com(1, 9600, "none", 8, 1);
state = fx3u_readV2(1, 1, "M",1,1);</td></tr>
</table>

10.16.4. FX2N 写驱动

<table>
<tr><td colspan="3">fx2n_write()</td></tr>
<tr><td>参数</td><td>类型</td><td>说明</td></tr>
<tr><td>Com</td><td>int</td><td>1，2，3，4 分别对应网关的串口 1-串口 4</td></tr>
<tr><td>Code</td><td>Str</td><td>
<table>
<tr><td>数据区名称</td><td>数据区类型</td><td>读写类型</td><td>推荐使用数据格式</td></tr>
<tr><td>开关量输入 (X)</td><td>位继电器</td><td>只读</td><td>位操作</td></tr>
<tr><td>开关量输出 (Y)</td><td>位继电器</td><td>读写</td><td>位操作</td></tr>
<tr><td>辅助继电器 (M)</td><td>位继电器</td><td>读写</td><td>位操作</td></tr>
<tr><td>状态继电器 (S)</td><td>位继电器</td><td>读写</td><td>位操作</td></tr>
<tr><td>特殊辅助继电器 (M)</td><td>位继电器</td><td>只读</td><td>位操作</td></tr>
<tr><td>定时器接点 (TS)</td><td>位继电器</td><td>只读</td><td>位操作</td></tr>
<tr><td>计数器接点 (CS)</td><td>位继电器</td><td>只读</td><td>位操作</td></tr>
<tr><td>数据寄存器 (D)</td><td>16 位寄存器</td><td>读写</td><td>16 位有符号数、位操作</td></tr>
<tr><td>文件寄存器（D）</td><td>16 位寄存器</td><td>读写</td><td>16 位有符号数、位操作</td></tr>
<tr><td>特殊数据寄存器（D）</td><td>16 位寄存器</td><td>只读</td><td>16 位有符号数、位操作</td></tr>
<tr><td>定时器当前值 (TN)</td><td>16 位寄存器</td><td>只读</td><td>16 位有符号数</td></tr>
<tr><td>计数器当前值 (CN)</td><td>16 位寄存器</td><td>只读</td><td>16 位有符号数</td></tr>
</table>
目前只支持：“M”、“D”</td></tr>
<tr><td>RegAddr</td><td>Int</td><td>寄存器地址</td></tr>
<tr><td>Value</td><td>Int</td><td>寄存器数值</td></tr>
<tr><td>返回值</td><td>Int</td><td>发送失败返回：-1；
成功：返回反馈字符数，一般为 5</td></tr>
<tr><td>备注：</td><td colspan="2"></td></tr>
<tr><td colspan="3">示例代码：</td></tr>
<tr><td colspan="3">--数值 PLC 地址为 400 的辅助继电器，值为 1。
sys_set_com(1, 9600, "none", 7, 1);
fx2n_write(1, "M", 400, 1);</td></tr>
</table>

10.16.5. FX2N 读驱动

<table>
<tr><td colspan="3">fx2n_read()</td></tr>
<tr><td>参数</td><td>类型</td><td>说明</td></tr>
<tr><td>Com</td><td>int</td><td>1，2，3，4 分别对应网关的串口 1-串口 4</td></tr>
<tr><td>Code</td><td>Str</td><td>
<table>
<tr><td>数据区名称</td><td>数据区类型</td><td>读写类型</td><td>推荐使用数据格式</td></tr>
<tr><td>开关量输入 (X)</td><td>位继电器</td><td>只读</td><td>位操作</td></tr>
<tr><td>开关量输出 (Y)</td><td>位继电器</td><td>读写</td><td>位操作</td></tr>
<tr><td>辅助继电器 (M)</td><td>位继电器</td><td>读写</td><td>位操作</td></tr>
<tr><td>状态继电器 (S)</td><td>位继电器</td><td>读写</td><td>位操作</td></tr>
<tr><td>特殊辅助继电器 (M)</td><td>位继电器</td><td>只读</td><td>位操作</td></tr>
<tr><td>定时器接点 (TS)</td><td>位继电器</td><td>只读</td><td>位操作</td></tr>
<tr><td>计数器接点 (CS)</td><td>位继电器</td><td>只读</td><td>位操作</td></tr>
<tr><td>数据寄存器 (D)</td><td>16 位寄存器</td><td>读写</td><td>16 位有符号数、位操作</td></tr>
<tr><td>文件寄存器（D）</td><td>16 位寄存器</td><td>读写</td><td>16 位有符号数、位操作</td></tr>
<tr><td>特殊数据寄存器（D）</td><td>16 位寄存器</td><td>只读</td><td>16 位有符号数、位操作</td></tr>
<tr><td>定时器当前值 (TN)</td><td>16 位寄存器</td><td>只读</td><td>16 位有符号数</td></tr>
<tr><td>计数器当前值 (CN)</td><td>16 位寄存器</td><td>只读</td><td>16 位有符号数</td></tr>
</table>
目前只支持：“M”、“D”</td></tr>
<tr><td>RegAddr</td><td>Int</td><td>寄存器地址</td></tr>
<tr><td>返回值</td><td>Int</td><td>发送失败返回：-1；
成功：返回寄存器值</td></tr>
<tr><td>备注：</td><td colspan="2"></td></tr>
<tr><td colspan="3">示例代码：</td></tr>
<tr><td colspan="3">--数值 PLC 地址为 400 的辅助继电器，值为 1
sys_set_com(1, 9600, "none", 7, 1);
stat = fx2n_read(1, "M", 400);</td></tr>
</table>

10.17. dlt645-97 协议水表驱动

10.17.1. 读水表数信息

DLT645_97_read		
参数	类型	说明
Com	int	1，2，3，4 分别对应网关的串口 1-串口 4
dev_id	Str	水表 ID 号，12 字节，不足应补 0，如"000000850249" 表号在水表上可读出
Code	Str	功能码 4 字节，要读取信息的功能码： "9010"—读水表用水量值 更多功能代码请查阅 dlt645-97 协议文档
返回值	Int	功能码对应数值
备注：	如果通过 M-bus 采集器采集水表读数，应先调用 M_bus_collector 打开端口，然后在 dlt645_97_read 函数前，应先调用 sys_set_com 函数配置串口，水表一般数值波特率为：sys_set_com(3, 2400, "none", 8, 2);	
示例代码：		

--将编号为"201612002319"的 M-bus 水表连接采集器端口 1，采集器 485 连接网关 com2，读取当前水表值
sys_set_com(2,2400,"none",8,2);
M_bus_collector(2,"15130410", 1, 1);
delay(500);
value= DLT645_97_read (2, "201612002319", "9010");		

10.17.2. M-bus 水表采集器操作

M_bus_collector		
参数	类型	说明
Com	int	1，2，3，4 分别对应网关的串口 1-串口 4
dev_id	Str	采集器 ID 号,8 字节,不足应补 0,如"15130410" ID 号在采集器上可读出
Port	Int	采集器端口号，1 或 2
Cmd	Int	采集器端口开关操作，1 为开，0 为关
返回值	Int	功能码对应数值
备注：	如果通过 M-bus 采集器采集水表读数，应先调用	

	M_bus_collector 打开端口，方可以进行读表操作，打开后如果 40 秒没有操作，端口会自动断开
示例代码：	

--将编号为"15130410"的采集器端口连接网关 com2，打开端口 1
sys_set_com(2,2400,"even",8,1);
M_bus_collector(2,"15130410", 1, 1);
delay(500);	

10.18. 小米智能家居产品对接

10.18.1. 开启小米智能家居网关服务

aqara_service_start();		
参数	类型	说明
无		
返回值	Int	创建失败返回：-1 创建成功返回：1
备注：	该函数用于开启组播服务，接收智能家居网关定时上报的状态，IP 信息，注意该函数必须调用，后面的读写函数才能正常工作	
示例代码：		
--开启开启小米智能家居网关服务 aqara_service_start();		

10.18.2. 读智能家居产品状态

aqara_read ( )		
参数	类型	说明
gateway_sid	Str	网关 sid
device_sid	Str	目标设备 sid
Num	Int	读取数值的数量

Val_name	Str	读取的数值名称，如果有多个则顺序填入 注意：该名称根据读取的设备类型，填入对应的名称： 温湿度传感器： 电池电压"voltage"，温度"temperature"，湿度"humidity"； 门磁传感器：电池电压"voltage",开关状态 "status"； Zigbee 插座： 状态"status",电器使用 "inuse"，电力消耗"power_consumed",负载功率 "load_power"；
返回值	Int	根据读取数值的数量，决定返回的数值数量 发送失败返回：-1； 成功：返回读到的数值
备注：		
示例代码：		

--读取门磁状态
voltage,status = aqara_read("7811dcdf0599", "158d00020e18cd", 2, "voltage", "status");
--读取插座状态
status,inuse,power_consumed,load_power = aqara_read("7811dcdf0599", "158d0001f3f780", 4, "status", "inuse", "power_consumed", "load_power");		

10.18.3. 写智能家居产品状态

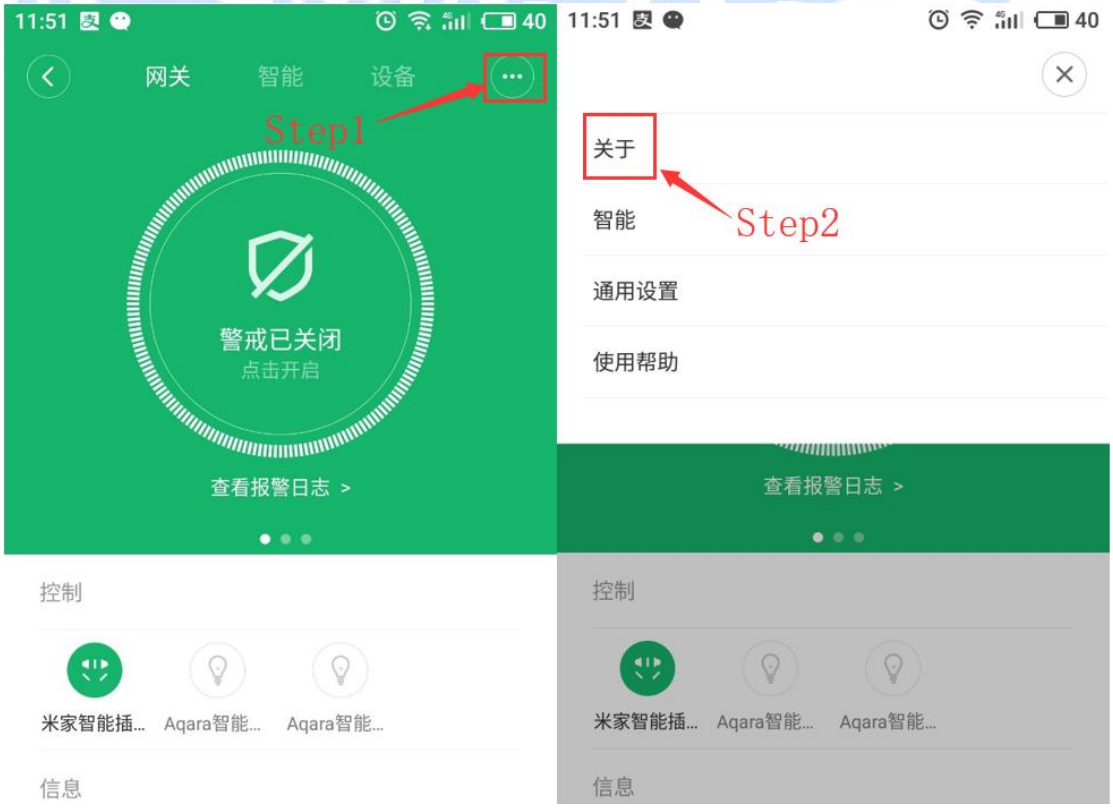
aqara_write ( )		
参数	**类型**	**说明**
gateway_sid	Str	网关 sid
gateway_key	Str	网关密码
device_sid	Str	目标设备 sid
name	Str	目标设备名称,该名称根据读取的设备类型，填入对应的名称： 插座开关: "plug" 86 开关双位开关左键：”ctrl_ln2-0” 86 开关双位开关右键：”ctrl_ln2_1” 86 开关单位开关键：”ctrl_ln1” 魔方：”cube” 窗帘:”curtain” 无线开关按钮:”switch”
write_status	Str	写状态,该名称根据读取的设备类型，填入对应的名称： 插座开关: "on"/"off" 86 开关双位开关左键："on"/"off" 86 开关双位开关右键："on"/"off" 86 开关单位开关右键："on"/"off" 魔方："flip90"/"flip180"/"move"/"tap_twice"/"shake_air" (翻

		转 90 度/翻转 180 度/平移/双击/摇一摇) 窗帘："0"-"100" 无线开关按钮："click"/"double_click"
返回值	Int	根据读取数值的数量，决定返回的数值数量 发送失败返回：-1； 成功：返回 1
备注：		
示例代码：		

--双位 86 开关关闭
aqara_write("7811dcdf0599", "js2gc883ntrqj4pe","158d0002402b8d", "ctrl_ln2-1","off");
--插座关闭
aqara_write("7811dcdf0599", "js2gc883ntrqj4pe","158d0001f3f780", "plug", "off");
--模拟魔方翻转 90° 事件
aqara_write("7811dcdf0599", "js2gc883ntrqj4pe","158d000269e2c4", "cube", "flip90");		

附：网关密码和各个设备 sid 号获取办法

1、在米家智能家庭 app 中开启局域网协议，操作步骤如下。记下 step5 上设置的密码，该密码就是网关密码。



进入关于页面后，狂点插件版本号，开启局域网协议，并出现局域网通信协议的菜单项。

2、在关于页面下，点击“网关信息”，将可以看到网关信息和各个连接到网关的设备信息。

10.19. Tlink 云平台对接

Modbus_client_service()		
参数	**类型**	**说明**
IP	Str	云服务器 IP 地址
Port	Int	云服务器服务端口
REG	Str	连接注册信息
返回值：无		
备注：	改函数开启后，网关向指定服务器发起连接，建立连接后，发送握手信号，后面网关工作在 modbus TCP 工作模式，云端可以读写网关所有寄存器。	
示例代码：		
Modbus_client_service ("119.23.117.28", 8655, "K3S9M276IWP4O34K");		

10.20. 阿里云平台对接

ALiYun_service()		
参数	**类型**	**说明**
DeviceName	Str	在阿里云平台添加设备生成的：DeviceName
DeviceSecret	Str	在阿里云平台添加设备生成的：DeviceSecret
ProductKey	Str	在阿里云平台添加设备生成的：ProductKey
cycle_time	int	上传数据的周期，单位是秒
返回值：无		
备注：		
示例代码：		
ALiYun_service (“Device001”,”5e252791fd2db6dc8081cb7ff6bb420b”,”a1b05MRdHma”,5)		

10.21. 私有云（MQTT）平台对接

10.21.1. 开启 MQTT 服务函数

MQTT_service		
MQTT 主动推送		
参数	**类型**	**说明**
Ip	Str	MQTT 服务器 IP
Port	Int	**端口**
user	Str	MQTT 服务器用户名
pwd	Str	MQTT 服务器密码
sub	Str	预留，默认填“ “
cycle_time	Int	MQTT 上传数据量周期，单位：秒
返回值	Int	无
函数说明	网关只能连接一个 mqtt 服务器，建立一个 mqtt 连接，调用 MQTT_service 函数后，会创建服务连接到 mqtt 服务器，并订阅主题，同时会按 cycle_time 的时间定时推送所有数据流。 订阅主题固定为：sys/网关 mac 地址/密码/sub	

	定时推送的主题固定为：sys/网关 mac 地址/密码/pub 其中网关 mac 地址，可以通过搜索工具搜索到，密码可以通过 SetPasswd 函数设置
示例代码：	
set_web_server(2);--设置上传数据格式 SetPasswd("123456");--设置密码 --连接上 192.168.1.252:1884 服务器，用户名：Test 密码：123，上报周期 5 秒 MQTT_service("192.168.1.252", 1884, "Test", "123", "", 5) --运行以上函数后，如果服务器和网络正常，其他 mqtt 客户端将 可以通过“sys/网关 mac 地址/主题密码/sub”主题向网关推送数据； 通过订阅“sys/网关 mac 地址/主题密码/pub”主题接收网关的变量信息。	

10.21.2. 设置主题密码函数

SetPasswd()		
参数	**类型**	**说明**
PD	Str	主题密码
返回值：无		
备注：		
示例代码：		
SetPasswd(“123456”)		

10.22. 迅通云（TCP）平台对接

XTCloud_service		
TCP 主动推送至迅通云。迅通云链接：https://cloud.rpdiot.com/		
参数	**类型**	**说明**
Ip	Str	TCP 服务器 IP,迅通云的 IP 为：116.198.225.132
Port	Int	端口，迅通云的 TCP 端口为：8888
Device_ID	Str	迅通云的设备编号
cycle_time	Int	TCP 上传数据量周期，单位：秒
返回值	Int	无
函数说明	推送的数据以 Add_DataPoint 添加的云变量为例，当读取到数据，	

	将数据存储到云变量。随后网关统一推送数据给迅通云。

示例代码：

```
function mode1()
Add_DataPoint("wendu",1020,"int"); --温度（只读）
Add_DataPoint("shidu",1021,"int"); --湿度（只读）
while 1 do
sd,wd,ret = ModbusRTU_read_hold_register(1, 1, 0, 2, "int"); -- --读取温湿度的寄存器的数据写到网关变量
if ret==1 then
Write_DataPoint("wendu",wd/10);
Write_DataPoint("shidu", sd/10);
print(string.format("温度:%d, 湿度:%d", wd, sd))
delay(100);
end
delay(500);
end
end
release(); --配置为发布版本
sys_set_com(1, 9600, "none", 8, 1); --485 串口
HTTP_service(8000, 2);--设置上传数据格式
XTCloud_service("116.198.225.132", 8888, "设备编号", 3)--TCP 接入迅通云
mode1()
```

10.23. Home Assistant（MQTT）平台对接

10.23.1. 开启 MQTT 服务函数

HA_service		
开启 Home Assistant 的 MQTT 服务		
参数	**类型**	**说明**
Ip	Str	HA 的 MQTT 服务器 IP
Port	Int	端口，HA 的 MQTT 端口为：8888
user	Str	MQTT 用户名
Password	Str	MQTT 密码
Device_Name	Str	想要设定的项目名称
返回值	Int	无
函数说明	网关连接 Home Assistant 的 MQTT 服务并创建 MQTT 项目。	

示例代码：
HA_service("192.168.1.114",1883,"mqtt123456", "mqtt123456","智能家居系统")

10.23.2. 添加 HA 云变量

10.23.2.1.传感器类型

注意：HA_Add_DataPoint 要放置到 Add_DataPoint 之前
HA_Add_DataPoint 实体标识名要与 Add_DataPoint 变量名称一致

HA_Add_DataPoint（传感器）		
添加 HA 云变量（传感器）		
参数	类型	说明
ID	Str	实体标识名，要与 Add_DataPoint 一致
Name	Str	显示到 HA 的名称
Type	Str	设备类型传感器固定为 sensor
Unit	Str	单位
返回值	Int	无
函数说明	创建 HA 传感器变量。	
示例代码：		
HA_Add_DataPoint("wendu", "温度", "sensor", "° C") HA_Add_DataPoint("shidu", "湿度", "sensor", "%RH") Add_DataPoint("wendu", 1001, "float"); Add_DataPoint("shidu", 1003, "float");		

10.23.2.2.DO 开关量输出类型

注意：HA_Add_DataPoint 要放置到 Add_DataPoint 之前
HA_Add_DataPoint 实体标识名要与 Add_DataPoint 变量名称一致

HA_Add_DataPoint（DO 输出）		
添加 HA 云变量（DO 输出）		
参数	类型	说明
ID	Str	实体标识名，要与 Add_DataPoint 变量名称一致
Name	Str	显示到 HA 的名称
Type	Str	设备类型传感器固定为 switch

返回值	Int	无
函数说明	创建 HA 传感器变量。 -- 读取风扇的数据点状态 s, r = Read_DataPoint("fan") -- 如果读取状态 r 为 2,表示有新的有效数据,将风扇状态值 s 打印输出，并将该状态写回数据点，用于后续上报给 HA。 if r == 2 then print(string.format("fan=%d\n", s)) Write_DataPoint("fan", s) end	
示例代码：		
HA_Add_DataPoint("fan", "风扇", "switch") Add_DataPoint("fan", 1006, "int");		

10.23.2.3.DI 开关量输入类型

注意：HA_Add_DataPoint 要放置到 Add_DataPoint 之前
HA_Add_DataPoint 实体标识名要与 Add_DataPoint 变量名称一致

HA_Add_DataPoint（DI 输入）		
添加 HA 云变量（DI 输入）		
参数	类型	说明
ID	Str	实体标识名，要与 Add_DataPoint 变量名称一致
Name	Str	显示到 HA 的名称
Type	Str	设备类型传感器固定为 binary_sensor
返回值	Int	无
函数说明	创建 HA 传感器变量。	
示例代码：		
HA_Add_DataPoint("renti", "人体", "binary_sensor") Add_DataPoint("renti", 1005, "int");		

10.23.2.4.窗帘类型

注意：HA_Add_DataPoint 要放置到 Add_DataPoint 之前
HA_Add_DataPoint 实体标识名要与 Add_DataPoint 变量名称一致

HA_Add_DataPoint（窗帘）

添加 HA 云变量（窗帘）		
参数	**类型**	**说明**
ID	Str	**实体标识名，要与 Add_DataPoint 变量名称一致**
Name	Str	显示到 HA 的名称
Type	Str	设备类型传感器固定为 cover
返回值	Int	无
函数说明	如果使用的是 Read_DataPoint 读取云变量，一共返回 2 个数值，第一个数值返回的是控制下发的数值，第一个返回状态值返回状态值 2 说明 HA 下发控制的是 0-100 数值，返回状态值 3 为停止电机移动。	

```
    -- 读取窗帘的数据点状态
    s, r = Read_DataPoint("curn")
    -- 当读取状态 r 为 2 时，表示窗帘有移动操作（移动范围 0 - 100），打印输出状态信息，并将该状态写回数据点
    if r == 2 then
        print(string.format("s=%d r=%d\n", s, r))
        Write_DataPoint("curn", s)
        -- 如果读取状态 r 为 3，表示窗帘停止移动操作，同样打印输出信息并写回数据点
    elseif r == 3 then
            print(string.format("s=%d r=%d\n", s, r))
            Write_DataPoint("curn", s)

    end
```

示例代码：

```
HA_Add_DataPoint("curn", "窗帘", "cover")
Add_DataPoint("curn", 1007, "int");
```

10.23.3. HA 更新上报数据

注意：放置到死循环内更新循环上报

HA_Update_Data()		
上报云变量数据		
参数	**类型**	**说明**
无	无	**无**
函数说明	创建 HA 传感器变量。	
示例代码：		

```
while 1 do
 Write_DataPoint("wendu", wendu);
 Write_DataPoint("shidu", shidu);
 shidu=shidu+0.1
 wendu=wendu+0.1
 Write_DataPoint("renti", s1);
 if s1==0 then s1=1
 else s1=0 end
 s,r= Read_DataPoint("fan")
 if r==2 then
    print(string.format("fan=%d\n", s))
    Write_DataPoint("fan", s);
 end

 s,r= Read_DataPoint("curn")
 if r ==2 then
 print(string.format("s=%d r=%d\n", s, r))
 Write_DataPoint("curn", s);
 end
 HA_Update_Data()
    delay(1000);
end
```

10.23.4. Home Assistant 代码例程

```
-- 配置 Home Assistant 服务，连接到指定的 IP 地址、端口，使用用户名和密码进行认证，并设置设备名称为 "智能家居系统"
HA_service("192.168.1.114", 1883, "mqtt123456", "mqtt123456", "智能家居系统")
-- 配置 HTTP 服务，设置端口号为 8000，参数 2 为JSON 类型显示
HTTP_service(8000, 2)
-- 向 Home Assistant 添加各种类型的数据点，用于表示智能家居系统中的不同传感器和控制设备
-- "wendu" 表示温度数据点，类型为传感器，单位为 °C
HA_Add_DataPoint("wendu", "温度", "sensor", "°C")
-- "shidu" 表示湿度数据点，类型为传感器，单位为 %RH
HA_Add_DataPoint("shidu", "湿度", "sensor", "%RH")
-- "renti" 表示人体传感器数据点，类型为二进制传感器
HA_Add_DataPoint("renti", "人体", "binary_sensor")
-- "fan" 表示风扇控制数据点，类型为开关
HA_Add_DataPoint("fan", "风扇", "switch")
-- "curn" 表示窗帘控制数据点，类型为覆盖设备（可控制窗帘的开合等操作）
HA_Add_DataPoint("curn", "窗帘", "cover")
-- 在本地系统中添加数据点，用于存储和处理对应设备的数据
```

```
-- 每个数据点有名称、编号和数据类型（float 表示浮点数，int 表示整数）
Add_DataPoint("wendu", 1001, "float")
Add_DataPoint("shidu", 1003, "float")
Add_DataPoint("renti", 1005, "int")
Add_DataPoint("fan", 1006, "int")
Add_DataPoint("curn", 1007, "int")
-- 初始化温度和湿度的初始值
wendu = 22.3
shidu = 64.1
-- 初始化人体传感器和窗帘控制的状态变量
s1 = 0
s2 = 0
-- 主循环，持续运行以不断更新和处理数据
while 1 do
    -- 将当前温度值写入到温度数据点，向 Home Assistant 和本地系统更新温度数据
    Write_DataPoint("wendu", wendu)
    -- 将当前湿度值写入到湿度数据点
    Write_DataPoint("shidu", shidu)
    -- 模拟温度和湿度的缓慢变化，每次循环增加 0.1
    shidu = shidu + 0.1
    wendu = wendu + 0.1
    -- 将人体传感器的状态写入到对应数据点，s1 的值在 0 和 1 之间交替变化，模拟人体检测状态的变化
    Write_DataPoint("renti", s1)
    if s1 == 0 then
        s1 = 1
    else
        s1 = 0
    end
    -- 读取风扇的数据点状态
    s, r = Read_DataPoint("fan")
    -- 如果读取状态 r 为 2，表示有新的有效数据，将风扇状态值 s 打印输出，并将该状态写回数据点（可能用于同步状态或更新显示）
    if r == 2 then
        print(string.format("fan=%d\n", s))
        Write_DataPoint("fan", s)
    end
    -- 读取窗帘的数据点状态
    s, r = Read_DataPoint("curn")
    -- 当读取状态 r 为 2 时，表示窗帘有移动操作（移动范围 0 - 100），打印输出状态信息，并将该状态写回数据点
    if r == 2 then
        print(string.format("s=%d r=%d\n", s, r))
        Write_DataPoint("curn", s)
```

```
        -- 如果读取状态 r 为 3，表示窗帘停止移动操作，同样打印输出信息并写回数据点
    elseif r == 3 then
            print(string.format("s=%d r=%d\n", s, r))
            Write_DataPoint("curn", s)

    end
    -- 向 Home Assistant 更新所有数据点的最新数据，确保数据的同步和一致性
    HA_Update_Data()
    -- 添加延迟，使主循环每 1000 毫秒（1 秒）执行一次，避免过于频繁的数据更新和系统资源占用
    delay(1000)
end
```

10.24. 窗帘电机驱动

10.24.1. 窗帘电机控制

curtain_485_control		
控制窗帘		
参数	**类型**	**说明**
Com	Int	1，2，3，4 分别对应网关的串口 1-串口 4
Command_type	Str	command_type 命令类型：1-打开 2-关闭 3-停止 4-设置百分比
返回值	Int	无
函数说明	command_type 命令类型：1-打开 2-关闭 3-停止 4-设置百分比 percentage 百分比值(0-100)，仅在 command_type 为 4 时有效 curtain_485_control(1, 1) curtain_485_control(1, 2) curtain_485_control(1, 3) curtain_485_control(1, 4, 21)	
示例代码：		

```
val, ret=Read_DataPoint("curtain_val")
 if ret == 2 then
 curtain_485_control(3, 4, val)
 Write_DataPoint("curtain_val",val);
 print(string.format("val:%d, ret:%d", val, ret))
 delay(50);
 elseif ret ==3 then
 curtain_485_control(3, 3)
```

```
delay(50);
print(string.format("val:%d, ret:%d", val, ret))
end
```

10.24.2. 读取窗帘电机位置状态

curtain_485_read_position		
读取窗帘位置信息		
参数	**类型**	**说明**
Com	Int	1，2，3，4 分别对应网关的串口 1-串口 4
返回值	Int	窗帘位置信息 0-100
函数说明	**无**	
示例代码：		

```
local position=curtain_485_read_position(3)
Write_DataPoint("curtain_val", position)
print(string.format("窗帘位置:%d",position))
```

10.25. LoRa 网关驱动

10.25.1. 网关启动 TCP 模式

lora_debug_mode		
读取窗帘位置信息		
参数	**类型**	**说明**
Com	Int	TCP 端口
Baud	int	波特率，9600，19200， 57600，115200
Parity	char	校验，"even"，"odd" or "none"
Dbit	int	数据位，5，6，7，8
Sbit	int	停止位，1，2
返回值	Int	无
函数说明	**无**	
注意	**代码烧入后要在搜索页面强行重启，或者物理断电**	
示例代码：		

```
release();
```

```
sys_set_com(3, 9600, "none", 8, 1); --初始化 LoRa 串口
Add_DataPoint("wendu", 1000, "int");--温度
Add_DataPoint("shidu", 1001, "int");--湿度
lora_debug_mode(3353, 9600, "NONE", 8, 1)--网关 LoRa TCP 模式
ModbusTCP_service (502, 1)--启动 modbus tcp 服务
HTTP_service(8000, 2);--HTTP 服务
while 1 do
  delay(500);
end
```

10.25.2. 设定 LoRa 数据接收响应时长

set_lora_delay_times		
读取窗帘位置信息		
参数	**类型**	**说明**
Delay	Int	延迟毫秒数，单位:ms
返回值	Int	无
函数说明	无	
注意	代码烧入后要在搜索页面强行重启，或者物理断电	
示例代码：		

```
release();
sys_set_com(3, 9600, "none", 8, 1); --初始化 LoRa 串口
Add_DataPoint("wendu", 1000, "int");--温度
Add_DataPoint("shidu", 1001, "int");--湿度
set_lora_delay_times(180)--设定 LoRa modbus 等待接收响应时长
ModbusTCP_service (502, 1)--启动 modbus tcp 服务
HTTP_service(8000, 2);--HTTP 服务
while 1 do
  delay(500);
  shidu,wendu, ret=ModbusRTU_read_hold_register(3, 1, 0, 2,"int");
  if ret==1 then
    print(string.format("温度:%.1f, 湿度:%.1f", wendu/10, shidu/10));
    Write_DataPoint("wendu", wendu)
    Write_DataPoint("shidu", shidu)
  else
    print(string.format("读取失败"));
  end
   delay(500);
end
```

10.26. 小智官方 MCP 连接

10.26.1. 网关 MCP 客户端连接到官方 MCP 接入点

MCP_service		
读取窗帘位置信息		
参数	**类型**	**说明**
MCP 接入点地址	Str	小智官方获取接入点地址（小智官方 URL） WebSocket 地址
配置变量名对应中文名称	Str	以键值对形式配置，将云变量的英文标识映射为中文名称，用于小智平台识别设备类型及功能
返回值	Int	无
函数说明	使用前需完成两项前置操作： 调用 HTTP_service(8000, 4); 开启 HTTP 模式配置，数据上传格式； 通过 Add_DataPoint 函数添加所需监控的云变量（需定义设备对应的英文标识、地址及数据类型）。	
注意	**代码烧入后要在搜索页面强行重启，或者物理断电**	
示例代码：		

```
release();  -- 配置为发布版本
-- 设置串口 1 参数：波特率 9600，无校验位，数据位 8，停止位 1
sys_set_com(1, 9600, "none", 8, 1);
HTTP_service(8000, 4);  -- 配置 HTTP 服务：端口 8000，指定数据上传格式
ModbusTCP_service(502, 1);  -- 配置 ModbusTCP 服务：端口 502
-- 卫生间设备：添加云变量（英文标识、Modbus 寄存器地址、数据类型）
Add_DataPoint("led1", 1001, "int");     -- 卫生间-照明灯
Add_DataPoint("wendu", 1004, "int");    -- 卫生间-温度
Add_DataPoint("shidu", 1005, "int");    -- 卫生间-湿度
-- 客厅设备
Add_DataPoint("zhuled2", 1006, "int"); -- 客厅-主灯
Add_DataPoint("curtain2", 1008, "int");-- 客厅-窗帘
Add_DataPoint("huoyan", 1011, "int");  -- 客厅-火焰
-- 厨房设备
Add_DataPoint("fan3", 1013, "int");     -- 厨房-风扇
Add_DataPoint("led3", 1015, "int");     -- 厨房-照明灯
Add_DataPoint("smog", 1016, "int");     -- 厨房-烟雾
-- 卧室设备
```

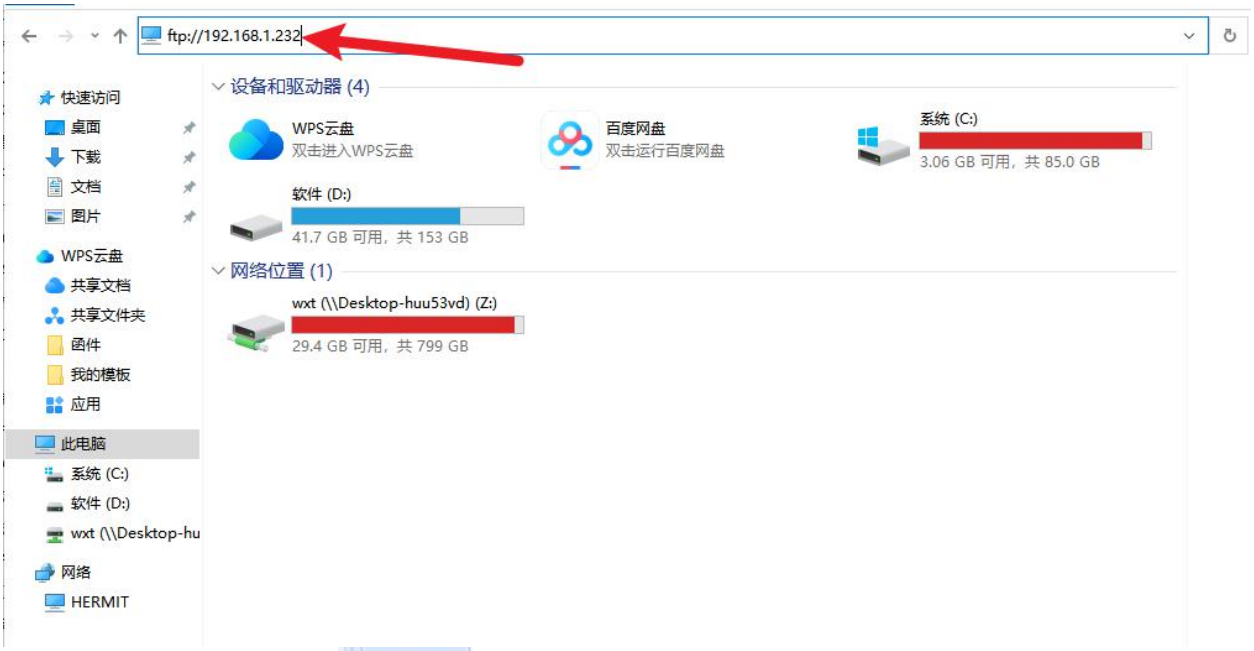
```
Add_DataPoint("zhuled4", 1019, "int"); -- 卧室-主灯
Add_DataPoint("guangzhao", 1021, "int");-- 卧室-光照度
Add_DataPoint("curtain4", 1022, "int");-- 卧室-窗帘
Add_DataPoint("renti", 1023, "int");   -- 卧室-人体传感器
-- 配置变量名映射：中文名称 -> 云变量英文标识（供小智平台识别）
local config = {
    ["卫生间照明灯"]   = "led1",
    ["卫生间温度"]     = "wendu",
    ["卫生间湿度"]     = "shidu",

    ["客厅窗帘"]       = "curtain2",
    ["客厅主灯"]       = "zhuled2",
    ["客厅火焰"]       = "huoyan",

    ["厨房风扇"]       = "fan3",
    ["厨房照明灯"]     = "led3",
    ["厨房烟雾"]       = "smog",

    ["卧室主灯"]       = "zhuled4",
    ["卧室光照度"]     = "guangzhao",
    ["卧室窗帘"]       = "curtain4",
    ["卧室人体"]       = "renti",

    ["192.168.1.227:8000"] = "HTTP_service_URL"  -- HTTP 服务地址映射
}
-- 接入小智 MCP 平台（传入接入点地址及变量映射配置）
MCP_service("wss://api.xiaozhi.me/mcp/?token=xxx", config)
```

10.27. AI 主机扬声器播放

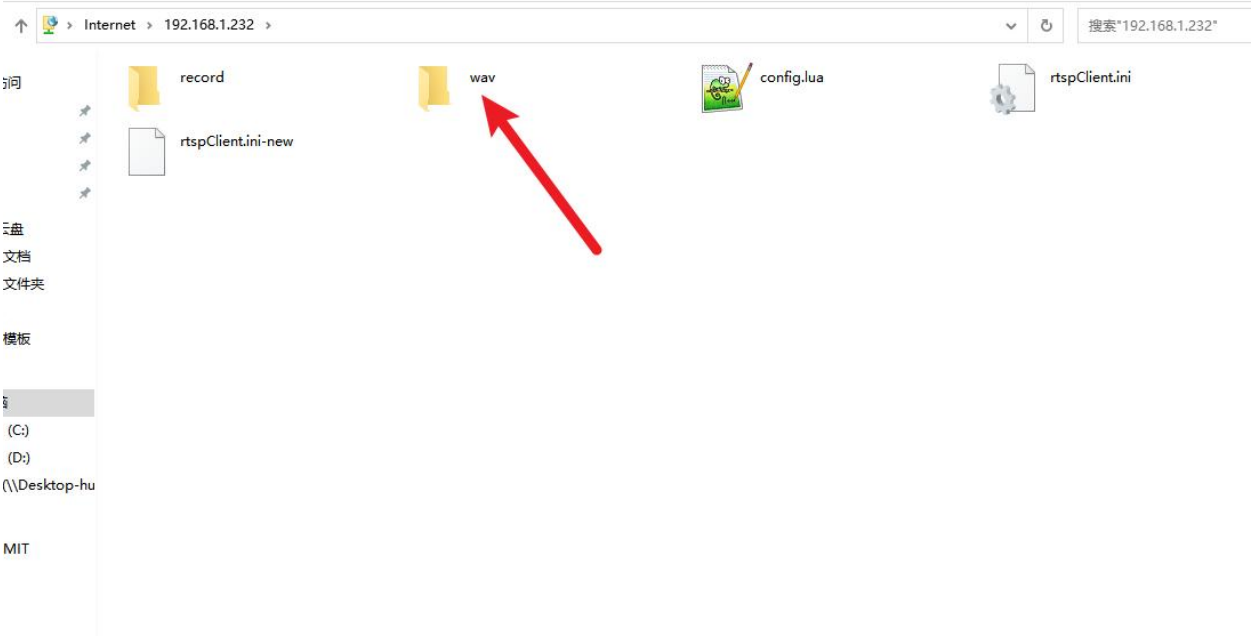
10.27.1. 音频播放函数

1. 使用网关搜索工具搜索出来 A60 的网关 IP 地址

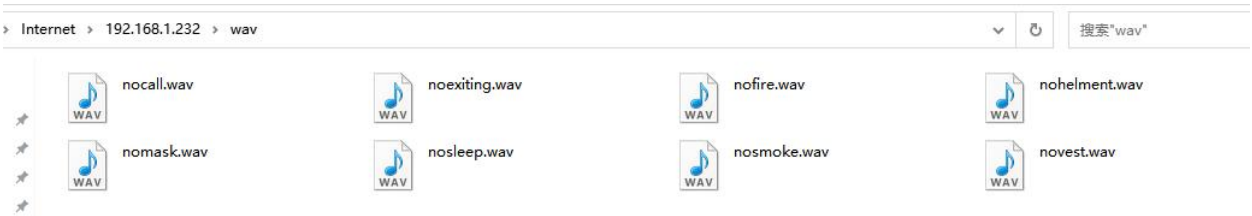
2．使用 ftp 的方式，拉入音频文件。打开我的电脑，搜索 ftp://网关 IP



3．进入 wav 文件夹



4. 拉入 wav 的音频文件，即可在函数中调用



playwav()		
读取窗帘位置信息		
参数	**类型**	**说明**
Wav 音频文件名称	Str	放置到 ftp://网关 IP 的 wav 文件的音频文件
阻塞模式	Int	0：非阻塞模式，无须将此音频文件播放完，下一次函数触发即可中断。 1：阻塞模式，必须将此音频文件播放完
返回值	Int	无
函数说明	**无**	
注意	**代码烧入后要在搜索页面强行重启，或者物理断电**	
示例代码：		

```
        -- -------------------- 10. 远程Modbus TCP 设备报警检测（跌倒、明
火） --------------------
        -- 注意：需提前定义"ip"变量为远程Modbus TCP 设备的IP（如
local ip = "1102.168.1.xxx"）
        -- 函数：ModbusTCP_read_hold_register(设备IP, 端口, 设备ID, 寄存器地址, 数
量, 类型)
        -- 读远程设备寄存器10（模式值）：1=正常，4=跌倒，6=明火
        ret, mode = ModbusTCP_read_hold_register(ip, 5002, 1, 10, 1, "int");
        if ret == 1 then
            if mode == 4 then  -- 模式4=检测到跌倒
                -- 读远程设备寄存器1000（跌倒次数）、1001（状态）
                ret, num, s = ModbusTCP_read_hold_register(ip, 5002, 1, 1000, 2,
"int");
                -- 跌倒次数>0 且状态s==0，播放跌倒报警音频
                if num > 0 and s == 0 then
                    playwav("/wav/down.wav", 1);
                end
            elseif mode == 6 then  -- 模式6=检测到明火
```

```
                -- 读远程设备寄存器1000（明火状态）
                ret, fire = ModbusTCP_read_hold_register(ip, 5002, 1, 1000, 1, "i
nt");
                if fire ~= 0 then  -- 明火状态非0=检测到明火，播放火灾报警音频
                    playwav("/wav/fire.wav", 1);
                end
            end
        end
```

11. 网关高级编程

在做一些复杂一些的网关应用项目时候，我们会发现太多的处理需要放在主循环轮询控制，导致主循环代码臃肿，控制命令响应延时的问题，这种情况下可以考虑使用网关的多线程功能或变量响应函数功能，多线程即是可以创建多个独立运行的函数。变量响应函数即是添加变量的响应事件，当变量值被改变时，网关后台自动调用变量的响应函数。

使用线程功能脚本编写要点：

1、脚本的任何语句必须放置在函数内部，脚本运行就必须执行的语句，必须放置在 main 函数内部；

2、使用 pthread_create("thread")函数创建线程，“thread”为线程函数名称，线程要做的工作，只需在“thread”函数内定义即可；

3、如果涉及多线程之间的通信，可以使用云变量或者网关内部寄存器实现数值传递，不可以在脚本中定义全局变量方式；

4、pthread_create 需放在 main 函数内，避免在循环内部调用，循环创建线程将会导致资源分配过大，导致网关死机；

5、创建一个线程将会占用一定的内存资源，在设计时尽量减少线程创建的数量；

例如现在要设计一个脚本，由主函数创建 2 个线程，主函数和 2 个线程分别做循环打印信息操作，脚本代码如下：

```
function main()
  sys_set_eth0("192.168.1.79", "255.255.255.0", "192.168.1.168");
  pthread_create("thread1");
  pthread_create("thread2");
  k=0;
while 1 do
print("main thread "..k.."\n" );
delay(500);
    k= k+1;
end
end

function thread1()
```

```
  k=0;
while 1 do
print("thread1 thread "..k.."\n" );
delay(500);
  k= k+1;
end
end

function thread2()
  k=0;
while 1 do
print("thread2 thread "..k.."\n" );
delay(500);
  k= k+1;
end
end
```

添加变量的响应事件，可以实现当变量值被改变时，网关后台自动调用变量的响应函数，以提高控制响应的实时性。使用该功能时脚本编写要点有：

1、脚本的任何语句必须放置在函数内部，脚本运行就必须执行的语句，必须放置在 main 函数内部；

2、使用 Add_DataPoint()函数先添加好云变量，然后使用 add_cloud_event()函数添加云变量的响应函数。

3、根据响应需求设计云变量响应函数，注意云变量响应函数的参数即是云变量的值，lua 语言是弱类型语言，参数变量类型会根据云变量类型自动适应，使用在函数定义时，输入参数只能有 2 位，分别是云变量值和云变量名称，并且不能指定类型。

4、变量响应函数尽量精简，杜绝出现死循环情况。

例如现在要设计一个脚本，有 onoff 和 light 两个变量，分别创建这 2 个变量的响应函数，当数值为 257 时闭合继电器，当数值为 256 时断开继电器，脚本代码如下：

```
function onoff_handle( value,cloud_name )
    ifvalue== 257   then
SetDO(1, 0);
Write_DataPoint(cloud_name, 0);
   elseifvalue== 256 then
SetDO(1, 1);
Write_DataPoint(cloud_name, 1);
   end
end

function light_handle(value, cloud_name )
  if   value == 256 then
SetDO(2, 0);
Write_DataPoint(cloud_name, 0);
  elseifvalue== 257 then
```

```
SetDO(2, 1);
Write_DataPoint(cloud_name, 1);
 end
end

function main()
release();
   sys_set_eth0("192.168.1.79", "255.255.255.0", "192.168.1.168");
   sys_set_com(1, 9600, "none", 7, 1);
   HTTP_service(8000,1);
   Add_DataPoint("onoff", 1001, "int");
   Add_DataPoint("light", 1002, "int");
   add_cloud_event("onoff", "onoff_handle");
   add_cloud_event("light", "light_handle");
   k=0;
while 1 do
print("main thread "..k.."\n" );
delay(500);
      k= k+1;
end
end
```

12. 网关固件更新

网关固件会定时更新，以兼容驱动市面上更多的智能设备，扩大网关的应用范围，网关固件更新方法如下文。

打开 IOTGatewayConfig.exe 工具，点击“搜索”按钮搜索网关，在搜索的列表上，选中要升级的网关，右击在弹出的菜单上选择“固件升级”，选择正确的固件路径，点击“升级”即可。

13. 脚本编写案例

13.1. 远程抄表案例

本案例通过将网关连接 dlt645-2007 电表和 modbus RTU 水表。通过网关设计脚本实现对电表、水表的数据采集，设置中移动云平台连接，将数据上传至中移动物联网平台。具体配置脚本如下：

```
HTTP_service(8000,1); --启动 web 服务
Add_DataPoint("dianliu", 1001, "float"); --添加云变量
Add_DataPoint("dianya", 1003, "float");
Add_DataPoint("gongluyin", 1005, "float");
Add_DataPoint("gonglu", 1007, "float");
Add_DataPoint("kwh", 1009, "float");
Add_DataPoint("water", 1011, "float");
ALiYun_service("183.230.40.33", 876, "283507", "zSBY2M5qhptNXP6JT4rsH5Bz7EgA" ,  4);
--设置云服务器
sys_set_com(1, 9600, "none", 8, 1);
sys_set_com(2, 1200, "none", 8, 1);
sys_set_com(3, 9600, "none", 8, 1);
sys_set_com(4, 2400, "even", 8, 1);
```

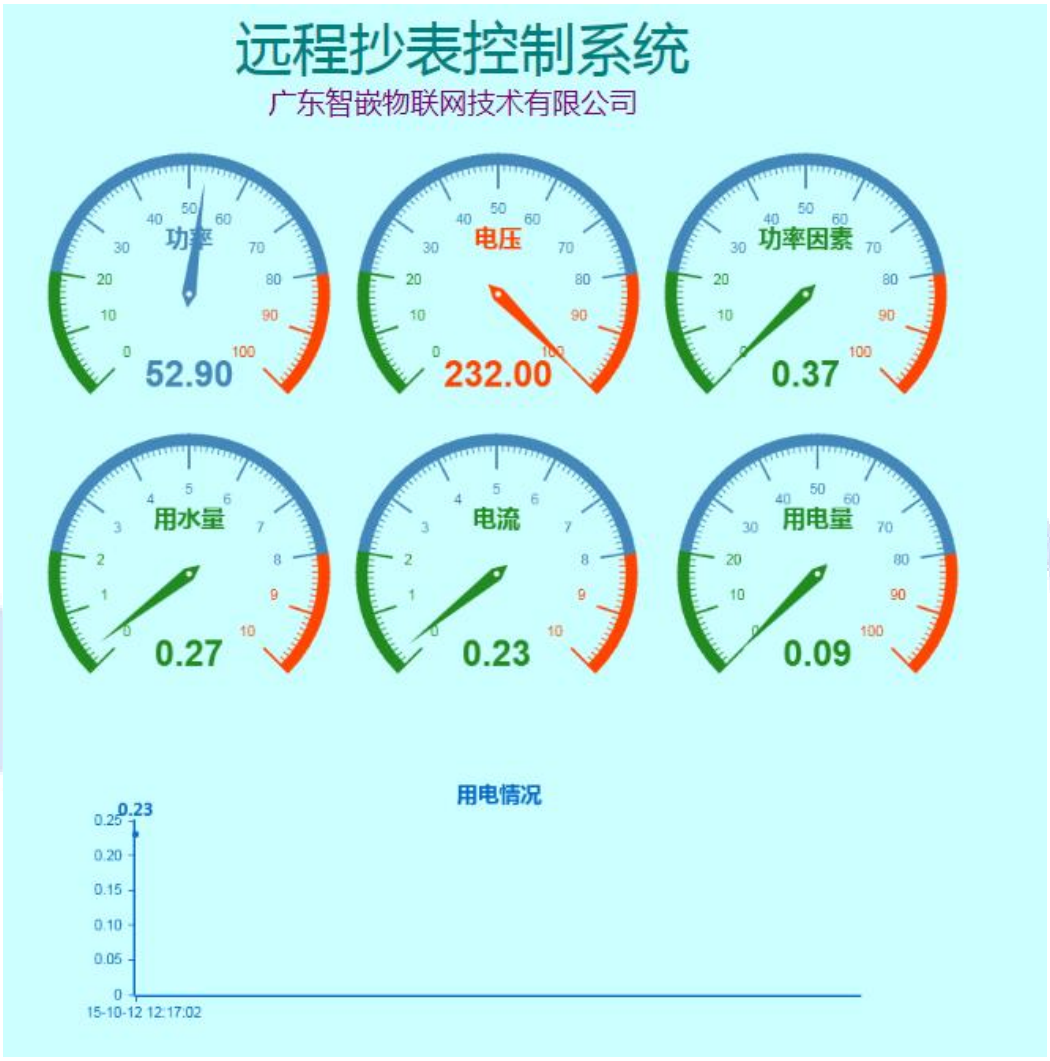
```
ModbusRTU_force_single_coil(2, 1, 1, 1); --开启供电–modbus R T U设备驱动
Write_sys_register(1021, 1, "int");   --写系统寄存器
Write_sys_register(1022, 1, "int");
stat=0;
k=1;
while k>0 do
v1,v2 =   ModbusRTU_read_hold_register(1,3, 0x0000, 4, "BCD"); --流量

v2 = v2 /100;
curent = DLT645_07_read(3,"000000850249", "02020100");   --读取电流值
val = DLT645_07_read(3,"000000850249", "02010100"); --读取电压值
kwh = DLT645_07_read(3,"000000850249", "00010000"); --读取有功总量
power = DLT645_07_read(3,"000000850249", "02060000");--功率
curent = curent / 1000.0; --数值调整
val = val / 10.0;
power = power /1000;
kwh = kwh/100;
gonglu = curent *val;

Write_sys_register(1001, curent, "float");
Write_sys_register(1003, val, "float");
Write_sys_register(1005, power, "float");
Write_sys_register(1007, gonglu, "float");
Write_sys_register(1009, kwh, "float");
Write_sys_register(1011, v2, "float");
reg1 = Read_sys_register(1021, "int");
if reg1 == 1 then
    ModbusRTU_force_single_coil(2, 1, 0, 0); --允许供水
else
    ModbusRTU_force_single_coil(2, 1, 0, 1); --停止供水
end
reg2 = Read_sys_register(1022, "int");
if reg2 == 1 then
    ModbusRTU_force_single_coil(2, 1, 1, 1); --开启供电
else
```

ModbusRTU_force_single_coil(2, 1, 1, 0); --停止供电

end

delay(2000);

在中移动物联网云平台上将可以实时看到网关上传的数据。



14. Node-red 对接

14.1. 基于 Modbus 实现 Node-red 组态监测

14.1.1. 实验介绍

在 Node-RED 中，可以通过添加 Modbus 节点来与 Modbus 设备进行通信。这些节点允许你配置 Modbus 主设备或从设备的参数，如设备地址、端口号、串行参数等。
一旦 Modbus 节点配置完成，你就可以在 Node-RED 流程中创建数据流，以读取或写入 Modbus 设备的数据。例如，你可以使用函数节点来处理从 Modbus 设备读取的数据，或使用调试节点来查看数据流的输出。
Node-RED 还提供了丰富的可视化组件，如仪表盘、图表和表格等，用于展示从 Modbus 设备采集的数据。这些组件可以与数据流相结合，以创建动态的数据可视化界面。

14.1.2. 实验准备

在智能家居实验台上，设备通过 RS485 转 USB 线连接到电脑，并使用配置软件进行设备地址配置，然后通过网关进行管理和控制，这通常涉及一系列步骤。以下是一个大致的流程：
硬件连接：

- 将 RS485 转 USB 线的一端接入智能家居设备的 console 口（或 RS485 接口）。
- 将 RS485 转 USB 线的另一端接入电脑的 USB 口。

配置软件设置：

- 打开智能家居设备的配置软件。
- 在软件中选择正确的串行端口（即 RS485 转 USB 模块对应的端口）。
- 根据设备的说明书或配置软件的指引，输入或选择正确的设备地址（如设备 ID、IP 地址等）。
- 配置其他必要的参数，如波特率、数据位、停止位等。

进入可视化编程，用 node-red 的 modbus 节点和 dashboard 实现数据采集和组态呈现。

14.1.3. 实验步骤

1) 网关通电后网线连接到路由器。

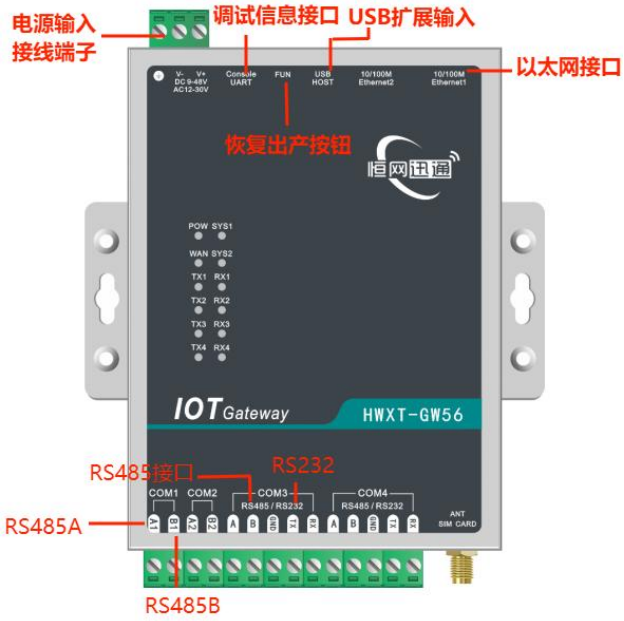


图 16 网关

2) 打开智能网关配置工具，点击搜索

图 17 网关配置工具

3) 右键网关，进入可视化编程

图 18 进入可视化编程

4) 利用 RS485 转 USB 连接传感器和控制器到电脑，利用配置软件配置设备地址。最后得到：（网关的串口为 com1）

传感器	设备地址	线圈/寄存器地址
湿度	1	0
温度	1	1
光照度	2	6
PM2.5	3	0
PM10	3	1

表 1 传感器地址

4 路 IO 控制器	4	
IN:对射	4	0
IN:门磁	4	1
IN:人体	4	2
IN:烟雾	4	3
ON:SOS	4	0
ON:风扇	4	1

表 2 连接 IO 控制器设备的地址

5) 回到 Node-red，拉入 Modbus-read，和 debug，配置读寄存器。（对于监控系统来说，设置合适的 Poll Rate 非常重要。如果 **Poll Rate 设置得太低（即查询频率太高）。**相反，如果 Poll Rate 设置得太高（即查询频率太低），则可能会导致系统无法及时响应设备状态的变化或数据更新。如果“Poll Rate”被设置为 5 秒，那么这意味着系统每 5 秒钟会查询一次设备或传感器的状态或数据。(要在 debug 中判断系统给传感器赋值的数组是什么类型的)

传感器/执行器	Poll Rate	数组
温湿度	4s	msg.payload[i]
光照度	5s	msg.payload[i]
空气质量	6s	msg.payload[i]
执行器	7s	msg.payload[i]

表 3 node-red modbus 配置

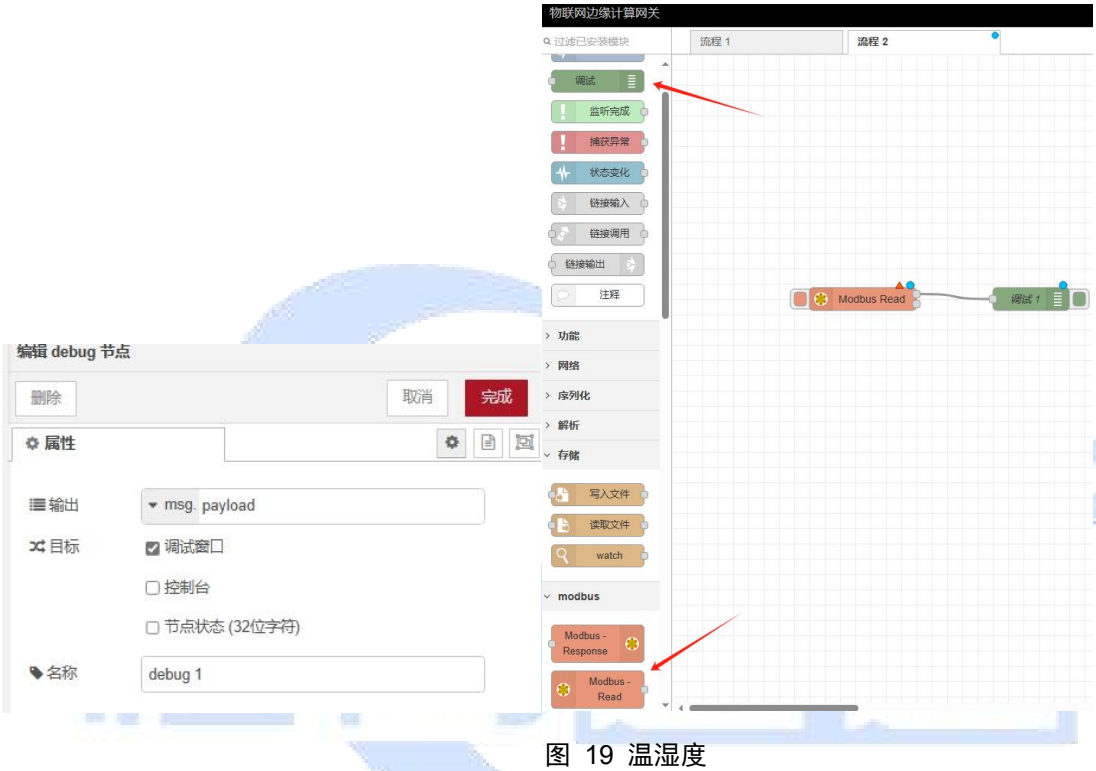


图 19 温湿度

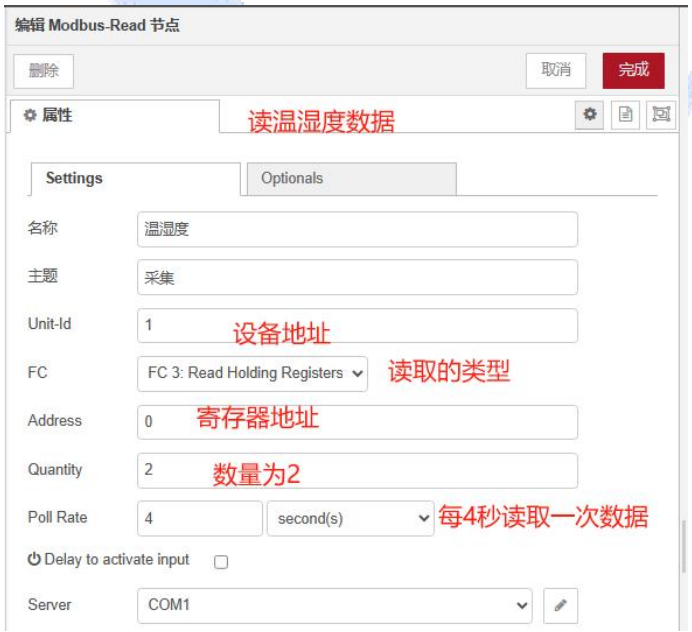


图 20 配置温湿度 read 节点

创建 Server

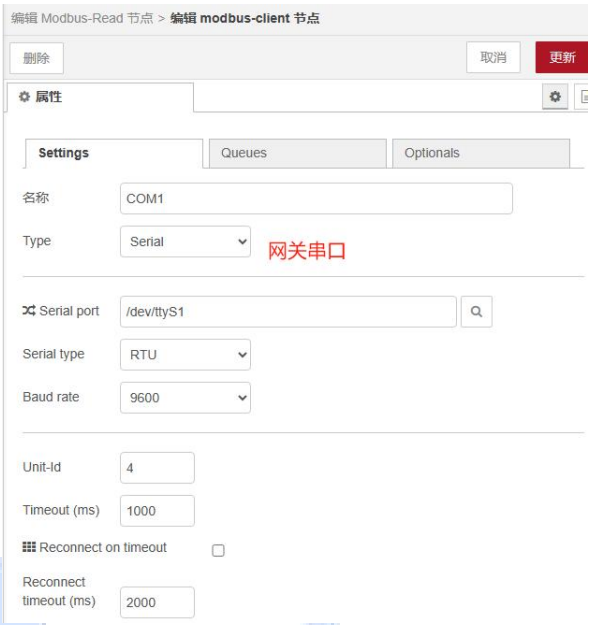


图 21 通信连接

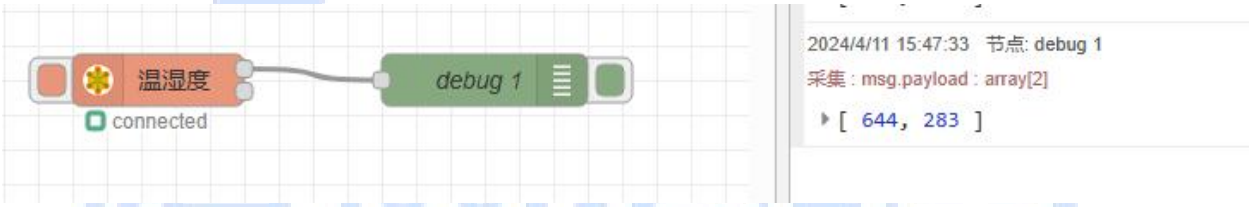


图 22 温湿度输出

6) 点击部署后，可以看到读取的数据都大了 10 倍，所以我们拉个 function 函数，并编辑数组内每个数都除以 10，并将数组内的温度，湿度进行赋值。。

图 23 数据除以 10

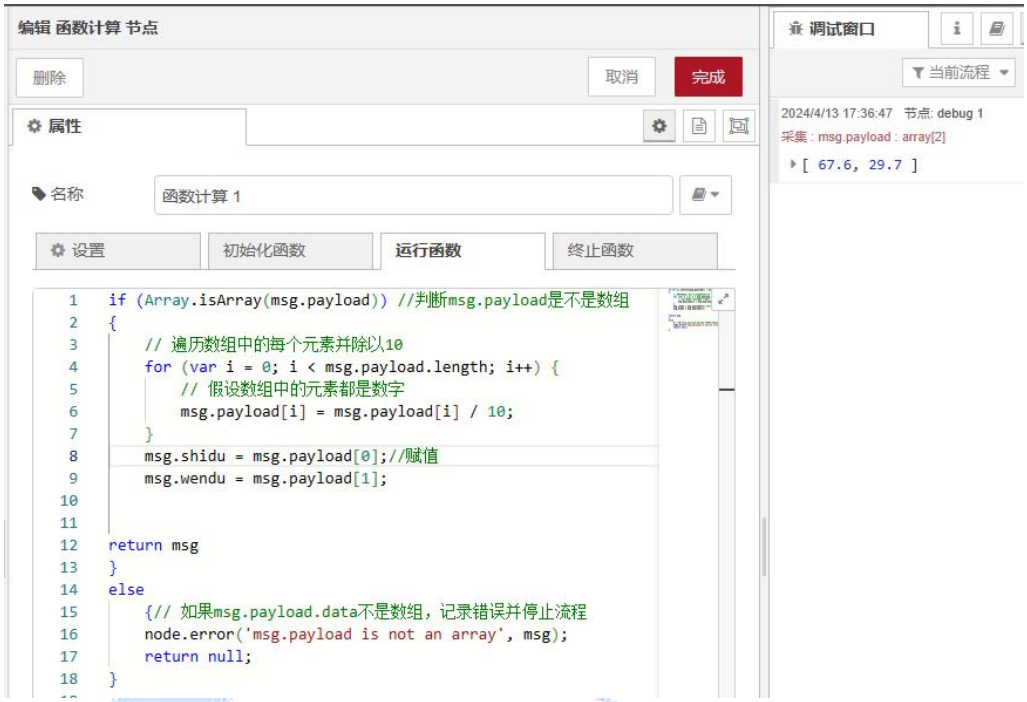


图 24 除十后输出

7) 同理根据 5）6）7）的信息和过程，可以全部配置

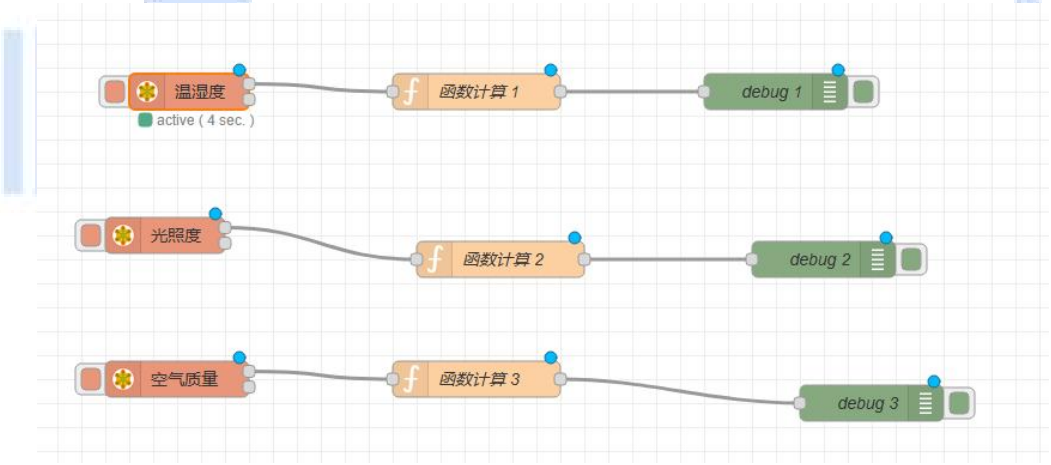


图 25 根据此图依次配置函数

图 26 光照度函数

图 27 空气质量函数

拉入在 modbus 类型中拉入 modbus-read 和 modbus-write，在 dashboard 中拉入 gague（仪表盘）switch（开关），根据 9.1.5 例如 sos 以下配置：

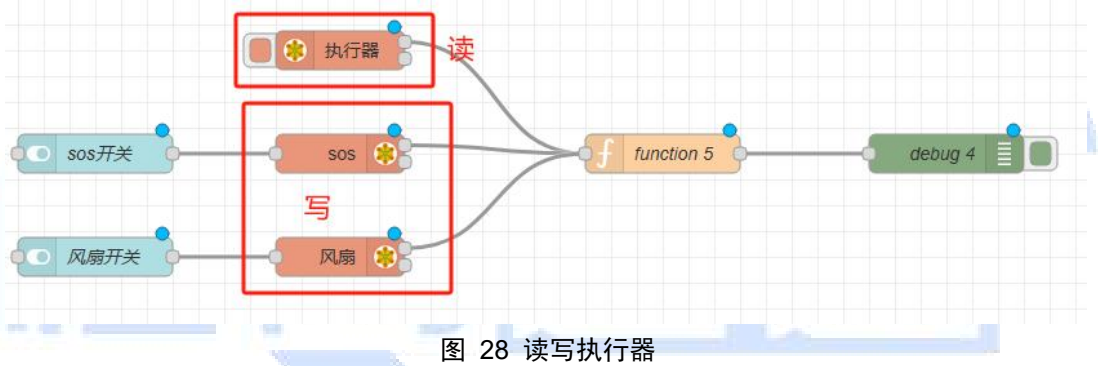


图 28 读写执行器

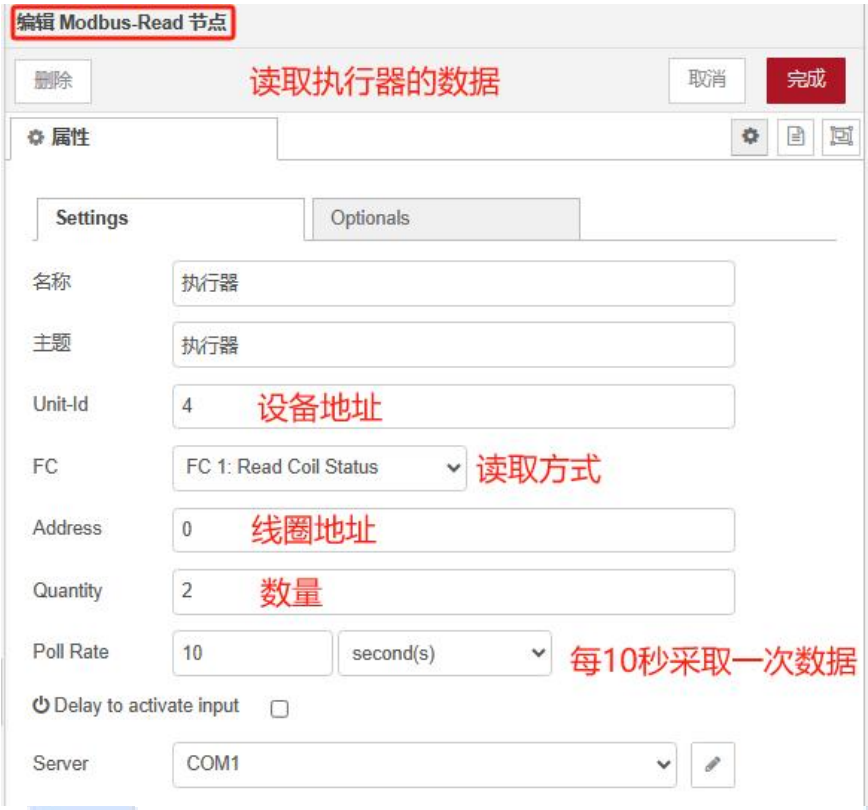


图 29 读执行器的数据配置

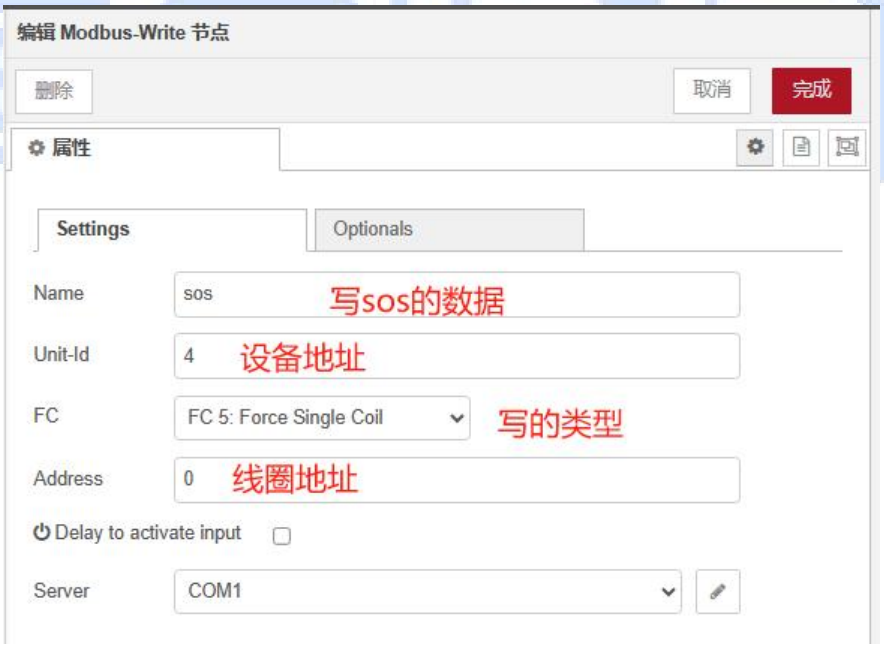


图 30 写 sos 的数据

图 31 执行器的函数

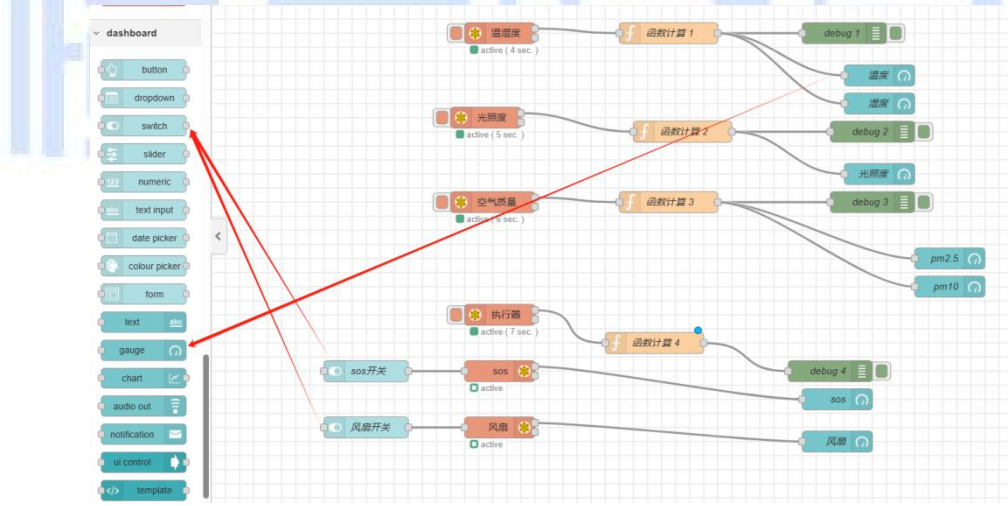


图 32 拉入 dashborad 节点

在 dashborad 中创建组

图 33 创建组

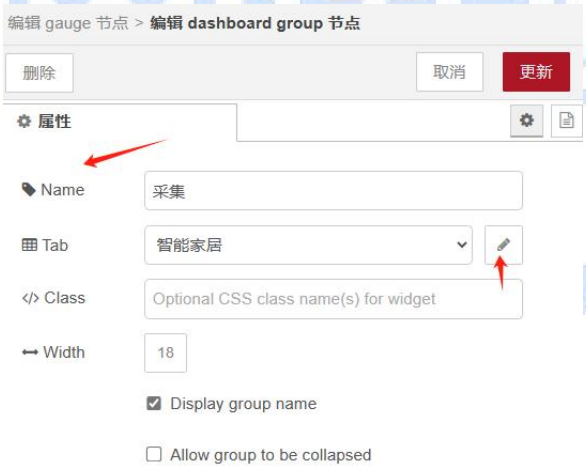


图 34 创建组

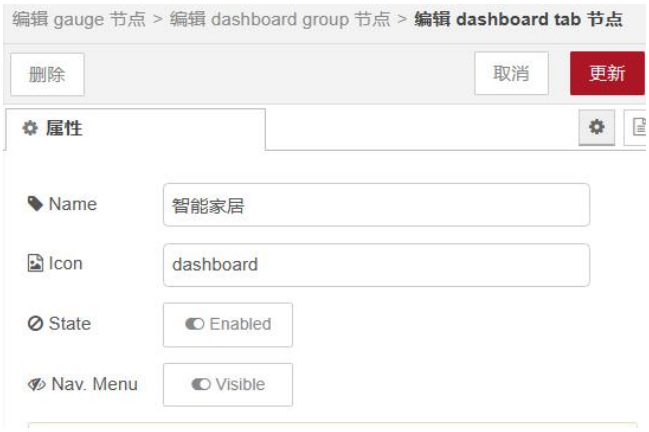


图 35 创建组

添加完成后依次配置采集传感器，湿度，光照度，PM2.5,PM10

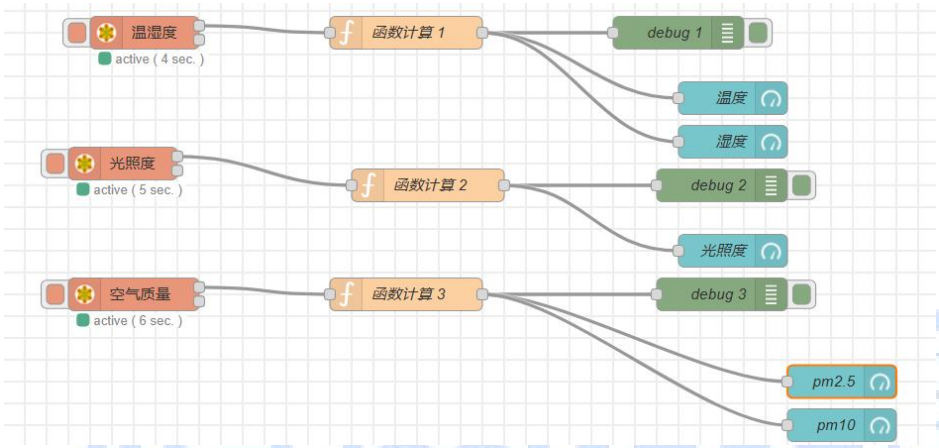


图 36 添加 dashboard

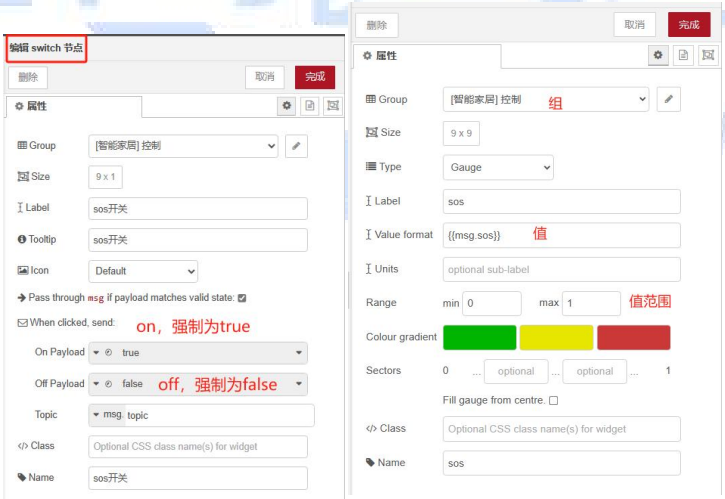


图 37 配置按钮和 sos 仪表盘

部署后进入调试界面

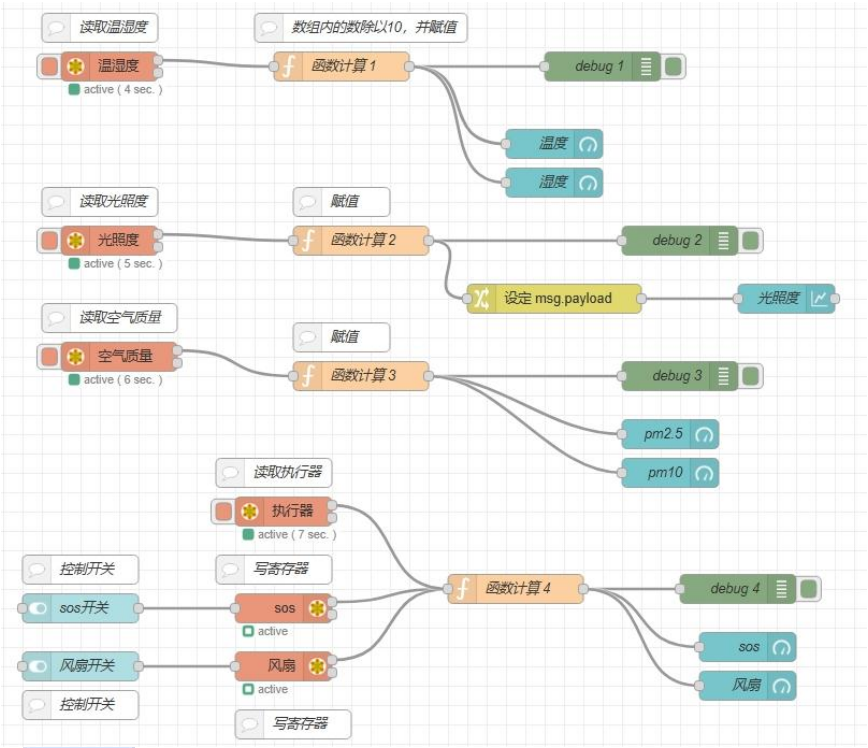


图 38 配置完后部署

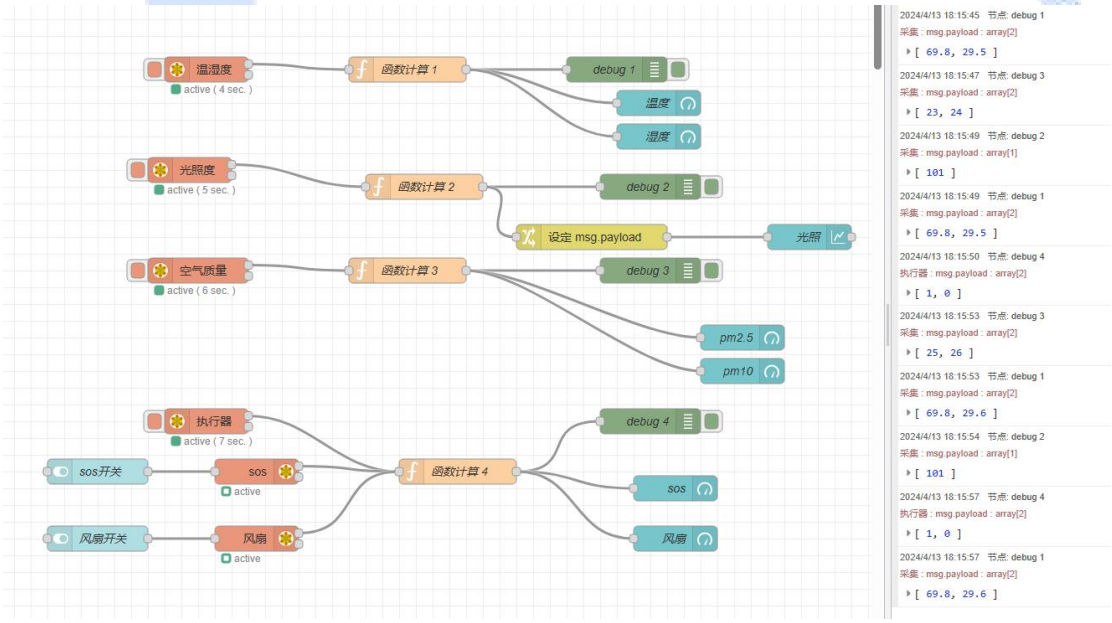


图 39 输出结果

点击右上角的下拉，进入 dashboard，进入组态界面

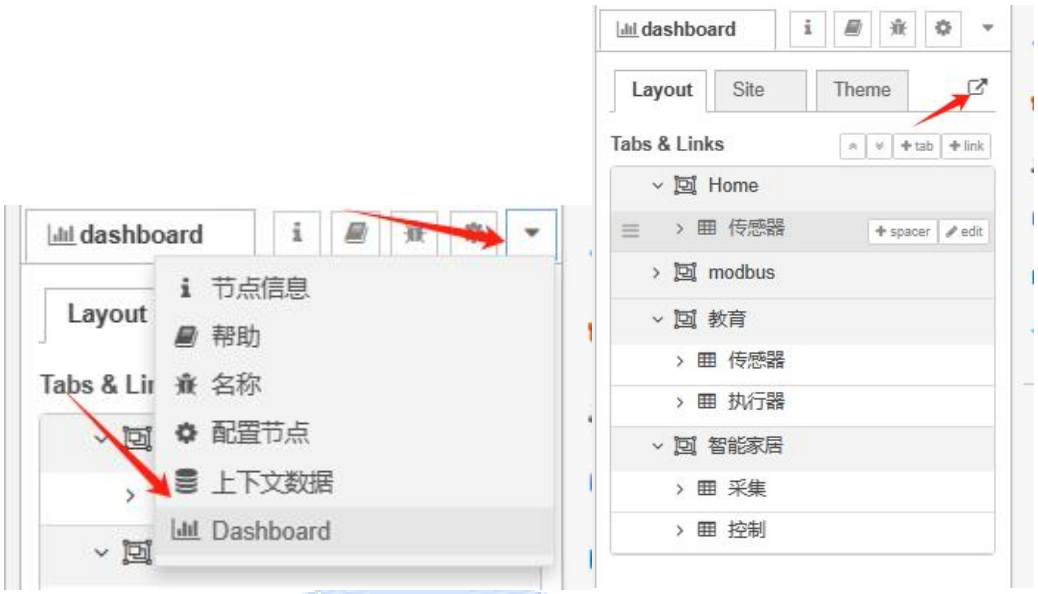


图 40 进入组态

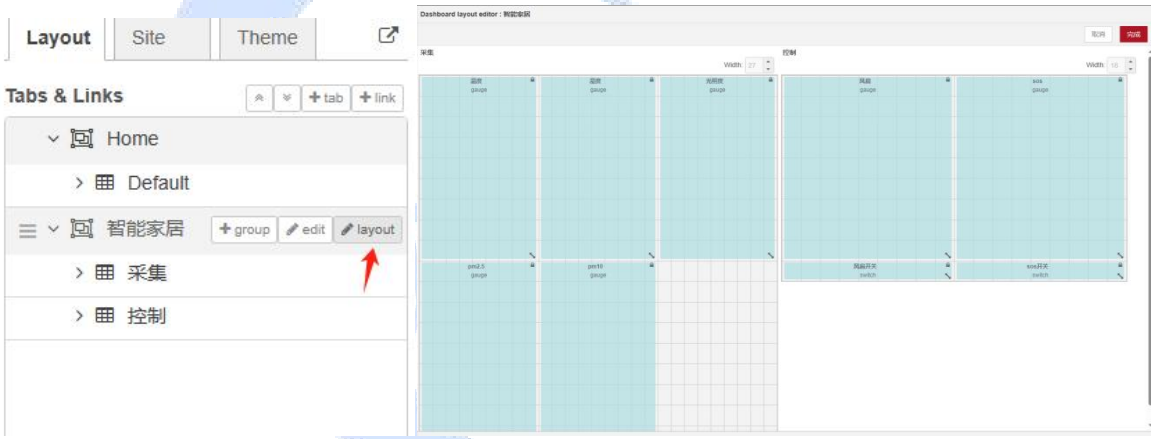


图 41 设计页面

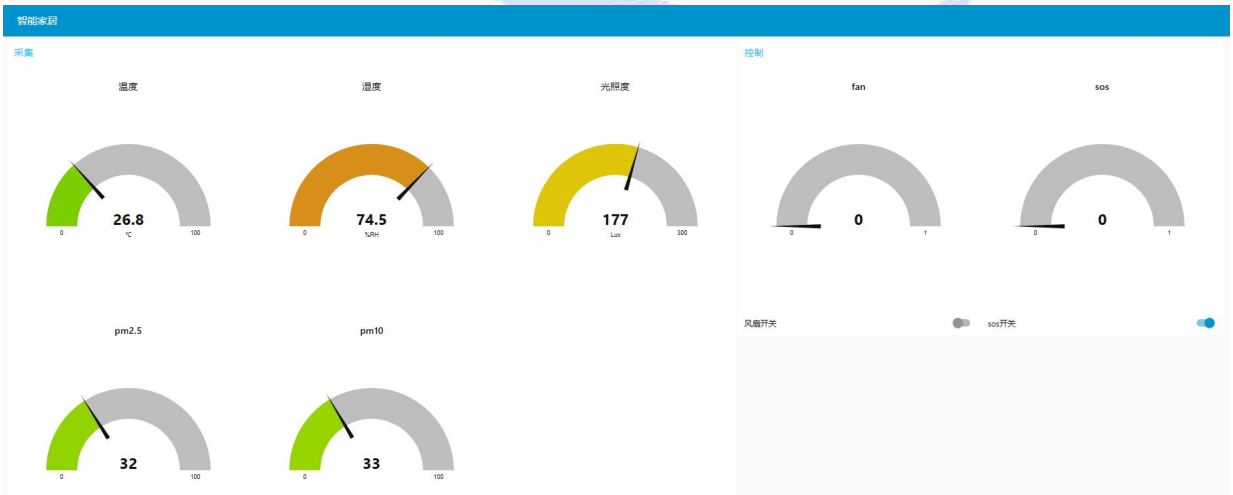


图 42 实现环境采集和控制

也可以把仪表盘更换成折线图

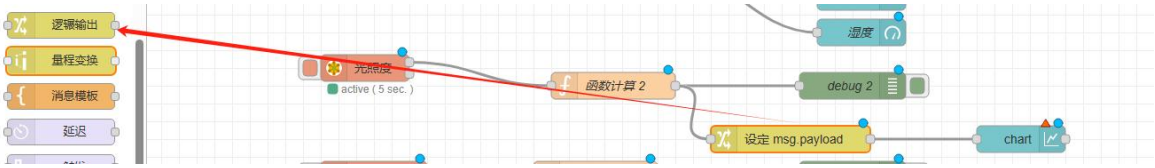


图 43 折线图

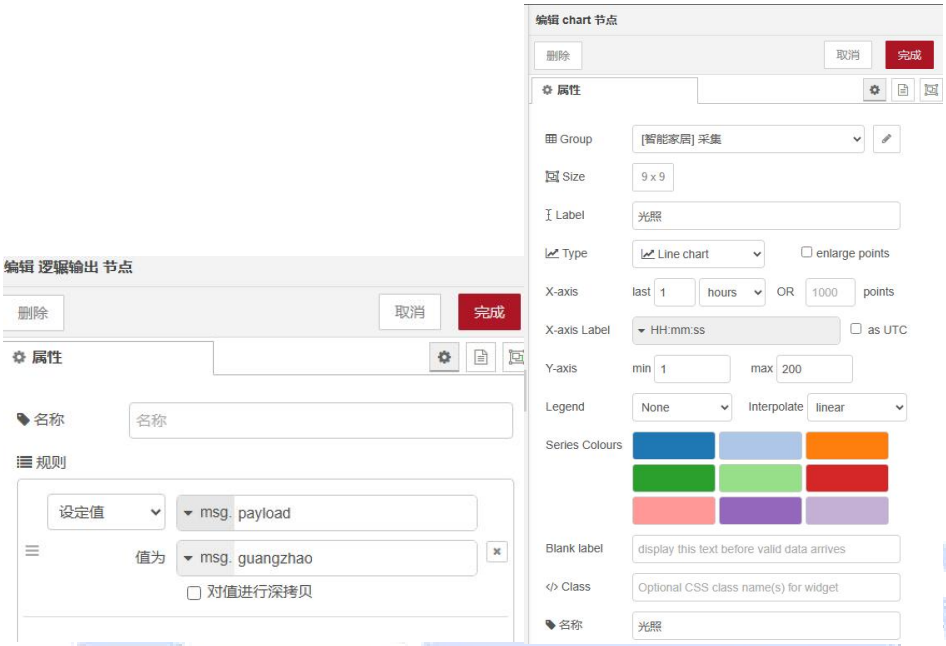


图 44 配置折线图

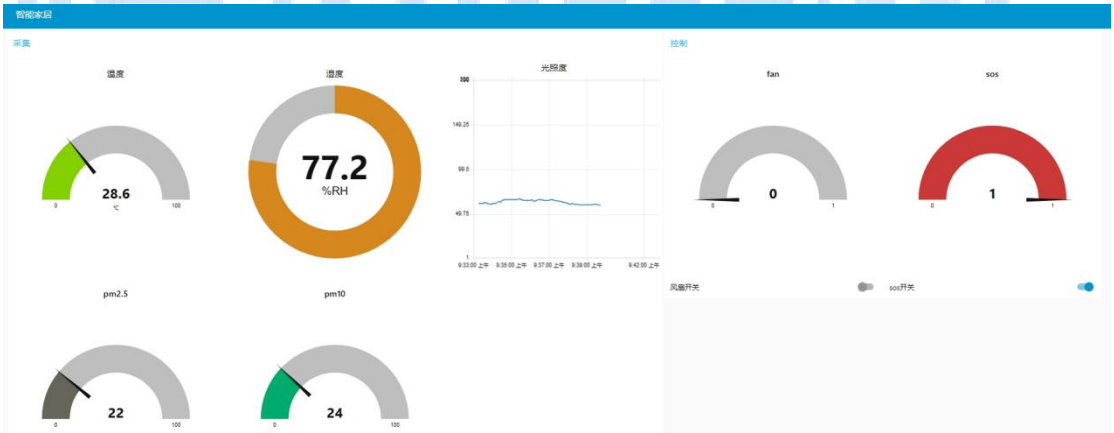


图 45 最终页面

14.1.4. 实验总结

我们成功地通过 RS485 转 USB 接口将设备连接到电脑，并利用配置软件对设备地址进行了精确的配置。随后，通过编写 Lua 脚本，我们将配置信息写入网关，为后续的数据采集和组态呈现打下了坚实的基础。

在数据采集环节，我们使用了 Node-RED 的 Modbus 节点。Modbus 是一种广泛应用的通信协议，特别适用于工业自动化和智能家居系统。通过 Modbus 节点，我们能

够轻松地从设备中读取所需的数据，如温度、湿度、光照度等。Node-RED 的灵活性和模块化设计使得我们可以轻松地构建数据处理流程，对采集到的数据进行清洗、转换和存储。

为了更直观地展示采集到的数据，我们利用了 Node-RED 的 dashboard 功能。Dashboard 提供了一个友好的用户界面，允许我们创建自定义的仪表板，用于实时显示各种环境 参数。通过 dashboard，我们可以直观地监控智能家居设备的运行状态，及时发现潜在 问题，并进行相应的调整和优化。

在整个过程中，我们深刻体会到了 Node-RED 在智能家居领域的强大能力。它不仅能够 实现设备之间的互联互通，还能够通过可视化的编程方式简化数据处理和呈现的流程。此外，Lua 脚本的编写也让我们对智能家居系统的底层逻辑有了更深入的了解。

最终 json 源码。[金山文档] 环境采集.json

14.2. 基于 zigbee2mqtt 实现 Node-red 组态监测

14.2.1. 实验介绍

Zigbee2MQTT 是一款基于开源硬件和软件的智能家居网关，它的主要作用是将低功耗无线网络协议 Zigbee 信号转换为标准的 MQTT 协议进行通讯。这使得 Zigbee 设备能够与 MQTT 服务器进行连接，从而实现对智能家居设备的集中控制。

Zigbee2MQTT 具有强大的兼容性，可以与众多的智能家居设备和平台进行兼容，包括 Node-red,Home Assistant、openHAB 等，这为用户提供了方便的集成和管理体验。同时，Zigbee2MQTT 的功能也十分丰富。通过与温度传感器连接，它可以实现对室内温度的实时监测和控制；通过与门窗传感器、烟雾报警器等设备连接，它可以实现对家庭安全的实时监控和报警。

Zigbee2MQTT 网关通常由硬件和软件两部分组成。硬件部分是一个用于连接 Zigbee 设备和计算机的 USB 适配器，而软件部分则负责实现与 MQTT 服务器的通信。由于其开源的特性，Zigbee2MQTT 提供了一种灵活且可定制的智能家居解决方案，用户可以根据自己的需求进行定制和扩展。

总的来说，Zigbee2MQTT 是一个功能强大、易于使用且兼容性强的智能家居网关，它在智能家居领域中得到了广泛的应用。通过它，用户可以方便地将 Zigbee 设备集成到智能家居系统中，实现对设备的远程控制和自动化管理。

14.2.2. 实验准备

- AI 主机边缘计算网关（包含 Node-red，mqtt 服务器）
- Zigbee 模块
- 智能插座
- 智能窗帘
- 门磁、人体传感器

14.2.3. 实验过程

1. AI 主机边缘计算网关支持 Zigbee 模块传输，网关也具备 Zigbee2MQTT 服务器，打开网关配置工具，得到网关 IP 地址后，在网页搜索后加入端口 8080。例如（网关 IP：192.168.1.230 网页搜索：192.168.1.230：8080）

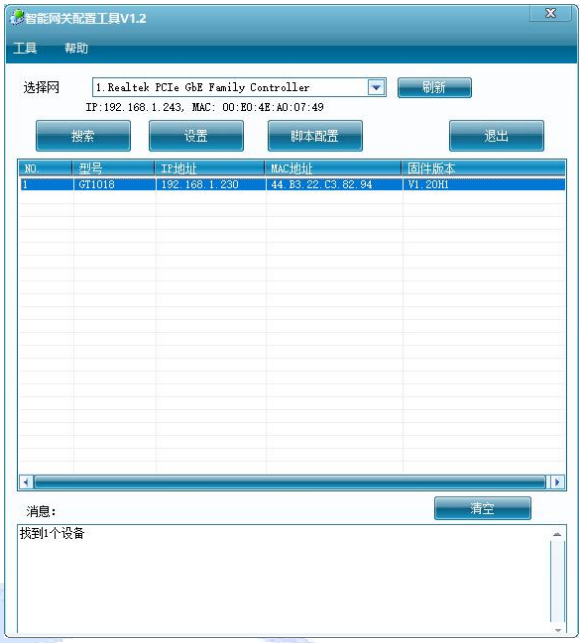


图 46 记录网关 ip 地址

2. 进入 Zigbee2mqtt 服务器后，可以添加设备，从中可得到设备标识，（设备长按 set 键，进入配网模式，点击）

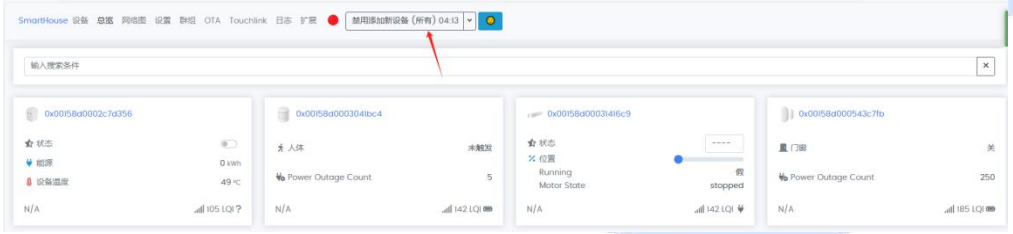


图 47 进入网址后，添加设备

控制下，是否能够通信

3. 打开 MQTT 的软件

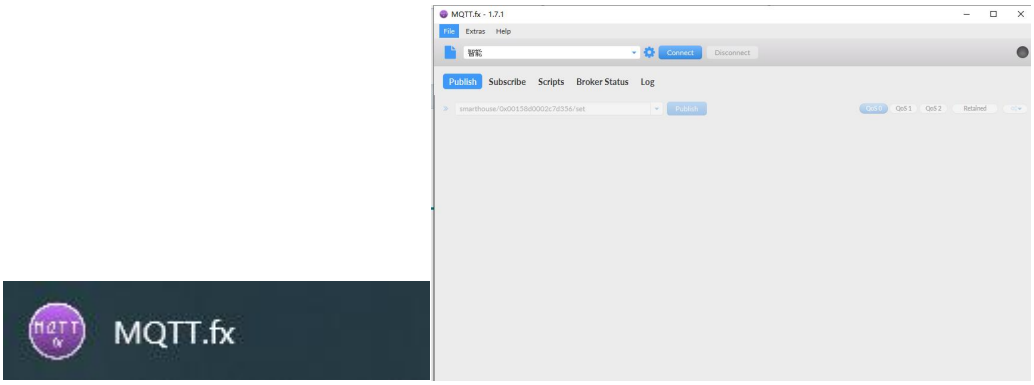


图 48 进入 MQTT 软件测试

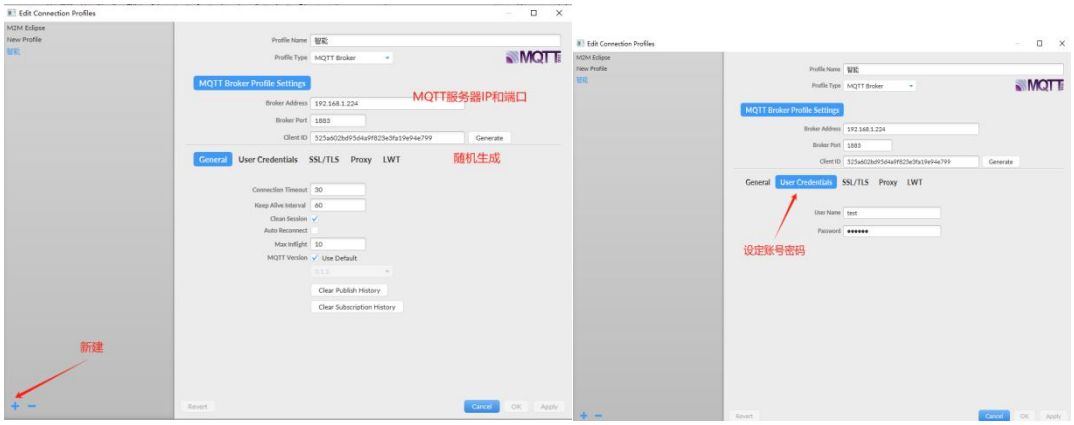


图 49 添加客户端

4. 连接成功后，订阅设备，前缀加设备标识。例如：smarthouse/0x00158d0002c7d356（智能插座）。

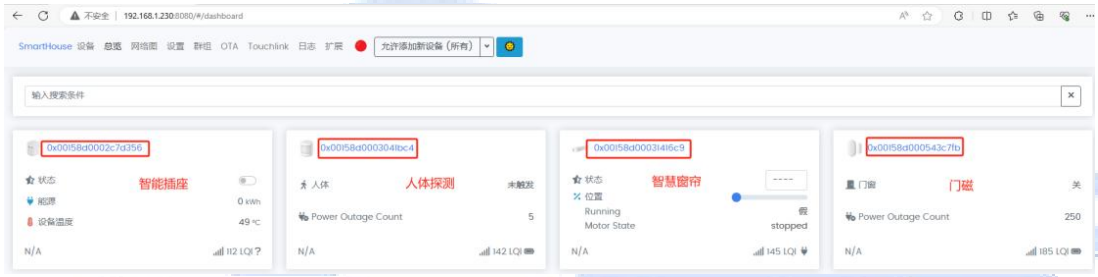


图 50 记录设备标识

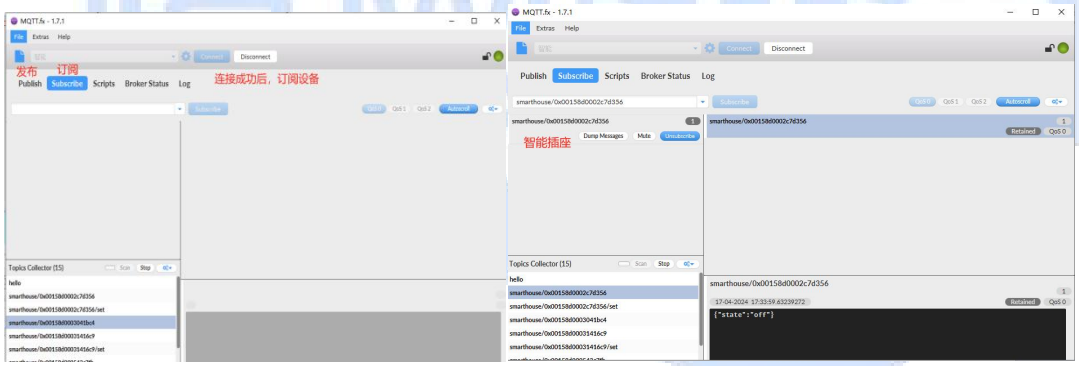


图 51 订阅智能插座

5. 回到 Zigbee2MQTT 控制下智能插座开和关，MQTT 软件收到消息，并且记录下改变的信息。

图 52 控制

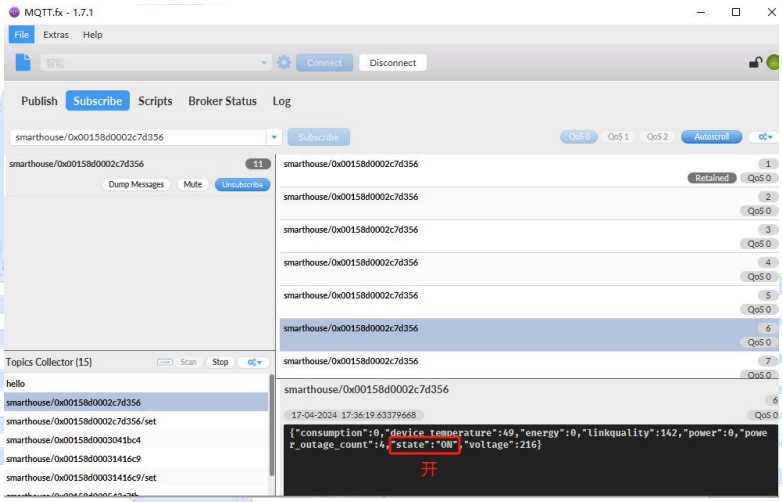


图 53 读取到状态改变

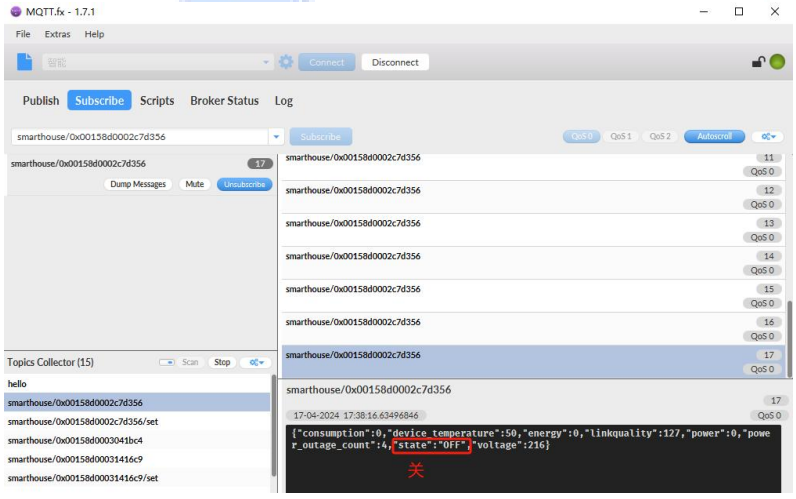


图 54 读取到状态改变

6. 因此我们知道智能插座的开和关的数据都存放到 state 里面，我们可以用 MQTT 软件发布信息控制。

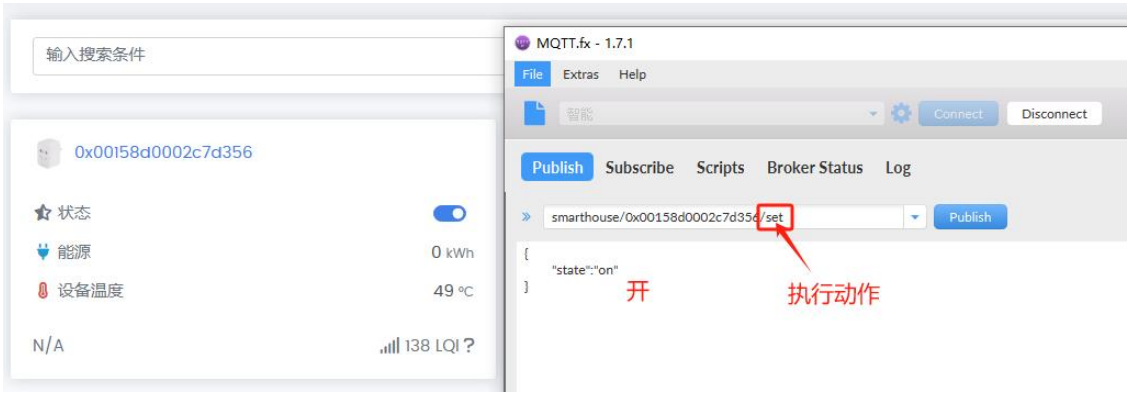


图 55 发布消息控制开

{"state":"on"} 开

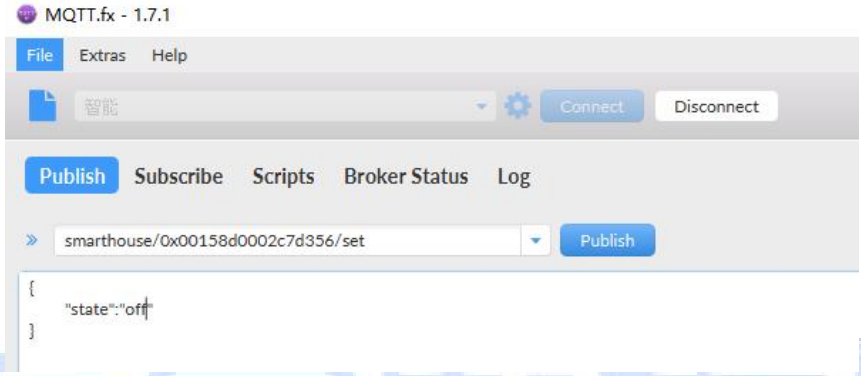


图 56 发布消息控制关

{"state":"off"}关

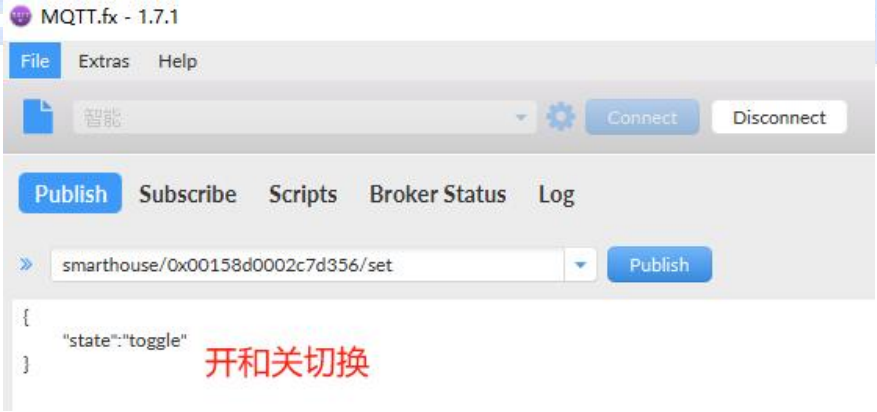


图 57 发布消息控制开和关

{"state":"toggle"}切换

7. 因此我们可以把剩余设备全部添加，能依次控制添加后，提取关键字。

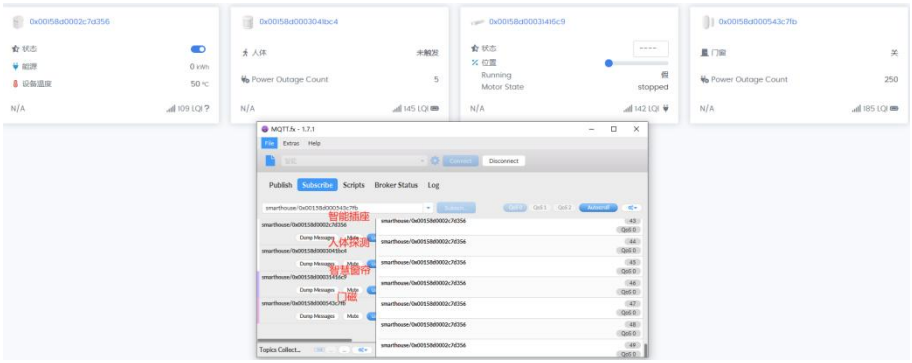


图 58 订阅全部设备

发布需用加/set，订阅（接收信息）不需要添加/set

设备	主题	关键字
智能插座	smarthouse/0x00158d0002c7d356/set	State（开关）
智能窗帘	smarthouse/0x00158d00031416c9/set	Position(位置)
人体探测	smarthouse/0x00158d0003041bc4	occupancy
门磁传感	smarthouse/0x00158d000543c7fb	contact

表 4 设备信息表

8. 进入网关内可视化编程（**Node-red**：例如网关 **IP:1880**）。拉入按钮和 **MQTT** 发布。（前面讲了发布控制主题才需要添加/set）

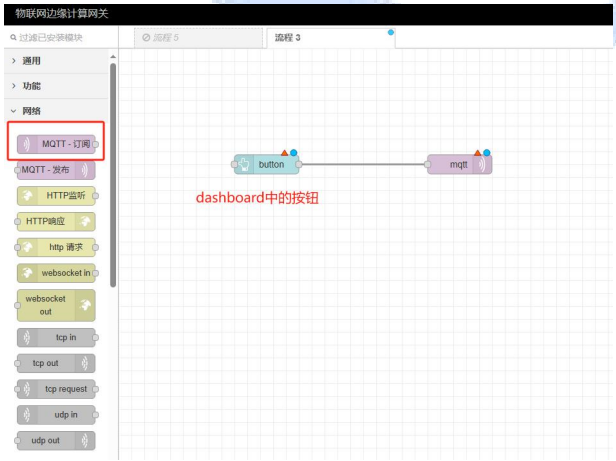


图 59 订阅主题

双击按钮和 **MQTT**-订阅进行配置。

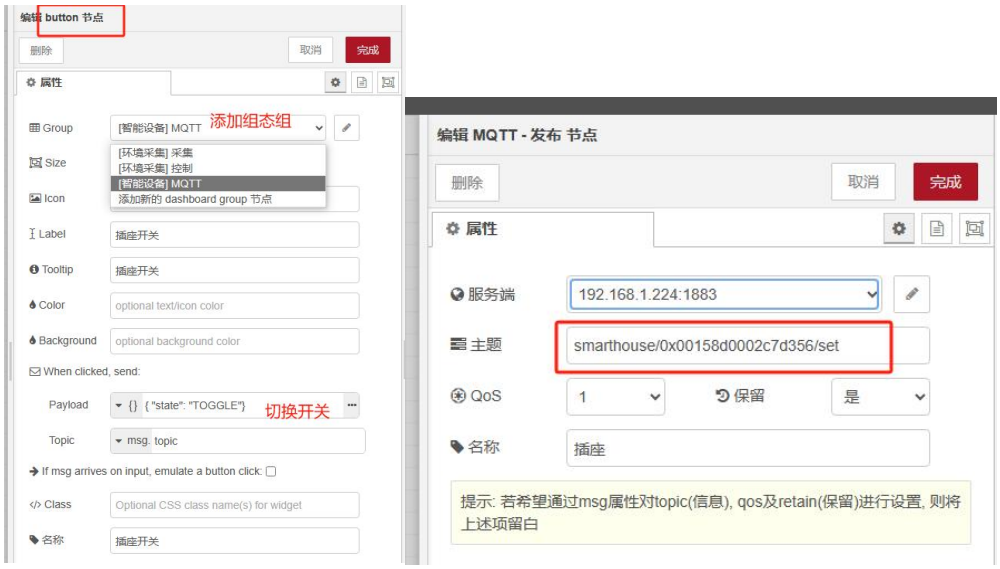


图 60 配置 MQTT 节点

配置客户端

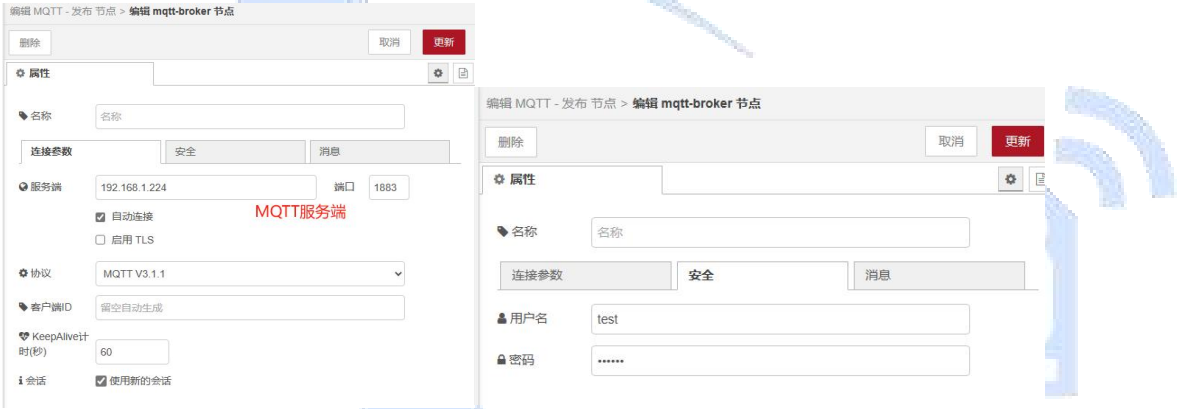


图 61 MQTT 服务器配置

9. 配置好后连接，点去组态测试下是否能够控制，添加所有设备。

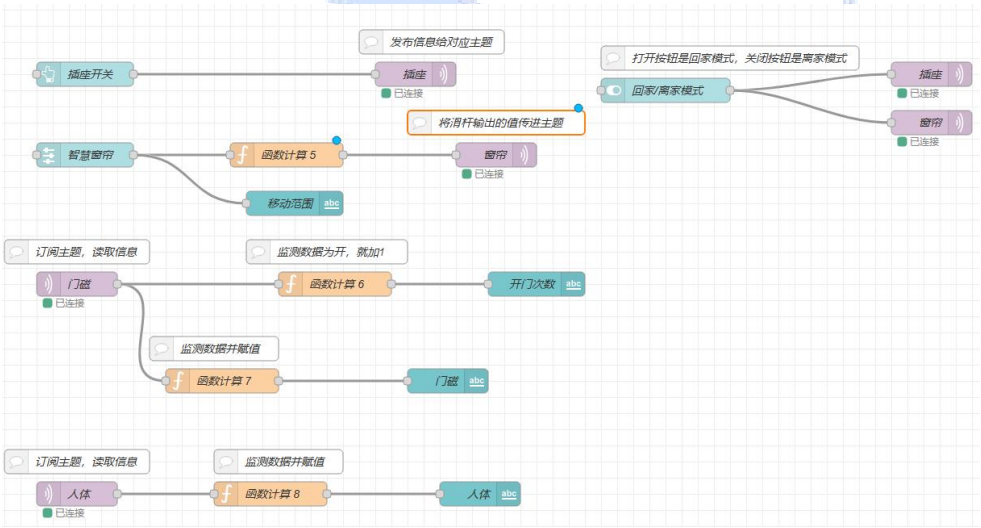


图 62 成果图

● 智能窗帘

移动滑杆时，数值为 msg.payload 改变，利用函数发送 json 格式，position 的值为移动滑杆的值。最后再赋值给 msg.payload 发送给 MQTT。

图 63 滑杆输出（发送 json 文本给 MQTT）

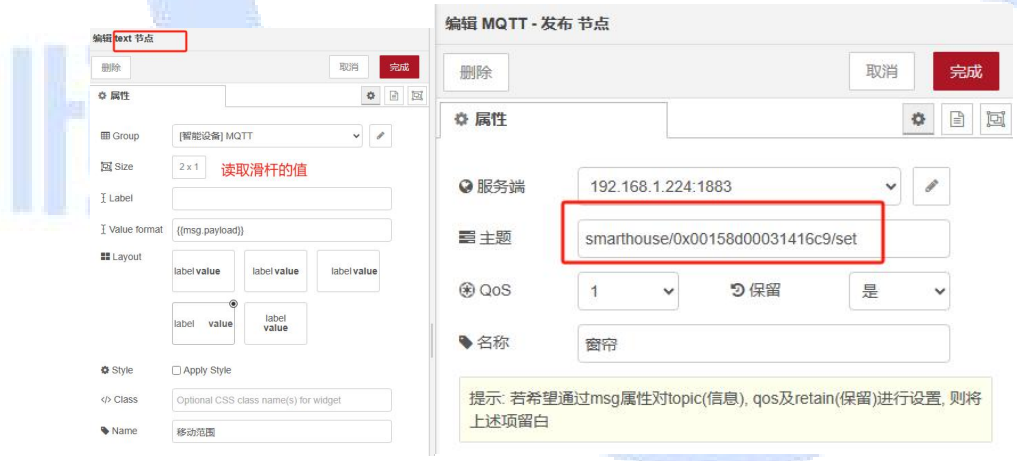


图 64 移动显示数值

图 65 智能窗帘

● 人体探测

人体探测是可读不可写的，订阅人体探测主题，检测到 true 是有人，没人为 false。

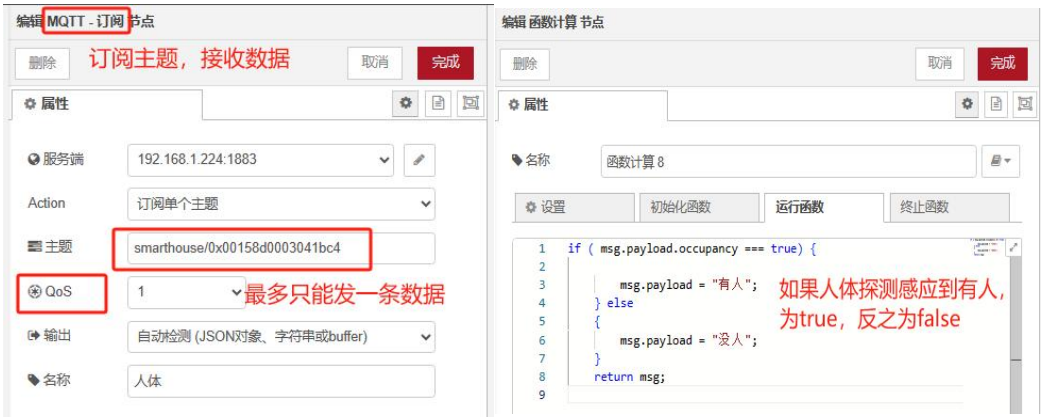


图 66 人体探测配置

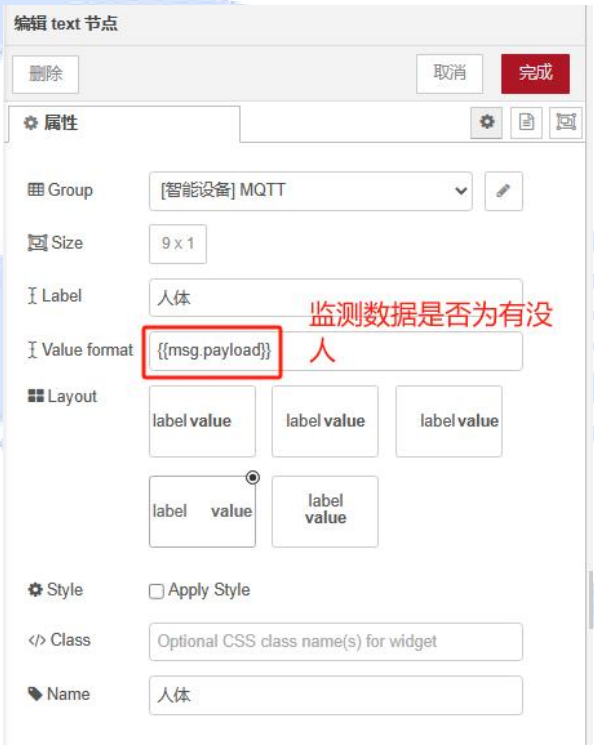


图 67 组态输出

图 68 人体探测

● 门磁

订阅门磁，函数来计数，并输出门状态

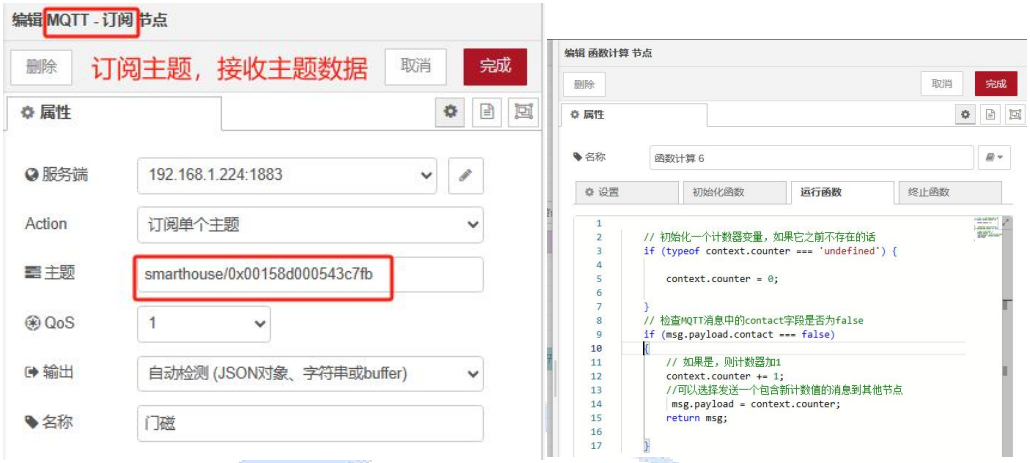


图 69 计数开了几次门

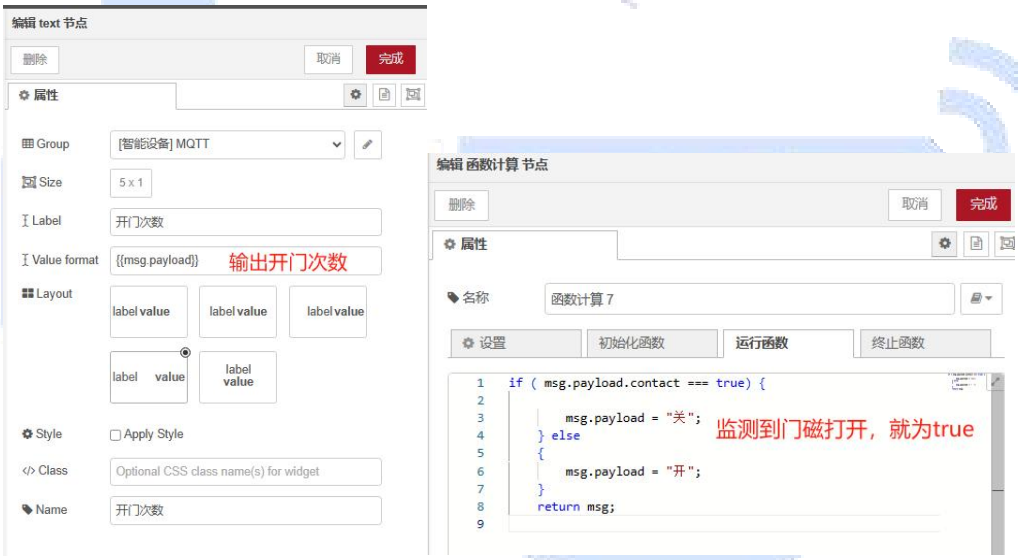


图 70 门状态输出

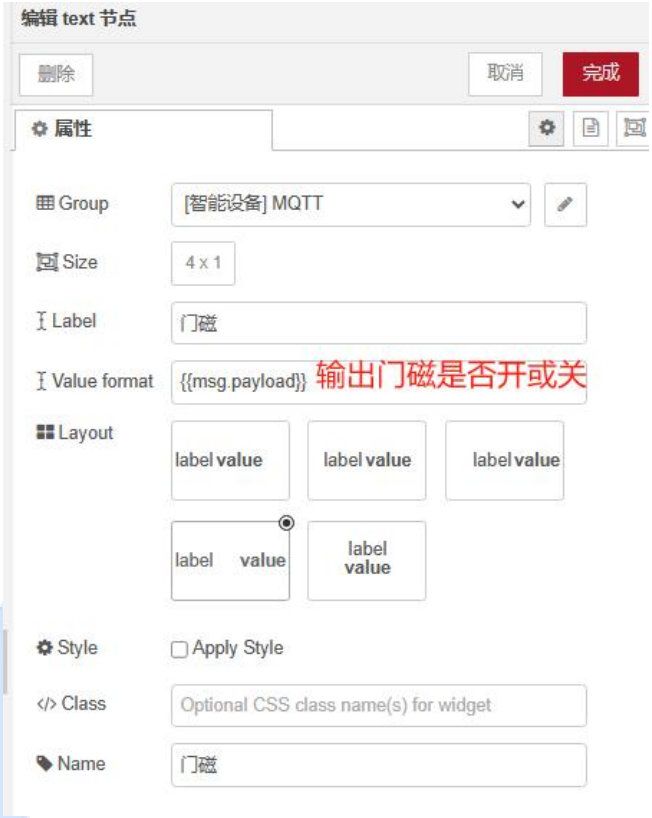


图 71 组态状态输出

图 72 门磁

- **回家模式**

如果打开开关，开启回家模式后，发送信息打开智能窗帘和智能插座

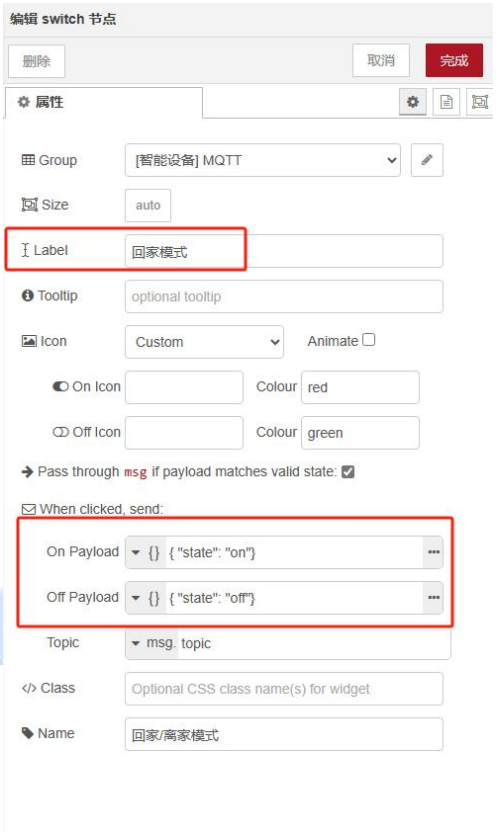


图 73 回家模式开关

图 74 回家模式

14.2.4. 实验总结

Zigbee 技术具有低速率、延迟短、近距离、容量大和高安全性等优势，MQTT 则是一种轻量级的发布/订阅消息传输协议。

Zigbee2MQTT 的主要功能包括：

设备发现与连接：自动发现并连接 Zigbee 设备，将其纳入 MQTT 网络中。

设备控制：通过 MQTT 消息控制 Zigbee 设备的状态和行为。

设备状态监控：实时获取 Zigbee 设备的状态信息，并将其发布到 MQTT 网络中。

Zigbee2MQTT 与 Node-RED 的结合使用为物联网项目提供了强大的组态控制和监控功能。通过可视化编程的方式，用户可以轻松地构建复杂的控制流程和监控逻辑。这种组合方式不仅降低了开发难度和成本，还提高了项目的灵活性和可扩展性。因此，在智能家居、工业自动化等物联网领域，Zigbee2MQTT 与 Node-RED 的组合具有广泛的应用前景。

最终 json 源码。[金山文档] 智能设备(1).json

14. 3. 基于红外实现 node-red 组态控制

14.3.1. 实验介绍

首先利用红外接收器接收射频码，记录下射频码后，将红外接收器接入网关，进入网关可视化编程，利用串行通信和组态设计实现组态控制。

本实验旨在通过红外技术实现 Node-RED 组态控制，

掌握红外通信的基本原理，熟悉 Node-RED 的编程模型，并能够将两者结合应用于实际的控制场景中。

14.3.2. 实验准备

首先将我们的红外设备连接上电脑 USB，打开我们的 sscom 软件，波特率选择 115200，并选择对应红外设备打开串口，并设置为 HEX 接收和发送（红外码是 HEX 格式），将配套空调遥控对着红外设备按相应的按键，即可收到一串射频码，收到并保存好串码，方便后续使用。

图 75 红外设备连接电脑

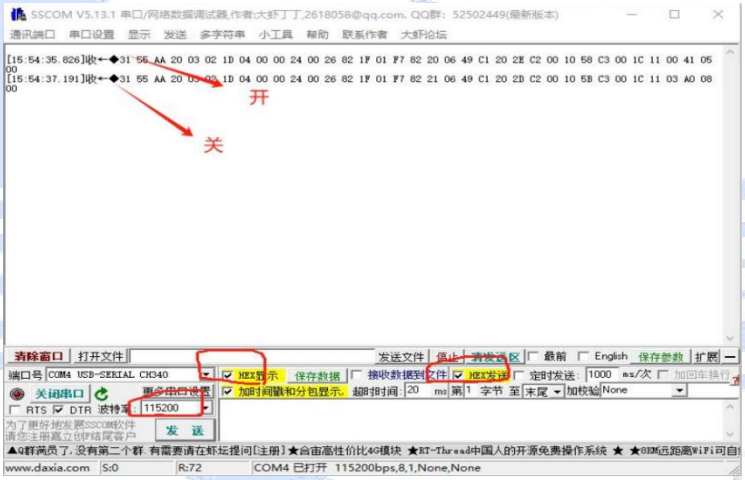


图 76 接收红外码

接下来验证这两个码是否能开启空调，我们将刚才接受复制的开那一串码放到 sscom 发送右边的框（切记需要勾选 HEX 发送才可以发送成功），然后点击发送，可看到空调开启，关空调同理；

图 77 发送射频码

我们可以设置开空调默认的温度，例如（先打开空调，设置成 22 度，先关闭，再打开获得的射频码就是设置的温度）。其它功能都是一样设置。

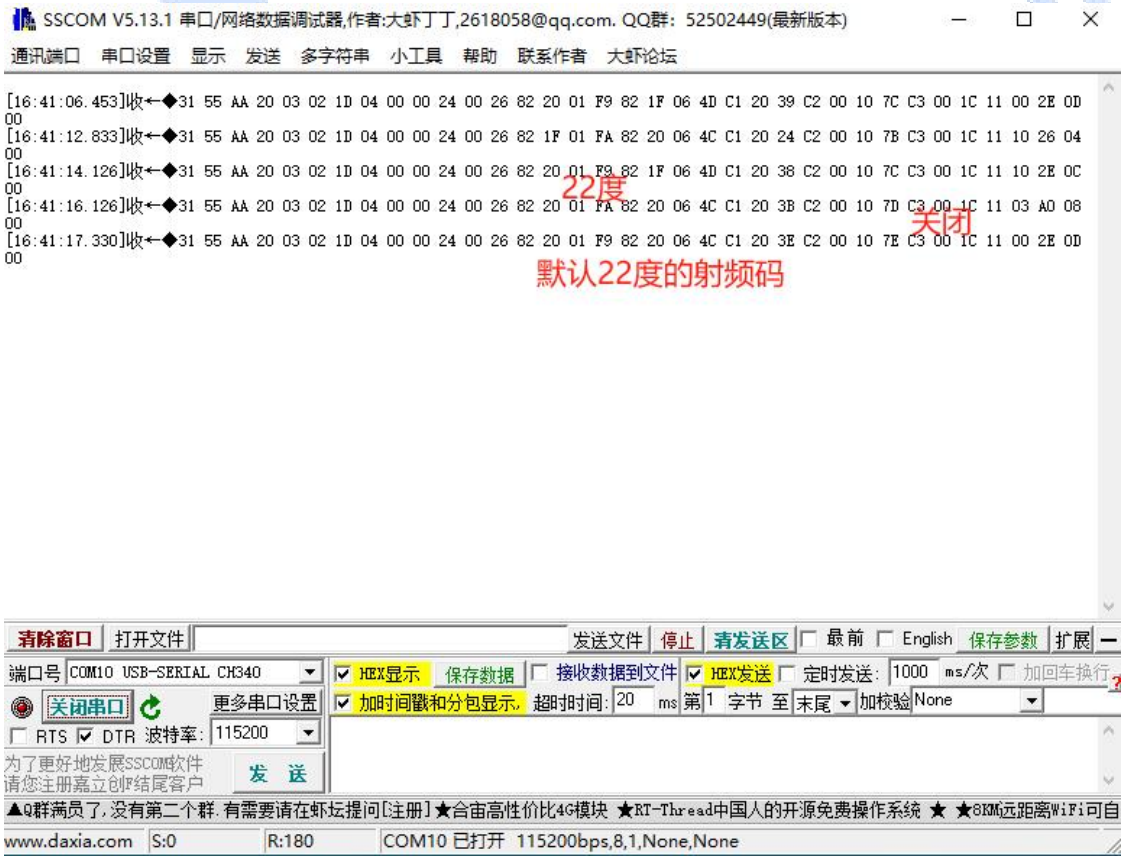
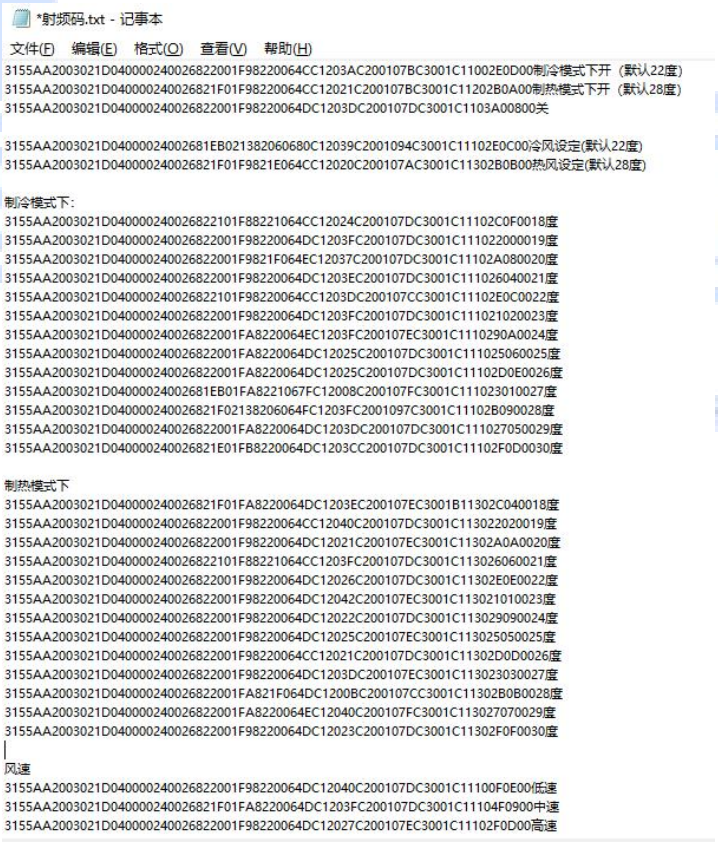


图 78 设置温度

图 79

经测试，关闭也成功。



*射频码.txt - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

3155AA2003021D04000024002682200 1F98220064CC1203AC200107BC3001C11002E0D00制冷模式下开（默认22度）
3155AA2003021D04000024002682101 F01F98220064CC12021C200107BC3001C11202B0A00制热模式下开（默认28度）
3155AA2003021D04000024002682200 1F98220064DC1203DC200107DC3001C1103A00800关

3155AA2003021D04000024002681EB02138206068 0C12039C2001094C3001C11102E0C00冷风设定(默认22度)
3155AA2003021D04000024002682101 F01F9821E064CC12020C200107AC3001C11302B0B00热风设定(默认28度)

制冷模式下:
3155AA2003021D04000024002682210 1F88221064CC12024C200107DC3001C11102C0F0018度
3155AA2003021D04000024002682200 1F98220064DC1203FC200107DC3001C11102200019度
3155AA2003021D04000024002682200 1F9821F064EC12037C200107DC3001C11102A080020度
3155AA2003021D04000024002682200 1F98220064DC1203EC200107DC3001C11102604021度
3155AA2003021D04000024002682210 1F98220064CC1203DC200107CC3001C11102E0C0022度
3155AA2003021D04000024002682200 1F98220064DC1203FC200107DC3001C11102102023度
3155AA2003021D04000024002682200 1FA8220064EC1203FC200107EC3001C1110290A0024度
3155AA2003021D04000024002682200 1FA8220064DC12025C200107DC3001C11102506025度
3155AA2003021D04000024002682200 1FA8220064DC12025C200107DC3001C11102D0E0026度
3155AA2003021D04000024002681EB01FA8221067FC12008C200107FC3001C11102301027度
3155AA2003021D04000024002682101 F02138206064FC1203FC2001097C3001C11102B090028度
3155AA2003021D04000024002682200 1FA8220064DC1203DC200107DC3001C11102705029度
3155AA2003021D04000024002682101 E01FB8220064DC1203CC200107DC3001C11102F0D0030度

制热模式下
3155AA2003021D04000024002682101 F01FA8220064DC1203EC200107EC3001B11302C040018度
3155AA2003021D04000024002682200 1F98220064CC12040C200107DC3001C11302202019度
3155AA2003021D04000024002682200 1F98220064DC12021C200107EC3001C11302A0A0020度
3155AA2003021D04000024002682210 1F88221064CC1203FC200107DC3001C11302606021度
3155AA2003021D04000024002682200 1F98220064DC12026C200107DC3001C11302E0E0022度
3155AA2003021D04000024002682200 1F98220064DC12042C200107EC3001C11302102023度
3155AA2003021D04000024002682200 1F98220064DC12022C200107DC3001C11302909024度
3155AA2003021D04000024002682200 1F98220064DC12025C200107EC3001C11302505025度
3155AA2003021D04000024002682200 1F98220064CC12021C200107DC3001C11302D0D0026度
3155AA2003021D04000024002682200 1F98220064DC1203DC200107EC3001C11302303027度
3155AA2003021D04000024002682200 1FA821F064DC1200BC200107CC3001C11302B0B0028度
3155AA2003021D04000024002682200 1FA8220064EC12040C200107FC3001C11302707029度
3155AA2003021D04000024002682200 1F98220064DC12023C200107DC3001C11302F0F0030度

风速
3155AA2003021D04000024002682200 1F98220064DC12040C200107DC3001C11100F0E00低速
3155AA2003021D04000024002682101 F01FA8220064DC1203FC200107DC3001C11104F0900中速
3155AA2003021D04000024002682200 1F98220064DC12027C200107EC3001C11102F0D00高速

图 80 射频码

空调射频码为[金山文档] 射频码.txt

14.3.3. 实验过程

控制开关：

1. 将红外接收器连接到网关上，进入网关可视化编码，拉入节点

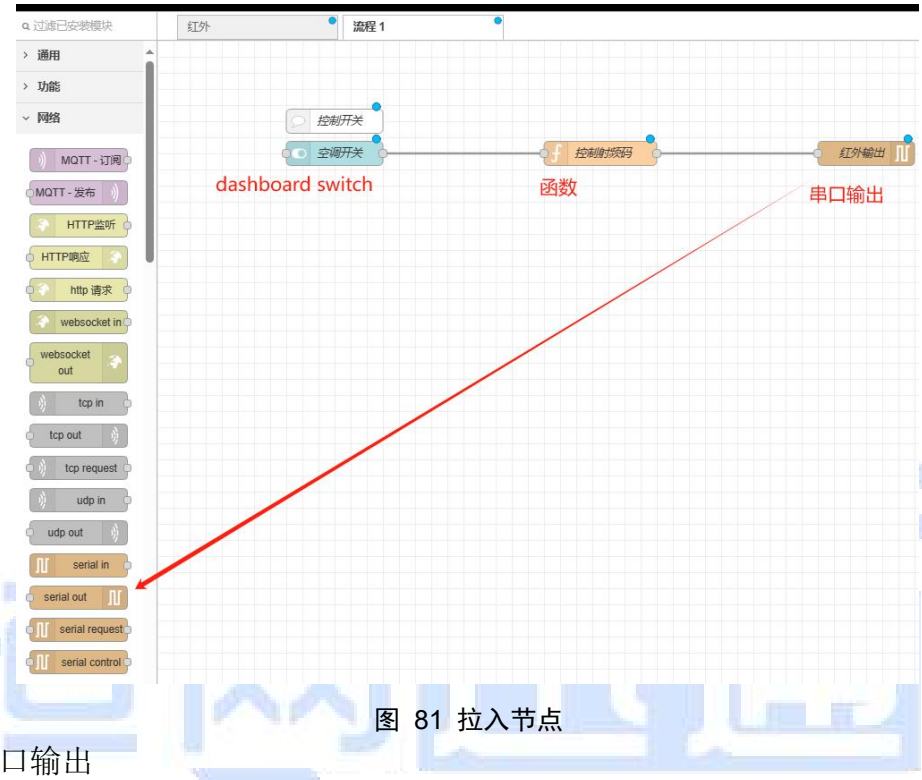


图 81 拉入节点

2. 配置串口输出

图 82 配置

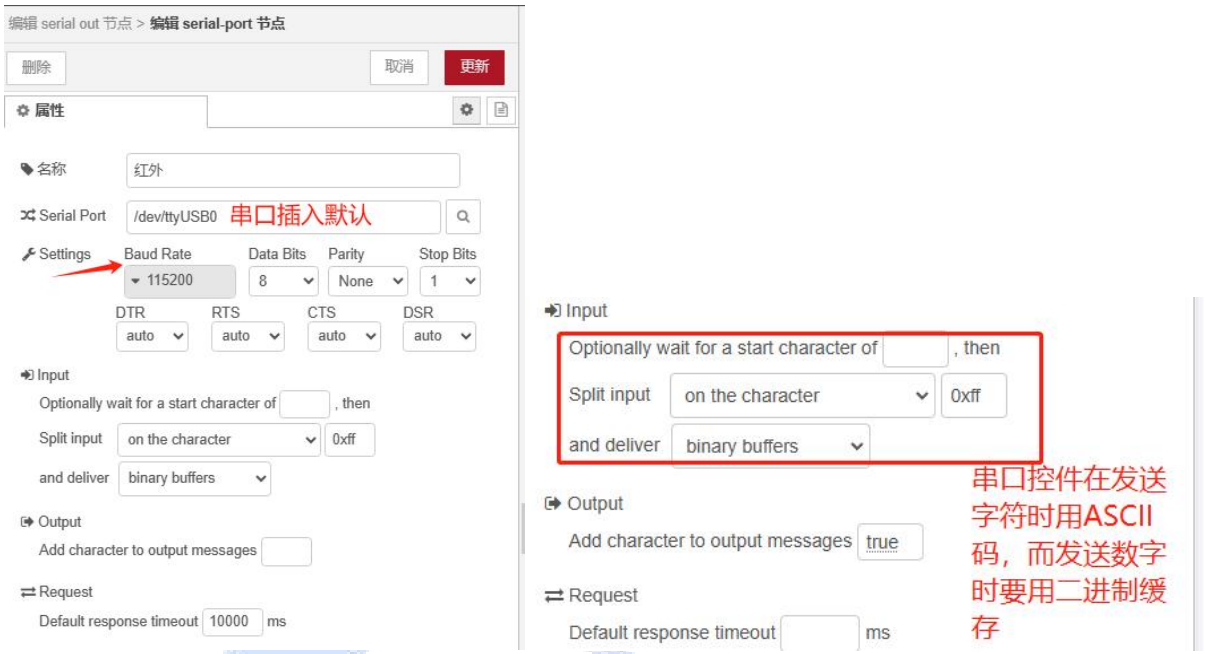


图 83 配置串口输出

3. 配置 dashboard switch 节点

首先创建组和组员。

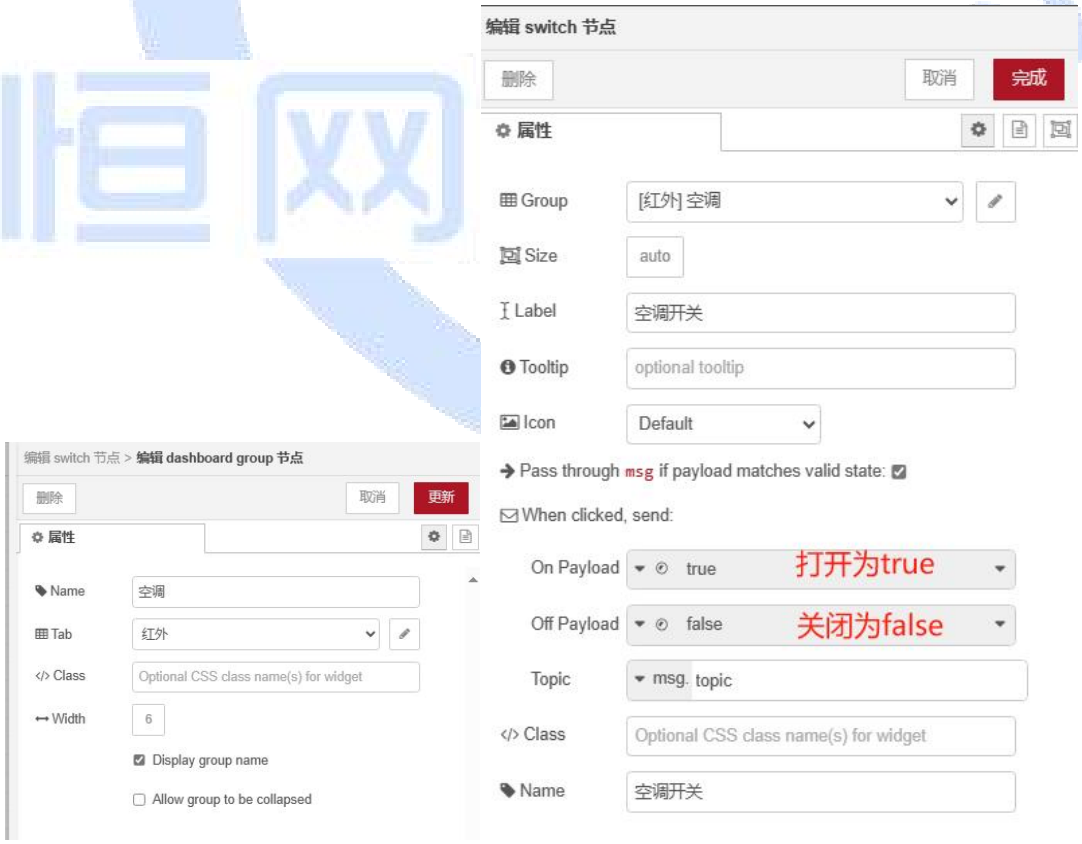


图 84 设计组态控制

4. 配置开关函数

判断条件:

if (msg.payload === true): 检查 msg.payload 是否严格等于 true。
else if(msg.payload===false): 如果 msg.payload 不是 true，则检查它是否严格等于 false。
设置新的 msg.payload:
如果 msg.payload 是 true，则将变量 open 的值（一个十六进制字符串）转换为 Buffer 对象，并赋值给 msg.payload。
如果 msg.payload 是 false，则将变量 close 的值（另一个十六进制字符串）转换为 Buffer 对象，并赋值给 msg.payload。

Buffer.from()方法:
Buffer.from(string, encoding): 这是 Node.js 中 Buffer 类的一个静态方法，用于创建一个新的 Buffer 实例，其内容由指定的字符串和编码组成。在这里，字符串是 open 或 close，编码是'hex'，意味着字符串应该被解释为十六进制数。
返回值:
最后，函数返回更新后的 msg 对象。

编辑 函数计算 节点 > JavaScript 编辑器

```
if (msg.payload === true)
{
    var open = '3155AA2003021D040000240026821F01F98220064CC12036C2001078C3001C11002E0D00';
    msg.payload = Buffer.from(open, 'hex');

}

else if(msg.payload===false)
{
    var close = '3155AA2003021D040000240026821E01FA821E064CC12019C2001079C3001C1103A00800';
    msg.payload = Buffer.from(close, 'hex');

}


return msg
```

图 85 开关函数

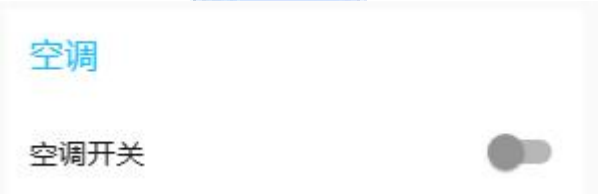


图 86 空调开关

控制模式：

1）拉入节点，进行连线

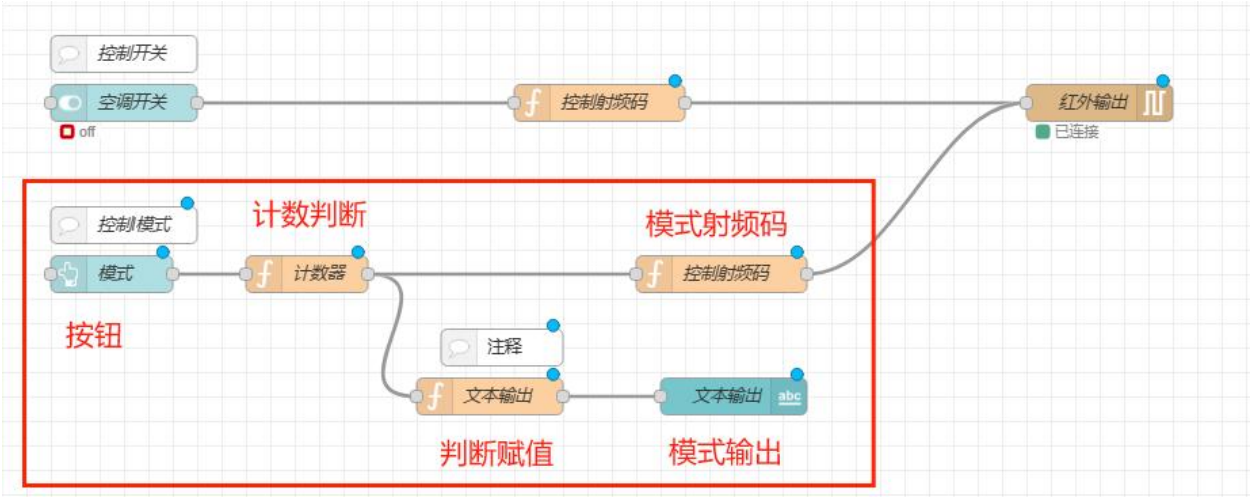


图 87 制冷制热切换

2) 按钮：作用是进行从触发

图 88 切换按钮

函数（计数器）：作用是当按钮按一次加 1

函数解释：我们每次点击按钮时都会增加 count 的值。如果 count 的值超过了 2，我们就将其重置为 1。这样，无论按钮被点击多少次，count 的值都将在 1 到 2 之间循环。

利用上下文（context）和流程（flow）中保存这个计数器的值，并在消息

（msg）中传递更新后的值，具有保存和检索的作用。

图 89 计数器(函数)

3) 模式射频码

同理开关函数射频码进行设置。

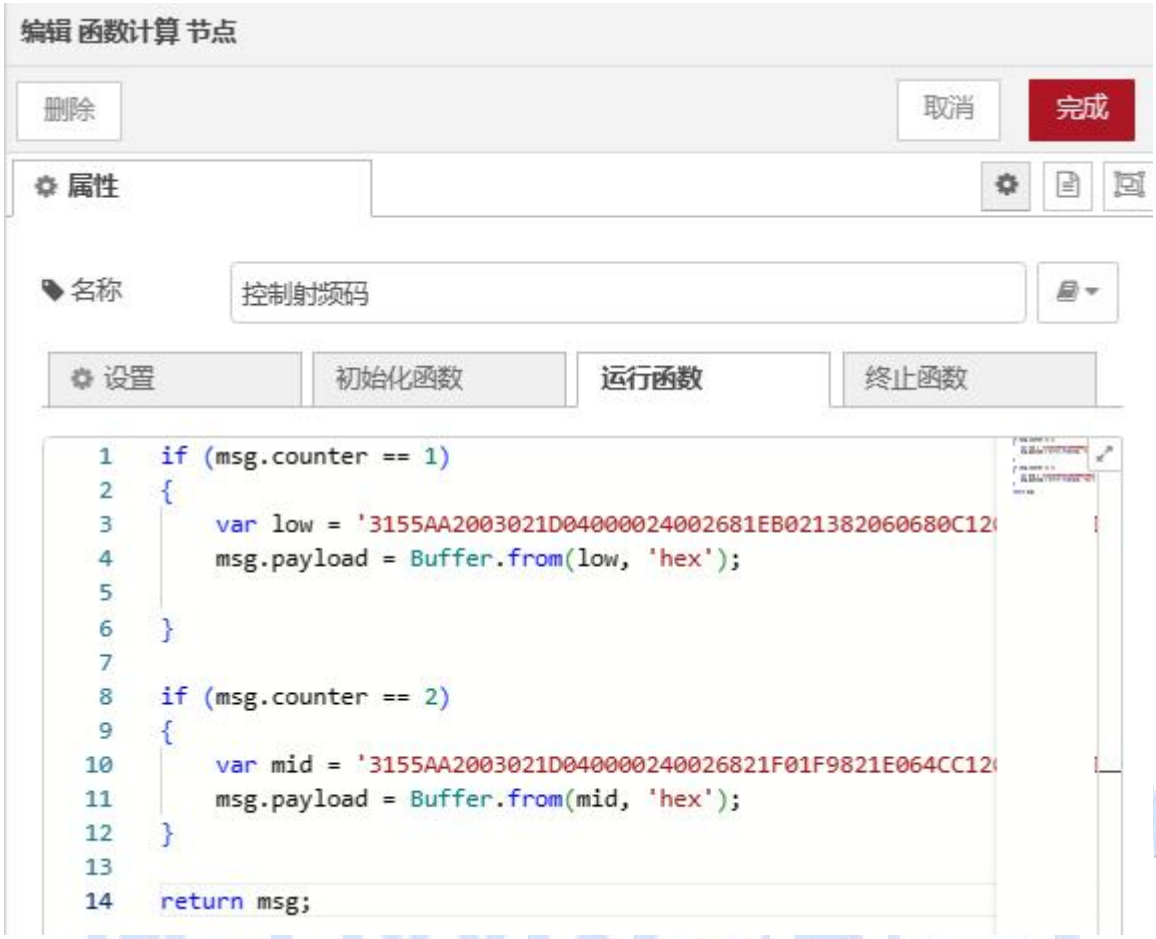


图 90 空调模式射频码（函数）

4) 文本输出，当按钮按一次，计数器计数加 1，msg.payload 赋值为制冷，反之为制热

图 91 输出

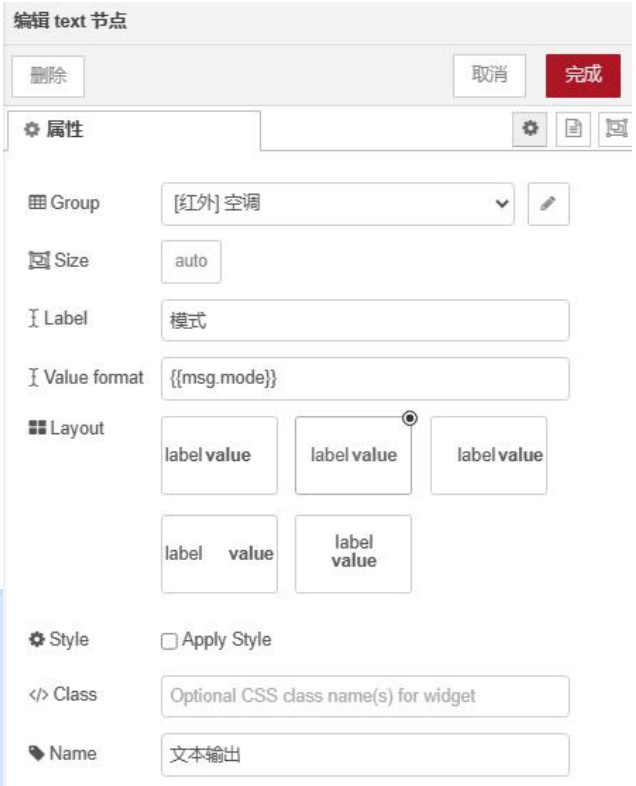


图 92 输出

进入 UI 组态测试

图 93 测试

控制温度

利用按钮调节温度，利用计步器设定，text 节点输出，函数射频码发送 hex 射频码，串口输出。

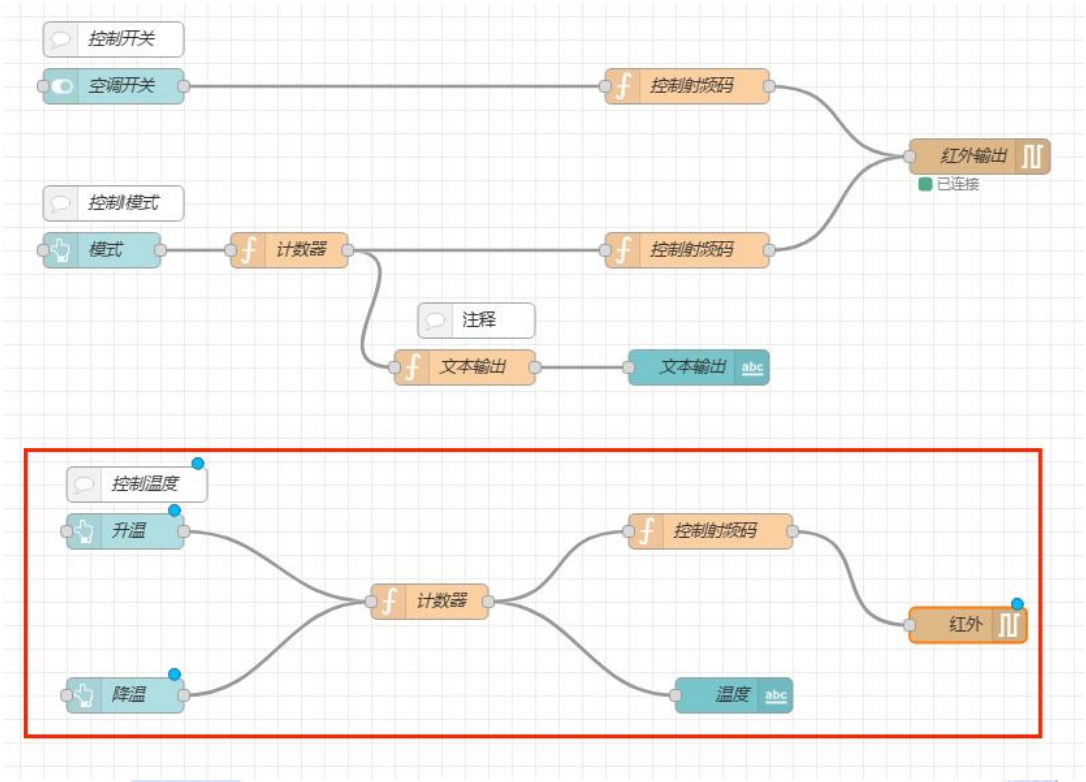


图 94 节点设计

1. 按钮：

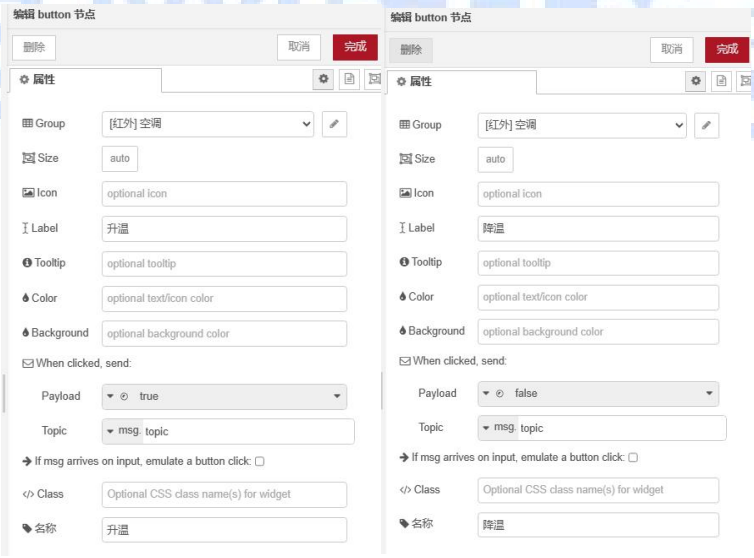


图 95 控制温度输出

2. 计数器函数：首先初始化 count 值为 18，判断如果按下按钮就执行，增加减少的范围是[18，30],利用上下文（context）和流程（flow）中保存这个计数器的值。

图 96 添加计数器

3. 射频码函数：创建数组，把记录到的各个温度射频码保存到数值内，例如（如果计数器计数到 22，函数数组内有计数器的值，并将数组内 22 的值（射频码）赋值到 msg.payload 发送射频码）。如果检查到计数器没有数组内的值，则报错。

```
// 假设有一个对象来存储不同模式和计数值对应的十六进制字符串
    const payloadMap =
    {
            18: '3155AA2003021D0400002400268221101F88221064CC12024C200107DC3001C11102C0F00',
            19: '3155AA2003021D0400002400268221001F98220064DC1203FC200107DC3001C1110220000',
            20: '3155AA2003021D0400002400268221001F9821F064EC12037C200107DC3001C11102A0800',
            21: '3155AA2003021D0400002400268221001F98220064DC1203EC200107DC3001C1110260400',
            22: '3155AA2003021D0400002400268221101F98220064CC1203DC200107CC3001C11102E0C00',
            23: '3155AA2003021D0400002400268221001F98220064DC1203FC200107DC3001C1110210200',
            24: '3155AA2003021D0400002400268221001FA8220064EC1203FC200107EC3001C1110290A00',
            25: '3155AA2003021D0400002400268221001FA8220064DC12025C200107DC3001C1110250600',
            26: '3155AA2003021D0400002400268221001FA8220064DC12025C200107DC3001C11102D0E00',
            27: '3155AA2003021D0400002400268221EB01FA8221067FC12008C200107FC3001C1110230100',
            28: '3155AA2003021D0400002400268221F02138206064FC1203FC2001097C3001C11102B0900',
            29: '3155AA2003021D0400002400268221001FA8220064DC1203DC200107DC3001C1110270500',
            30: '3155AA2003021D0400002400268221E01FB8220064DC1203CC200107DC3001C11102F0D00'
            // ... 其他制热模式下的计数值和对应的十六进制字符串
    };
    // 检查msg.count，并设置msg.payload
    if (msg.count in payloadMap)
        {
            msg.payload = Buffer.from(payloadMap[msg.count], 'hex');
        }
        else
        {
            // 处理msg.count未找到对应值的情况，例如记录日志或发送错误消息
            console.error(`未找到{msg.count}的对应payload`);
        }
    // 返回更新后的msg对象，以便在后续流程中使用
    return msg;
```

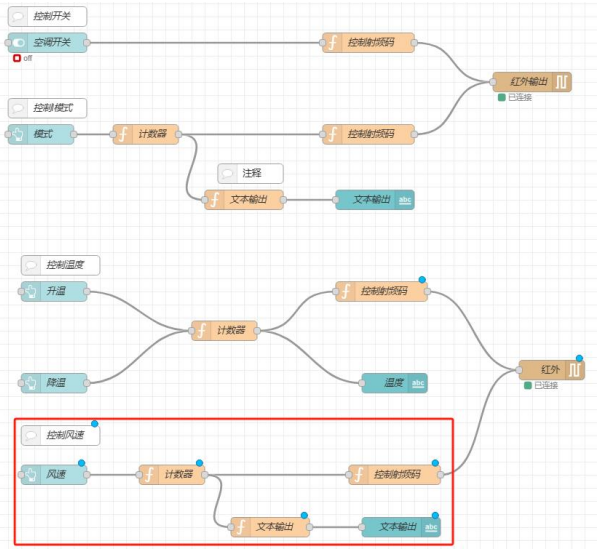
图 97 发送 hex

4. 文本输出

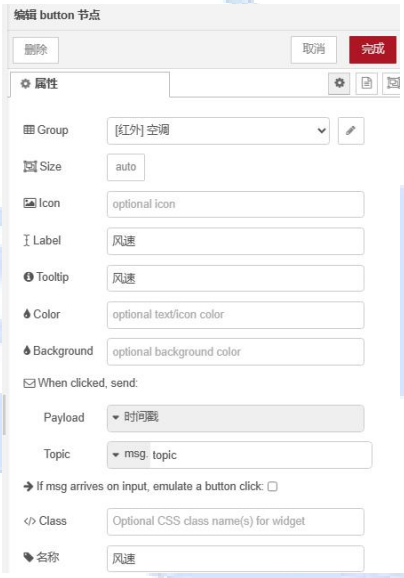
图 98 组态呈现

控制风速：

1) 与控制模式相似，拉入节点



2) 按钮：做为触发动作，改变风速大小



3) 函数（计数器）：作用是当按钮按一次加 1

函数解释：我们每次点击按钮时都会增加 count 的值。如果 count 的值超过了 3，我们就将其重置为 1。这样，无论按钮被点击多少次，count 的值都将在 1 到 3 之间循环。

利用上下文（context）和流程（flow）中保存这个计数器的值，并在消息（msg）中传递更新后的值

var count = context.get('count') || 0，这行代码的目的是尝试从 context 对

象中获取'count'属性的值，并将这个值赋给 count 变量。如果'count'属性的值不存在或是一个“falsy”值（即 0），那么 count 变量就会被初始化为 0。

图 99 控制风速操作

4) 控制射频码（函数）：计数器计数，控制风速。发送射频

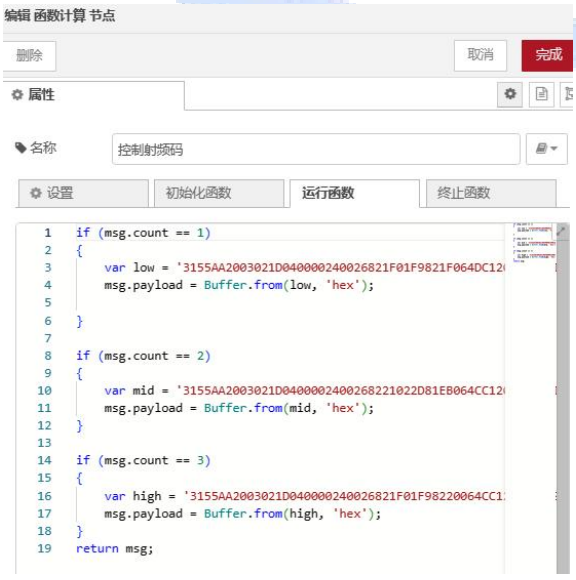


图 100 发送射频

5) 文本输入和输出

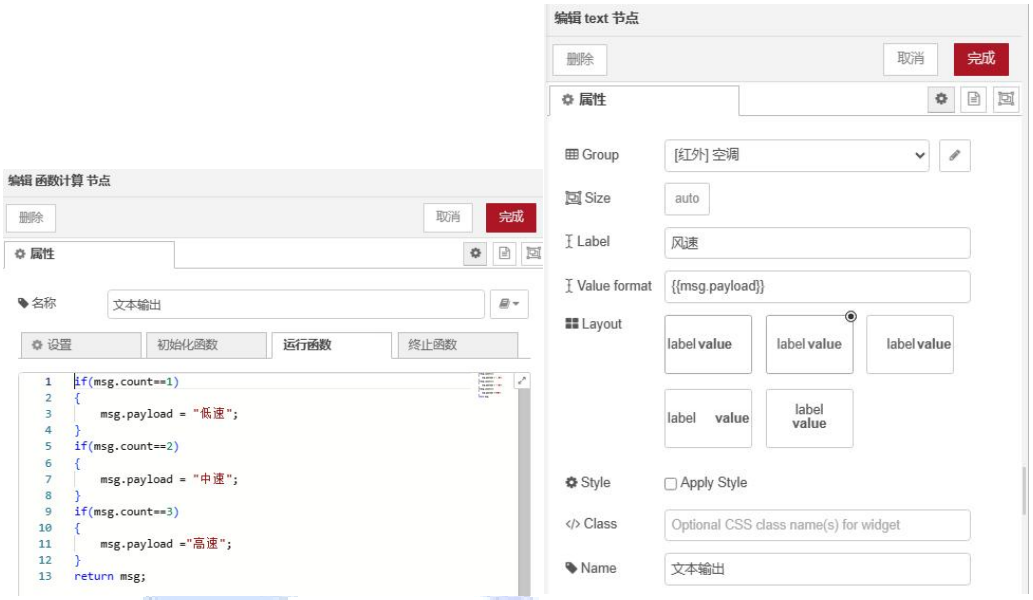


图 101 判断赋值函数

图 102 设计成果

14.3.4. 实验总结

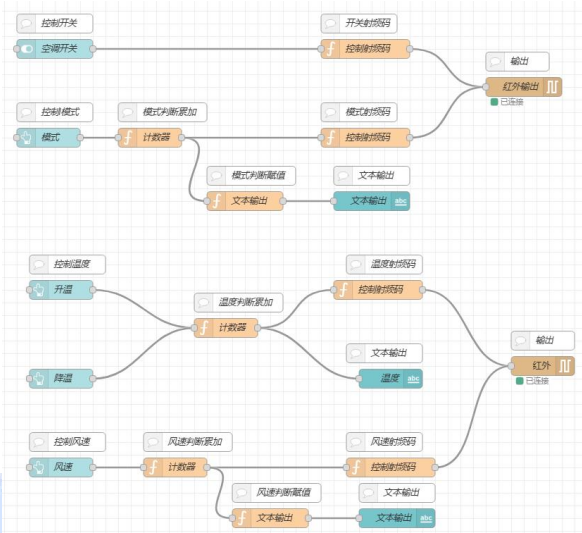


图 103 最终设计

利用红外接收器接收射频码，并通过网关实现组态控制的过程，涉及硬件和软件两个层面的操作。以下是整个过程的总结：

硬件层面：

红外接收器设置：首先，需要确保红外接收器能够正确接收射频码。这通常涉及到将红外接收器对准射频信号源，并调整其接收灵敏度，以确保能够准确捕获射频码。

射频码记录：一旦红外接收器成功接收到射频码，就需要将这些射频码记录下来。这可以通过串口助手等工具，将红外接收器连接至电脑，并通过串口通信将接收到的射频码数据传输到电脑上进行保存。

红外接收器与网关连接：接下来，将红外接收器接入网关。这通常需要使用适当的连接线将两者连接在一起，确保连接稳定且可靠。

软件层面：

网关可视化编程：进入网关的可视化编程环境，这是实现组态控制的关键步骤。可视化编程环境提供了直观易用的界面，方便用户进行各种配置和编程操作。

串行通信配置：在网关的可视化编程环境中，需要配置串行通信参数，以便与红外接收器进行通信。这些参数通常包括波特率、数据位、停止位等，需要确保与红外接收器的设置相匹配。

组态设计：利用网关的可视化编程工具，进行组态设计。这包括创建控制界面、配置控制逻辑、设置触发条件等。通过组态设计，可以实现对红外接收器的远程控制，以及基于射频码的自动化控制逻辑。

调试与测试：完成组态设计后，需要进行调试和测试。这包括检查通信是否正常、控制逻辑是否正确、射频码是否能够被正确识别和执行等。通过不断的调试和优化，确保整个系统的稳定性和可靠性。

整个过程实现了从硬件层面的红外接收和射频码记录，到软件层面的网关可视化编程和组态设计的完整流程。通过这种方法，可以方便地将红外接收器接入网关，实现基于射频码的组态控制。这不仅提高了控制的灵活性和自动化程度，也降低了人工操作的复杂性和错误率。同时，可视化编程和组态设计的方式使得整个过程更加直观和易于理解，降低了技术门槛，使得更多用户能够轻松地进行相关操作和应用。**最终 json 源码。**[金山文档] 红

外(1).json

14.4. Node-RED 实现西门子 S7 PLC 的数据采集及反控

14.4.1. 实验介绍

Node-RED 是一个基于流的编程工具，可方便地连接硬件设备、API 和在线服务。对于西门子 S7 PLC 的数据采集，Node-RED 提供了 node-red-contrib-s7 节点，支持直接读取和写入 PLC 数据。通过简单的配置和拖拽操作，用户能轻松实现 PLC 数据的实时采集、监控和处理，为工业自动化提供灵活且高效的解决方案。Node-RED 的可视化编程界面使得数据采集过程直观易懂，大大提高了工作效率

14.4.2. 实验准备

本次实验使用的是西门子 S7-200 SMART

S7 通信准备工作：
只可以使用没有优化的数据块，可以通过 TIA-Portal 在数据块的属性中进行设置。 请确保没有勾选该选项。

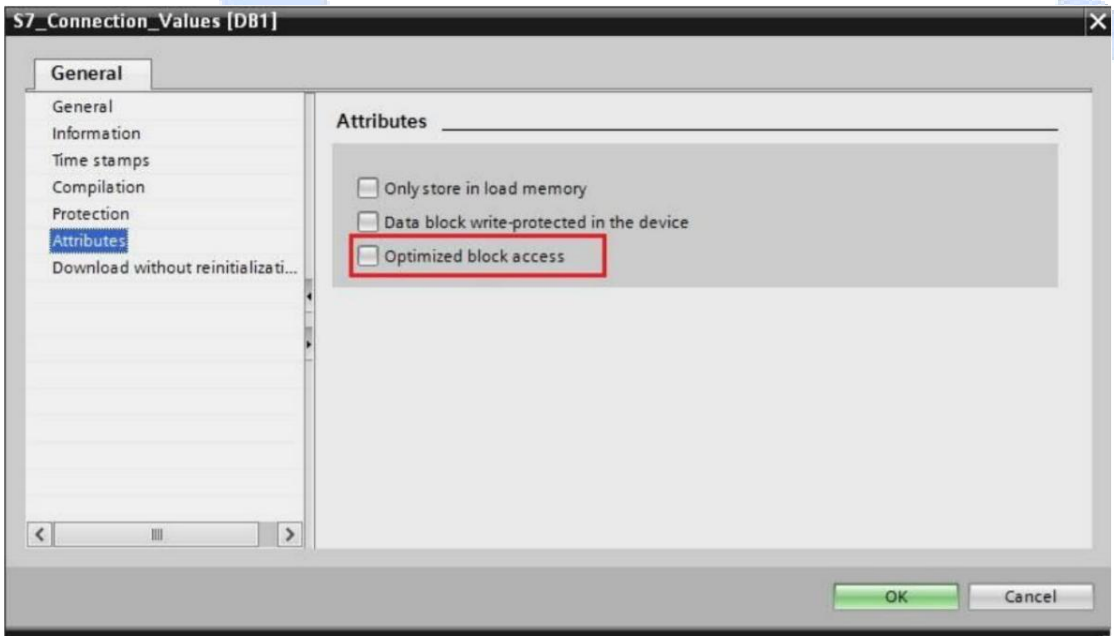


图 104 通信准备

1. 必须在 S7-300/400/1200/1500 PLC CPU 配置中启用 PUT/GET 通信，请确保勾选该选项。

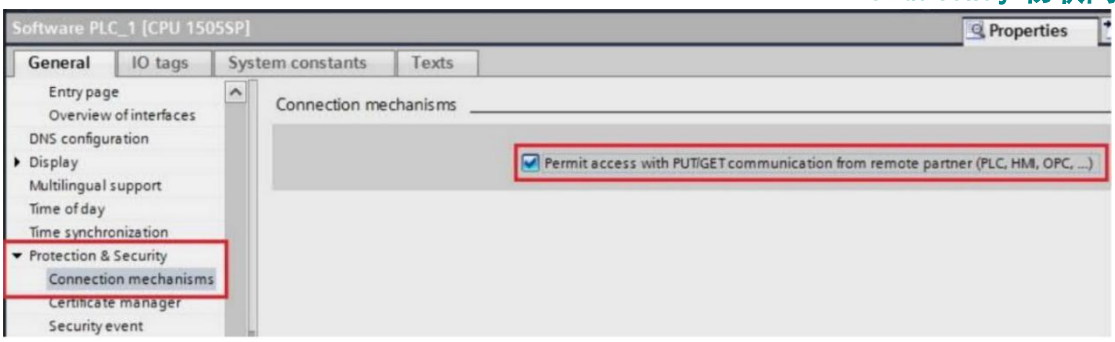


图 105 通信准备

14.4.3. 实验过程

1. 编辑 S7 in 节点

利用“s7 in”节点可以基于 S7 协议从 S7 PLC 读取数据。

在左侧节点栏中选择“s7 in”节点，拖动添加至编辑区域

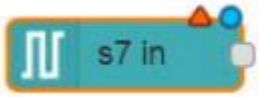


图 106 S7 读变量

2. 双击“s7 in” 节点。将 Mode 选为 All variables。点击图中图标编辑 S7 端点。

图 107 节点说明

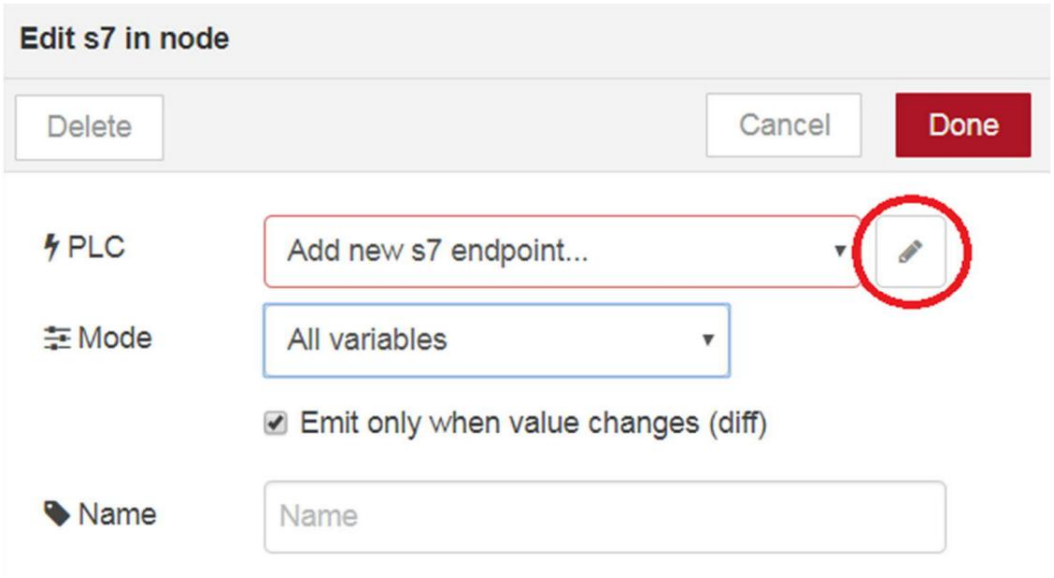


图 108 PLC 配置

3. 在通信节点配置框内，S7200 SMART PLC 的配置与 S7-1200 配置稍有不同，这里需要按照图示配置即可进行通信

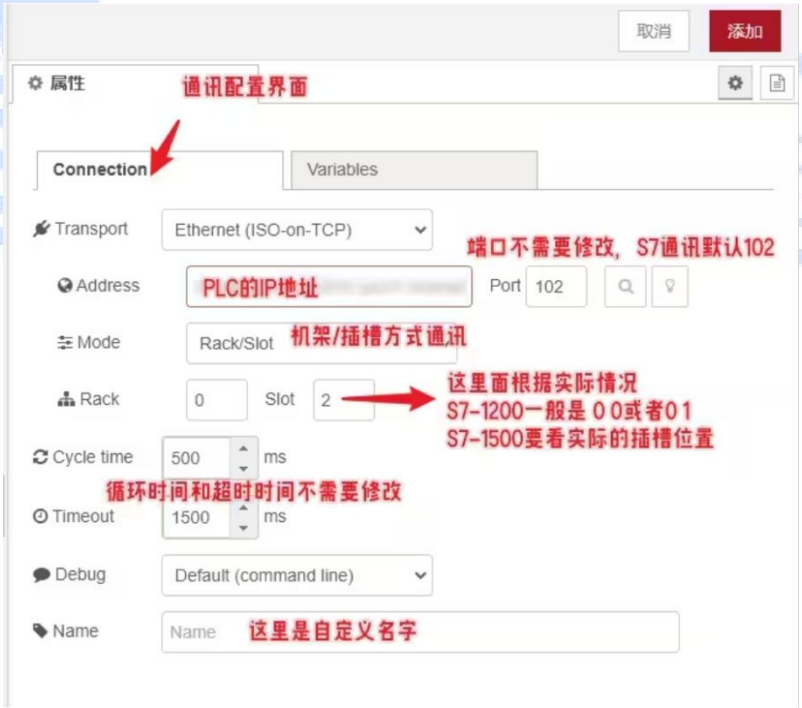


图 109 槽口 机架模式通信

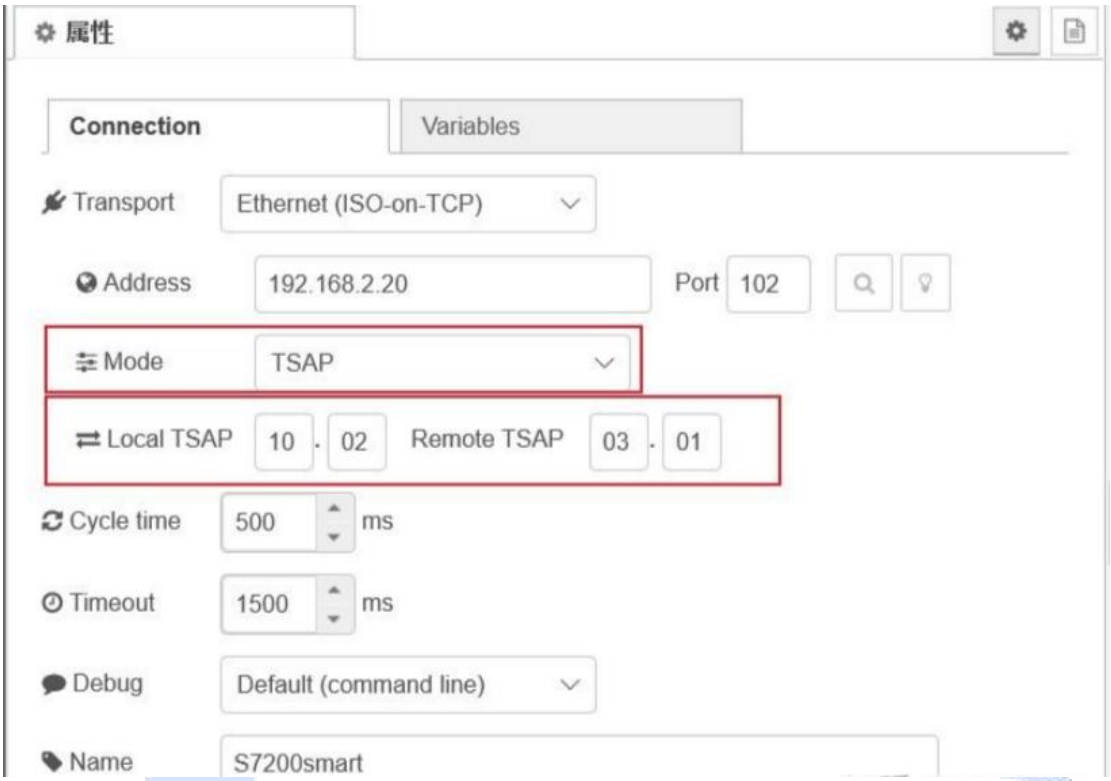


图 110 S7200 SMART PLC 的配置

如果您使用的不是 S7200 SMART PLC（因为 S7200 的 PLC 版本过低，不能用 Rack/Slot 的模式去通信），使用其它版本的 PLC 的话具体需要看产商的使用手册来填写。
S7 端点的连接信息包括 S7 PLC 的 IP 地址、端口号、机架号、槽号、读取周期等信息。其中，默认的端口号是 102。不同的 S7 PLC，槽号也不同。

4. S7 端点的变量信息包括变量的寻址方式以及变量名称。使用“+Add”按钮来添加新的变量，“Export”按钮可以将变量列表导出至 .csv 文件中，“Import”按钮可以通过 .csv 文件导入变量列表

图 111 添加 PLC 变量

5. S7 PLC 数据寻址方式及 Node-red 对应寻址方式

下表给出了 S7 PLC 中数据寻址方式及 Node-red 对应寻址方式。

（1）S7-200 Smart

	数据区域	数据类型	PLC 寻址方式	Node-red 寻址方式
PLC 变量	I	BOOL	Ix.y	Ix.y
		BYTE	IBx	IBx
		WORD,INT	IWx	IWx
		DWORD,DINT	IDx	IDx
	Q	BOOL	Qx.y	Qx.y
		BYTE	QBx	QBx
		WORD,INT	QWx	QWx
		DWORD,DINT	QDx	QDx
	M	BOOL	Mx.y	Mx.y
		BYTE	MBx	MBx
		WORD,INT	MWx	MWx
		DWORD,DINT	MDx	MDx
	T			
	C			
	SM			
	HC			
	L			
	S			
用户数据块	DB(V)	DWORD	DBn VDx	DBn,DWORDx
		DINT	DBn VDx	DBn,DINTx
		INT	DBn VWx	DBn,INTx
		WORD	DBn VWx	DBn,WORDx
		STRING or S	DBn VBx	DBn,B(x-1).length
		BYTE or B	DBn VBx	DBn,Bx
用户数据块	DB(V)	BOOL		DBn,Xx.y

表 5 S7-200 Smart 寻址说明

说明：

n 为数据区域偏移量

x 为数据字节偏移量
y 为数据位偏移量
确保 STRING 或“S”的数据字节偏移量非 0，length 替换为字符串长度

（2）S7-300/400/1200/1500

	数据区域	数据类型	PLC 寻址方式	Node-red 寻址方式
PLC 变量	I	BOOL	Ix.y	Ix.y
		BYTE,CHAR	IBx	IBx
		WORD,INT	IWx	IWx
		DWORD,DINT, REAL	IDx	IDx
	Q	BOOL	Qx.y	Qx.y
		BYTE,CHAR	QBx	QBx
		WORD,INT	QWx	QWx
		DWORD,DINT, REAL	QDx	QDx
	M	BOOL	Mx.y	Mx.y
		BYTE,CHAR	MBx	MBx
		WORD,INT	MWx	MWx
		DWORD,DINT, ,REAL	MDx	MDx
	T	TIMER (only support S7-300/400)	Tn	
	C	COUNTER (only support S7-300/400)	Cn	
用户数据块	DB	REAL		DBn,REALx
		DWORD		DBn,DWORDx
		DINT		DBn,DINTx
		INT		DBn,INTx
		WORD		DBn,WORDx
		BYTE or B		DBn,Bx
		STRING or S		DBn,S (x -1).length
		CHAR or C		DBn,Cx
用户数据块	DB(V)	BOOL		DBn,Xx.y

表 6 S7-300/400/1200/1500 寻址说明

说明：
n 为数据区域偏移量
x 为数据字节偏移量
y 为数据位偏移量
确保 STRING 或“S”的数据字节偏移量非 0，length 替换为字符串长度

6. 实例读取数据（s7200 smart）

图 112 PLC 变量

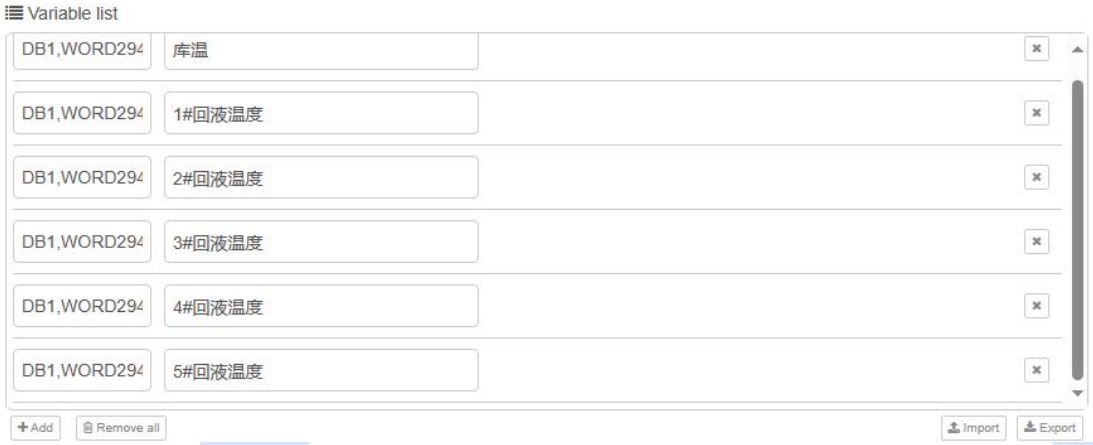


图 113 node-red 变量寻址

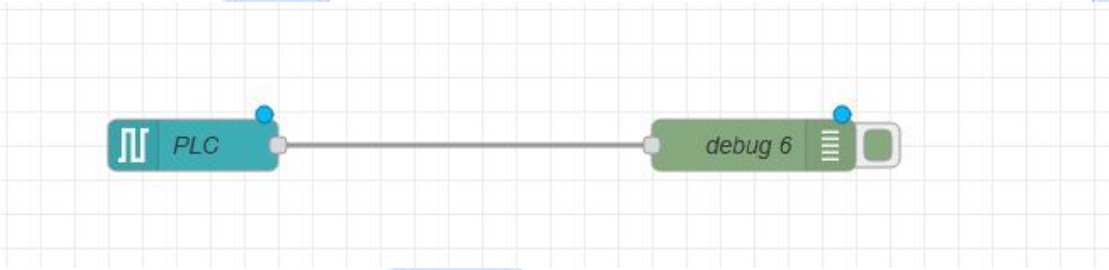


图 114 拖入 debug 输出

图 115 正常读取温度

7. 实例控制开关（s7200 smart）

控制需要用到 S7 out 实现，写变量是将要写的值通过 msg.payload 写到对应的变量中.
在左侧节点栏中选择 “s7 out” 节点，拖动添加至编辑区域。

图 116 输出节点

双击 “s7 out” 节点，编辑节点。

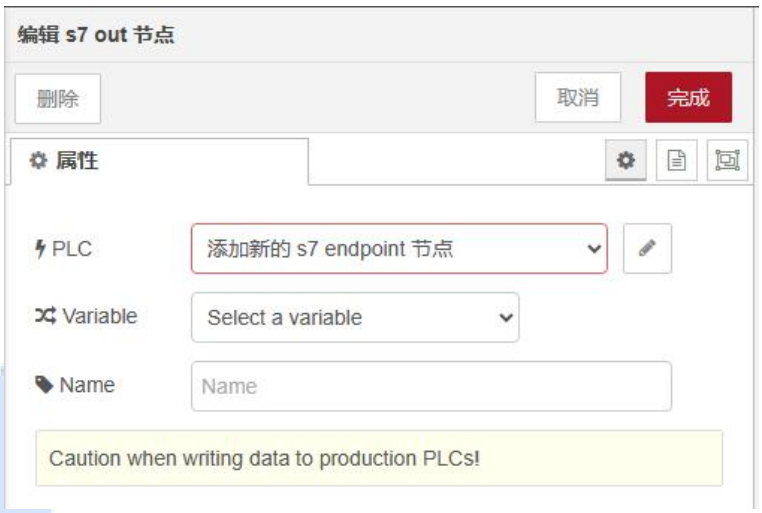


图 117 编辑节点

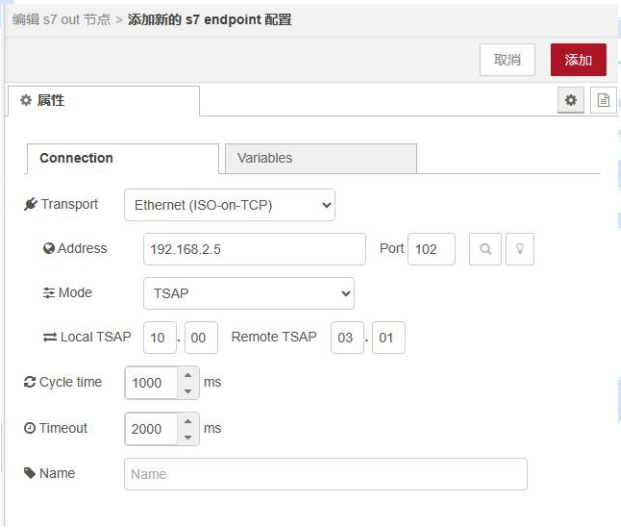


图 118 配置

添加可读写的变量，可以选择时间戳设置成布尔值，接上 S7 out。

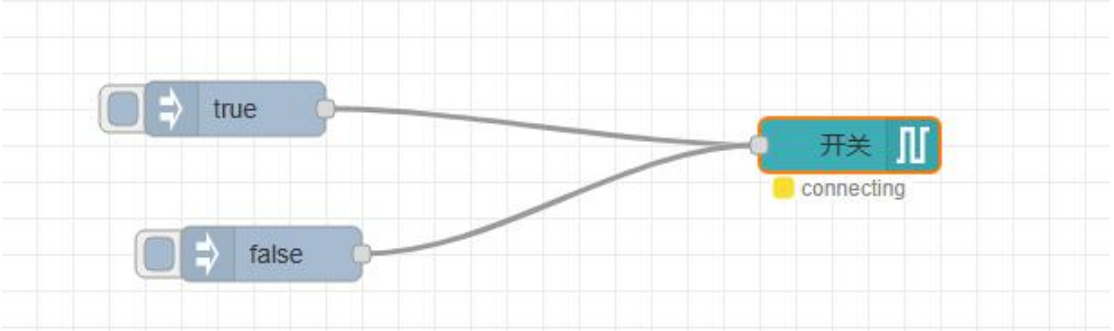


图 119 控制

实现的效果是：当点击注入时，能改变变量的值，从而控制设备开或关。当变量被改变时，S7 in 能输出变量的值，可以观察变量值是否改变。

14.4.4. 实验总结

Node-RED 在实现西门子 S7 PLC 的数据采集及反控方面提供了强大的支持。用户能够轻松地与 S7 PLC 进行通信，实现数据的实时读取和写入。Node-RED 的流编程方式使得整个数据采集和反控过程变得直观且易于管理。用户只需通过简单的拖拽和配置操作，即可构建出复杂的数据流，实现 PLC 与其他系统或服务之间的交互。此外，Node-RED 的可视化界面大大提高了工作效率，使得用户能够快速地监控和调整数据采集和反控过程。总之，Node-RED 为西门子 S7 PLC 的数据采集及反控提供了灵活、高效且易于使用的解决方案。

14.5. 采集 modbus 设备数据到 mqtt 服务器，APP inventor 通过 mqtt 进行采集与反控

14.5.1. 实验介绍

Node-RED 是一个流编程工具，能够方便地连接硬件设备、API 和服务。当需要采集 Modbus 设备的数据时，可以通过 Node-RED 中的 Modbus 节点来轻松实现。这些节点可以配置为从 Modbus 设备读取数据，并将这些数据发送到 MQTT 服务器。

MQTT 服务器作为消息代理，能够接收并转发这些来自 Modbus 设备的数据。同时，APP Inventor 作为强大的应用开发平台，能够利用 MQTT 协议与服务器进行通信，实时采集 Modbus 设备的数据。不仅如此，APP Inventor 还能通过 MQTT 向 MQTT 服务器发送控制指令，实现对 Modbus 设备的远程反控。

整个流程高效、便捷，不仅简化了 Modbus 设备与 APP 之间的通信，还提高了工业自动化系统的灵活性和可维护性。

14.5.2. 实验准备

（modbus 设备接入网关的串口为 com1）

1. 根据 9.1.2 实验准备配置连接好 modbus 设备，利用 RS485 转 USB 连接传感器和控制器到电脑，利用配置软件配置设备地址。最后得到：

传感器	设备地址	线圈/寄存器地址
湿度	1	0
温度	1	1
光照度	2	6
PM2.5	3	0
PM10	3	1

表 7 传感器地址

4 路 IO 控制器	4	
ON:SOS	4	0
ON:风扇	4	1

表 8 执行器地址

2. 根据此文档搭建本地服务器[金山文档] 本地 MQTT 服务器的搭建.docx，内置数据库内创建客户端，示例(用户名:node-red 密码:123456)

图 120 创建客户端

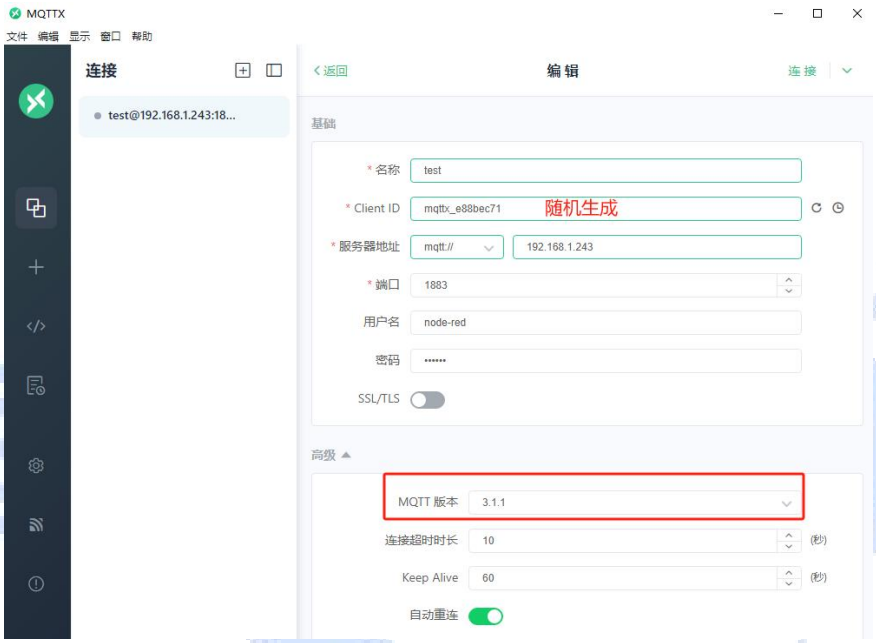


图 121 MQTTX 上创建主题

14.5.3. 实验过程

1. 将网关通过网线连接入路由器，打开智能网关配置工具搜索网关

图 122 网关配置工具

2．进入脚本配置，上传脚本，保存后重启。

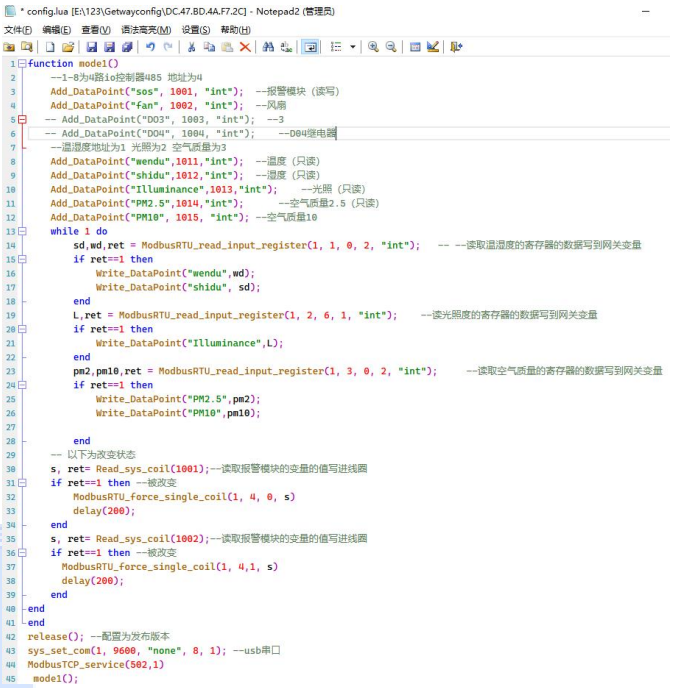


图 123 脚本

3．进入 Node-red，拉入 Modbus-read，和 debug，配置读寄存器。（对于监控系统来说，设置合适的 Poll Rate 非常重要。如果 **Poll Rate 设置得太低（即查询频率太高）**。相反，如果 Poll Rate 设置得太高（即查询频率太低），则可能会导致系统无法及时响应设备状态的变化或数据更新。如果“Poll Rate”被设置为 5 秒，那么这意味着系统每 5 秒钟会查询一次设备或传感器的状态或数据。(要在 debug 中判断系统给传感器赋值的数组是什么类型的)

传感器/执行器	Poll Rate	数组
温湿度	4s	msg.payload[i]
光照度	5s	msg.payload[i]
空气质量	6s	msg.payload[i]
执行器	7s	msg.payload[i]

图 124 读取数据

温湿度读取：

1. 拉入 Modbus Read 和调式

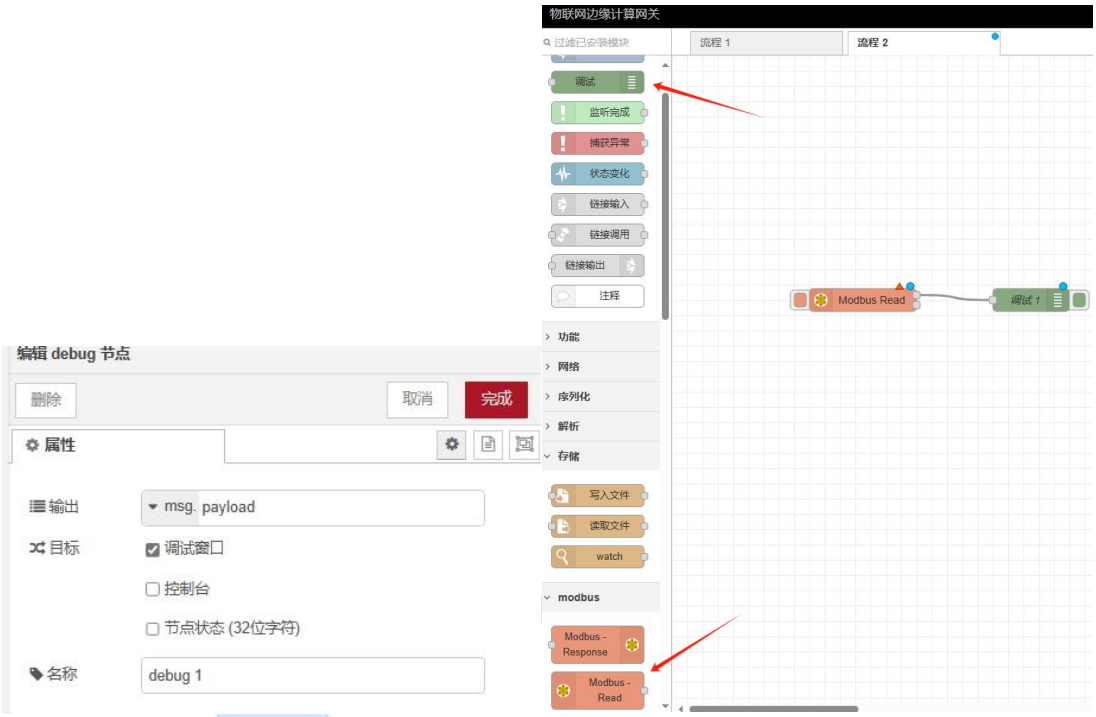


图 125 温湿度读取

2. 根据**表 4 温湿度**配置节点

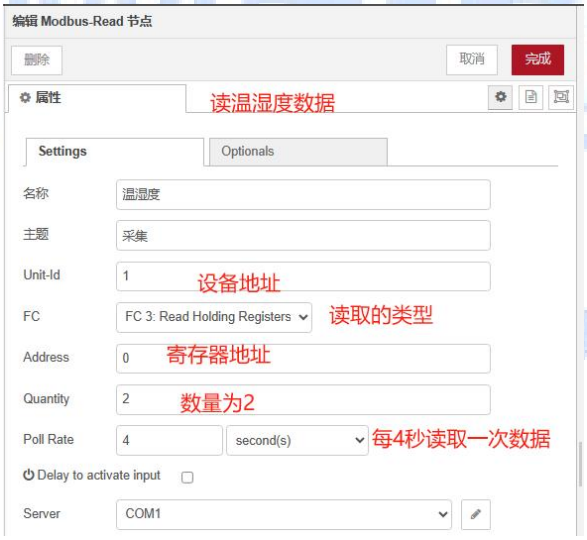


图 126 温湿度读取

3. 创建 Server

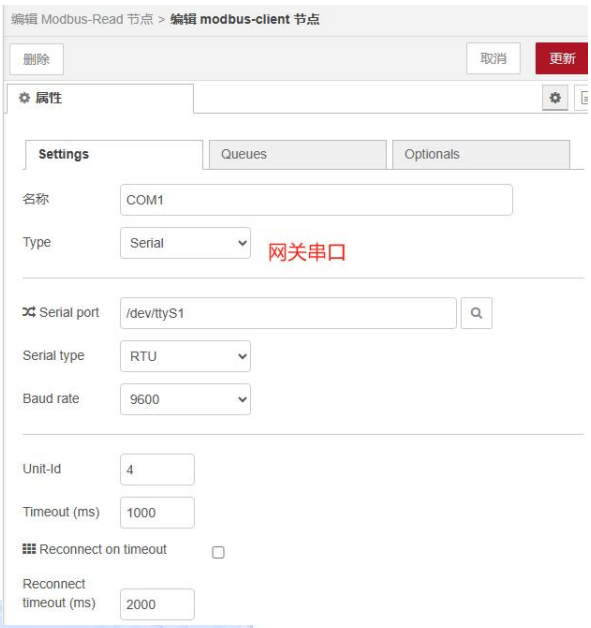


图 127 配置通信

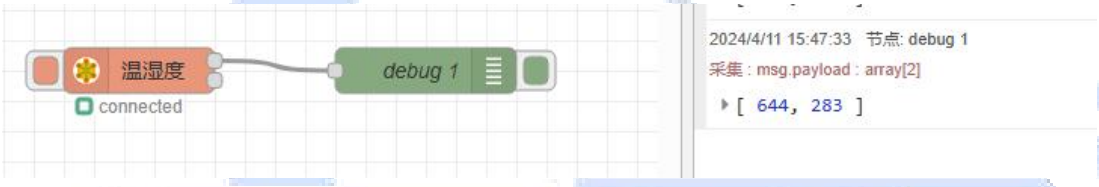


图 128 输出数值

4. 点击部署后，可以看到读取的数据都大了 10 倍，所以我们拉个 function 函数，并编辑数组内每个数都除以 10，并将数组内的温度，湿度进行赋值。。

图 129 数值除以 10

5. 同理，我们你可以在基础上加个单位。

图 130 添加单位

6. 部署后看输出

图 131 输出结果

7. 里面包含湿度和温度是值，我们要分别输出，以便后面 MQTT 读取数据。在功能里拉入 2 个逻辑输出。

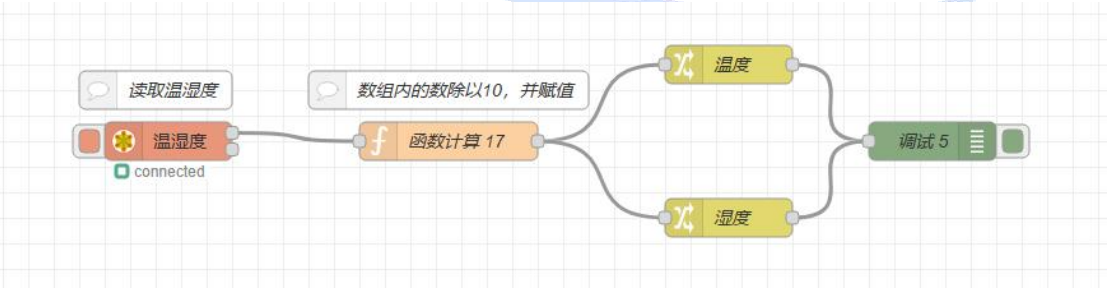


图 132 分别输出

图 133 温度输出

图 134 湿度输出

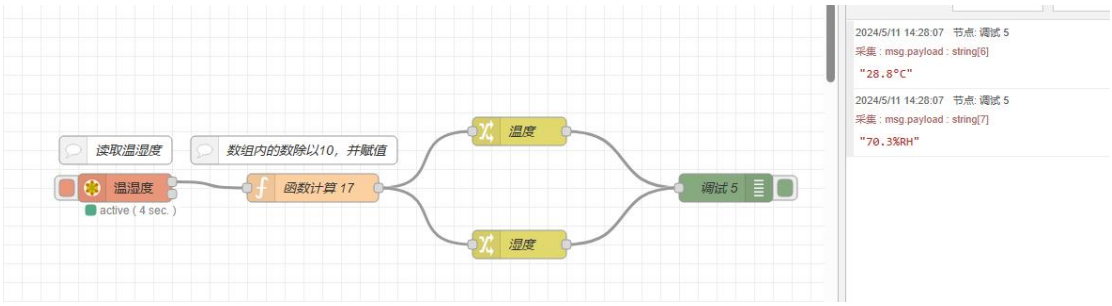
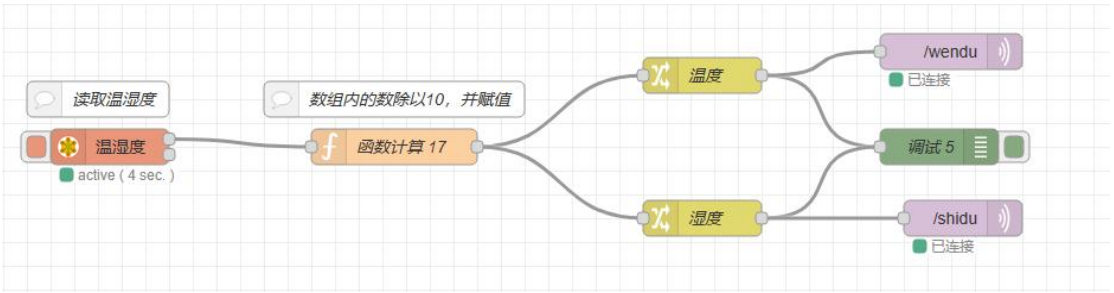


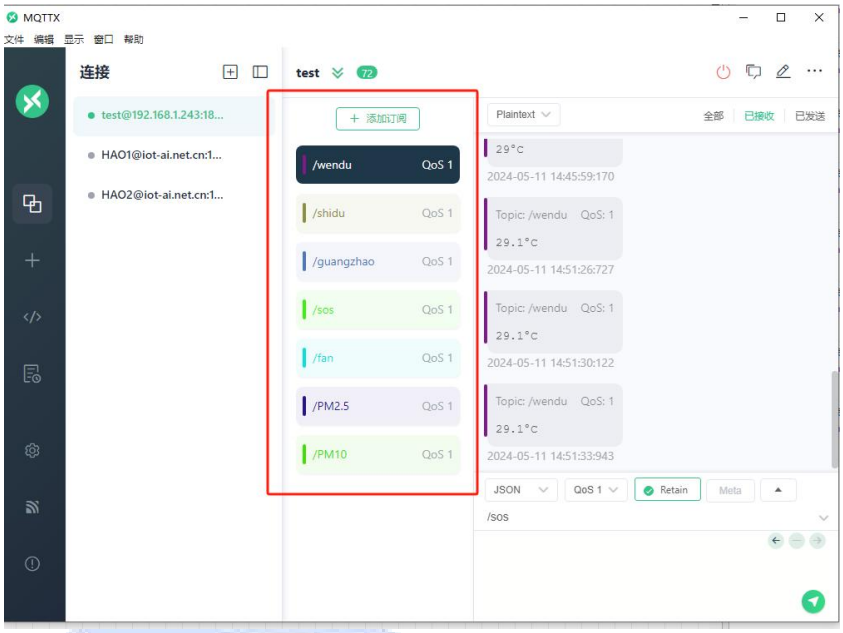
图 135 输出结果

8. 拉入 MQTT 节点，配置主题和服务器。



双击 MQTT -发布，编辑 mqtt-broker

9. 打开 MQTTX，创建主题，收到数据



10. 同理，读取其它传感器和执行器。并且逐个创建主题，创建完主题，依次拉入 MQTT 节点，配置 MQTT 节点。

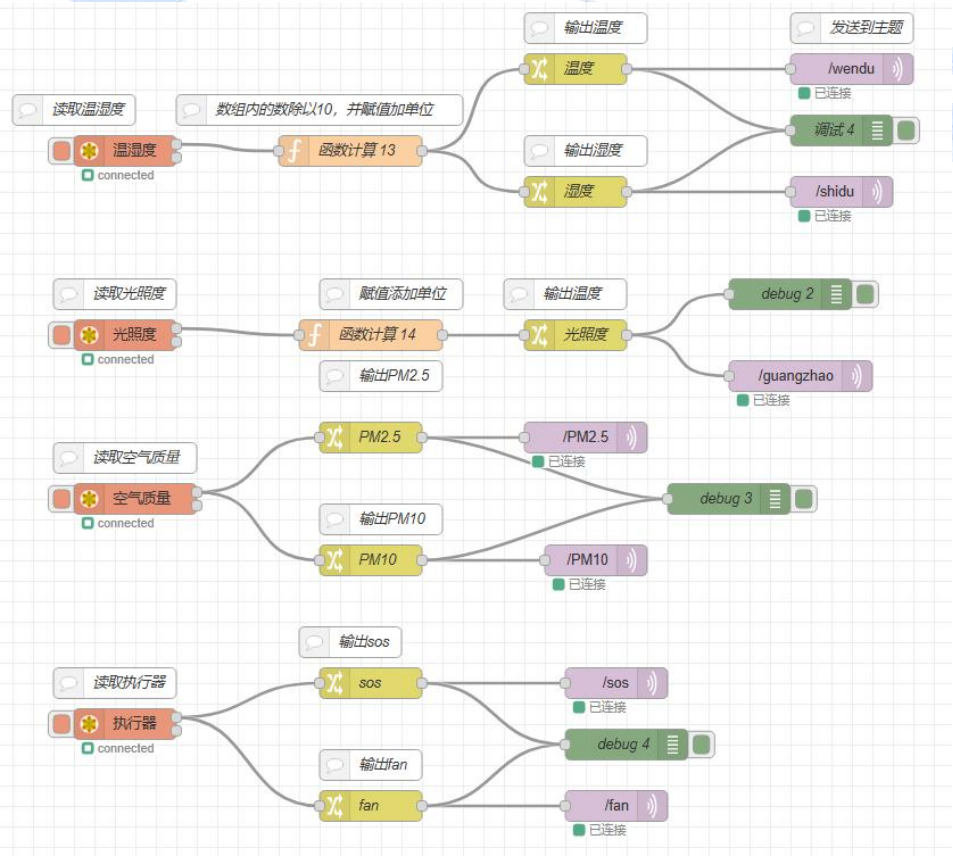


图 136 读取传感器和执行器

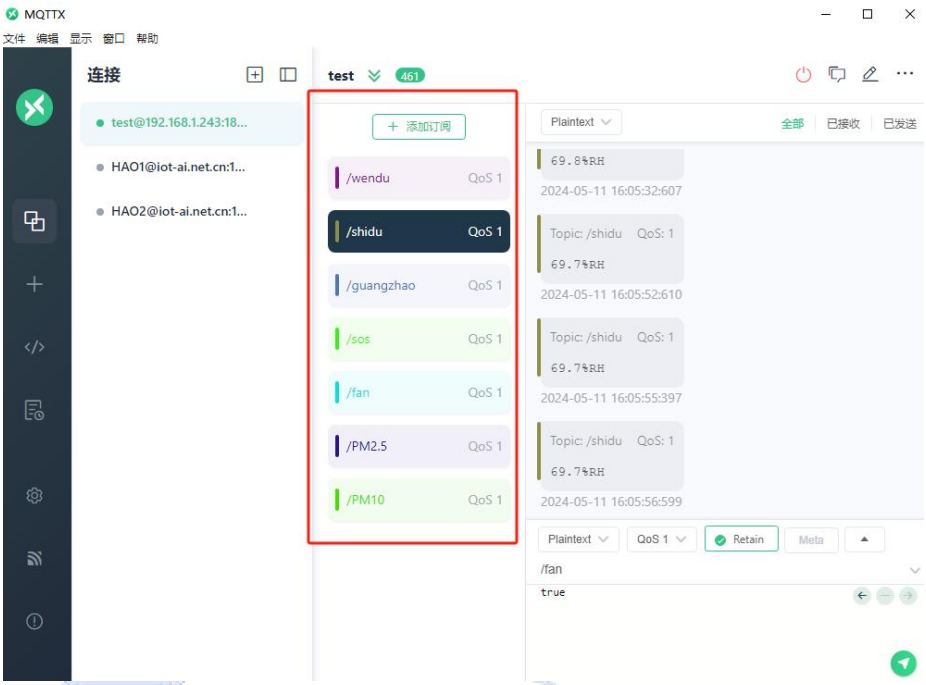


图 137 创建主题

11. 实现执行器反控，拉入 Modbus-Write 节点和 MQTT 订阅节点。（思路是：服务器发送消息给执行器的主题，执行器主题收到消息把消息传给 Modbus-write 节点实现控制）

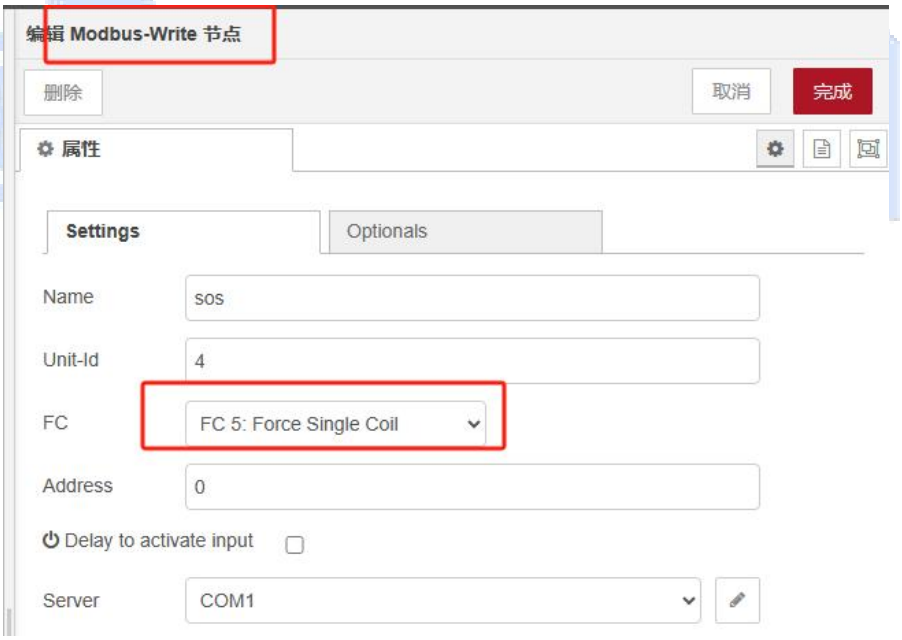


图 138 Modbus-Write 配置

图 139 反控节点

12. 最终设计

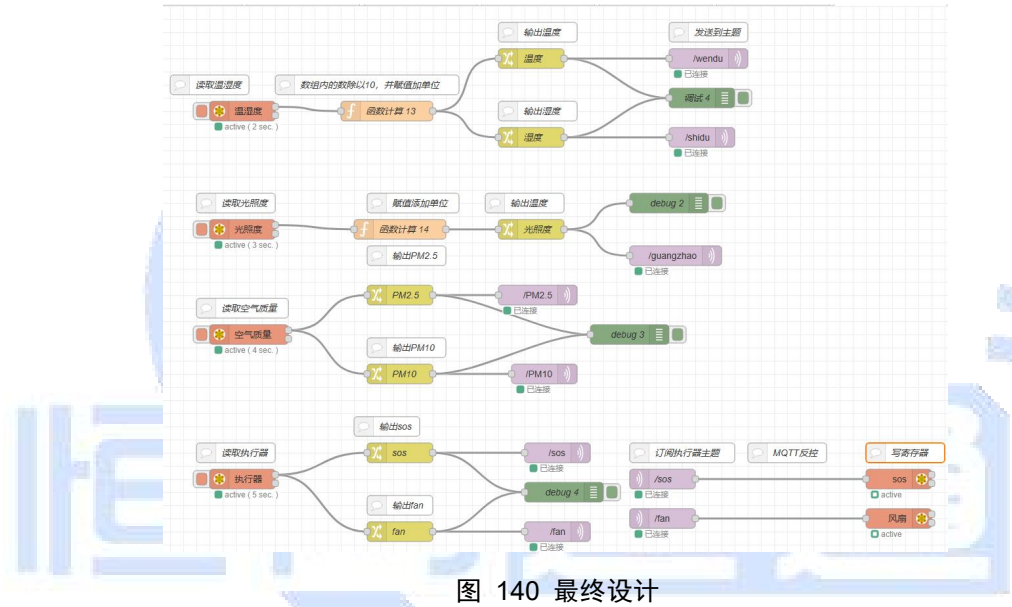


图 140 最终设计

最终 JSON 流[金山文档] MQTT.json

13. 进入 APP Inventor，创建项目，进行设计

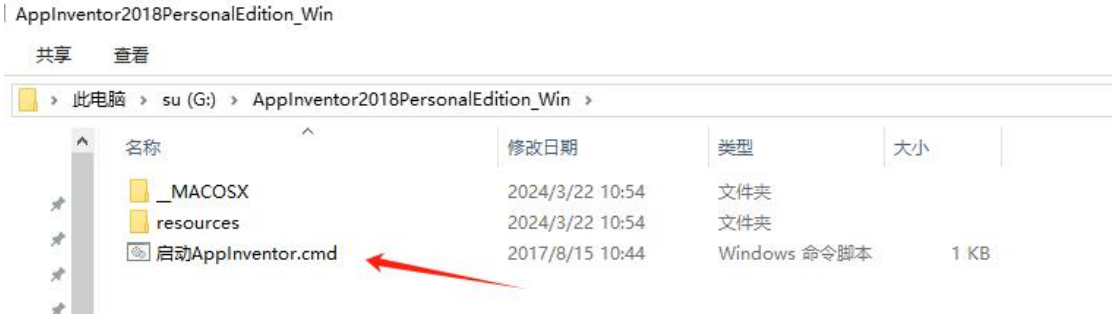


图 141 打开 APP Inventor

搜索 http://localhost:8888/进入。添加项目

图 142 添加项目

14. 导入 MQTT 插件

图 143 导入插件

15. 最终页面设计

图 144 设计页面

图 145 设计页面

16. 进入编程，拉入 MQTT 连接代码块，原理是创建一个新的客户端，连接的是同个账号密码。

17. 每一秒检测 MQTT 是否连接成功。

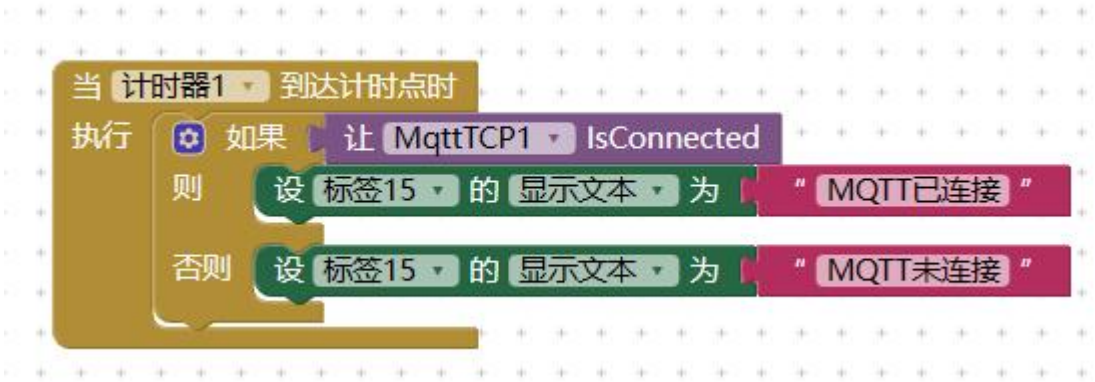


图 146 检测 MQTT 是否连接成功

18. 当连接成果 MQTT 后，读取 Modbus 数据

图 147 连接读取数据

19. 如果读取到相对应主题的数据，就输出数据。

图 148 数据判断

20. MQTT 反控，按钮被点击后，发布主题（true 为开，false 为关）

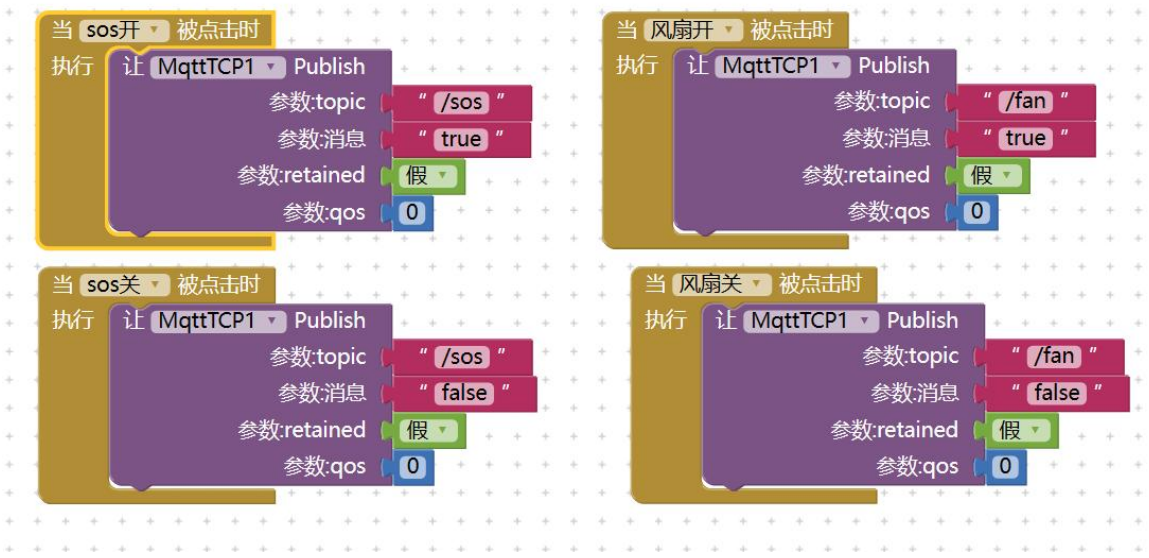


图 149 反控

21. 设定初始值。如果没用接收到相对应的数据，则就一直显示无数据。

图 150 设定初始值

图 151

22. 最终展示

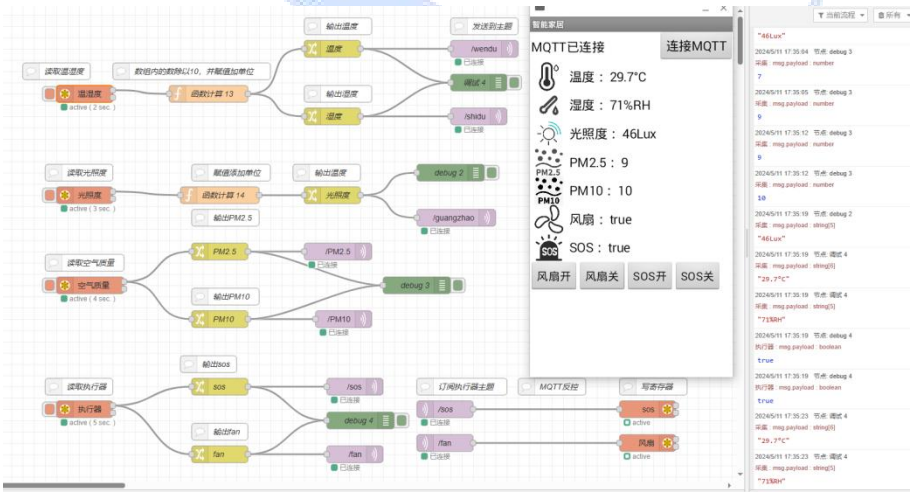


图 152 最终呈现

APP Inventor 的 aia [Modbus.aia]

14.5.4. 实验总结

总结来说，通过使用 Node-RED、MQTT 服务器和 APP Inventor，可以构建一个高效、灵活的工业自动化解决方案，实现 Modbus 设备数据的采集与远程反控。Node-RED 作为流编程工具，简化了 Modbus 设备与 MQTT 服务器之间的连接和数据传输。MQTT 服务器作为消息代理，确保数据的实时性和可靠性。而 APP Inventor 则提供了易于使用的应用开发平台，允许用户通过 MQTT 协议实时采集 Modbus 设备数据，并实现远程控制。这种解决方案不仅提高了工业自动化系统的效率，还增强了系统的灵活性和可维护性，为工业物联网应用提供了强大的技术支撑。

14.6. node-red 采集 modbus 设备数据到 mqtt 服务器，云平台通过 mqtt 进行采集与反控

14.6.1. 实验介绍

Node-RED 作为一个流编程工具，可以轻松实现 Modbus 设备数据的采集，并将其传输到 MQTT 服务器。通过 Node-RED 的 modbus 节点，可以直接与 Modbus 设备通信，读取或写入数据。一旦数据被采集，Node-RED 可以进一步通过 MQTT 节点将数据发布到 MQTT 服务器上。
云平台通过订阅 MQTT 服务器上相应的主题，可以实时接收 Modbus 设备的数据，实现远程监控。同时，云平台也可以通过 MQTT 协议向 Modbus 设备发送控制指令，实现远程反控。这种基于 MQTT 的通信方式，不仅实现了数据的实时传输，还保证了数据的安全性和可靠性。整个系统结构简洁，易于扩展和维护，为物联网应用提供了强大的数据采集与控制能力。

14.6.2. 实验准备

1. 搭建本地 MQTT 服务器。
2. 创建 MQTT 客户端。
3. 测试 MQTT 服务器。
4. 初步学会 MQTT 服务器与私有云平台对接收发数据。
5. 学会用 node-red 采集 Modbus 设备数据。根据 9.1.3 实验步骤进行学习。
6. 采集数据上发到云平台，云平台下发到 node-red 控制。

教学文档：

7. [金山文档] 本地 MQTT 服务器与私有云平台对接.docx

重点知识

- **创建用户**
  如下图所示，进入 emqx（127.0.0.1:18083）创建认证后，点击“用户管理”，进入用户管理界面。

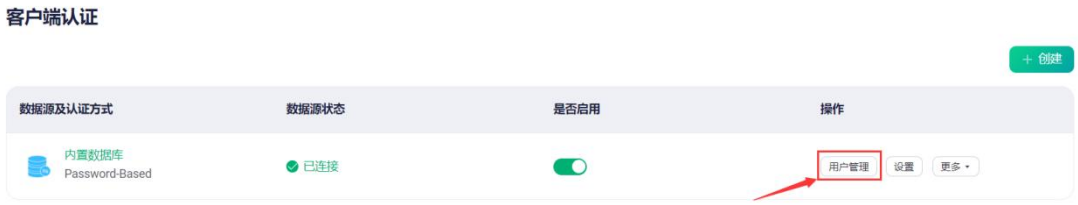


图 153

进入用户管理界面之后，点击“+”按钮，分别添加“MQTT1”和“MQTT2”用户，如下图所示。

图 154 创建客户端

配置传感器地址

设备通过 RS485 转 USB 线连接到电脑，并使用配置软件进行设备地址配置，然后通过网关进行管理和控制，这通常涉及一系列步骤。以下是一个大致的流程：

- 硬件连接：
- 将 RS485 转 USB 线的一端接入智能家居设备的 console 口（或 RS485 接口）。
- 将 RS485 转 USB 线的另一端接入电脑的 USB 口。
- 配置软件设置：
- 打开智能家居设备的配置软件。
- 在软件中选择正确的串行端口（即 RS485 转 USB 模块对应的端口）。根据设备的说明书或配置软件的指引，输入或选择正确的设备地址（如设备 ID、IP 地址等）。
- 配置其他必要的参数，如波特率、数据位、停止位等。

MQTT 与云平台收发主题

- “ClientID”就在创建 MQTT 设备时的 ClientID(此时我们的 ClientID 为（nodered)
- //订阅主题，必须要包含${ClientID}参数，即推送数据到物联网云平台组态软件 。**/sys/${ClientID}/post**
- //发布主题，必须要包含${ClientID}参数，即接收物联网云平台组态软件下发的数据。**/sys/${ClientID}/pub**
- 上发数据的格式示例：**{“properties”:{“wendu”:11}}**
“wendu”为 MQTT 设备的数据标识。

下发数据的格式：

- {{.ClientID}}为设备的 MQTT 客户端 ID，用于多个 MQTT 设备区分那个设备。
- {{.Identifier}}为设备的变量标识名（也可以直接理解为变量名）。
- {{.SetValue}}为要设置的设备的变量标识的值。

自定义格式：

- {"ClientID":"{{.ClientID}}","Identifier":"{{.Identifier}}","SetValue":{{.SetValue}}}

填写到下发格式可自由设定

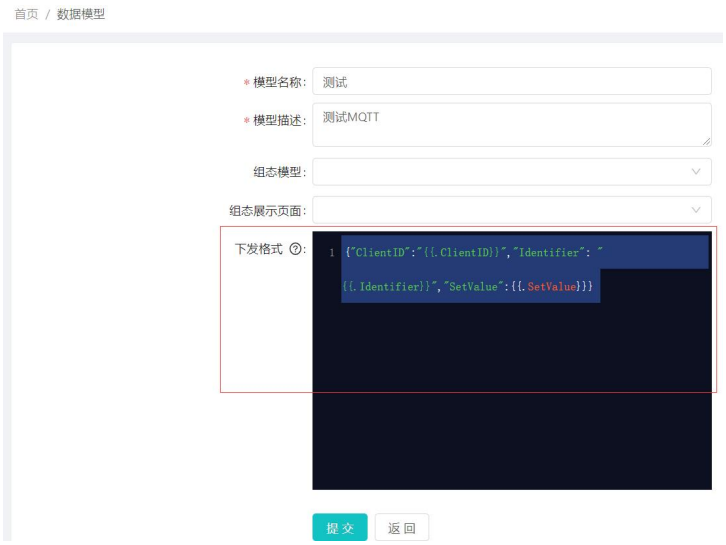


图 155 下发格式

14.6.3. 实验过程

1. 利用 RS485 转 USB 连接传感器和控制器到电脑，利用配置软件配置设备地址。最后得到：（网关的串口为 com1）

传感器	设备地址	线圈/寄存器地址
湿度	1	0
温度	1	1
光照度	2	6
PM2.5	3	0
PM10	3	1

表 9 传感器地址

4 路 IO 控制器	4	
IN:对射	4	0
IN:门磁	4	1
IN:人体	4	2
IN:烟雾	4	3
ON:SOS	4	0
ON:风扇	4	1

表 10 连接 IO 控制器设备的地址

2. 回到 Node-red，拉入 Modbus-read，和 debug，配置读寄存器。（对于监控系统来说，设置合适的 Poll Rate 非常重要。如果 Poll Rate 设置得太低（即查询频率太高）。相反，如果 Poll Rate 设置得太高（即查询频率太低），则可能会导致系统无法及时响应设备状态的变化或数据更新。如果“Poll Rate”被设置为 5 秒，那么这意味着系统每 5 秒钟会查询一次设备或传感器的状态或数据。(要在 debug 中判断系统给传感器赋值的数组是什么类型的)

传感器/执行器	Poll Rate	数组
温湿度	4s	msg.payload[i]
光照度	5s	msg.payload[i]
空气质量	6s	msg.payload[i]
执行器	7s	msg.payload[i]

表 11 node-red modbus 配置

3．进入网迅通物联网云平台，点击应用管理，创建应用，可以设计组态展示。

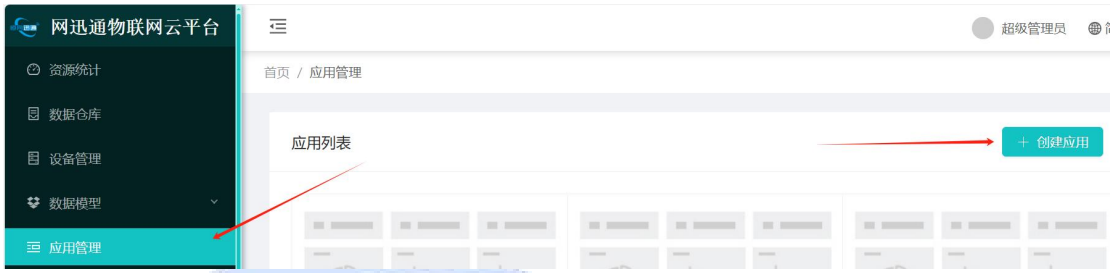


图 156 创建组态

4．添加 MQTT 设备进行数据采集。
点击数据模型，MQTT 设备，点击新建。

图 157 添加 MQTT 设备

图 158 添加 MQTT 设备

图 159 添加数据

图 160 添加数据

图 161 添加数据

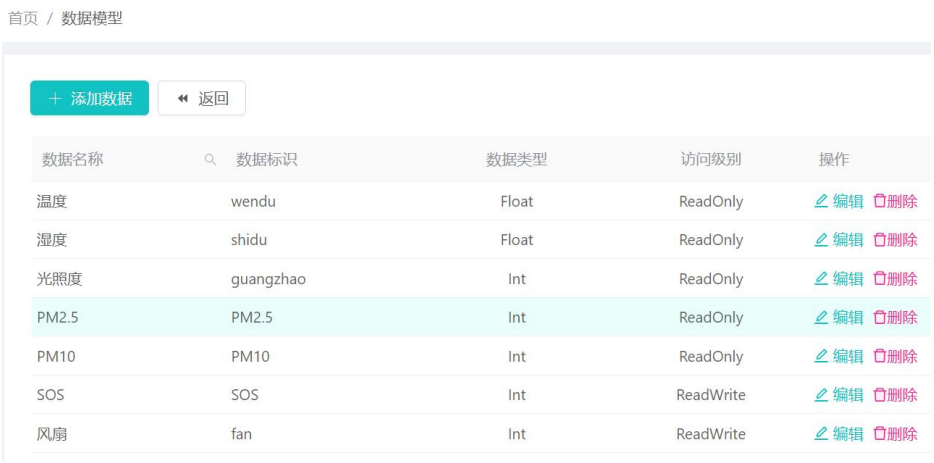


图 162 全部数据

设置下发格式。

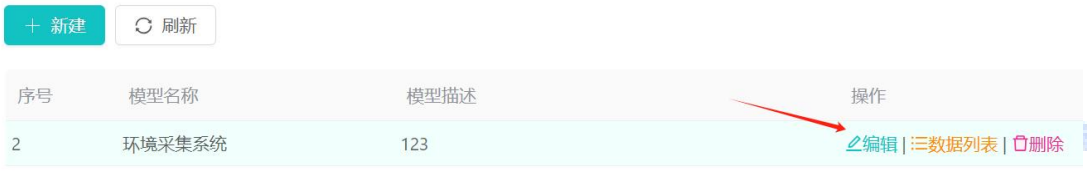


图 163 设置下发格式

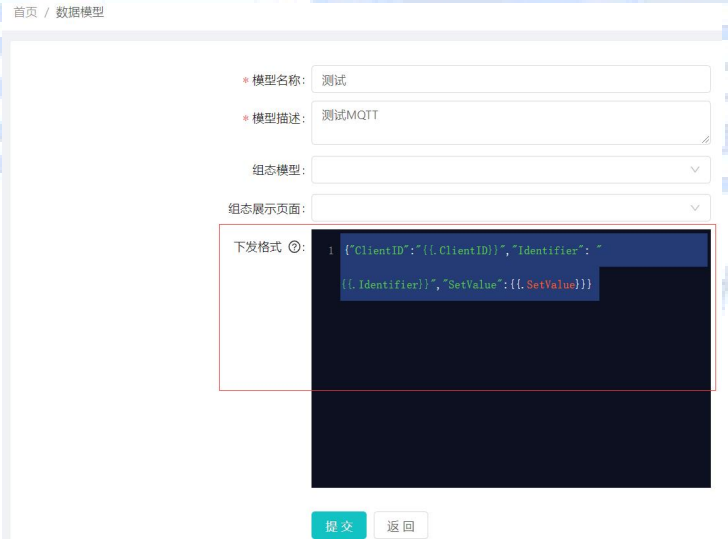


图 164 下发格式

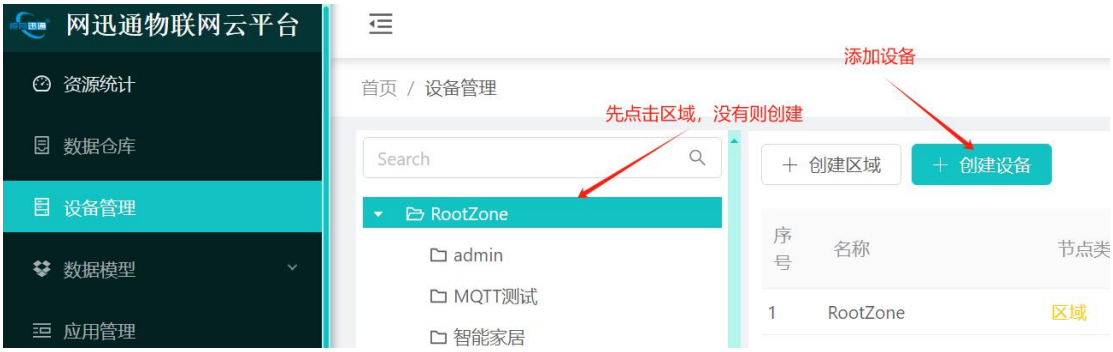


图 165 创建设备

图 166 添加设备

5. 读取温湿度

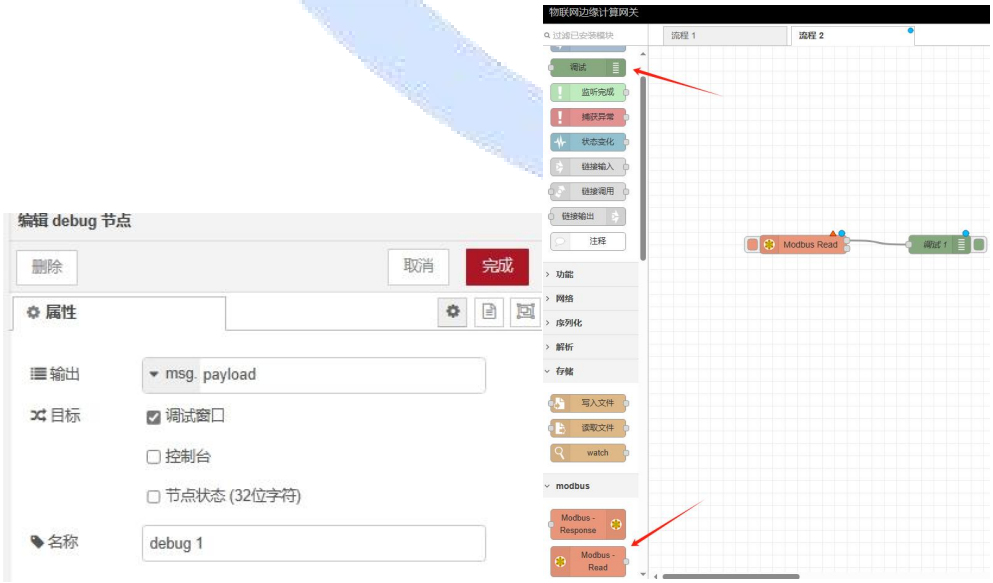


图 167 温湿度

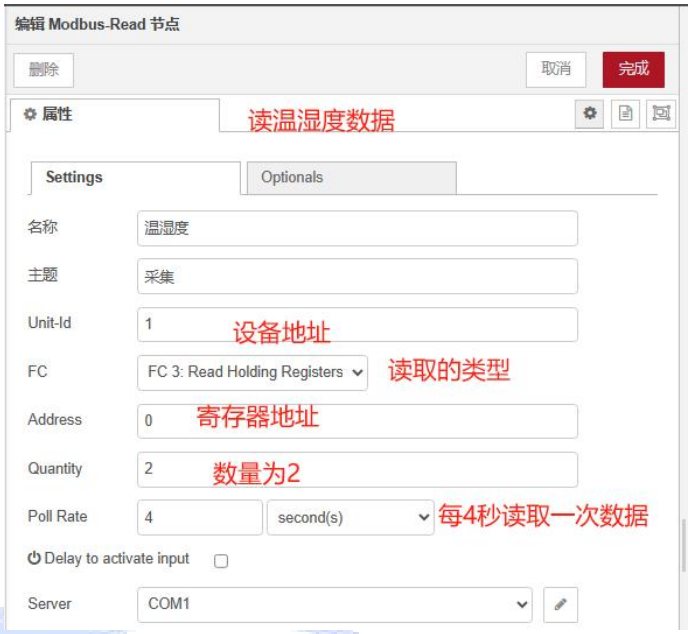


图 168 配置温湿度 read 节点

创建 Server

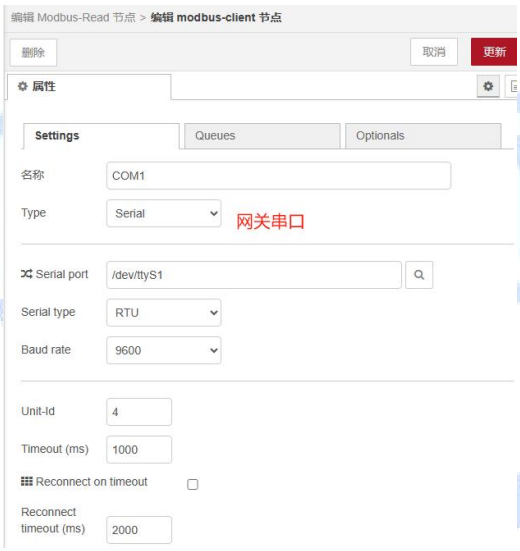


图 169 通信连接

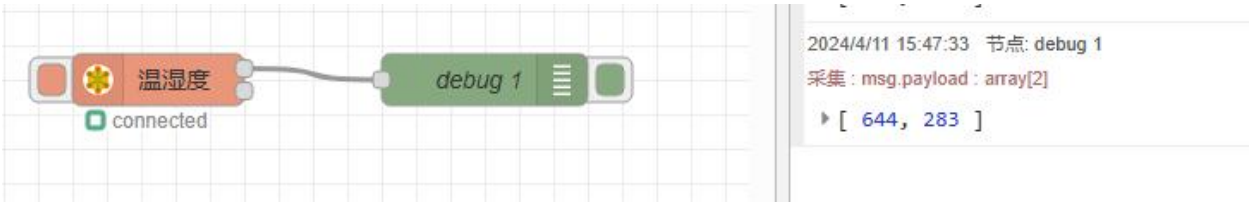


图 170 温湿度输出

由此可见,msg.payload[0]为湿度，msg.payload[1]为温度。

所以要将 2 个数值提取出来，分别以云平台支持的 JSON 格式发到 MQTT。

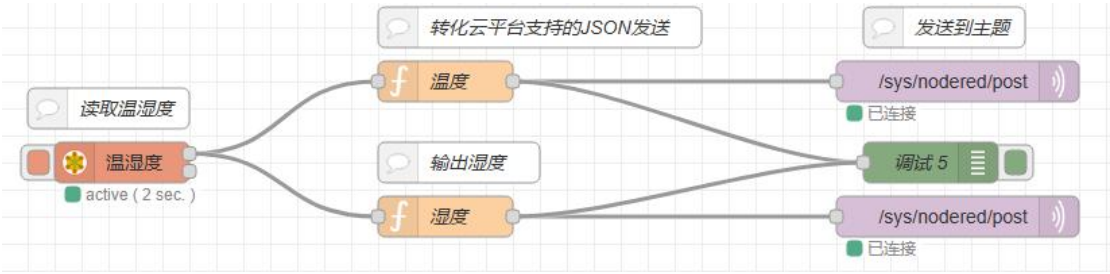


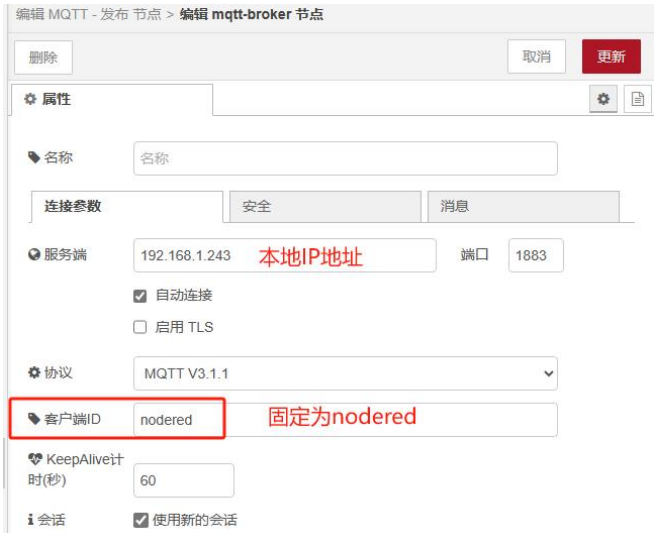
图 171 发送主题

温度函数:

图 172 温度函数

温度 MQTT 节点:

图 173 节点配置



配置服务器:

图 174 配置服务器

图 175 配置完点击更新

点击部署，查看调式结果，返回云平台数据仓库查看数据。

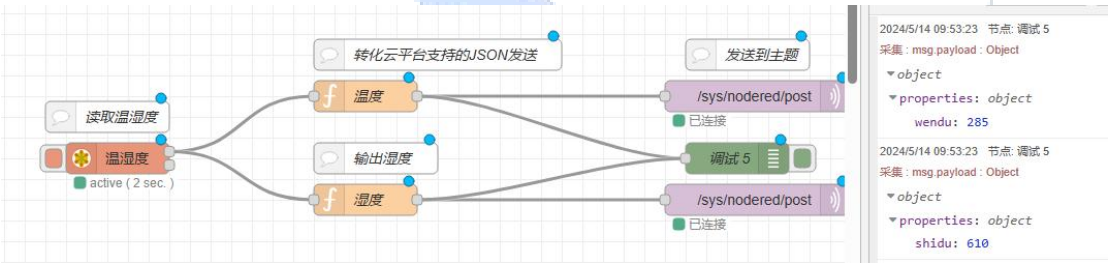


图 176 结果输出

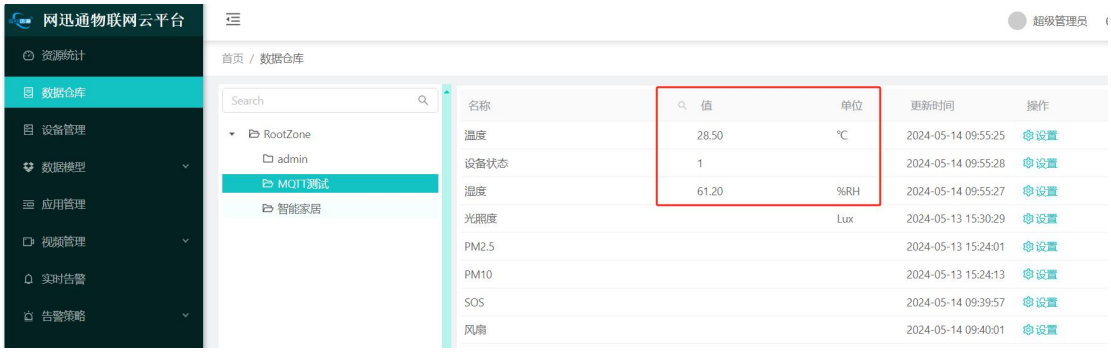


图 177 云平台接收到数据

6．综上所述，可以整体规划出节点设计。

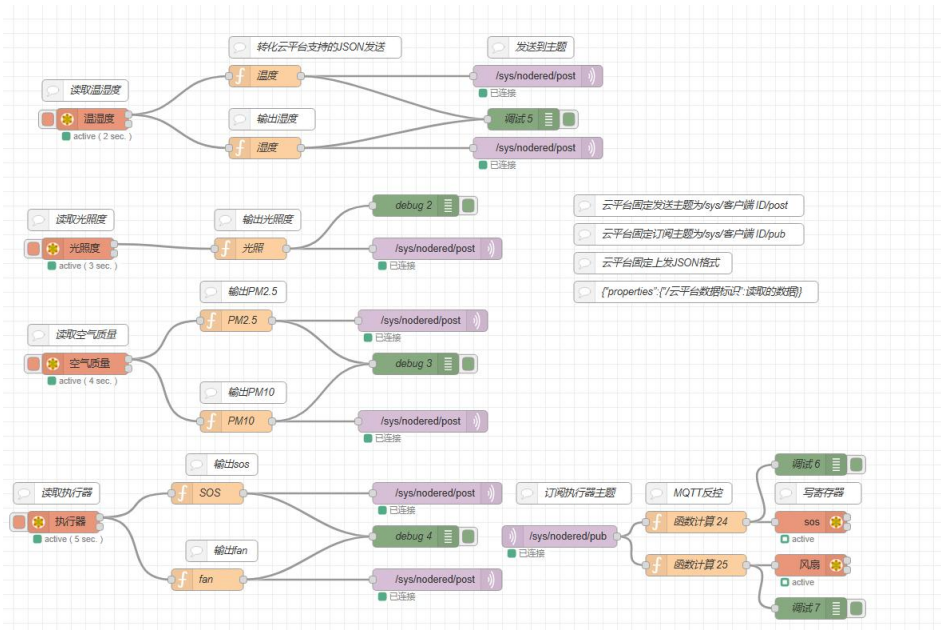


图 178 整体设计

- 光照度:

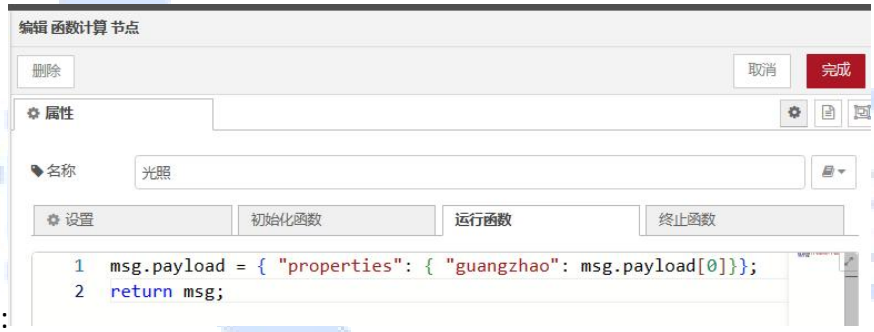


图 179 光照度

光照度 MQTT 节点配置一致。

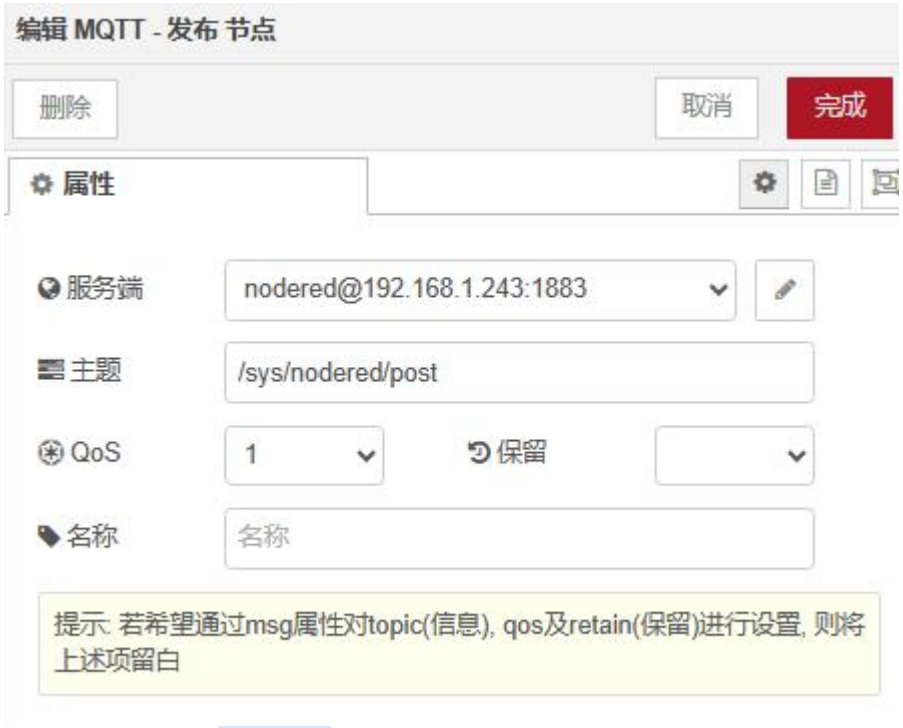


图 180 MQTT 节点

- PM2.5 函数：

图 181 PM2.5 函数

MQTT 节点配置一致。

- PM10 函数:

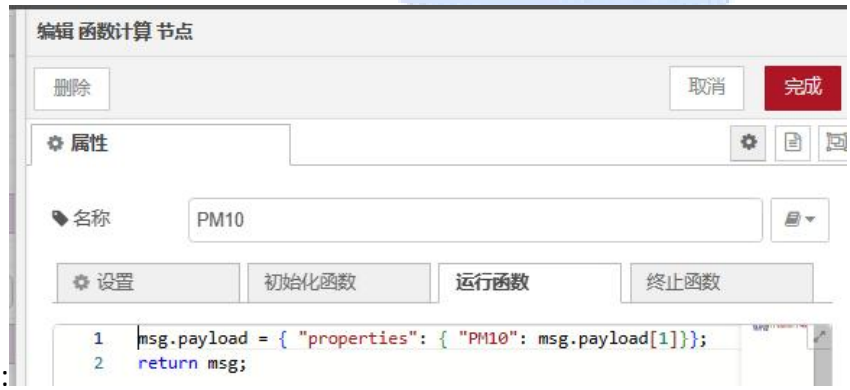


图 182 PM10 函数

MQTT 节点配置一致。

- SOS 函数:

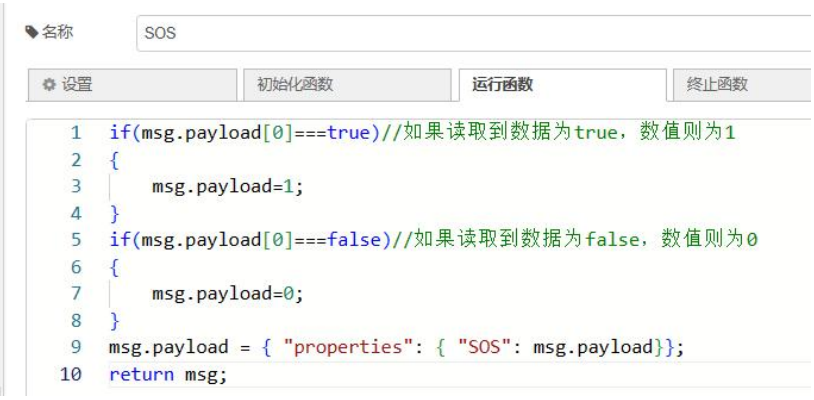


图 183 SOS 函数

MQTT 节点配置一致。

- fan 函数：

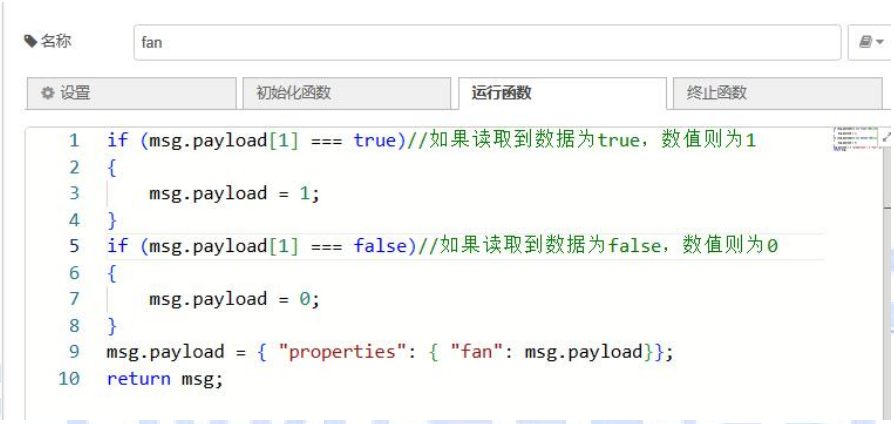


图 184 fan 函数

MQTT 节点配置一致。

7. 执行器反控。（云平台控制设备）

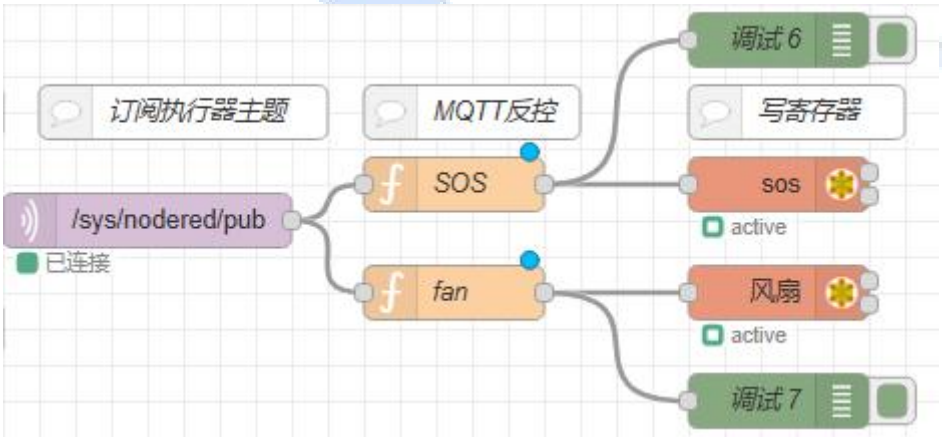


图 185 执行器反控

云平台下发主题固定为:/sys/客户端 ID/pub。下发格式

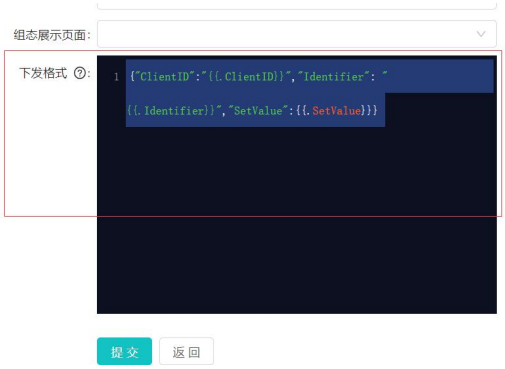


图 186 格式

MQTT 节点:

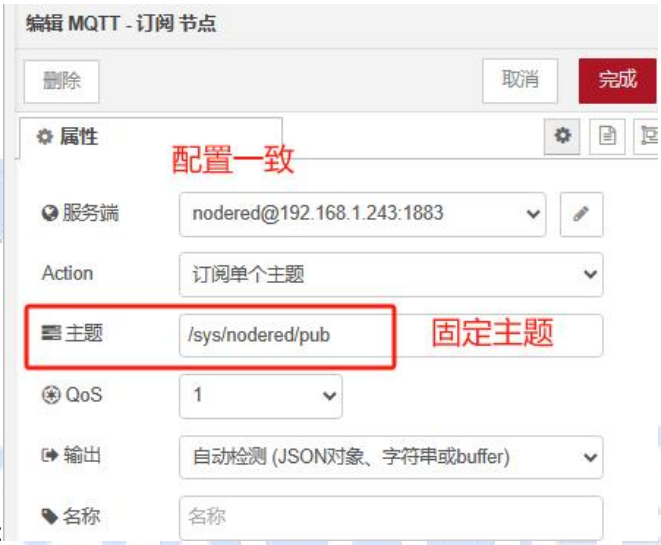


图 187 MQTT 节点

SOS 函数：

图 188 SOS 函数

fan 函数:

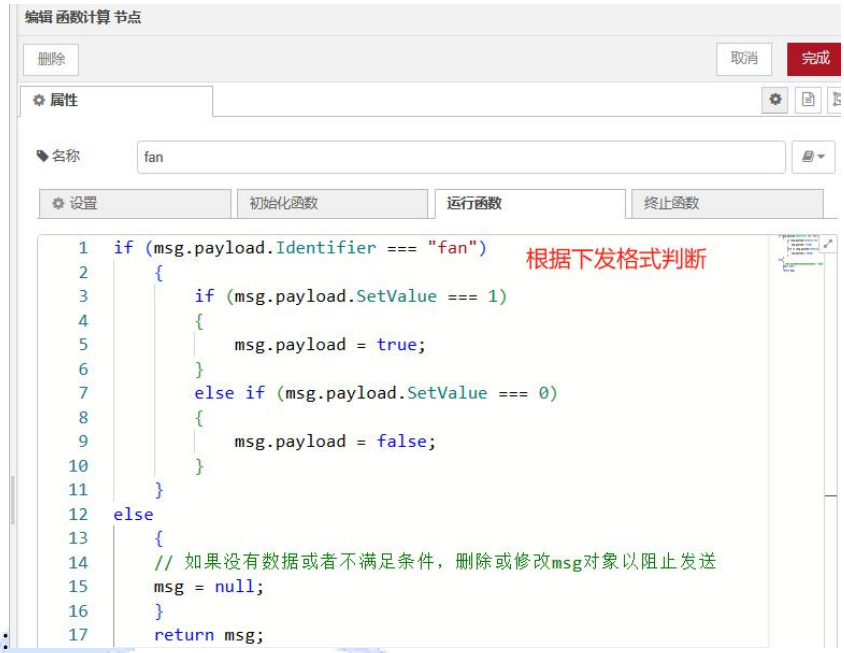


图 189 fan 函数

最后传入 Modbus-Write 节点写寄存器,有数据就开，没数据就关。

8. 回到云平台,进入数据仓库观察数据是否正常，测试是否能够进行反控。

图 190 云平台接收与下发

14.6.4. 实验总结

Node-RED、Modbus 与 MQTT 在工业自动化中协同工作的总结如下：

数据采集：

Node-RED 通过 Modbus 节点与 Modbus 设备通信，实时读取设备数据。

配置数据流，将数据通过 MQTT 节点发布到 MQTT 服务器。

MQTT 服务器：

作为消息代理，MQTT 服务器接收 Node-RED 发布的数据，并将其转发给订阅了该主题的客户端。

云平台：

云平台通过 MQTT 客户端订阅 MQTT 服务器上 Modbus 数据的主题。

实时接收数据，并进行存储、分析或触发相应操作。

反控：

云平台通过 MQTT 发布控制命令到特定主题。
Node-RED 监听这些主题，并根据接收到的命令通过 Modbus 节点控制设备。
优势：
实时性：数据实时采集与传输。
灵活性：易于配置和扩展，支持多种设备和系统。
可靠性：MQTT 协议适用于不稳定网络环境。
最终的 JSON 流:MQTT 云平台.json